

Rosa A. S
M. Shalahuddin

REKAYASA PERANGKAT LUNAK

TERSTRUKTUR dan BERORIENTASI OBJEK

Bahasan meliputi:

- Analisis dan Desain Sistem ■
- SDLC ■
- Basis Data ■
- Pemrograman Terstruktur ■
- Pemrograman Berorientasi Objek ■
- Analisis dan Desain Berorientasi Objek ■
- Pemodelan dan UML ■
- Studi Kasus UML ■
- Perancangan Pola Berorientasi Objek ■
- Manajemen Proyek Perangkat Lunak ■

Penerbit **INFORMATIKA**



Nama : Afriliyana
NIM : A1316012

Rekayasa Perangkat Lunak

Terstruktur dan Berorientasi Objek

Rosa A. S.

M. Shalahuddin



INFORMATIKA
Bandung

**Rekayasa Perangkat Lunak
(Terstruktur dan Berorientasi Objek)**

Penyusun : Rosa A.S.
M. Shalahuddin

Penerbit : Informatika Bandung

Pemasaran : **BI-Obses**
Pasar Buku Palasari No. 82
Bandung 40264
Telp. (022) 7317812
Fax. (022) 7317896

Cetakan Keempat : September 2016

ISBN : 978- 602-1514-05-4

Copyright © 2016 pada Penerbit INFORMATIKA Bandung

Kata Pengantar

Segala puji bagi Allah SWT yang telah melimpahkan nikmat-Nya sehingga buku **Rekayasa Perangkat Lunak** ini bisa diselesaikan. Buku ini diharapkan dapat menunjang mahasiswa dalam memahami mata kuliah Rekayasa Perangkat Lunak dan juga bagi para pengembang perangkat lunak agar memahami korelasi rekayasa perangkat lunak dan pembangunan/pengembangan perangkat lunak. Buku ini dibuat berdasarkan sumber-sumber yang banyak digunakan dan pengalaman penulis selama sekitar satu dekade menjadi *software engineer* profesional.

Pada buku ini akan membahas tentang analisis dan desain sistem dengan disertai suatu studi kasus untuk memudahkan dalam pemahaman. Analisis dan desain sistem itu dimulai dengan analisis dan desain basis data, analisis dan desain sistem untuk pemrograman terstruktur dengan menggunakan DFD, dan analisis dan desain sistem untuk pemrograman berorientasi objek dengan menggunakan UML.

Setelah membaca studi kasus tersebut, pembaca diharapkan dapat memahami bagaimana melakukan analisis dan desain sistem untuk pemrograman terstruktur maupun pemrograman berorientasi objek. Untuk menghasilkan analisis dan desain perangkat lunak yang baik, seorang analis seharusnya memahami konsep pemrograman.

Tanpa pemahaman konsep pemrograman yang baik, analisis tidak mungkin menghasilkan perancangan perangkat lunak yang baik.

Pada buku ini juga dijelaskan mengenai tahapan-tahapan yang harus dilalui dalam rekayasa perangkat lunak serta penjelasan secara umum tentang manajemen proyek perangkat lunak.

Pada kesempatan ini, penulis menyampaikan terima kasih yang tulus kepada berbagai pihak atas segala bantuan dan dukungannya sehingga penulis dapat menyelesaikan penulisan buku ini. Akhirnya, penulis mohon maaf jika dalam tulisan ini masih banyak kekurangan. Sumbangan ide, saran, dan kritik yang membangun untuk perbaikan buku ini sangat penulis harapkan.

Bandung, April 2013

Penulis

Daftar Isi

Kata Pengantar	iii
Daftar Isi.....	v
1 Pendahuluan	1
1.1 Perangkat Lunak.....	2
1.2 Rekayasa Perangkat Lunak.....	4
1.3 Proses Rekayasa Perangkat Lunak	8
1.4 Teknologi Informasi Sosial.....	11
2 Analisis dan Desain Sistem.....	17
2.1 Definisi Analisis Sistem	18
2.2 Teknik Pengumpulan Data.....	19
2.2.1 Teknik Wawancara.....	19
2.2.2 Teknik Observasi.....	20
2.2.3 Teknik Kuisisioner.....	21
2.3 Jenis Kebutuhan.....	21
2.4 Definisi Desain Sistem.....	23
3 SDLC	25
3.1 Pengertian SDLC	25
3.2 Model SDLC.....	28
3.2.1 Model <i>Waterfall</i>	28
3.2.2 Model Prototipe	31
3.2.3 Model <i>Rapid Application Development</i> (RAD)	34
3.2.4 Model Iteratif.....	38
3.2.5 Model Spiral.....	39

4	Basis Data	43
4.1	Pengertian Basis Data.....	43
4.2	DBMS.....	44
4.3	SQL	46
4.4	Alur Hidup Basis Data	48
4.5	ERD.....	50
4.6	Studi kasus.....	53
4.7	Studi kasus ERD	55
4.7.1	Definisi Entitas dan atribut	55
4.7.2	Definisi Relasi	57
4.7.3	Diagram ER.....	58
4.8	CDM.....	58
4.9	Studi kasus CDM	62
4.10	PDM	63
4.11	Studi Kasus PDM	64
5	Pemrograman Terstruktur.....	67
5.1	Pengertian Pemrograman Terstruktur.....	67
5.2	DFD.....	69
5.3	Kamus Data	73
5.4	Studi Kasus DFD.....	74
5.5	Studi Kasus Kamus Data	87
5.6	Implementasi DFD ke dalam Kode Program Studi Kasus.....	91
6	Pemrograman Berorientasi Objek.....	99
6.1	Pengertian Pemrograman Berorientasi Objek	100
6.2	Konsep Dasar Berorientasi Objek.....	103
6.3	Perbandingan Pendekatan OO dan Terstruktur.....	110
7	Analisis dan Desain Berorientasi Objek	113
7.1	Analisis Berorientasi Objek	114
7.2	Desain Berorientasi Objek.....	120
7.3	CASE Tools	122
7.4	RUP.....	124
7.4.1	Kelebihan RUP.....	126
7.4.2	FaseRUP.....	128

8	Pemodelan dan UML	133
8.1	Kompleksitas Pengembangan Perangkat Lunak	134
8.2	Pemodelan	135
8.3	Pengenalan UML	137
8.4	Sejarah UML	138
8.5	Diagram UML	140
8.6	<i>Class Diagram</i>	141
8.7	<i>Object Diagram</i>	147
8.8	<i>Component Diagram</i>	148
8.9	<i>Composite Structure Diagram</i>	150
8.10	<i>Package Diagram</i>	153
8.11	<i>Deployment Diagram</i>	154
8.12	<i>Use Case Diagram</i>	155
8.13	<i>Activity Diagram</i>	161
8.14	<i>State Machine Diagram</i>	163
8.15	<i>Sequence Diagram</i>	165
8.16	<i>Communication Diagram</i>	168
8.17	<i>Timing Diagram</i>	169
8.18	<i>Interaction Overview Diagram</i>	171
9	Studi Kasus UML	175
9.1	<i>Use Case</i>	176
9.2	Diagram Kelas	205
9.3	Diagram Objek	207
9.4	Diagram Sekuen	208
9.5	Diagram Komunikasi	222
9.6	Diagram Kolaborasi	226
9.7	Diagram Status	229
9.8	Diagram Aktivitas	233
9.9	Diagram Komponen	235
9.10	Diagram <i>Deployment</i>	236
10	Perancangan Pola Berorientasi Objek	239
10.1	Design Pattern	240
10.2	Anti Pattern	256

11	Manajemen Proyek Perangkat Lunak.....	263
11.1	Pengertian Manajemen Proyek Perangkat Lunak...	264
11.2	Perencanaan Proyek.....	269
11.3	Pengujian Perangkat Lunak.....	271
11.3.1	Pengujian Unit.....	277
11.3.2	Pengujian Integrasi.....	278
	Glosarium	287
	Indeks	293
	Daftar Pustaka	295

1

Pendahuluan



Overview

Bab ini merupakan pendahuluan sebelum menjelaskan inti materi buku ini terkait dengan Rekayasa Perangkat Lunak (RPL) atau *software engineering*. Bab ini menjelaskan tentang hal-hal sebagai berikut:

1. Pengertian perangkat lunak atau *software*
2. Pengertian rekayasa perangkat lunak atau *software engineering*
3. Pengenalan proses rekayasa perangkat lunak
4. Pengenalan tentang faktor sosial yang berkaitan dengan teknologi informasi

Setelah membaca bab ini, pembaca diharapkan memahami pengertian bidang rekayasa perangkat lunak dan istilah-istilah yang berkaitan dengan hal tersebut.

1.1 Perangkat Lunak

Perangkat lunak (*software*) adalah program komputer yang terasosiasi dengan dokumentasi perangkat lunak seperti dokumentasi kebutuhan, model desain, dan cara penggunaan (*user manual*). Sebuah program komputer tanpa terasosiasi dengan dokumentasinya maka belum dapat disebut perangkat lunak (*software*). Sebuah perangkat lunak juga sering disebut dengan sistem perangkat lunak. Sistem berarti kumpulan komponen yang saling terkait dan mempunyai satu tujuan yang ingin dicapai.

Sistem perangkat lunak berarti sebuah sistem yang memiliki komponen berupa perangkat lunak yang memiliki hubungan satu sama lain untuk memenuhi kebutuhan pelanggan (*customer*). Pelanggan (*customer*) adalah orang atau organisasi yang memesan atau membeli perangkat lunak (*software*) dari pengembang perangkat lunak atau bisa dianggap bahwa pelanggan (*customer*) adalah orang atau organisasi yang dengan sukarela mengeluarkan uang untuk memesan atau membeli perangkat lunak. *User* atau pemakai perangkat lunak adalah orang yang memiliki kepentingan untuk memakai atau menggunakan perangkat lunak untuk memudahkan pekerjaannya.

Karakter perangkat lunak adalah sebagai berikut:

- Perangkat lunak dibangun dengan rekayasa (*software engineering*) bukan diproduksi secara manufaktur atau pabrikan.
- Perangkat lunak tidak pernah usang (*"wear out"*) karena kecacatan dalam perangkat lunak dapat diperbaiki.
- Barang produksi pabrikan biasanya komponen barunya akan terus diproduksi, sedangkan perangkat lunak biasanya terus diperbaiki seiring bertambahnya kebutuhan.

Aplikasi dari perangkat lunak adalah sebagai berikut:

- Perangkat lunak sistem (*system software*) adalah kumpulan program dalam hal ini program yang satu ditulis untuk memenuhi kebutuhan program lainnya.

- Perangkat lunak waktu nyata (*real-time software*) merupakan perangkat lunak yang memonitor, menganalisis, mengontrol sesuatu secara waktu nyata (*real-time*). Reaksi yang dibutuhkan pada perangkat lunak harus langsung menghasilkan respon yang diinginkan.
- Perangkat lunak bisnis (*business software*) merupakan perangkat lunak pengelola informasi bisnis (seperti akuntansi, penjualan, pembayaran, penyimpanan (*inventory*)).
- Perangkat lunak untuk keperluan rekayasa dan keilmuan (*engineering and scientific software*) merupakan perangkat lunak yang mengimplementasikan algoritma yang terkait dengan keilmuan ataupun perangkat lunak yang membantu keilmuan, misalkan perangkat lunak di bidang astronomi, di bidang matematika dan lain sebagainya.
- Perangkat lunak tambahan untuk membantu mengerjakan suatu fungsi dari perangkat lunak yang lainnya (*embedded software*) misalnya perangkat lunak untuk mencetak dokumen ditambahkan agar perangkat lunak yang memerlukan dapat mencetak laporan, maka perangkat lunak untuk mencetak dokumen ini disebut *embedded software*.
- Perangkat lunak komputer personal (*personal computer software*) merupakan perangkat lunak untuk PC misalnya perangkat lunak pemroses teks, pemroses grafik dan lain sebagainya.
- Perangkat lunak berbasis web (*web based software*) merupakan perangkat lunak yang dapat diakses dengan menggunakan *browser*.

- Perangkat lunak berintelijensia buatan (*artificial intelligence software*) merupakan perangkat lunak yang menggunakan algoritma tertentu untuk mengelola data sehingga seakan-akan memiliki intelijensia seiring bertambahnya data yang diproses.

Produk perangkat lunak yang dibuat oleh pengembang (*developer*) perangkat lunak terdiri dari dua jenis:

- Produk generik
produk perangkat lunak yang dibuat oleh pengembang perangkat lunak untuk dijual atau dipopulerkan (*open source*) tanpa ada yang memesan terlebih dahulu, perangkat lunak yang termasuk dalam produk generik misalnya perangkat lunak sistem operasi, perangkat lunak pendukung perkantoran untuk membuat dokumen, *slide* presentasi, atau perhitungan dalam bentuk *papersheet* dan lain sebagainya.
- Produk pemesanan
produk perangkat lunak yang dibuat karena ada pelanggan yang melakukan pemesanan, misalnya sebuah instansi memerlukan perangkat lunak untuk memenuhi proses bisnis yang terjadi di instansinya, maka instansi itu akan bekerja sama dengan pengembang untuk membuat perangkat lunak yang diinginkan.

1.2 Rekayasa Perangkat Lunak

Rekayasa perangkat lunak (*software engineering*) merupakan pembangunan dengan menggunakan prinsip atau konsep rekayasa dengan tujuan menghasilkan perangkat lunak yang bernilai ekonomi yang dipercaya dan bekerja secara efisien menggunakan mesin. Perangkat lunak banyak dibuat dan pada akhirnya sering tidak digunakan karena tidak memenuhi kebutuhan pelanggan atau bahkan karena masalah non-teknis seperti keengganan pemakai perangkat lunak (*user*) untuk mengubah cara kerja dari manual ke otomatis, atau ketidakmampuan *user* menggunakan komputer. Oleh karena itu,

rekayasa perangkat lunak dibutuhkan agar perangkat lunak yang dibuat tidak hanya menjadi perangkat lunak yang tidak terpakai.

Rekayasa perangkat lunak lebih fokus pada praktik pengembangan perangkat lunak dan mengirimkan perangkat lunak yang bermanfaat kepada pelanggan (*customer*). Adapun ilmu komputer lebih fokus pada teori dan konsep dasar perangkat komputer. Rekayasa perangkat lunak lebih fokus pada bagaimana membuat perangkat lunak yang memenuhi kriteria berikut:

- dapat terus dipelihara setelah perangkat lunak selesai dibuat seiring berkembangnya teknologi dan lingkungan (*maintainability*)
- dapat diandalkan dengan proses bisnis yang dijalankan dan perubahan yang terjadi (*dependability* dan *robust*)
- efisien dari segi sumber daya dan penggunaan
- kemampuan untuk dipakai sesuai dengan kebutuhan (*usability*)

Dari kriteria di atas maka perangkat lunak yang baik adalah perangkat lunak yang dapat memenuhi kebutuhan pelanggan (*customer*) atau *user* (pemakai perangkat lunak) atau berorientasi pada pelanggan atau pemakai perangkat lunak, bukan berorientasi pada pembuat atau pengembang perangkat lunak.



Perangkat lunak yang baik adalah perangkat lunak yang fokus pada pengguna atau pelanggan

Pekerjaan yang terkait dengan rekayasa perangkat dapat dikategorikan menjadi tiga buah kategori umum tanpa melihat area dari aplikasi, ukuran proyek perangkat lunak, atau kompleksitas perangkat lunak yang akan dibuat. Setiap fase dialamatkan pada satu atau lebih pertanyaan yang diajukan sebelumnya.

Fase pendefinisian fokus pada "*what*" yang artinya harus mencari tahu atau mengidentifikasi informasi apa yang harus diproses, seperti

apa fungsi dan performansi yang diinginkan, seperti apa perilaku sistem yang diinginkan, apa kriteria validasi yang dibutuhkan untuk mendefinisikan sistem.

Fase pengembangan yang fokus dengan "*how*" yang artinya selama tahap pengembangan perangkat lunak seorang perekayasa perangkat lunak (*software engineer*) berusaha untuk mendefinisikan bagaimana data distrukturkan dan bagaimana fungsi-fungsi yang dibutuhkan diimplementasikan didalam arsitektur perangkat lunak, bagaimana detail prosedural diimplementasikan, bagaimana karakter antarmuka tampilan, bagaimana desai ditranslasikan ke bahasa pemrograman, dan bagaimana pengujian akan dijalankan.

Fase pendukung (*support phase*) fokus pada perubahan yang terasosiasi pada perbaikan kesalahan (*error*), adaptasi yang dibutuhkan pada lingkungan perangkat lunak yang terlibat, dan perbaikan yang terjadi akibat perubahan kebutuhan pelanggan (*customer*). Fase pendukung terdiri dari empat tipe perubahan antara lain:

- Koreksi (*correction*)
walaupun dengan jaminan kualitas yang terbaik, akan selalu ada kecacatan atau keinginan pelanggan (*customer*) yang tidak tertangani oleh perangkat lunak. Pemeliharaan dengan melakukan perbaikan terhadap kecacatan perangkat lunak.
- Adaptasi (*adaptation*)
pada saat tertentu lingkungan asli (seperti CPU, sistem operasi, aturan bisnis, karakteristik produk luar) dimana perangkat lunak dikembangkan akan mengalami perubahan. Pemeliharaan adaptasi merupakan tahap untuk memodifikasi perangkat lunak guna mengakomodasi perubahan lingkungan luar dimana perangkat lunak dijalankan.
- Perbaikan (*enhancement*)
sejalan dengan digunakannya perangkat lunak, maka pelanggan (*customer*) atau pemakainya (*user*) akan mengenali fungsi tambahan yang dapat mendatangkan manfaat.

Pemeliharaan perfektif atau penyempurnaan melakukan ekstensi atau penambahan pada kebutuhan fungsional sebelumnya.

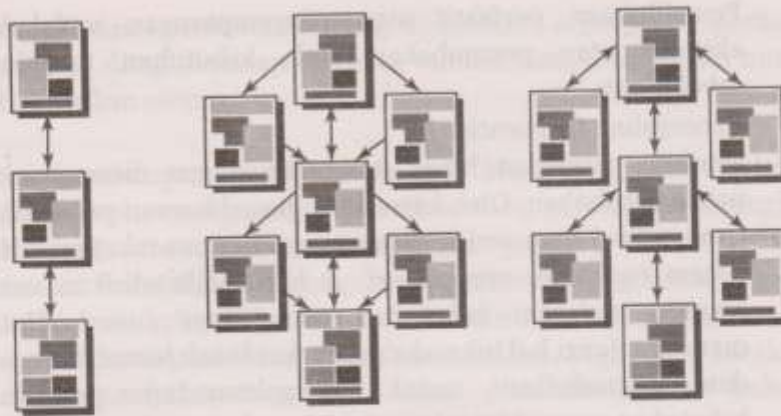
- Pencegahan (*prevention*)

keadaan perangkat lunak komputer sangat dimungkinkan untuk perubahan. Oleh karena itu, pemeliharaan pencegahan (*preventive*) atau sering disebut juga dengan rekayasa ulang sistem (*software reengineering*) harus dikondisikan untuk mampu melayani kebutuhan pemakainya (*user*). Untuk menanggulangi hal ini maka perangkat lunak harus dirancang dan dikondisikan untuk mengakomodasi perubahan kebutuhan yang diinginkan oleh pemakainya (*user*). Di lain sisi biasanya setelah perangkat lunak dikirimkan ke *user* maka masih dibutuhkan asistensi dan *help desk* dari pengembang perangkat lunak.

Tantangan yang dihadapi dari proses rekayasa perangkat lunak adalah sebagai berikut:

- tantangan warisan dimana perangkat lunak dikembangkan selama bertahun-tahun oleh orang-orang yang berbeda, hal ini dapat menyebabkan ketidakpahaman atau perubahan tujuan pembuatan perangkat lunak;
- tantangan heterogenitas dimana perangkat lunak harus dapat beradaptasi dengan teknologi yang terus berkembang dengan semakin luasnya lingkungan distribusi perangkat lunak;
- tantangan pengiriman bahwa perangkat lunak dengan skala besar dan kompleks sekalipun dapat sampai ke tangan pelanggan (*customer*) atau *user* dengan cepat dan kualitas tetap terjaga.

Ada suatu hal tambahan dalam rekayasa perangkat lunak yang berbasis *web* pada tahap desainnya yaitu bagaimana menentukan aliran halaman *web*. Beberapa aliran halaman *web* misalnya adalah sebagai berikut:



Gambar 1 Contoh aliran halaman web

1.3 Proses Rekayasa Perangkat Lunak

Proses rekayasa perangkat lunak dilakukan selama pembangunan perangkat lunak. Proses-proses yang dilakukan dalam rekayasa perangkat lunak secara garis besar adalah sebagai berikut:



Gambar 2 Tahapan Umum Rekayasa Perangkat Lunak

Proses-proses pada gambar di atas dapat dilakukan berulang kali sampai perangkat lunak memenuhi kebutuhan pelanggan atau *user*. Untuk membangun perangkat lunak yang benar-benar baik maka diperlukan tahapan-tahapan rekayasa perangkat lunak. Perangkat lunak yang dibangun tanpa melalui tahapan rekayasa perangkat lunak yang baik maka akan seperti membuat baju tanpa pola dan rencana model baju yang baik. Dalam membuat sebuah baju, maka dibutuhkan adanya perencanaan model baju dan pembuatan pola baju yang baik. Setelah pola baju terbentuk maka kain mulai dipotong sesuai pola, baru kemudian dijahit. Jika baju dibuat tanpa pola, maka yang terjadi adalah baju hasil tambal sulam. Seperti halnya pembuatan perangkat lunak tanpa melalui proses rekayasa perangkat lunak, maka hanya akan menghasilkan perangkat lunak "tambal sulam", begitu kurang maka akan ditambah. Tentu saja hasil perangkat lunak "tambal sulam" tidak akan bagus. Contoh lain misalnya sebuah bangunan tanpa rencana yang matang, maka yang terjadi adalah bangunan "tambal sulam" yang akhirnya dirobohkan atau bahkan roboh karena pembuatannya tanpa perencanaan yang matang. Misalkan awalnya dibuat tanpa jendela, lalu butuh jendela, maka ditambahkan jendela. Jika butuh tambahan lantai ke atas, maka ditambahkan lantai, tanpa melihat kekuatan lantai di bawahnya. Maka yang terjadi adalah bangunan yang ambruk jika tidak mampu menahan kebutuhan tanpa perencanaan yang baik. Berikut adalah ilustrasi celana yang dibuat tanpa perencanaan yang baik. Apakah Anda mau memakainya? Dari segi bentuk memang masih celana, tapi tidak enak untuk dipakai. Seperti halnya perangkat lunak, bentuknya akan tetap perangkat lunak, tapi tanpa rekayasa yang baik, maka tidak akan enak untuk digunakan.



Gambar 3 Celana tanpa rekayasa tidak akan menjadi celana yang berkualitas untuk dipakai, maukah Anda memakainya?

Perkembangan teknologi informasi saat ini sangat mempermudah dan mempercepat proses rekayasa perangkat lunak dengan adanya bermacam-macam *tools* desain, *tools* pengembangan seperti IDE (*Integrated Development Environment*), *application framework*, dan lain-lain. Hal tersebut sering disalahartikan oleh pengembang perangkat lunak pemula dan pelanggan/pengguna perangkat lunak bahwa pengembangan perangkat lunak bisa diselesaikan dalam satu malam.

Anggapan tersebut sangat tidak tepat. Rekayasa perangkat lunak terdiri dari beberapa kegiatan yang harus dilakukan. Jika tahapan-tahapan tersebut tidak dilalui dengan baik, hampir bisa dipastikan perangkat lunak yang dihasilkan tidak akan mempunyai kualitas yang baik. Jadi, tidak ada perangkat lunak yang bisa dihasilkan dengan baik hanya dalam satu malam seperti mitos pembuatan candi Prambanan atau mitos pembuatan Gunung Tangkuban Perahu.



Rekayasa perangkat lunak bukan seperti sulap yang bisa diselesaikan dalam sekejap, ada beberapa proses yang harus dilalui agar menghasilkan perangkat lunak yang berkualitas

Proses perangkat lunak (*software process*) adalah sekumpulan aktifitas yang memiliki tujuan untuk mengembangkan atau mengubah perangkat lunak. Secara umum proses perangkat lunak terdiri dari:

- Pengumpulan Spesifikasi (*Specification*)
mengetahui apa saja yang harus dapat dikerjakan sistem perangkat lunak dan batasan pengembangan perangkat lunak.
- Pengembangan (*Development*)
pengembangan perangkat lunak untuk menghasilkan sistem perangkat lunak.
- Validasi (*Validation*)
memeriksa apakah perangkat lunak sudah memenuhi keinginan pelanggan (*customer*).
- Evolusi (*Evolution*)
mengubah perangkat lunak untuk memenuhi perubahan kebutuhan pelanggan (*customer*).

1.4 Teknologi Informasi Sosial

Teknologi informasi sosial atau *social information technology* (*social IT*) adalah faktor atau aspek sosial yang berkaitan dengan implementasi suatu teknologi informasi. Sebuah produk perangkat lunak tidak hanya berkutat di masalah teknologi dan teknis. Sering sebuah produk perangkat lunak hanya dibuat berdasarkan sudut pandang pengembang perangkat lunak yang memiliki kecenderungan berpola pikir teknis.

Sebuah perangkat lunak dianggap berkualitas jika memenuhi kebutuhan pelanggan (*customer*) dan sesuai keinginan pelanggan (*customer*). Sering kendala dari pengembangan perangkat lunak bukan berada pada masalah teknis (teknologi perangkat lunak dan perangkat keras) tapi pada kondisi lingkungan pelanggan, misalnya ternyata yang akan menjadi *user* atau pemakai perangkat lunak belum bisa mengoperasikan komputer, atau keengganan *user* untuk mengubah kebiasannya bekerja menggunakan perangkat lunak yang baru atau dari manual menjadi menggunakan perangkat lunak, lingkungan yang tidak mendukung disiplin (misalnya untuk aplikasi keuangan) maka dari itu perlu adanya analisis terlebih dahulu di lingkungan pelanggan apakah dapat dijalankan perangkat lunak yang akan dikembangkan, karena sebaik apapun perangkat lunak jika tidak mampu digunakan di lingkungan pelanggan maka perangkat lunak itu hanya akan menjadi "artifak" yang disimpan di gudang atau bahkan dibuang.

Hal-hal yang harus dilakukan sebelum mengembangkan perangkat lunak di lingkungan tertentu maka harus dicari tahu:

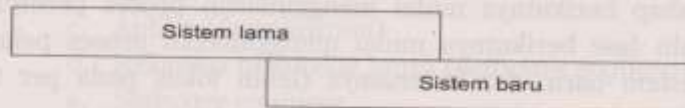
- pengetahuan lingkungan tentang teknologi informasi dan komputer
- "*social knowledge*" atau "*local knowledge*" (pengetahuan mengenai budaya lokal) di lingkungan yang akan dikembangkan perangkat lunak, apakah memungkinkan untuk dikembangkan perangkat lunak
- pengetahuan tentang apa saja yang bisa dibatasi dan yang tidak, sehingga saat pengembangan perangkat lunak dapat mendefinisikan aturan main dari perangkat lunak

Setelah perangkat lunak dikembangkan tetap masih diperlukan adanya sosialisasi perangkat lunak dengan mengadakan pelatihan secara bertahap, karena mengubah kebiasaan sebuah lingkungan ke sesuatu yang baru tidaklah gampang dan yang akan menggunakan perangkat lunak memiliki karakter yang berbeda-beda. Memang tidak harus menuruti semua karakter, tapi setidaknya dapat dicari titik tengah yang dapat diterima semua pihak. Komunikasi yang baik

antara pengembang dan pelanggan atau *user* sangat dibutuhkan agar terjalin kerja sama yang baik dan saling menguntungkan.

Melakukan konversi dari cara kerja yang lama ke cara kerja yang baru menggunakan perangkat lunak yang dikembangkan perlu dilakukan secara bertahap, karena perubahan yang ekstrim akan menghabiskan lebih banyak sumber daya, dana, dan waktu. Beberapa cara konversi adalah sebagai berikut:

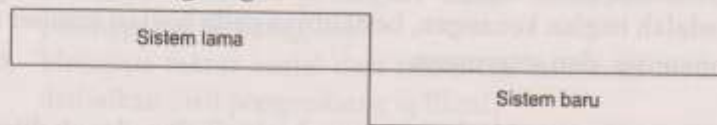
- konversi paralel



Gambar 4 Ilustrasi konversi paralel

konversi paralel dilakukan dengan melakukan beberapa waktu transisi dimana ada waktu dimana kedua sistem (sistem lama dan sistem baru) berjalan bersama untuk keperluan transisi sampai sistem baru dapat berjalan mandiri. Sumber daya yang dibutuhkan pada konversi paralel akan banyak terkuras pada waktu transisi.

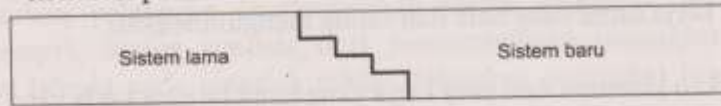
- konversi langsung



Gambar 5 Ilustrasi konversi langsung

konversi langsung dilakukan karena sistem lama secara ekstrim langsung diganti dengan sistem yang baru. Konversi ini akan mengalami waktu yang sangat sulit di awal berjalannya sistem baru.

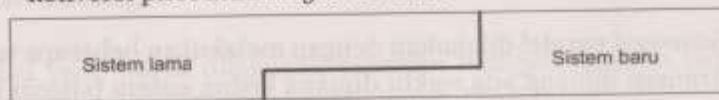
- konversi per fase



Gambar 6 Ilustrasi konversi per fase

konversi per fase dilakukan dengan berpindah per fase dari sistem lama ke sistem baru misalkan pada awal konversi hanya pada pekerjaan memasukkan data-data saja, pada tahap berikutnya mulai menggunakan proses perhitungan, lalu fase berikutnya mulai menggunakan proses pelaporan sistem baru, dan seterusnya (lebih fokus pada per fungsi sistem).

- konversi pilot atau *single location*



Gambar 7 Ilustrasi konversi pilot

konversi pilot dilakukan dengan melakukan konversi per unit kerja atau per lokasi di dalam sebuah lingkungan kerja. Misalnya pada tahap awal unit kerja yang sistemnya berubah adalah bagian keuangan, berikutnya pada bagian sumber daya manusia, dan seterusnya.

Pengembangan perangkat lunak yang berhasil dan dapat diterima dengan baik tidak hanya memperhatikan masalah teknis, tapi memperhatikan masalah nonteknis seperti permasalahan sosial dan mencermati isu-isu yang sedang berkembang di masyarakat.



1. Sebutkan definisi atau pengertian dari istilah-istilah sebagai berikut:
 - a. Perangkat lunak (*software*)
 - b. Sistem (*system*)
 - c. Aplikasi (*application*)
 - d. Rekayasa perangkat lunak (*software engineering*)
 - e. *Software engineer*
 - f. *Software developer*
 - g. *Programmer*
 - h. *Social IT*
2. Apakah proses produksi perangkat lunak identik atau serupa dengan proses produksi pada pabrik/manufaktur pembuatan mobil? Jelaskan alasannya!
3. Bidang rekayasa perangkat lunak apakah sebagai bagian dari seni atau bagian dari teknik? Jelaskan alasannya!
4. Mengapa ada proses-proses atau tahapan-tahapan yang harus dilakukan dalam rekayasa perangkat lunak?
5. Mengapa rekayasa perangkat lunak sebaiknya fokus pada pelanggan atau pengguna?
6. Mengapa faktor sosial dari teknologi informasi sering sekali diabaikan oleh pengembang aplikasi?
7. Mengapa faktor sosial dari teknologi informasi perlu untuk diperhatikan?
8. Sebutkan kelebihan dan kekurangan masing-masing metode konversi sistem yang sudah disebutkan di atas!

2

Analisis dan Desain Sistem



Overview

Bab ini menjelaskan mengenai kegiatan analisis sistem. Batasan mengenai hal-hal apa saja yang dilakukan pada tahap analisis berbeda-beda tergantung literatur yang digunakan. Pada bab ini yang dimaksud analisis sistem adalah mendefinisikan kebutuhan terkait sistem yang akan dikembangkan. Jadi, hasil akhir dari tahap analisis di sini adalah sebuah dokumen yang menjelaskan mengenai spesifikasi kebutuhan sistem informasi atau SRS (*Software Requirement Specification*)

Desain sistem informasi merupakan tahapan yang harus dilakukan berikutnya setelah analisis desain. Bab desain sistem ini berisi konsep dasar untuk melakukan perancangan sistem dengan menggunakan pendekatan berorientasi objek. Selain itu, pada bab ini juga ada perbandingan metode desain sistem berorientasi objek dengan pendekatan terstruktur.

2.1 Definisi Analisis Sistem

Kegiatan analisis sistem adalah kegiatan untuk melihat sistem yang sudah berjalan, melihat bagian mana yang bagus dan tidak bagus, dan kemudian mendokumentasikan kebutuhan yang akan dipenuhi dalam sistem yang baru. Hal tersebut terlihat sederhana, namun sebenarnya tidak. Banyak hambatan yang akan ditemui dalam proses tersebut.

Pada banyak proyek sistem informasi, proses analisis dan desain sering kali berjalan bersama-sama. Jadi selama kegiatan analisis, kegiatan desain juga dilakukan. Hal ini dilakukan karena pada banyak kasus, *user* sering kesulitan untuk mendefinisikan kebutuhan mereka. Jadi mereka akan lebih mudah mendefinisikan kebutuhan, jika mereka telah melihat gambar rancangan sistem yang baru, khususnya rancangan antarmuka.

Oleh karena itu, sering kali batasan mengenai bagian mana yang dianggap sebagai analisis dan bagian mana yang dianggap sebagai desain banyak terjadi perbedaan.

Misalnya untuk pemrograman berorientasi objek ada yang mengatakan bahwa *use case*, *analysis class*, dan *sequence diagram* merupakan bagian dari analisis. Namun, ada juga pihak lain yang menyatakan bahwa *use case* dan *sequence diagram* merupakan bagian dari desain, dan *analysis class* tidak ada karena sudah ada *design class*.

Pada buku ini yang dibahas pada bagian analisis adalah bagaimana metode pengumpulan data dan bagaimana mendokumentasikannya. Sedangkan *use case*, *class diagram*, dan *sequence diagram* dianggap merupakan bagian dari desain sistem dan akan dibahas pada bab yang terkait dengan UML.

2.2 Teknik Pengumpulan Data

Hal pertama yang dilakukan dalam analisis sistem adalah melakukan pengumpulan data. Ada beberapa teknik pengumpulan data yang sering dilakukan yaitu sebagai berikut.

- Teknik Wawancara
- Teknik Observasi
- Teknik Kuisioner

2.2.1 Teknik Wawancara

Pengumpulan data dengan menggunakan wawancara mempunyai beberapa keuntungan sebagai berikut.

- Lebih mudah dalam menggali bagian sistem mana yang dianggap baik dan bagian mana yang dianggap kurang baik
- Jika ada bagian tertentu yang menurut anda perlu untuk digali lebih dalam, anda dapat langsung menanyakan kepada narasumber
- Dapat menggali kebutuhan *user* secara lebih bebas
- *User* dapat mengungkapkan kebutuhannya secara lebih bebas.

Selain mempunyai beberapa kelebihan tersebut, teknik wawancara juga mempunyai beberapa kelemahan. Berikut ini adalah beberapa kelemahan dari teknik wawancara.

- Wawancara akan sulit dilakukan jika narasumber kurang dapat mengungkapkan kebutuhannya
- Pertanyaan dapat menjadi tidak terarah, terlalu fokus pada hal-hal tertentu dan mengabaikan bagian lainnya

Berikut ini adalah beberapa panduan dalam melakukan kegiatan wawancara agar memperoleh data yang diharapkan.

- Buatlah jadwal wawancara dengan narasumber dan beritahukan maksud dan tujuan wawancara
- Buatlah panduan wawancara yang akan anda jadikan arahan agar pertanyaan dapat fokus kepada hal-hal yang dibutuhkan
- Gunakan pertanyaan yang jelas dan mudah dipahami

- Cobalah untuk menggali mengenai kelebihan dan kekurangan sistem yang telah berjalan sebelumnya
- Anda boleh berimprovisasi dengan mencoba menggali bagian-bagian tertentu yang menurut anda penting
- Catat hasil wawancara tersebut

2.2.2 Teknik Observasi

Pengumpulan data dengan menggunakan observasi mempunyai keuntungan yaitu

- Analis dapat melihat langsung bagaimana sistem lama berjalan
- Mampu menghasilkan gambaran lebih baik jika dibanding dengan teknik lainnya

Sedangkan kelemahan dengan menggunakan teknik observasi adalah

- Membutuhkan waktu cukup lama karena jika observasi waktunya sangat terbatas maka gambaran sistem secara keseluruhan akan sulit untuk diperoleh
- Orang-orang yang sedang diamati biasanya perilakunya akan berbeda dengan perilaku sehari-hari (cenderung berusaha terlihat baik). Hal ini akan menyebabkan gambaran yang diperoleh selama observasi akan berbeda dengan perilaku sehari-hari
- Dapat mengganggu pekerjaan orang-orang pada bagian yang sedang diamati

Berikut ini adalah beberapa petunjuk untuk melakukan observasi

- Tentukan hal-hal apa saja yang akan diobservasi agar kegiatan observasi menghasilkan sesuai dengan yang diharapkan
- Mintalah izin kepada orang yang berwenang pada bagian yang akan diobservasi
- Berusaha sesedikit mungkin agar tidak mengganggu pekerjaan orang lain

- Jika ada yang Anda tidak mengerti, cobalah bertanya. Jangan membuat asumsi sendiri

2.2.3 Teknik Kuisioner

Pengumpulan data dengan menggunakan kuisioner mempunyai keuntungan yaitu

- Hasilnya lebih objektif, karena kuisioner dapat dilakukan kepada banyak orang sekaligus
- Waktunya lebih singkat

Sedangkan kelemahan pengumpulan data dengan menggunakan kuisioner adalah sebagai berikut

- Responden cenderung malas untuk mengisi kuisioner
- Sulit untuk membuat pertanyaan yang singkat, jelas, dan mudah dipahami

Berikut ini adalah beberapa cara yang dapat dilakukan untuk membuat teknik kuisioner menghasilkan data yang baik

- Hindari pertanyaan isian, lebih baik pilihan ganda, karena responden biasanya malas untuk menulis banyak, dan jika responden menuliskan sesuatu sering kali susah untuk dipahami. Dan juga dengan pertanyaan pilihan ganda, akan memudahkan Anda untuk melakukan rekapitulasi data hasil kuisioner
- Buatlah pertanyaan yang tidak terlalu banyak
- Buatlah pertanyaan yang singkat, padat, dan jelas

2.3 Jenis Kebutuhan

Kebutuhan (*requirement*) yang dikumpulkan dengan menggunakan wawancara, observasi, kuisioner, atau gabungan dari ketiga hal tersebut dapat dikelompokkan menjadi beberapa kategori sebagai berikut (tidak semua kebutuhan ini harus ada).

- *Functional requirement*
Kebutuhan yang terkait dengan fungsi produk, misalnya sistem informasi harus mampu mencetak laporan, sistem informasi harus mampu menampilkan grafik, dan lain-lain.
- *Development requirement*
Kebutuhan yang terkait *tools* untuk pengembangan sistem informasi baik perangkat keras maupun perangkat lunak, misalnya sistem informasi dikembangkan dengan menggunakan alat bantu Eclipse untuk pengembangan dan StarUML untuk pemodelan.
- *Deployment requirement*
Kebutuhan terkait dengan lingkungan di mana sistem informasi akan digunakan baik perangkat lunak maupun perangkat keras. Contoh kebutuhan ini misalnya sistem informasi harus mampu berjalan pada *server* dengan spesifikasi perangkat keras *memory* 4 GB DDR3, *processor* Intel Xeon Quad Core, dan spesifikasi sistem operasi Ubuntu Server 9.
- *Performance requirement*
Kebutuhan yang terkait dengan ukuran kualitas maupun kuantitas, khususnya terkait dengan kecepatan, skalabilitas, dan kapasitas. Misalnya sistem informasi tersebut harus mampu diakses oleh minimal 1000 orang pada waktu yang bersamaan.
- *Documentation requirement*
Kebutuhan ini terkait dengan dokumen apa saja yang akan disertakan pada produk akhir. Dokumen yang biasanya dihasilkan pada tahap akhir pengembangan sistem informasi antara lain dokumen teknis (mulai dari dokumen perencanaan proyek, analisis, desain, sampai pengujian), *user manual*, dan dokumen pelatihan.

- *Support requirement*

Kebutuhan yang terkait dukungan yang diberikan setelah sistem informasi digunakan. Dukungan teknis tersebut misalnya adanya pelatihan bagi calon pengguna.

- *Miscellaneous requirement*

Kebutuhan ini adalah kebutuhan-kebutuhan tambahan lainnya yang belum tercakup pada beberapa kategori kebutuhan yang telah terdefinisi di atas.

2.4 Definisi Desain Sistem

Desain atau perancangan dalam pembangunan perangkat lunak merupakan upaya untuk mengonstruksi sebuah sistem yang memberikan kepuasan (mungkin informal) akan spesifikasi kebutuhan fungsional, memenuhi target, memenuhi kebutuhan secara implisit atau eksplisit dari segi performansi maupun penggunaan sumber daya, kepuasan batasan pada proses desain dari segi biaya, waktu, dan perangkat. Kualitas perangkat lunak biasanya dinilai dari segi kepuasan pengguna perangkat lunak terhadap perangkat lunak yang digunakan.

Analisis dan desain sistem akan dijelaskan lebih lengkap pada bab-bab selanjutnya, mulai dari analisis dan desain basis data, pemrograman terstruktur, maupun pemrograman berorientasi objek.



1. Apakah yang dimaksud dengan analisis sistem?
2. Kegiatan apa saja yang dilakukan pada saat analisis sistem?
3. Sebutkan dan jelaskan macam-macam teknik pengumpulan data!
4. Sebutkan jenis-jenis kebutuhan pengembangan sistem informasi!
5. Apakah yang dimaksud dengan dokumen SRS?
6. Apa saja isi dokumen SRS?
7. Buatlah sebuah dokumen SRS untuk pengembangan sistem informasi apotek!
8. Apakah yang dimaksud dengan desain sistem?
9. Hal apa saja yang dilakukan pada tahap desain sistem?
10. Sebutkan dan jelaskan karakteristik apa saja yang terdapat pada sistem berorientasi objek!
11. Mengapa berkembang metodologi berorientasi objek?
12. Apa yang dimaksud dengan kelas dan objek? Gambarkan keterhubungan antara kelas dan objek!
13. Apa yang dimaksud dengan pendekatan terstruktur?
14. Apa perbedaan pendekatan terstruktur dengan pendekatan berorientasi objek?
15. Sebutkan minimal 2 metodologi lain (selain pendekatan terstruktur dan pendekatan berorientasi objek). Sebutkan karakteristiknya dan kemudian carilah kelebihan dan kekurangannya jika dibandingkan dengan pendekatan terstruktur dan pendekatan berorientasi objek!

3

SDLC



Overview

Bab ini berisi penjelasan mengenai SDLC. SDLC singkatan dari *Software Development Life Cycle* atau kadang disebut juga *System Development Life Cycle*. Pada bab ini juga dijelaskan beberapa model SDLC yang sering digunakan. Setelah membaca bab ini, diharapkan pembaca dapat memilih model SDLC mana yang tepat untuk rekayasa perangkat lunak yang mereka lakukan.

3.1 Pengertian SDLC

Pada awal pengembangan perangkat lunak, para pembuat program (*programmer*) langsung melakukan pengodean perangkat lunak tanpa menggunakan prosedur atau tahapan pengembangan perangkat lunak. Dan ditemuilah kendala-kendala seiring dengan perkembangan skala sistem-sistem perangkat yang semakin besar.

SDLC dimulai dari tahun 1960-an, untuk mengembangkan sistem skala usaha besar secara fungsional untuk para konglomerat pada jaman itu. Sistem-sistem yang dibangun mengelola informasi kegiatan dan rutinitas dari perusahaan-perusahaan yang berpotensi memiliki data yang besar dalam perkembangannya.

SDLC atau *Software Development Life Cycle* atau sering disebut juga *System Development Life Cycle* adalah proses mengembangkan atau mengubah suatu sistem perangkat lunak dengan menggunakan model-model dan metodologi yang digunakan orang untuk mengembangkan sistem-sistem perangkat lunak sebelumnya (berdasarkan *best practice* atau cara-cara yang sudah teruji baik). Seperti halnya proses metamorfosis pada kupu-kupu, untuk menjadi kupu-kupu yang indah maka dibutuhkan beberapa tahap untuk dilalui, sama halnya dengan membuat perangkat lunak, memiliki daur tahapan yang dilalui agar menghasilkan perangkat lunak yang berkualitas.

Tahapan-tahapan yang ada pada SDLC secara global adalah sebagai berikut:

- Inisiasi (*initiation*)
tahap ini biasanya ditandai dengan pembuatan proposal proyek perangkat lunak.
- Pengembangan konsep sistem (*system concept development*)
mendefinisikan lingkup konsep termasuk dokumen lingkup sistem, analisis manfaat biaya, manajemen rencana, dan pembelajaran kemudahan sistem.
- Perencanaan (*planning*)
mengembangkan rencana manajemen proyek dan dokumen perencanaan lainnya. Menyediakan dasar untuk mendapatkan sumber daya (*resources*) yang dibutuhkan untuk memperoleh solusi.

- Analisis kebutuhan (*requirements analysis*) menganalisis kebutuhan pemakai sistem perangkat lunak (*user*) dan mengembangkan kebutuhan *user*. Membuat dokumen kebutuhan fungsional.
- Desain (*design*) mentransformasikan kebutuhan detail menjadi kebutuhan yang sudah lengkap, dokumen desain sistem fokus pada bagaimana dapat memenuhi fungsi-fungsi yang dibutuhkan.
- Pengembangan (*development*) mengonversi desain ke sistem informasi yang lengkap termasuk bagaimana memperoleh dan melakukan instalasi lingkungan sistem yang dibutuhkan; membuat basis data dan mempersiapkan prosedur kasus pengujian; mempersiapkan berkas atau *file* pengujian, pengodean, pengompilasian, memperbaiki dan membersihkan program; peninjauan pengujian.
- Integrasi dan pengujian (*integration and test*) mendemonstrasikan sistem perangkat lunak bahwa telah memenuhi kebutuhan yang dispesifikasikan pada dokumen kebutuhan fungsional. Dengan diarahkan oleh staf penjamin kualitas (*quality assurance*) dan *user*. Menghasilkan laporan analisis pengujian.
- Implementasi (*implementation*) termasuk pada persiapan implementasi, implementasi perangkat lunak pada lingkungan produksi (lingkungan pada *user*) dan menjalankan resolusi dari permasalahan yang teridentifikasi dari fase integrasi dan pengujian.
- Operasi dan pemeliharaan (*operations and maintenance*) mendeskripsikan pekerjaan untuk mengoperasikan dan memelihara sistem informasi pada lingkungan produksi

(lingkungan pada *user*), termasuk implementasi akhir dan masuk pada proses peninjauan.

- Disposisi (*disposition*) mendeskripsikan aktifitas akhir dari pengembangan sistem dan membangun data yang sebenarnya sesuai dengan aktifitas *user*.

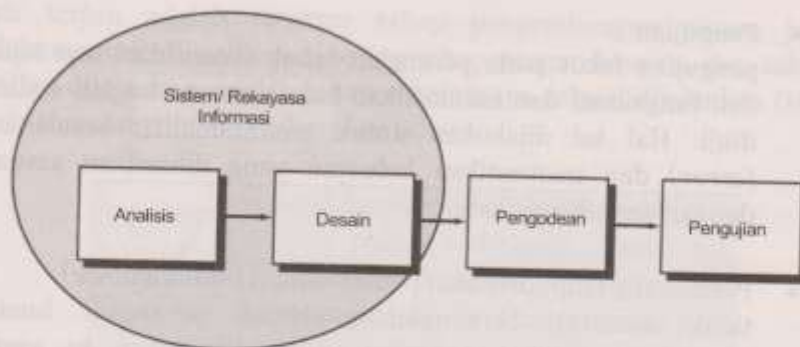
Analisis dan Desain sering dikelompokkan sebagai proses sistem/rekayasa informasi karena pada tahapan inilah informasi mengenai kebutuhan perangkat lunak banyak dikumpulkan dan diintegrasikan. Ada beberapa model SDLC yang dapat digunakan. Semuanya memiliki kelemahan dan kelebihan pada setiap model SDLC. Hal terpenting adalah mengenali tipe pelanggan (*customer*) dan memilih menggunakan model SDLC yang sesuai dengan karakter pelanggan (*customer*) dan sesuai dengan karakter pengembang.

3.2 Model SDLC

SDLC memiliki beberapa model dalam penerapan tahapan prosesnya. Beberapa model dasar akan dibahas pada subbab-subbab berikutnya. Selain model-model dasar yang akan dibahas, masih banyak model-model yang muncul dengan memodifikasi model-model SDLC dasar.

3.2.1 Model *Waterfall*

Model SDLC air terjun (*waterfall*) sering juga disebut model sekuensial linier (*sequential linear*) atau alur hidup klasik (*classic life cycle*). Model air terjun menyediakan pendekatan alur hidup perangkat lunak secara sekuensial atau terurut dimulai dari analisis, desain, pengodean, pengujian, dan tahap pendukung (*support*). Berikut adalah gambar model air terjun:



Gambar 8 Ilustrasi model waterfall

- **Analisis kebutuhan perangkat lunak**
proses pengumpulan kebutuhan dilakukan secara intensif untuk mespesifikasikan kebutuhan perangkat lunak agar dapat dipahami perangkat lunak seperti apa yang dibutuhkan oleh *user*. Spesifikasi kebutuhan perangkat lunak pada tahap ini perlu untuk didokumentasikan.
- **Desain**
desain perangkat lunak adalah proses multi langkah yang fokus pada desain pembuatan program perangkat lunak termasuk struktur data, arsitektur perangkat lunak, representasi antarmuka, dan prosedur pengodean. Tahap ini mentranslasi kebutuhan perangkat lunak dari tahap analisis kebutuhan ke representasi desain agar dapat diimplementasikan menjadi program pada tahap selanjutnya. Desain perangkat lunak yang dihasilkan pada tahap ini juga perlu didokumentasikan.
- **Pembuatan kode program**
desain harus ditranslasikan ke dalam program perangkat lunak. Hasil dari tahap ini adalah program komputer sesuai dengan desain yang telah dibuat pada tahap desain.

- **Pengujian**
pengujian fokus pada perangkat lunak secara dari segi logik dan fungsional dan memastikan bahwa semua bagian sudah diuji. Hal ini dilakukan untuk meminimalisir kesalahan (*error*) dan memastikan keluaran yang dihasilkan sesuai dengan yang diinginkan.
- **Pendukung (*support*) atau pemeliharaan (*maintenance*)**
tidak menutup kemungkinan sebuah perangkat lunak mengalami perubahan ketika sudah dikirimkan ke *user*. Perubahan bisa terjadi karena adanya kesalahan yang muncul dan tidak terdeteksi saat pengujian atau perangkat lunak harus beradaptasi dengan lingkungan baru. Tahap pendukung atau pemeliharaan dapat mengulangi proses pengembangan mulai dari analisis spesifikasi untuk perubahan perangkat lunak yang sudah ada, tapi tidak untuk membuat perangkat lunak baru.

Dari kenyataan yang terjadi sangat jarang model air terjun dapat dilakukan sesuai alurnya karena sebab berikut:

- perubahan spesifikasi perangkat lunak terjadi di tengah alur pengembangan.
- sangat sulit bagi pelanggan untuk mendefinisikan semua spesifikasi di awal alur pengembangan. Pelanggan sering kali butuh contoh (*prototype*) untuk menjabarkan spesifikasi kebutuhan sistem lebih lanjut.
- pelanggan tidak mungkin bersabar mengakomodasi perubahan yang diperlukan di akhir alur pengembangan.

Dengan berbagai kelemahan yang dimiliki model air terjun tapi model ini telah menjadi dasar dari model-model yang lain dalam melakukan perbaikan model pengembangan perangkat lunak.

Model air terjun sangat cocok digunakan kebutuhan pelanggan sudah sangat dipahami dan kemungkinan terjadinya perubahan kebutuhan selama pengembangan perangkat lunak kecil. Hal positif dari model

air terjun adalah struktur tahap pengembangan sistem jelas, dokumentasi dihasilkan di setiap tahap pengembangan, dan sebuah tahap dijalankan setelah tahap sebelumnya selesai dijalankan (tidak ada tumpang tindih pelaksanaan tahap).



Model *waterfall* adalah model SDLC yang paling sederhana. Model ini hanya cocok untuk pengembangan perangkat lunak dengan spesifikasi yang tidak berubah-ubah

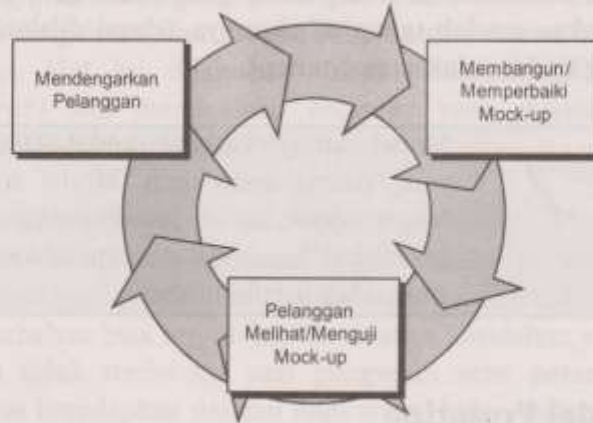
3.2.2 Model Prototipe

Sering pelanggan (*customer*) membayangkan kumpulan kebutuhan yang diinginkan tapi tidak terspesifikasikan secara detail dari segi masukan (*input*), proses, maupun keluaran (*output*). Di sisi lain seorang pengembang perangkat lunak harus menspesifikasikan sebuah kebutuhan secara detail dari segi teknis dimana pelanggan sering kurang mengerti mengenai hal teknis ini.

Model prototipe dapat digunakan untuk menyambungkan ketidakpahaman pelanggan mengenai hal teknis dan memperjelas spesifikasi kebutuhan yang diinginkan pelanggan kepada pengembang perangkat lunak.

Model prototipe (*prototyping model*) dimulai dari mengumpulkan kebutuhan pelanggan terhadap perangkat lunak yang akan dibuat. Lalu dibuatlah program prototipe agar pelanggan lebih terbayang dengan apa yang sebenarnya diinginkan. Program prototipe biasanya merupakan program yang belum jadi. Program ini biasanya menyediakan tampilan dengan simulasi alur perangkat lunak sehingga tampak seperti perangkat lunak yang sudah jadi. Program prototipe ini dievaluasi oleh pelanggan atau *user* sampai ditemukan spesifikasi yang sesuai dengan keinginan pelanggan atau *user*.

Berikut adalah gambar dari model prototipe:



Gambar 9 Ilustrasi model prototipe

Mock-up adalah sesuatu yang digunakan sebagai model desain yang digunakan untuk mengajar, demonstrasi, evaluasi desain, promosi, atau keperluan lain. Sebuah *mock-up* disebut sebagai prototipe perangkat lunak jika menyediakan atau mampu mendemonstrasikan sebagian besar fungsi sistem perangkat lunak dan memungkinkan pengujian desain sistem perangkat lunak. Iterasi terjadi pada pembuatan prototipe sampai sesuai dengan keinginan pelanggan (*customer*) atau *user*.

Seiring dengan mengembangkan prototipe maka sistem perangkat lunak yang sebenarnya dikembangkan juga sehingga sesuai dengan kebutuhan pelanggan (*customer*) atau *user*.

Model prototipe juga memiliki kelemahan sebagai berikut:

- Pelanggan dapat sering mengubah-ubah atau menambah-tambah spesifikasi kebutuhan karena menganggap aplikasi sudah dengan cepat dikembangkan, karena adanya iterasi ini dapat menyebabkan pengembang banyak mengalah dengan pelanggan karena perubahan atau penambahan spesifikasi kebutuhan perangkat lunak.

- Pengembang lebih sering mengambil kompromi dengan pelanggan untuk mendapatkan prototipe dengan waktu yang cepat sehingga pengembang lebih sering melakukan segala cara (tanpa idealis) guna menghasilkan prototipe untuk didemonstrasikan. Hal ini dapat menyebabkan kualitas perangkat lunak yang kurang baik atau bahkan menyebabkan iteratif tanpa akhir.

Permasalahan dapat terjadi pada model prototipe, hal ini dapat diatasi dengan melakukan perjanjian antara pengembang perangkat lunak dengan pelanggan (*customer*) atau *user* agar model prototipe hanya digunakan untuk mendefinisikan spesifikasi kebutuhan perangkat lunak, tapi tidak untuk seluruh proses pengembangan seluruh sistem perangkat lunak.

Model prototipe cocok digunakan untuk menjabarkan kebutuhan pelanggan secara lebih detail karena pelanggan sering kali kesulitan menyampaikan kebutuhannya secara detail tanpa melihat gambaran yang jelas. Untuk mengantisipasi agar proyek dapat berjalan sesuai dengan target waktu dan biaya di awal, maka sebaiknya spesifikasi kebutuhan sistem harus sudah disepakati oleh pengembang dengan pelanggan secara tertulis. Dokumen tersebut akan menjadi patokan agar spesifikasi kebutuhan sistem masih dalam ruang lingkup proyek.

Model prototipe kurang cocok untuk aplikasi dengan skala besar karena membuat prototipe untuk aplikasi skala besar akan sangat memakan waktu dan tenaga.

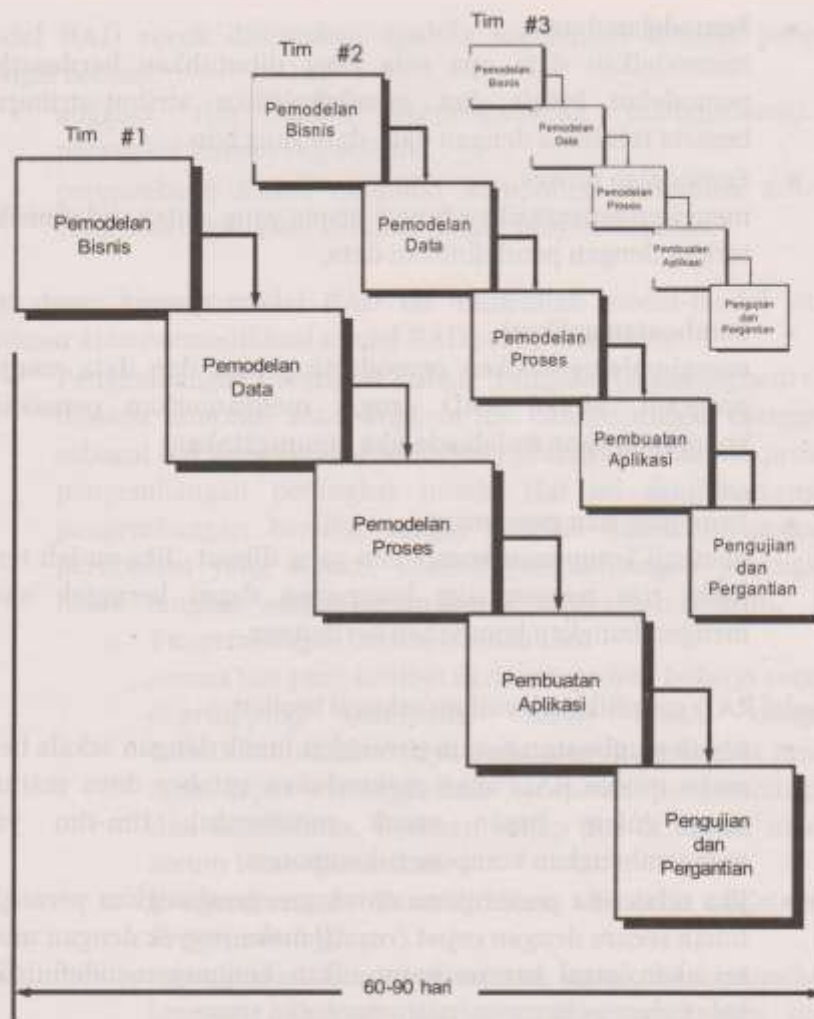


Model prototipe cocok digunakan untuk menggali spesifikasi kebutuhan pelanggan secara lebih detail tetapi beresiko tinggi terhadap membengkaknya biaya dan waktu proyek.

3.2.3 Model *Rapid Application Development* (RAD)

Rapid Application Development (RAD) adalah model proses pengembangan perangkat lunak yang bersifat inkremental terutama untuk waktu pengerjaan yang pendek. Model RAD adalah adaptasi dari model air terjun versi kecepatan tinggi dengan menggunakan model air terjun untuk pengembangan setiap komponen perangkat lunak.

Jika kebutuhan perangkat lunak dipahami dengan baik dan lingkup perangkat lunak dibatasi dengan baik sehingga tim dapat menyelesaikan pembuatan perangkat lunak dengan waktu yang pendek. Model RAD membagi tim pengembang menjadi beberapa tim untuk mengerjakan beberapa komponen masing-masing tim pengerjaan dapat dilakukan secara paralel. Berikut adalah gambar dari model RAD:



Gambar 10 Ilustrasi model RAD

- **Pemodelan bisnis**
pemodelan yang dilakukan untuk memodelkan fungsi bisnis untuk mengetahui informasi apa yang terkait proses bisnis, informasi apa saja yang harus dibuat, siapa yang harus membuat informasi itu, bagaimana alur informasi itu, proses apa saja yang terkait informasi itu.

- **Pemodelan data**
memodelkan data apa saja yang dibutuhkan berdasarkan pemodelan bisnis dan mendefinisikan atribut-atributnya beserta relasinya dengan data-data yang lain.
- **Pemodelan proses**
mengimplementasikan fungsi bisnis yang sudah didefinisikan terkait dengan pendefinisian data.
- **Pembuatan aplikasi**
mengimplementasikan pemodelan proses dan data menjadi program. Model RAD sangat menganjurkan pemakaian komponen yang sudah ada jika dimungkinkan.
- **Pengujian dan pergantian**
menguji komponen-komponen yang dibuat. Jika sudah teruji maka tim pengembang komponen dapat beranjak untuk mengembangkan komponen berikutnya.

Model RAD memiliki kelemahan sebagai berikut:

- untuk pembuatan sistem perangkat lunak dengan skala besar maka model RAD akan memerlukan sumber daya manusia yang cukup besar untuk membentuk tim-tim yang mengembangkan komponen-komponen;
- jika tidak ada persetujuan untuk mengembangkan perangkat lunak secara dengan cepat (*rapid*) maka proyek dengan model ini akan gagal karena hanya akan bingung mendefinisikan kebutuhan pelanggan (*customer*) atau *user*;
- jika sistem perangkat lunak yang akan dibuat tidak bisa dimodulkan (dibagi-bagi menjadi beberapa komponen) maka model RAD tidak dapat digunakan untuk membuat sistem perangkat lunak ini karena terlalu banyak campur tangan antar tim;
- model RAD tidak cocok digunakan untuk sistem perangkat lunak yang memiliki risiko teknis sangat tinggi, misalnya menggunakan teknologi baru yang belum banyak dikenal dan dikuasai pengembang.

Model RAD cocok diterapkan apabila memenuhi kriteria proyek sebagai berikut:

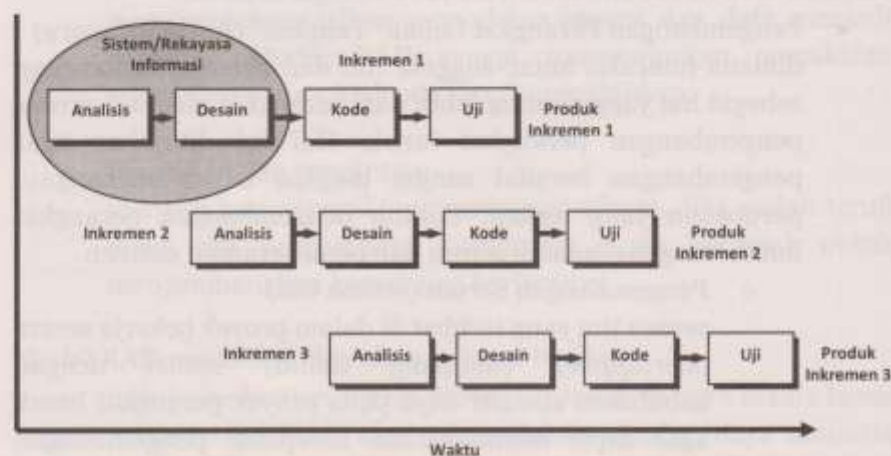
- anggota tim sudah berpengalaman mengembangkan perangkat lunak yang sejenis;
- pengembang sudah memiliki komponen-komponen sistem yang bisa digunakan kembali dalam proyek tersebut.

Dari dasar konsep model RAD ini muncullah model-model yang berdasar atau memodifikasi model RAD sebagai berikut:

- Pengembangan Perangkat Lunak "Tangkas" (*Agile Software*)
dimana interaksi antar-anggota tim dan pelanggan dianggap sebagai hal yang penting lebih dari perangkat ataupun proses pengembangan perangkat lunak. Hal ini ditujukan agar pengembangan bersifat sangat tangkas dalam menangani perubahan yang terjadi. Contoh pengembangan perangkat lunak "tangkas" adalah scrum dan pemrograman ekstrim.
 - Pengembangan Scrum (semua tim)
semua tim yang terlibat di dalam proyek bekerja secara *overlapping* (tumpang tindih) sesuai dengan kebutuhan sumber daya pada proyek perangkat lunak agar dapat meningkatkan kecepatan pengembangan dan fleksibilitas. Peranan setiap tim di dalam model scrum telah ditentukan.
 - Pengembangan Pemrograman Ekstrim (*Extreme Programming*)
mengizinkan tim pengembang untuk berkomunikasi langsung dengan pelanggan (*customer*) atau *user* maupun sesama pembuat program (*programmer*). Ciri khas dari model ini adalah komunikasi yang dilakukan setiap hari atau setiap ditemukan hal-hal yang kurang jelas. Model ini sangat mengandalkan adanya umpan balik (*feedback*) sehingga dibutuhkan anggota-anggota tim yang berkualitas.

3.2.4 Model Iteratif

Model iteratif (*iterative model*) mengkombinasikan proses-proses pada model air terjun dan iteratif pada model prototipe. Model inkremental akan menghasilkan versi-versi perangkat lunak yang sudah mengalami penambahan fungsi untuk setiap pertambahannya (inkremen/*increment*). Berikut adalah gambar dari model inkremental:



Gambar 11 Ilustrasi model iteratif

Model inkremental dibuat untuk mengatasi kelemahan dari model air terjun yang tidak mengakomodasi iterasi, dan mengatasi kelemahan dari metode prototipe yang memiliki proses terlalu pendek dan setiap iteratif prosesnya tidak selalu menghasilkan produk (bisa jadi hanya prototipe). Model inkremental menghasilkan produk/aplikasi untuk setiap tahapan inkremen.

Model inkremental sangat cocok digunakan jika staf yang dimiliki memiliki pergantian (*turnover*) yang tinggi sehingga staf tidak dapat terus ikut dalam pengembangan perangkat lunak. Mekanisme tahapan inkremental perlu direncanakan terlebih dahulu agar hasil produk dan pengerjaan setiap tahapan inkremen menjadi lebih baik.



Model iteratif merupakan gabungan dari model *waterfall* dan model prototipe. Model ini cocok digunakan pengembang dengan *turnover* staf yang tinggi

3.2.5 Model Spiral

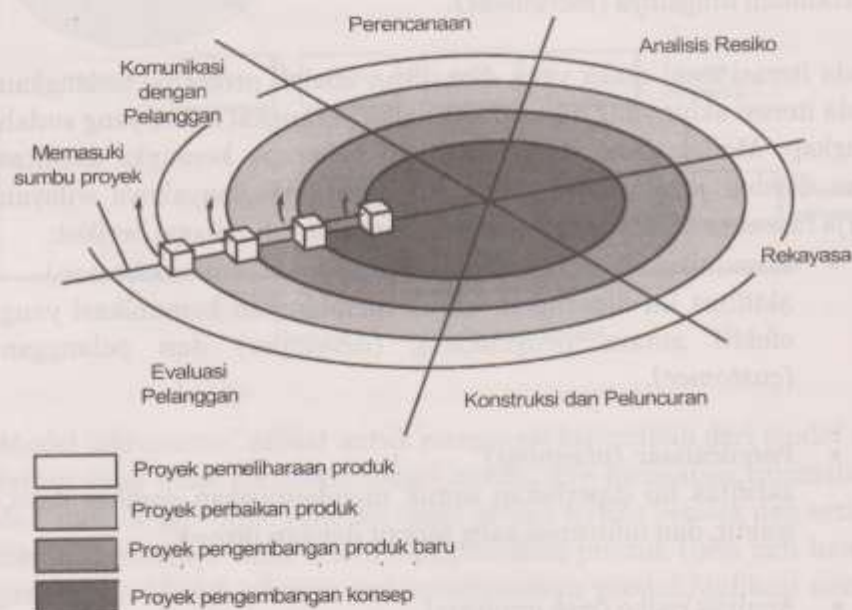
Model spiral (*spiral model*) memasangkan iteratif pada model prototipe dengan kontrol dan aspek sistematis yang diambil dari model air terjun. Model spiral menyediakan pengembangan dengan cara cepat dengan perangkat lunak yang memiliki versi yang terus bertambah fungsinya (*increment*).

Pada iterasi awal maka yang dihasilkan adalah prototipe sedangkan pada iterasi akhir yang dihasilkan adalah perangkat lunak yang sudah lengkap. Model spiral dibagi menjadi beberapa kerangka aktifitas atau disebut juga wilayah kerja (*task region*). Banyaknya wilayah kerja biasanya diantara tiga sampai enam wilayah sebagai berikut:

- Komunikasi dengan pelanggan (*customer communication*)
aktifitas ini diperlukan untuk membangun komunikasi yang efektif antara pengembang (*developer*) dan pelanggan (*customer*).
- Perencanaan (*planning*)
aktifitas ini diperlukan untuk mendefinisikan sumber daya, waktu, dan informasi yang terkait dengan proyek.
- Analisis risiko (*risk analysis*)
aktifitas ini diperlukan untuk memperkirakan risiko dari segi teknis maupun manajemen.
- Rekayasa (*engineering*)
aktifitas ini diperlukan untuk membangun satu atau lebih representasi dari aplikasi perangkat lunak (dapat juga berupa prototipe).

- Konstruksi dan peluncuran (*construction and release*) aktifitas ini dibutuhkan untuk mengonstruksi, menguji, melakukan instalasi, dan menyediakan dukungan terhadap *user* (misalnya dari segi dokumentasi dan pelatihan).
- Evaluasi pelanggan (*customer evaluation*) aktifitas ini dibutuhkan untuk mendapatkan umpan balik berdasarkan evaluasi representasi perangkat lunak yang dihasilkan dari proses rekayasa dan diimplementasikan pada tahap instalasi.

Berikut adalah gambar model spiral:



Gambar 12.1 Ilustrasi model spiral

Setiap wilayah kerja terdiri dari kumpulan pekerjaan (*task set*) yang tergantung pada karakteristik proyek yang sedang dikerjakan. Semakin besar suatu proyek maka kumpulan pekerjaan di dalam setiap wilayah kerja juga semakin banyak.

Model spiral dilakukan searah dengan jarum jam dimulai dari sumbu proyek. Sumbu proyek dapat digunakan sebagai awal iterasi ataupun evaluasi dari iterasi yang sudah dilakukan. Pada gambar di atas setiap wilayah kerja dibedakan dengan warna wilayah, dimana setiap wilayah berputar dengan urutan kerja tertentu.

Pada model spiral, hasil akhir dan evaluasi dari sebuah wilayah kerja akan menjadi inisiasi dari wilayah kerja berikutnya. Model spiral cocok digunakan untuk mengembangkan sistem perangkat lunak berskala besar karena memiliki proses analisis risiko yang dapat sangat meminimalisir risiko yang mungkin terjadi. Model spiral memungkinkan pengembang untuk menggunakan prototipe pada setiap tahap untuk mengurangi risiko.

Dari beberapa model SDLC yang sudah disebutkan sebelumnya, model spiral merupakan model yang bisa memberikan jaminan kualitas yang paling baik untuk aplikasi berskala besar. Penerapan model spiral cocok digunakan untuk suatu proyek dengan target waktu dan biaya yang tidak terlalu ketat. Setiap perubahan spesifikasi pasti berisiko pada molornya waktu pengerjaan dan membengkaknya biaya proyek.



Model spiral cocok digunakan untuk mengembangkan aplikasi dengan skala besar tetapi target waktu dan biaya tidak terlalu mengikat.

4.1 Pengertian Basis Data

Basis data adalah suatu komputasi yang mampu menyimpan, mengelola data yang sudah terdistribusi informasi dan membuat informasi tersebut lebih efisien. Pada dasarnya basis data adalah media untuk menyimpan data yang dapat diakses dengan mudah dan cepat. Pada basis data terdistribusi basis data terdistribusi



1. Jelaskan pengertian SDLC dan sebutkan tahapan-tahapannya secara umum!
2. Apa kelebihan dan kekurangan masing-masing model sebagai berikut:
 - a. Model *waterfall*
 - b. Model prototipe
 - c. Model RAD
 - d. Model iteratif
 - e. Model spiral
3. Apa risiko yang dihadapi jika pengembangan aplikasi (rekayasa perangkat lunak) tidak mengikuti tahapan-tahapan SDLC?
4. Sebutkan alasan munculnya SDLC!
5. Mengapa model *waterfall* dianggap sebagai model SDLC yang paling sederhana dan hanya cocok digunakan untuk aplikasi skala kecil?
6. Mengapa ketika menerapkan metode prototipe, pada tahap awal harus benar-benar diperjelas batasan-batasan/ruang lingkup/ spesifikasi perangkat lunak secara umum?
7. Mengapa metode RAD bisa memberikan hasil yang lebih cepat dibandingkan metode *waterfall*?
8. Mengapa metode iteratif cocok digunakan untuk pengembang dengan *turnover* staf yang tinggi?

4

Basis Data



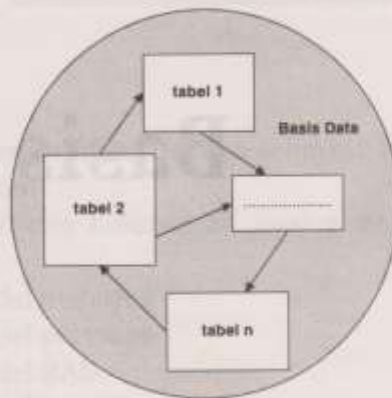
Overview

Pada bab ini, berisi penjelasan umum mengenai basis data dan tahap-tahap yang dilakukan untuk membuat desain basis data yang digunakan pada sistem informasi. Perancangan basis data merupakan salah satu bagian dalam rekayasa perangkat lunak. Pada bab ini hanya dijelaskan secara umum dan tidak terlalu detail tentang perancangan basis data. Perancangan basis data secara detail dipelajari dalam mata kuliah/buku yang lain yang lebih fokus pada basis data.

4.1 Pengertian Basis Data

Sistem basis data adalah sistem terkomputerisasi yang tujuan utamanya adalah memelihara data yang sudah diolah atau informasi dan membuat informasi tersedia saat dibutuhkan. Pada intinya basis data adalah media untuk menyimpan data agar dapat diakses dengan mudah dan cepat. Pada buku ini menggunakan basis data relasional

yang diimplementasikan dengan tabel-tabel yang saling memiliki relasi seperti pada gambar berikut.



Gambar 13 Ilustrasi Basis Data

Sistem informasi tidak dapat dipisahkan dengan kebutuhan akan basis data apapun bentuknya, entah berupa *file* teks ataupun *Database Management System* (DBMS).

Kebutuhan basis data dalam sistem informasi meliputi:

- memasukkan, menyimpan, dan mengambil data
- membuat laporan berdasarkan data yang telah disimpan

Tujuan dari dibuatnya tabel-tabel di sini adalah untuk menyimpan data ke dalam tabel-tabel agar mudah diakses. Oleh karena itu, untuk merancang tabel-tabel yang akan dibuat maka dibutuhkan pola pikir penyimpanan data nantinya jika dalam bentuk baris-baris data (*record*) dimana setiap baris terdiri dari beberapa kolom.

4.2 DBMS

DBMS (*Database Management System*) atau dalam bahasa Indonesia sering disebut sebagai Sistem Manajemen Basis Data adalah suatu sistem aplikasi yang digunakan untuk menyimpan,

mengelola, dan menampilkan data. Suatu sistem aplikasi disebut DBMS jika memenuhi persyaratan minimal sebagai berikut:

- Menyediakan fasilitas untuk mengelola akses data
- Mampu menangani integritas data
- Mampu menangani akses data yang dilakukan secara
- Mampu menangani *backup* data

Karena pentingnya data bagi suatu organisasi/perusahaan, maka hampir sebagian besar perusahaan memanfaatkan DBMS dalam mengelola data yang mereka miliki. Pengelolaan DBMS sendiri biasanya ditangani oleh tenaga ahli yang spesialis menangani DBMS yang disebut sebagai DBA (*Database Administrator*).

DBMS sudah mulai berkembang sejak tahun 1960an. Kemudian sekitar tahun 1970an mulai berkembang teknologi *Relational DBMS* yaitu DBMS berbasis relasional model. Relasional model pertama kali dikembangkan oleh Edgar J. Codd pada tahun 1970. Secara sederhana relasional model dapat dipahami sebagai suatu model yang memandang data sebagai sekumpulan tabel yang saling terkait. Hampir semua DBMS komersial dan *open source* saat ini berbasis *Relational DBMS* atau RDBMS.

Pada tahun 1980an mulai berkembang *Object Oriented DBMS* (OODBMS). OODBMS berkembang seiring dengan perkembangan teknologi pemrograman berorientasi objek. Secara umum dapat diartikan bahwa OODBMS merupakan DBMS yang memandang data sebagai suatu objek. Saat ini OODBMS juga cukup berkembang namun belum dapat menggeser kepopuleran RDBMS.

Berikut ini adalah 4 macam DBMS versi komersial yang paling banyak digunakan di dunia saat ini, yaitu:

- Oracle
- Microsoft SQL Server
- IBM DB2
- Microsoft Access

Sedangkan DBMS versi *open source* yang cukup berkembang dan paling banyak digunakan saat ini adalah sebagai berikut:

- MySQL
- PostgreSQL
- Firebird
- SQLite

Hampir semua DBMS mengadopsi SQL sebagai bahasa untuk mengelola data pada DBMS.

4.3 SQL

SQL (*Structured Query Language*) adalah bahasa yang digunakan untuk mengelola data pada RDBMS. SQL awalnya dikembangkan berdasarkan teori aljabar relasional dan kalkulus.

SQL mulai berkembang pada tahun 1970an. SQL mulai digunakan sebagai standar yang resmi pada tahun 1986 oleh ANSI (*American National Standards Institute*) dan pada tahun 1987 oleh ISO (*International Organization for Standardization*) dan disebut sebagai SQL-86. Pada perkembangannya, SQL beberapa kali dilakukan revisi. Berikut ini sejarah perkembangan SQL sampai saat ini:

No.	Tahun	Nama
1	1986	SQL-86
2	1989	SQL-89
3	1992	SQL-92
4	1999	SQL:1999
5	2003	SQL:2003
6	2006	SQL:2006
7	2008	SQL:2008
8	2011	SQL:2011

Meskipun SQL diadopsi dan diacu sebagai bahasa standar oleh hampir sebagian besar RDBMS yang beredar saat ini, tetapi tidak semua standar yang tercantum dalam SQL diimplementasikan oleh

seluruh DBMS tersebut. Sehingga kadang-kadang ada perbedaan perilaku (hasil yang ditampilkan) oleh DBMS yang berbeda padahal *query* yang dimasukkan sama.

Berikut ini adalah contoh pengaksesan data pada DBMS dengan SQL yang secara umum terdiri dari 4 hal sebagai berikut:

- Memasukkan data (*insert*)

```
INSERT INTO Tabel_mahasiswa
(nim, nama, tanggal_lahir)
VALUES
('13501058', 'Rosa', '1986-01-01');
```

Query di atas digunakan untuk memasukkan data mahasiswa dengan NIM 13501058, nama Rosa, dan tanggal lahir 1 Januari 1986 ke tabel "Tabel_mahasiswa".

- Mengubah data (*update*)

```
UPDATE Tabel_mahasiswa
SET
    tanggal_lahir = '1990-03-04'
WHERE
    nim = '13501058';
```

Query di atas digunakan untuk mengubah data tanggal lahir mahasiswa dengan NIM = 13501058 menjadi 4 Maret 1990 dalam tabel "Tabel_mahasiswa".

- Menghapus data (*delete*)

```
DELETE FROM Tabel_mahasiswa
WHERE
    nim = '13501058';
```

Query di atas digunakan untuk menghapus data mahasiswa dengan NIM = 13501058 dari tabel "Tabel_mahasiswa".

- Menampilkan data (*select*)

```
SELECT nim,  
nama  
FROM Tabel_mahasiswa  
WHERE  
nim = '13501058';
```

Query di atas digunakan untuk menampilkan data mahasiswa yang tersimpan dalam "Tabel_mahasiswa" dengan NIM = 13501058.



Hampir sebagian besar DBMS yang beredar saat ini memanfaatkan SQL sebagai bahasa untuk pengelolaan data

4.4 Alur Hidup Basis Data

Tidak hanya perangkat lunak yang memiliki alur hidup, dalam membuat perencanaan basis data juga memiliki alur hidup atau *Database Life Cycle* (DBLC). Alur hidup basis data dapat dilihat pada gambar berikut:



Gambar 14 Alur Hidup Basis Data

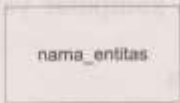
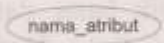
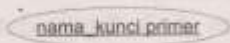
Fase-fase DBLC antara lain:

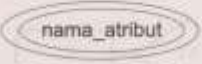
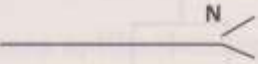
- Analisis kebutuhan / *requirement analysis*
Hal-hal yang harus dilakukan pada tahap ini adalah:
 - Didefinisikan dengan mewawancarai produsen dan pemakai data, data apa sajakah yang butuh untuk disimpan dan terkait dengan aplikasi komputer yang akan dikembangkan
 - Membuat kontrak spesifikasi basis data
 - *Entity Relationship Diagram* (ERD) sebagai bagian dari desain konseptual
- Desain logik basis data / *logical database design*
Pada tahap ini harus dibuat rancangan logik basis data. Biasanya pada tahap ini dibuat *Conceptual Data Model* (CDM).
- Desain fisik basis data / *physical database design*
Pada tahap ini harus dibuat rancangan fisik basis data. Biasanya pada tahap ini dibuat *Physical Data Model* (PDM).

- Implementasi
 - Membuat Query SQL
 - Aplikasi ke DBMS atau *file*

4.5 ERD

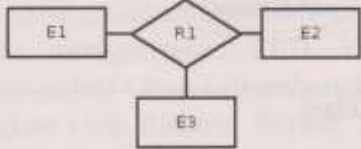
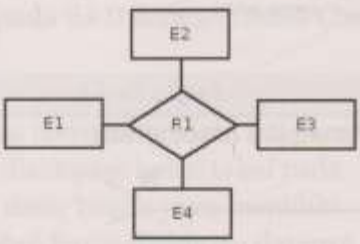
Pemodelan awal basis data yang paling banyak digunakan adalah menggunakan *Entity Relationship Diagram* (ERD). ERD dikembangkan berdasarkan teori himpunan dalam bidang matematika. ERD digunakan untuk pemodelan basis data relasional. Sehingga jika penyimpanan basis data menggunakan OODBMS maka perancangan basis data tidak perlu menggunakan ERD. ERD memiliki beberapa aliran notasi seperti notasi Chen (dikembangkan oleh Peter Chen), Barker (dikembangkan oleh Richard Barker, Ian Palmer, Harry Ellis), notasi Crow's Foot, dan beberapa notasi lain. Namun yang banyak digunakan adalah notasi dari Chen. Berikut adalah simbol-simbol yang digunakan pada ERD dengan notasi Chen:

Simbol	Deskripsi
Entitas / <i>entity</i> 	Entitas merupakan data inti yang akan disimpan; bakal tabel pada basis data; benda yang memiliki data dan harus disimpan datanya agar dapat diakses oleh aplikasi komputer; penamaan entitas biasanya lebih ke kata benda dan belum merupakan nama tabel
Atribut 	<i>Field</i> atau kolom data yang butuh disimpan dalam suatu entitas
Atribut kunci primer 	<i>Field</i> atau kolom data yang butuh disimpan dalam suatu entitas dan digunakan sebagai kunci akses <i>record</i> yang diinginkan; biasanya

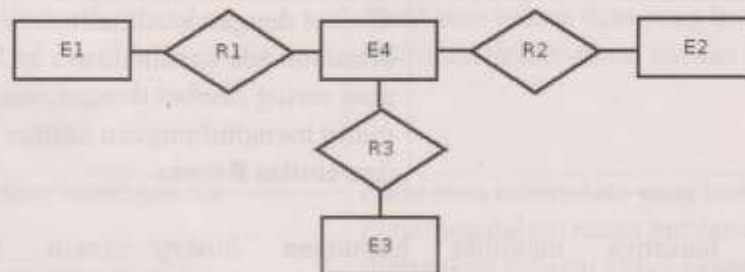
Simbol	Deskripsi
	berupa id; kunci primer dapat lebih dari satu kolom, asalkan kombinasi dari beberapa kolom tersebut dapat bersifat unik (berbeda tanpa ada yang sama)
Atribut multinilai / <i>multivalue</i> 	<i>Field</i> atau kolom data yang butuh disimpan dalam suatu entitas yang dapat memiliki nilai lebih dari satu
Relasi 	Relasi yang menghubungkan antar entitas; biasanya diawali dengan kata kerja
Asosiasi / <i>association</i> 	Penghubung antara relasi dan entitas di mana di kedua ujungnya memiliki <i>multiplicity</i> kemungkinan jumlah pemakaian Kemungkinan jumlah maksimum keterhubungan antara entitas satu dengan entitas yang lain disebut dengan kardinalitas. Misalkan ada kardinalitas 1 ke N atau sering disebut dengan <i>one to many</i> menghubungkan entitas A dan entitas B maka

ERD biasanya memiliki hubungan *binary* (satu relasi menghubungkan dua buah entitas). Beberapa metode perancangan ERD menoleransi hubungan relasi *ternary* (satu relasi

menghubungkan tiga buah relasi) atau *N-ary* (satu relasi menghubungkan banyak entitas), tapi banyak metode perancangan ERD yang tidak mengizinkan hubungan *ternary* atau *N-ary*. Berikut adalah contoh bentuk hubungan relasi dalam ERD:

Nama	Gambar
<i>binary</i>	
<i>ternary</i>	
<i>N-ary</i>	

Beberapa metode perencanaan ERD memberikan solusi untuk perencanaan ERD yang tidak *binary* diubah sebagai berikut:



Gambar 15 Cara Menghindari Relasi Ternary

Entitas E4 berasal dari relasi R1 yang dijadikan entitas, karena banyak metode perencanaan ERD yang menyatakan bahwa jika terjadi relasi *ternary*, maka sebenarnya relasinya lebih layak dijadikan entitas dibandingkan menjadi relasi.



ERD adalah bentuk paling awal dalam melakukan perancangan basis data relasional. Jika menggunakan OODBMS maka perancangan ERD tidak perlu dilakukan

4.6 Studi Kasus

Dalam buku ini, untuk mempermudah pemahaman tentang materi yang dijelaskan akan menggunakan sebuah studi kasus tentang Sistem Informasi Manajemen Perpustakaan. Studi kasus ini akan dijadikan studi kasus sampai di akhir buku. Diharapkan dengan adanya studi kasus yang menyatu ini pembaca dapat memahami proses rekayasa perangkat lunak dari awal dan akhir secara komprehensif / menyeluruh.

Nama aplikasi: Sistem Informasi Manajemen Perpustakaan

Deskripsi:

Sistem informasi manajemen perpustakaan merupakan sebuah sistem informasi untuk mengelola informasi yang diperlukan dalam suatu perpustakaan yang meliputi pendaftaran pustaka, anggota, dan proses peminjaman pustaka. Aturan perpustakaan yang harus diatasi pada sistem informasi manajemen perpustakaan yang akan dimodelkan adalah sebagai berikut:

1. Pustaka dapat memiliki lebih dari satu pengarang
2. Anggota dapat memiliki lebih dari satu nomor telepon
3. Seorang anggota dapat melakukan sebuah peminjaman dalam satu waktu dan boleh lebih dari satu pustaka

4. Seorang anggota dapat mengembalikan pustaka yang dipinjam tidak dalam waktu yang bersamaan walaupun pustaka-pustaka itu dipinjam pada waktu yang sama.
5. Pengunjung yang bukan anggota tidak diperbolehkan meminjam pustaka.
6. Proses pendaftaran pustaka, anggota, dan peminjaman dilakukan oleh petugas perpustakaan.
7. Anggota dan pengunjung dapat melakukan pencarian pustaka.
8. Satu pustaka akan disimpan sebagai satu data dengan id yang unik.

Manajemen perpustakaan meliputi fungsi-fungsi sebagai berikut:

1. Validasi Petugas
 - a. *Login*
 - b. *Logout*
2. Mengelola data pustaka, meliputi:
 - a. Memasukkan data pustaka
 - b. Mengubah data pustaka
 - c. Menghapus data pustaka
 - d. Mencari data pustaka
 - e. Melihat data pustaka
3. Mengelola data anggota, meliputi:
 - a. Memasukkan data anggota
 - b. Mengubah data anggota
 - c. Menghapus data anggota
 - d. Mencari data anggota
 - e. Melihat data anggota
4. Mengelola data peminjaman, meliputi:
 - a. Memasukkan data peminjaman
 - b. Mengubah data peminjaman (mekanisme pengembalian pustaka)
 - c. Mencari data peminjaman
 - d. Melihat data peminjaman
5. Mengelola data petugas untuk petugas yang memiliki hak akses untuk mengelola data petugas, meliputi:
 - a. Memasukkan data petugas baru
 - b. Mengubah data petugas

- c. Menghapus data petugas
- d. Mencari data petugas
- e. Melihat data petugas

4.7 Studi kasus ERD

Studi kasus untuk membuat ERD menggunakan sistem informasi manajemen perpustakaan dengan deskripsi seperti yang ada pada subbab di atas.

4.7.1 Definisi Entitas dan atribut

Berikut adalah definisi entitas dan atribut dari sistem informasi manajemen perpustakaan:

No	Entitas	Atribut
1.	Pustaka Entitas yang menyimpan data pustaka	id atribut yang menjadi identitas pustaka
		judul atribut judul pustaka
		jenis atribut jenis pustaka
		penerbit atribut penerbit pustaka
		tahun atribut tahun pustaka
		pengarang atribut pengarang pustaka (bisa lebih

No	Entitas	Atribut
		dari satu)
2.	Anggota Entitas yang menyimpan data anggota	id atribut yang menjadi identitas anggota
		nama atribut nama anggota
		alamat atribut alamat rumah anggota
		email atribut alamat email anggota
		telepon atribut nomor telepon anggota (bisa lebih dari satu)
3.	Peminjaman Entitas yang menyimpan data anggota	tgl_peminjaman atribut tanggal peminjaman pustaka
		tgl_kembali atribut tanggal kembali pustaka
4.	Petugas Entitas yang menyimpan data petugas yang berhak login ke aplikasi untuk mengelola data	username atribut untuk melakukan proses login
		password atribut sebagai kata sandi untuk melakukan login
		nama atribut nama petugas
		no_petugas atribut untuk nomor petugas

No	Entitas	Atribut
		hak_akses atribut untuk mengetahui hak akses petugas yang berhak mengelola data petugas atau tidak (biasanya disebut sebagai admin/administrator/yang mengurus administrasi)

4.7.2 Definisi Relasi

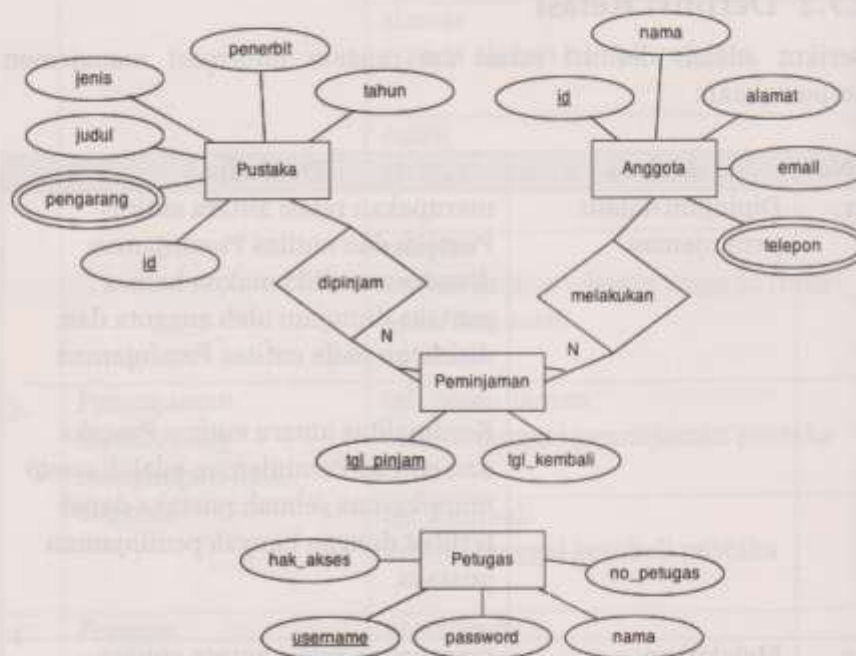
Berikut adalah definisi relasi dari sistem informasi manajemen perpustakaan:

No	Relasi	Deskripsi
1.	Dipinjam dalam peminjaman	merupakan relasi antara entitas Pustaka dan entitas Peminjaman dimana memiliki makna bahwa pustaka dipinjam oleh anggota dan disimpan pada entitas Peminjaman Kardinalitas antara entitas Pustaka dan entitas Peminjaman adalah <i>oneto many</i> karena sebuah pustaka dapat terlibat dengan banyak peminjaman pustaka
2.	Melakukan peminjaman	merupakan relasi antara entitas Anggota dan entitas Peminjaman dimana memiliki makna bahwa anggota melakukan peminjaman pustaka dan disimpan pada entitas Peminjaman Kardinalitas antara entitas Anggota dan entitas Peminjaman adalah <i>oneto many</i> karena seorang anggota dapat

No	Relasi	Deskripsi
		terlibat dengan banyak peminjaman pustaka

4.7.3 Diagram ER

Berikut ini gambar diagram ER untuk studi kasus Sistem Informasi Manajemen Perpustakaan di atas.



Gambar 16 Diagram ER Studi Kasus

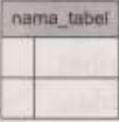
Entitas Petugas dalam perancangan ERD di atas tidak memiliki relasi dengan tabel lain karena data petugas tidak dikaitkan atau tidak masuk dalam data yang ada di entitas-entitas lain. Entitas Petugas diperlukan menyimpan data untuk keperluan proses *login*.

4.8 CDM

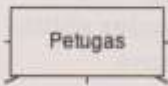
CDM (*Conceptual Data Model*) atau model konsep data merupakan konsep yang berkaitan dengan pandangan pemakai terhadap data yang disimpan dalam basis data. CDM dibuat sudah dalam bentuk tabel-tabel tanpa tipe data yang menggambarkan relasi antar tabel untuk keperluan implementasi ke basis data.

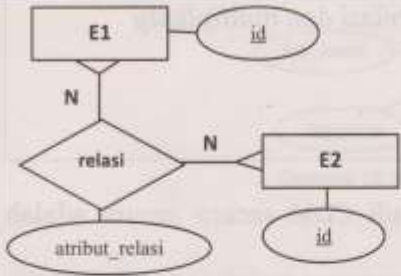
CDM merupakan hasil penjabaran lebih lanjut dari ERD. Ada aturan-aturan yang harus diikuti dalam melakukan konversi ERD menjadi CDM. Dalam buku ini aturan-aturan tersebut tidak dijabarkan karena diasumsikan hal tersebut bisa dipelajari pada buku-buku yang secara khusus membahas basis data.

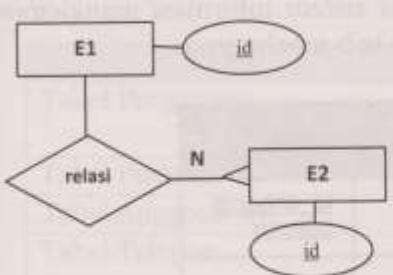
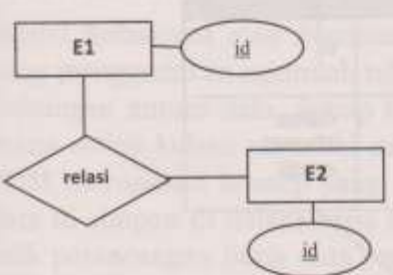
Berikut adalah simbol-simbol yang ada pada CDM:

Simbol	Deskripsi
Entitas / tabel 	entitas atau tabel yang menyimpan data dalam basis data
Relasi 	relasi antar tabel yang terdiri atas nama relasi dan <i>multiplicity</i>

Aturan untuk mengubah ERD menjadi CDM secara umum adalah sebagai berikut:

ERD	CDM
 entitas	

ERD	CDM								
	<table border="1"> <thead> <tr> <th colspan="2">Petugas</th> </tr> </thead> <tbody> <tr> <td>PK</td><td>username</td> </tr> <tr> <td></td><td>password nama no_petugas hak_akses</td> </tr> </tbody> </table> menjadi sebuah tabel tersendiri	Petugas		PK	username		password nama no_petugas hak_akses		
Petugas									
PK	username								
	password nama no_petugas hak_akses								
 atribut <i>multivalue</i>	<table border="1"> <thead> <tr> <th colspan="2">Pengarang</th> </tr> </thead> <tbody> <tr> <td>PK</td><td>id_pustaka</td> </tr> <tr> <td>PK</td><td>pengarang</td> </tr> </tbody> </table> menjadi sebuah tabel tersendiri dengan kunci primer (<i>primary key</i>) adalah kunci primer pada entitas dan memiliki atribut dengan nama seperti pada atribut entitas	Pengarang		PK	id_pustaka	PK	pengarang		
Pengarang									
PK	id_pustaka								
PK	pengarang								
 relasi dengan kardinalitas <i>many to many</i>	<table border="1"> <thead> <tr> <th colspan="2">Relasi</th> </tr> </thead> <tbody> <tr> <td>PK</td><td>id_E1</td> </tr> <tr> <td>PK</td><td>id_E2</td> </tr> <tr> <td></td><td>atribut_relasi</td> </tr> </tbody> </table> menjadi sebuah tabel tersendiri dengan kunci primer adalah atribut yang menjadi kunci primer di kedua entitas yang direlasikannya	Relasi		PK	id_E1	PK	id_E2		atribut_relasi
Relasi									
PK	id_E1								
PK	id_E2								
	atribut_relasi								

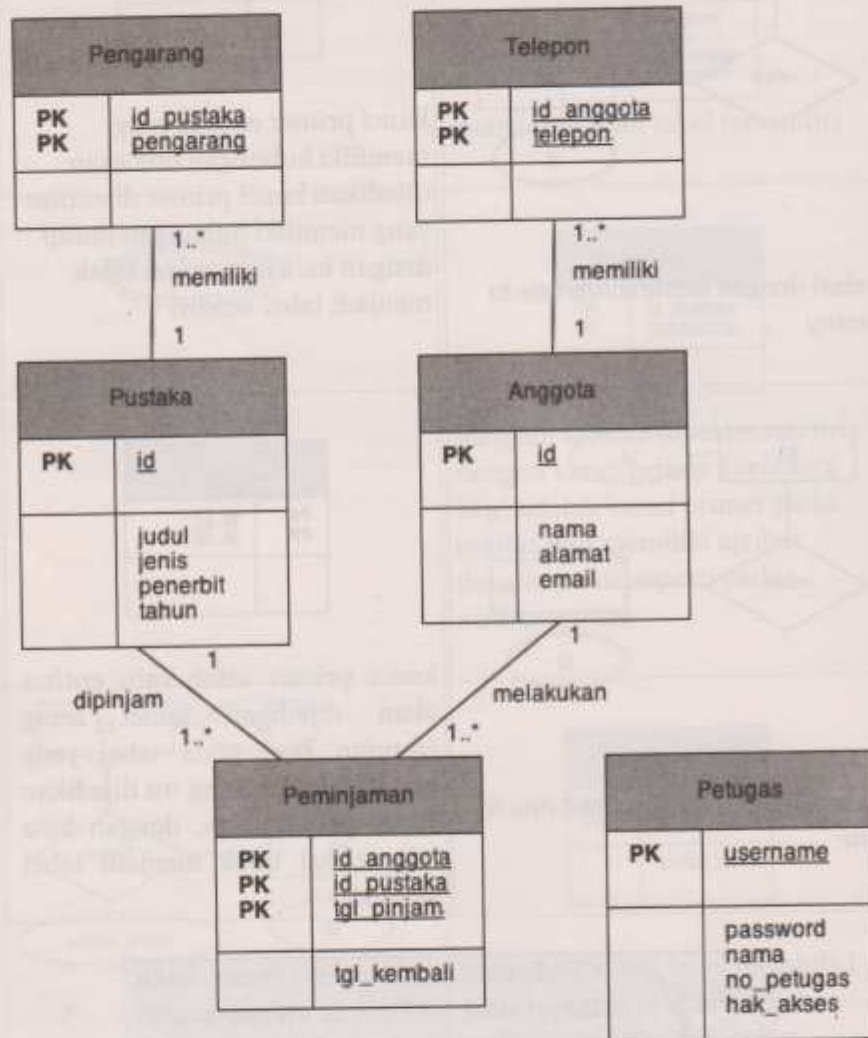
ERD	CDM								
 relasi dengan kardinalitas <i>one to many</i>	<table border="1"> <thead> <tr> <th colspan="2">E2</th></tr> </thead> <tbody> <tr> <td>PK</td><td>id_E1</td></tr> <tr> <td>PK</td><td>id_E2</td></tr> <tr> <td></td><td></td></tr> </tbody> </table> kunci primer entitas yang memiliki hubungan <i>one</i> akan dijadikan kunci primer di entitas yang memiliki hubungan <i>many</i> dengan kata lain, relasi tidak menjadi tabel sendiri	E2		PK	id_E1	PK	id_E2		
E2									
PK	id_E1								
PK	id_E2								
 relasi dengan kardinalitas <i>one to one</i>	<table border="1"> <thead> <tr> <th colspan="2">E2</th></tr> </thead> <tbody> <tr> <td>PK</td><td>id_E1</td></tr> <tr> <td>PK</td><td>id_E2</td></tr> <tr> <td></td><td></td></tr> </tbody> </table> kunci primer salah satu entitas akan dijadikan kunci asing (<i>foreign key</i>) pada tabel yang lain dan kunci asing itu dijadikan kunci primer juga, dengan kata lain, relasi tidak menjadi tabel sendiri.	E2		PK	id_E1	PK	id_E2		
E2									
PK	id_E1								
PK	id_E2								



CDM merupakan penjabaran lebih lanjut dari ERD. Ada aturan-aturan cara menjabarkan ERD menjadi CDM yang bisa dipelajari pada buku yang secara khusus membahas basis data.

4.9 Studi Kasus CDM

Berikut adalah CDM dari studi kasus sistem informasi manajemen perpustakaan seperti pada studi kasus bab sebelumnya:



Gambar 17 CDM Studi Kasus

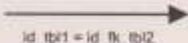
Tabel-tabel di atas merupakan hasil pertanggungjawaban dari diagram ER yang telah dibuat dengan rincian sebagai berikut:

CDM	ERD
Tabel Pengarang	atribut <i>multivalue</i> pengarang dari entitas Pustaka
Tabel Pustaka	entitas Pustaka
Tabel Anggota	entitas Anggota
Tabel Telepon	atribut <i>multivalue</i> telepon dari entitas Anggota
Tabel Peminjaman	entitas Peminjaman
Tabel Petugas	entitas Petugas

4.10 PDM

Model Relasional atau *Physical Data Model* (PDM) adalah model yang menggunakan sejumlah tabel untuk menggambarkan data serta hubungan antara data. Setiap tabel mempunyai sejumlah kolom di mana setiap kolom memiliki nama yang unik beserta tipe datanya. PDM merupakan konsep yang menerangkan detail dari bagaimana data di simpan di dalam basis data. PDM sudah merupakan bentuk fisik perancangan basis data yang sudah siap diimplementasikan ke dalam DBMS sehingga nama tabel juga sudah merupakan nama asli tabel yang akan diimplementasikan ke dalam DBMS.

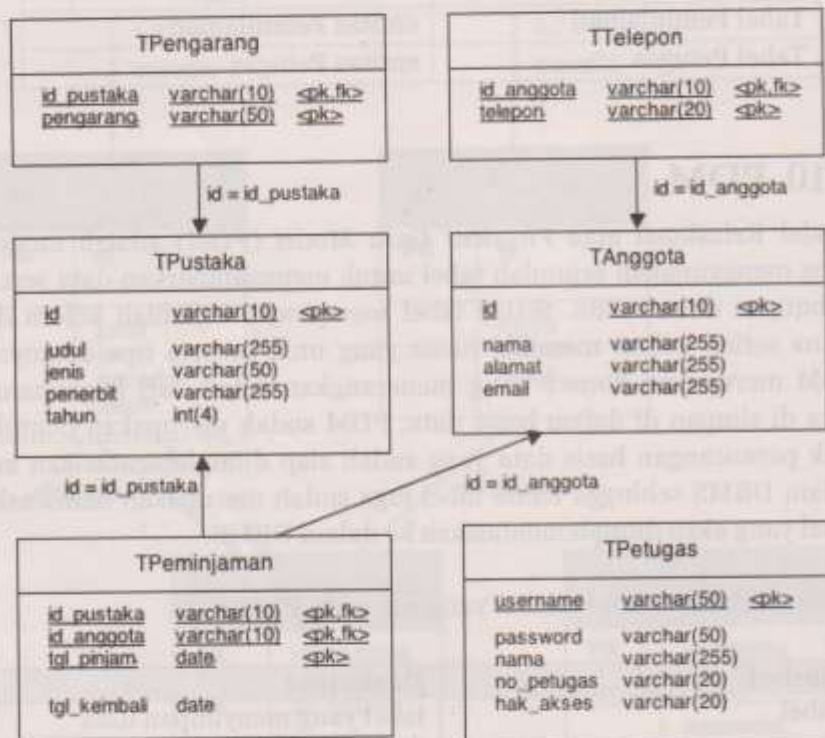
Berikut adalah simbol-simbol yang ada pada PDM:

Simbol	Deskripsi
Tabel nama_tabel	tabel yang menyimpan data dalam basis data
Relasi  id_tb1 = id_tb2	relasi antar tabel yang terdiri dari persamaan antara <i>primary key</i> (kunci primer) tabel yang diacu

Simbol	Deskripsi
	dengan kunci yang menjadi referensi acuan di tabel lain.

4.11 Studi Kasus PDM

Berikut adalah PDM dari studi kasus sistem informasi manajemen perpustakaan seperti pada studi kasus bab sebelumnya:



Gambar 18 PDM Studi Kasus



1. Apakah yang dimaksud dengan basis data?
2. Apakah fungsi basis data pada suatu sistem informasi?
3. Apakah yang dimaksud dengan DBMS dan apa fungsinya?
4. Sebutkan syarat-syarat suatu sistem bisa disebut sebagai DBMS!
5. Sebutkan kelebihan dan kekurangan masing-masing DBMS sebagai berikut!
 - a. Oracle
 - b. Microsoft SQL server
 - c. Microsoft Access
 - d. IBM DB2
 - e. MySQL
 - f. PostgreSQL
 - g. Firebird
 - h. SQLite
6. Jelaskan tentang organisasi ANSI dan ISO! Apa peran yang mereka lakukan?
7. Apa pengertian SQL dan apa kegunaannya?
8. Apa pengertian *query*?
9. Mengapa implementasi/adopsi standar SQL pada masing-masing DBMS bisa berbeda-beda?
10. Apa yang dimaksud dengan entitas, relasi, dan atribut pada ERD?
11. Bagaimana cara/aturan untuk melakukan pemetaan dari ERD ke CDM?
12. Apakah perbedaan antara CDM dan PDM?
13. Sebutkan dan jelaskan beberapa basis data non-relasional! Apa kelebihan dan kekurangan basis data tersebut jika dibandingkan dengan basis data relasional?
14. Buatlah desain basis data untuk Sistem Informasi Apotek!

5

Pemrograman Terstruktur



Overview

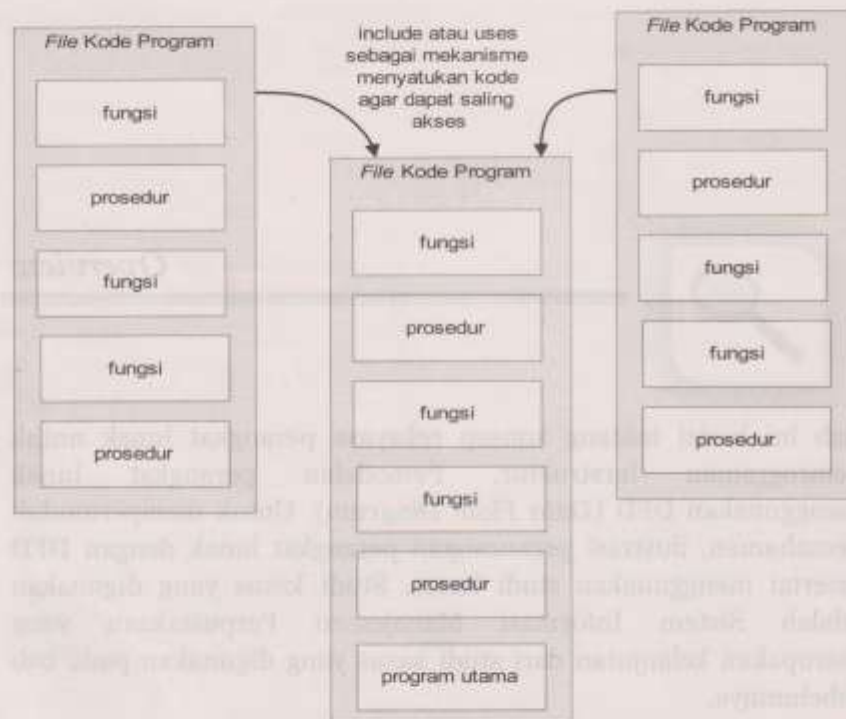
Bab ini berisi tentang konsep rekayasa perangkat lunak untuk pemrograman terstruktur. Pemodelan perangkat lunak menggunakan DFD (*Data Flow Diagram*). Untuk mempermudah pemahaman, ilustrasi perancangan perangkat lunak dengan DFD disertai menggunakan studi kasus. Studi kasus yang digunakan adalah Sistem Informasi Manajemen Perpustakaan yang merupakan kelanjutan dari studi kasus yang digunakan pada bab sebelumnya.

5.1 Pengertian Pemrograman Terstruktur

Pemrograman terstruktur adalah konsep atau paradigma atau sudut pandang pemrograman yang membagi-bagi program berdasarkan

fungsi-fungsi atau prosedur-prosedur yang dibutuhkan program komputer. Modul-modul (pembagian program) biasanya dibuat dengan mengelompokkan fungsi-fungsi dan prosedur-prosedur yang diperlukan sebuah proses tertentu.

Fungsi-fungsi dan prosedur-prosedur ditulis secara sekuensial atau terurut dari atas ke bawah sesuai dengan kebergantungan antar fungsi atau prosedur (fungsi atau prosedur yang dapat dipakai oleh fungsi atau prosedur dibawahnya harus yang sudah ditulis atau dideklarasikan di atasnya). Berikut adalah contoh ilustrasi untuk pemrograman terstruktur:



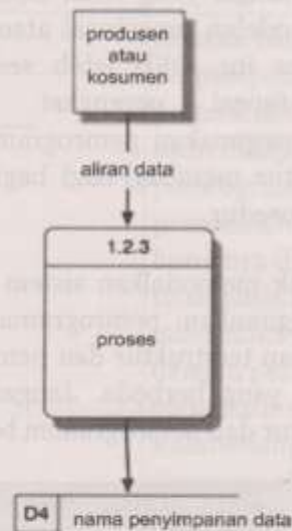
Gambar 19 Ilustrasi pemrograman terstruktur

Pemodulan pada pemrograman terstruktur dibagi berdasarkan fungsi-fungsi dan prosedur-prosedur. Oleh karena itu, pemodelan pada pemrograman terstruktur lebih fokus bagaimana memodelkan

data dan fungsi-fungsi atau prosedur-prosedur yang harus dibuat. Jenis paradigma pemrograman yang digunakan dapat dideteksi dari bahasa pemrograman apa yang akan digunakan untuk membuat program, baru setelah itu ditentukan paradigma pemrograman apa yang akan digunakan.

5.2 DFD

Data Flow Diagram (DFD) awalnya dikembangkan oleh Chris Gane dan Trish Sarson pada tahun 1979 yang termasuk dalam *Structured Systems Analysis and Design Methodology* (SSADM) yang ditulis oleh Chris Gane dan Trish Sarson. Sistem yang dikembangkan ini berbasis pada dekomposisi fungsional dari sebuah sistem. Berikut adalah contoh DFD yang dikembangkan oleh Chris Gane dan Trish Sarson:



Gambar 20 Contoh DFD yang dikembangkan Chris Gane & Trish Sarson

Edward Yourdon dan Tom DeMarco memperkenalkan metode yang lain pada tahun 1980-an di mana mengubah persegi dengan sudut

lengkung (pada DFD Chris Gane dan Trish Sarson) dengan lingkaran untuk menotasikan. DFD Edward Yourdon dan Tom DeMarco populer digunakan sebagai model analisis sistem perangkat lunak untuk sistem perangkat lunak yang akan diimplementasikan dengan pemrograman terstruktur.

Informasi yang ada di dalam perangkat lunak dimodifikasi dengan beberapa transformasi yang dibutuhkan. *Data Flow Diagram* (DFD) atau dalam bahasa Indonesia menjadi Diagram Alir Data (DAD) adalah representasi grafik yang menggambarkan aliran informasi dan transformasi informasi yang diaplikasikan sebagai data yang mengalir dari masukan (*input*) dan keluaran (*output*).

DFD dapat digunakan untuk merepresentasikan sebuah sistem atau perangkat lunak pada beberapa level abstraksi. DFD dapat dibagi menjadi beberapa level yang lebih detail untuk merepresentasikan aliran informasi atau fungsi yang lebih detail. DFD menyediakan mekanisme untuk pemodelan fungsional ataupun pemodelan aliran informasi. Oleh karena itu, DFD lebih sesuai digunakan untuk memodelkan fungsi-fungsi perangkat lunak yang akan diimplementasikan menggunakan pemrograman terstruktur karena pemrograman terstruktur membagi-bagi bagiannya dengan fungsi-fungsi dan prosedur-prosedur.

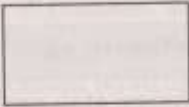
DFD tidak sesuai untuk memodelkan sistem perangkat lunak yang akan dibangun menggunakan pemrograman berorientasi objek. Paradigma pemrograman terstruktur dan pemrograman berorientasi objek merupakan hal yang berbeda. Jangan mencampurkan pemrograman terstruktur dan pemrograman berorientasi objek.



DFD tidak sesuai untuk memodelkan sistem yang menggunakan pemrograman berorientasi objek.

Notasi-notasi pada DFD (Edward Yourdon dan Tom DeMarco) adalah sebagai berikut:

Notasi	Keterangan
	proses atau fungsi atau prosedur; pada pemodelan perangkat lunak yang akan diimplementasikan dengan pemrograman terstruktur, maka pemodelan notasi inilah yang harusnya menjadi fungsi atau prosedur di dalam kode program catatan: nama yang diberikan pada sebuah proses biasanya berupa kata kerja
	<i>file</i> atau basisdata atau penyimpanan (<i>storage</i>); pada pemodelan perangkat lunak yang akan diimplementasikan dengan pemrograman terstruktur, maka pemodelan notasi inilah yang harusnya dibuat menjadi tabel-tabel basis data yang dibutuhkan, tabel-tabel ini juga harus sesuai dengan perancangan tabel-tabel pada basis data (<i>Entity Relationship Diagram</i> (ERD), <i>Conceptual Data Model</i> (CDM), <i>Physical Data Model</i> (PDM)) catatan: nama yang diberikan pada sebuah penyimpanan biasanya kata benda

Notasi	Keterangan
	entitas luar (<i>external entity</i>) atau masukan (<i>input</i>) atau keluaran (<i>output</i>) atau orang yang memakai/berinteraksi dengan perangkat lunak yang dimodelkan atau sistem lain yang terkait dengan aliran data dari sistem yang dimodelkan catatan: nama yang digunakan pada masukan (<i>input</i>) atau keluaran (<i>output</i>) biasanya berupa kata benda
	aliran data; merupakan data yang dikirim antar proses, dari penyimpanan ke proses, atau dari proses ke masukan (<i>input</i>) atau keluaran (<i>output</i>) catatan: nama yang digunakan pada aliran data biasanya berupa kata benda, dapat diawali dengan kata data misalnya "data siswa" atau tanpa kata data misalnya "siswa"

Berikut ini adalah tahapan-tahapan perancangan dengan menggunakan DFD:

1. Membuat DFD Level 0 atau sering disebut juga *Context Diagram*

DFD Level 0 menggambarkan sistem yang akan dibuat sebagai suatu entitas tunggal yang berinteraksi dengan orang maupun sistem lain. DFD Level 0 digunakan untuk

menggambarkan interaksi antara sistem yang akan dikembangkan dengan entitas luar.

2. Membuat DFD Level 1

DFD Level 1 digunakan untuk menggambarkan modul-modul yang ada dalam sistem yang akan dikembangkan. DFD Level 1 merupakan hasil *breakdown* DFD Level 0 yang sebelumnya sudah dibuat.

3. Membuat DFD Level 2

Modul-modul pada DFD Level 1 dapat di-*breakdown* menjadi DFD Level 2. Modul mana saja yang harus di-*breakdown* lebih detail tergantung pada tingkat kedetailan modul tersebut. Apabila modul tersebut sudah cukup detail dan rinci maka modul tersebut sudah tidak perlu untuk di-*breakdown* lagi. Untuk sebuah sistem, jumlah DFD Level 2 sama dengan jumlah modul pada DFD Level 1 yang di-*breakdown*.

4. Membuat DFD Level 3 dan seterusnya

DFD Level 3, 4, 5, dan seterusnya merupakan *breakdown* dari modul pada DFD Level di-atasnya. *Breakdown* pada level 3, 4, 5, dan seterusnya aturannya sama persis dengan DFD Level 1 atau Level 2.

Pada satu diagram DFD sebaiknya jumlah modul tidak boleh lebih dari 20 buah. Jika lebih dari 20 buah modul, diagram akan terlihat rumit dan susah untuk dibaca sehingga menyebabkan sistem yang dikembangkan juga menjadi rumit.

5.3 Kamus Data

Kamus data (*data dictionary*) dipergunakan untuk memperjelas aliran data yang digambarkan pada DFD. Kamus data adalah kumpulan daftar elemen data yang mengalir pada sistem perangkat lunak sehingga masukan (*input*) dan keluaran (*output*) dapat dipahami secara umum (memiliki standar cara penulisan). Kamus data dalam implementasi program dapat menjadi parameter masukan atau keluaran dari sebuah fungsi atau prosedur. Kamus data biasanya berisi:

- nama – nama dari data
- digunakan pada – merupakan proses-proses yang terkait data
- deskripsi – merupakan deskripsi data
- informasi tambahan – seperti tipe data, nilai data, batas nilai data, dan komponen yang membentuk data

Kamus data memiliki beberapa simbol untuk menjelaskan informasi tambahan sebagai berikut:

Simbol	Keterangan
=	disusun atau terdiri dari
+	dan
[]	baik ...atau...
{ } ⁿ	n kali diulang/ bernilai banyak
()	data opsional
...	batas komentar

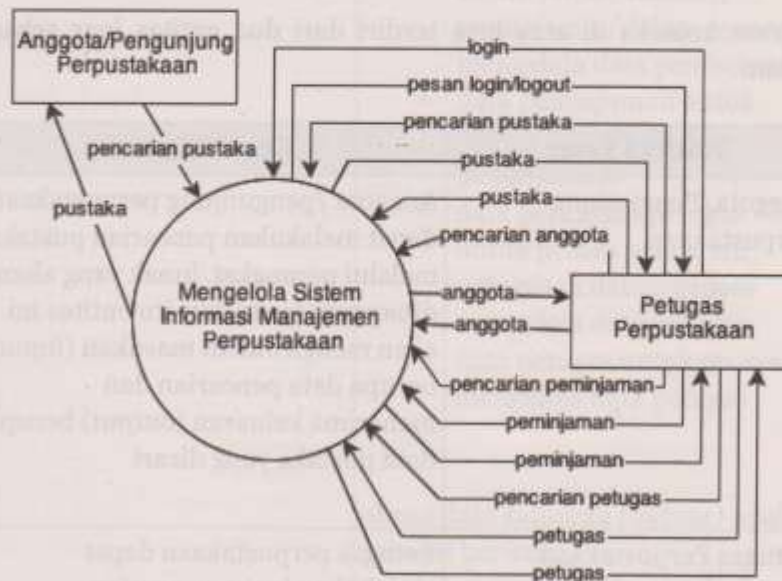
Kamus data pada DFD nanti harus dapat dipetakan dengan hasil perancangan basis data yang dilakukan sebelumnya. Jika ada kamus data yang tidak dapat dipetakan pada tabel hasil perancangan basis data berarti hasil perancangan basis data dengan perancangan dengan DFD masih belum sesuai, sehingga harus ada yang diperbaiki baik perancangan basis datanya, perancangan DFD-nya, atau keduanya.

Contoh penulisan kamus data dapat dilihat pada studi kasus pembuatan DFD.

5.4 Studi Kasus DFD

Studi kasus yang digunakan sama dengan saat membuat diagram relasi entitas atau *Entity Relationship Diagram* (ERD) yaitu sistem

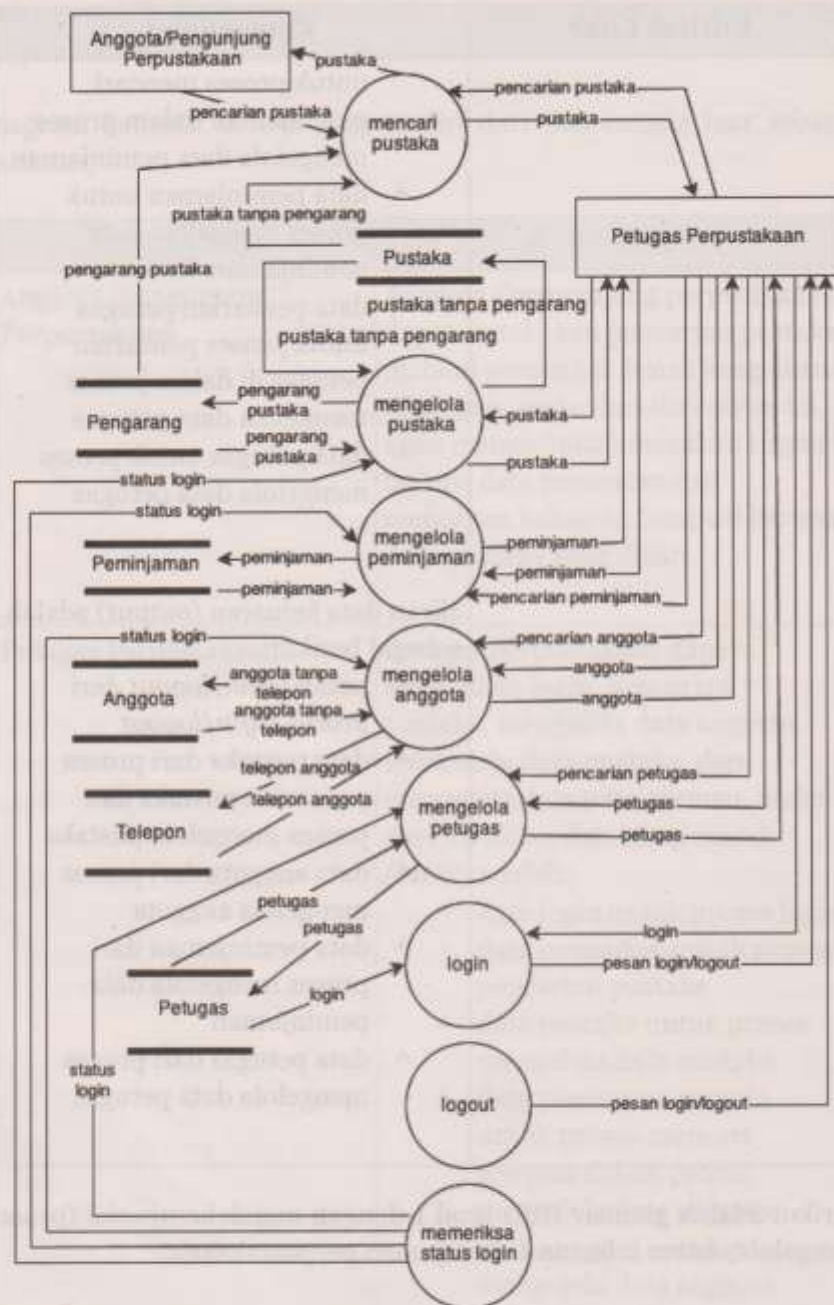
informasi manajemen perpustakaan. Berikut adalah DFD level 0 dari studi kasus di atas:



Gambar 21 DFD level 0/Context Diagram

DFD level 0 biasa disebut dengan diagram sistem inti (*fundamental system model*) atau model sistem inti (*fundamental system model*) atau biasa disebut juga diagram konteks (*context diagram*) atau model konteks (*context model*). Arah panah dari aliran data menunjukkan aliran data berupa data masukan (*input*) dan keluaran (*output*) ke dalam proses perangkat lunak yang dirancang.

Pada diagram konteks studi kasus ini memiliki sebuah proses besar yang nantinya akan didekomposisi menjadi proses-proses yang lebih detail. Proses ini adalah proses mengelola sistem informasi manajemen perpustakaan yang artinya adalah sebuah perangkat lunak, maka dari itu sering juga kata "mengelola" tidak dituliskan, sehingga isi dari proses pada diagram konteks hanya bertuliskan nama dari sistem perangkat lunak yang akan dibangun, dimana



Gambar 1 DFD level 1

Tempat penyimpanan (*storage*) yang digunakan pada perancangan DFD level 1 adalah:

Nama Penyimpanan	Keterangan
Petugas	sebuah tabel dalam basis data untuk menyimpan data petugas
Pustaka	sebuah tabel dalam basis data untuk menyimpan data pustaka
Pengarang	sebuah tabel dalam basis data untuk menyimpan data pengarang pustaka
Anggota	sebuah tabel dalam basis data untuk menyimpan data anggota
Telepon	sebuah tabel dalam basis data untuk menyimpan data telepon anggota
Peminjaman	sebuah tabel dalam basis data untuk menyimpan data peminjaman

penyimpanan yang digunakan mengacu pada perancangan tabel pada basis data di bab sebelumnya.

Proses-proses yang terlibat pada DFD level 1 adalah:

Nama Proses	Aliran Data Masuk (<i>input</i>)	Aliran Data Keluar (<i>output</i>)	Keterangan
mencari pustaka	pencarian pustaka	pustaka	petugas atau pengunjung perpustakaan melakukan pencarian pustaka dengan memasukkan data pencarian misalnya kata kunci dan

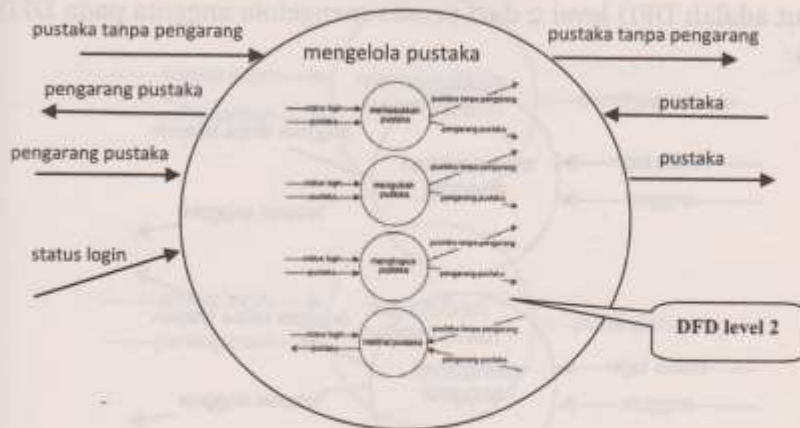
Nama Proses	Aliran Data Masuk (input)	Aliran Data Keluar (output)	Keterangan
			kategori pencarian dan menghasilkan data pustaka hasil pencarian
login	login	pesan login/logout	petugas perpustakaan melakukan <i>login</i> untuk validasi melakukan pengelolaan data anggota, pustaka, petugas, dan peminjaman
logout		pesan logi/logout	petugas perpustakaan melakukan <i>logout</i> untuk keluar dari akun aplikasi
memeriksa status login		status login	memeriksa apakah pengguna sudah melakukan <i>login</i> untuk dapat mengakses fungsi-fungsi tertentu yang harusnya hanya bisa diakses petugas
mengelola pustaka	• pustaka • status login	pustaka	petugas perpustakaan dapat melakukan memasukkan data pustaka, mengubah data pustaka, menghapus data

Nama Proses	Aliran Data Masuk (input)	Aliran Data Keluar (output)	Keterangan
			pustaka, atau melihat data pustaka jika telah melakukan <i>login</i>
mengelola anggota	• pencarian anggota • anggota • status login	anggota	petugas perpustakaan dapat melakukan memasukkan data anggota, mengubah data anggota, menghapus data anggota, mencari data anggota, atau melihat data anggota jika telah melakukan <i>login</i>
mengelola peminjaman	• pencarian peminjaman • peminjaman • status login	peminjaman	petugas perpustakaan dapat melakukan memasukkan data peminjaman, mengubah data peminjaman, menghapus data peminjaman, mencari peminjaman, atau melihat data peminjaman jika telah melakukan <i>login</i>
mengelola petugas	• pencarian petugas • petugas	petugas	petugas perpustakaan dapat melakukan memasukkan data

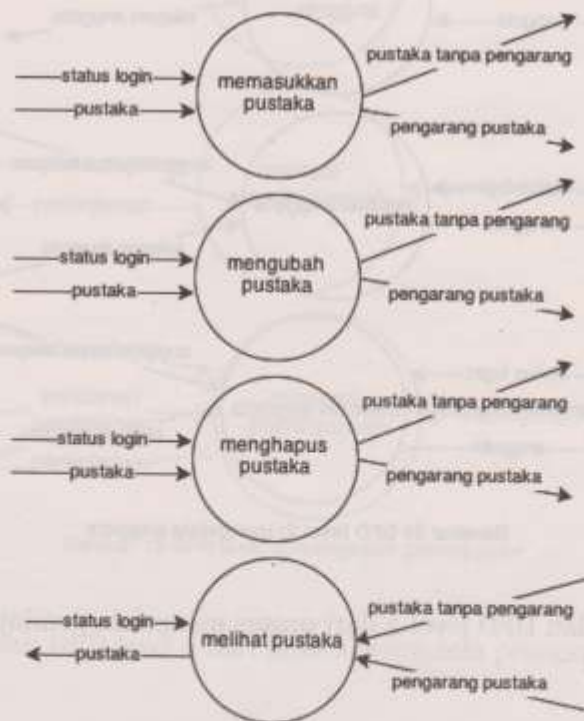
Nama Proses	Aliran Data Masuk (input)	Aliran Data Keluar (output)	Keterangan
	• status login		petugas, mengubah data petugas, menghapus data petugas, mencari petugas, atau melihat data petugas jika telah melakukan <i>login</i> dan memiliki hak akses untuk mengelola data petugas

Sebuah proses pada DFD dapat terus didekomposisi sampai tidak ditentukan, tapi jika terlalu banyak maka akan memperlihatkan ketidakandalan rancangan DFD karena harusnya dapat dirancang ruang lingkup proses agar tidak mencakup terlalu banyak proses detail pada sebuah proses global (harusnya dipecah lagi menjadi beberapa proses global).

Sebuah **proses dalam DFD dengan level paling tinggi akan menjadi sebuah fungsi atau prosedur dalam kode program aplikasi komputer**. Saat mendekomposisi sebuah proses maka harus dipastikan data yang masuk ataupun keluar sama dengan yang masuk atau keluar pada proses yang di dekomposisi, misalkan pada proses mengelola pustaka dan didekomposisi maka logikanya seperti gambar berikut:

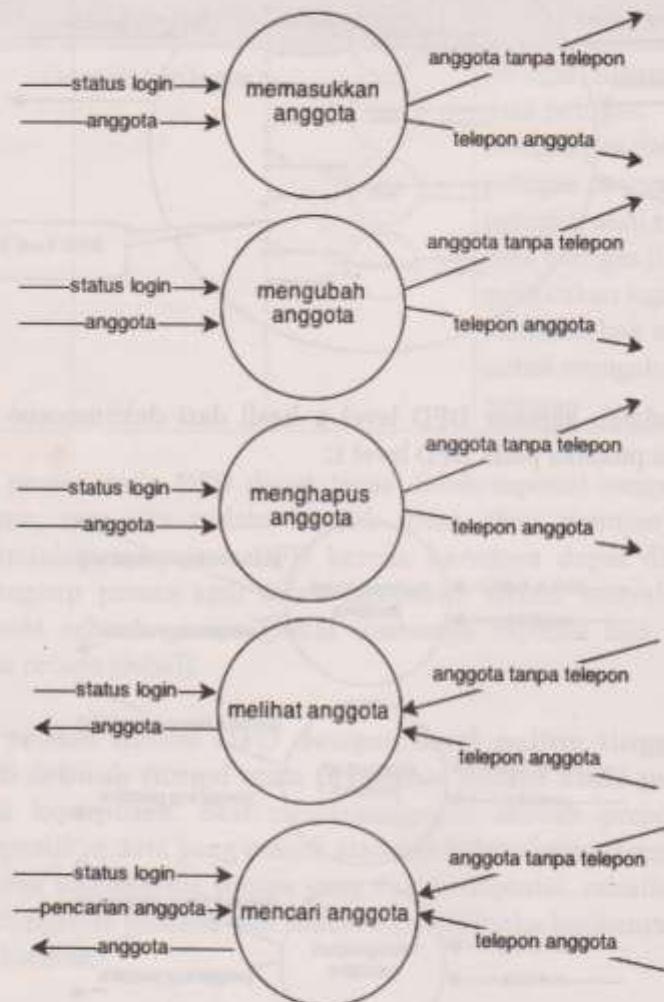


Berikut adalah gambar DFD level 2 hasil dari dekomposisi proses mengelola pustaka pada DFD level 1:



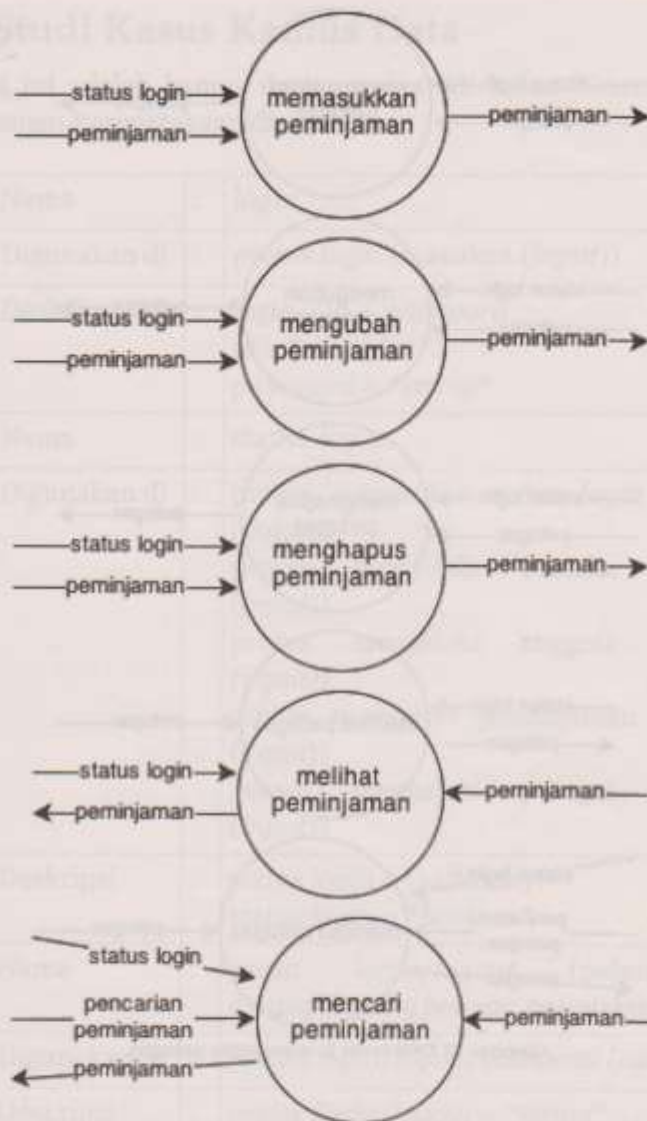
Gambar 2 DFD level 2: mengelola pustaka

Berikut adalah DFD level 2 dari proses mengelola anggota pada DFD level 1:



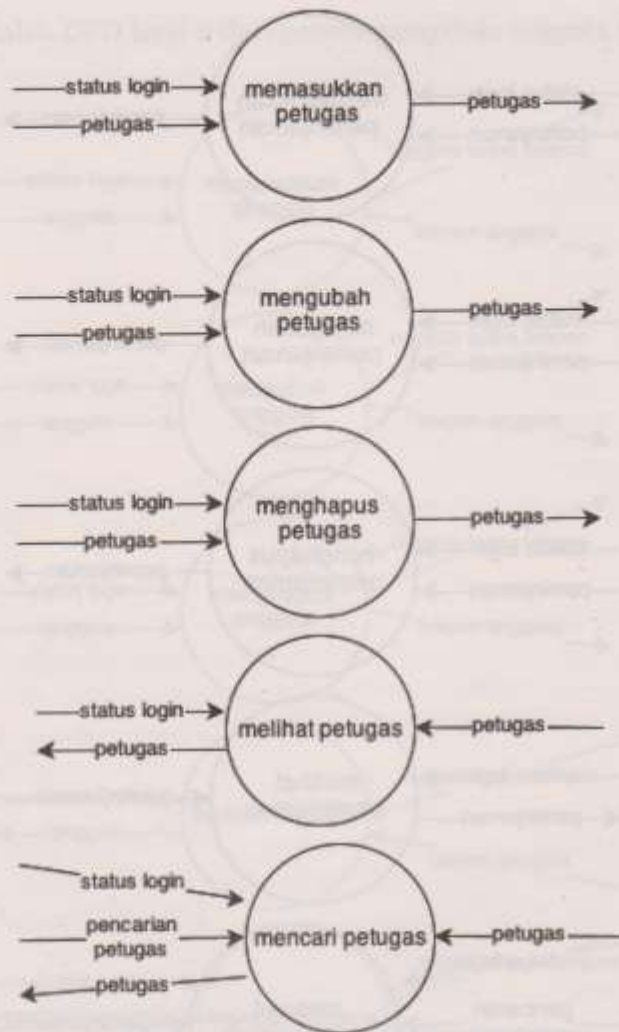
Gambar 24 DFD level 2: mengelola anggota

Berikut adalah DFD level 2 dari proses mengelola peminjaman pada DFD level 1:



Gambar 25 DFD level 2: mengelola peminjaman

Berikut adalah DFD level 2 dari proses mengelola petugas pada DFD level 1:



Gambar 26 DFD level 2: mengelola petugas

Breakdown level DFD ditentukan oleh kompleksitas sistem. Perancangan DFD yang baik adalah diagram tetap tergambar sederhana, tidak rumit, dan tidak terlalu banyak levelnya meskipun sistemnya sangat kompleks.

5.5 Studi Kasus Kamus Data

Berikut ini adalah kamus data untuk studi kasus Sistem Informasi Manajemen Perpustakaan di atas:

1.	Nama	:	<i>login</i>
	Digunakan di	:	proses login (masukan (<i>input</i>))
	Deskripsi	:	login = id + <i>password</i> id = <i>*string*</i> <i>password</i> = <i>*string*</i>
2.	Nama	:	status <i>login</i>
	Digunakan di	:	proses memeriksa status <i>login</i> (keluaran (<i>output</i>)) proses mengelola pustaka (masukan (<i>input</i>)) proses mengelola anggota (masukan (<i>input</i>)) proses mengelola peminjaman (masukan (<i>input</i>)) proses mengelola petugas (masukan (<i>input</i>))
	Deskripsi	:	status <i>login</i> = [ya tidak] status <i>login</i> = <i>*boolean*</i>
3.	Nama	:	pesan <i>login/logout</i> (pesan untuk ditampilkan ke petugas perpustakaan)
	Digunakan di	:	proses <i>login/logout</i> (keluaran (<i>output</i>))
	Deskripsi	:	pesan <i>login/logout</i> = <i>*string*</i>
4.	Nama	:	pencarian pustaka
	Digunakan di	:	proses mencari pustaka (masukan (<i>input</i>))
	Deskripsi	:	pencarian = kata kunci + kategori pencarian kata kunci = <i>*string*</i>

		kategori = <i>*string*</i>
5.	Nama	: pustaka
	Digunakan di	: proses mencari pustaka (masukan (<i>input</i>)/keluaran (<i>output</i>)) proses mengelola pustaka (masukan (<i>input</i>)/keluaran (<i>output</i>))
	Deskripsi	: pustaka = id + judul + jenis + {pengarang} ⁿ + penerbit + tahun id= <i>*string*</i> judul= <i>*string*</i> jenis= <i>*string*</i> pengarang= <i>*string*</i> penerbit= <i>*string*</i> tahun = <i>*4 digit integer*</i>
6.	Nama	: pustaka tanpa pengarang
	Digunakan di	: proses mencari pustaka (masukan (<i>input</i>)) proses mengelola pustaka (masukan (<i>input</i>)/keluaran (<i>output</i>))
	Deskripsi	: pustaka = id + judul + jenis + penerbit + tahun id= <i>*string*</i> judul= <i>*string*</i> jenis= <i>*string*</i> penerbit= <i>*string*</i> tahun = <i>*4 digit integer*</i>
7.	Nama	: pengarang pustaka
	Digunakan di	: proses mencari pustaka (masukan (<i>input</i>)) proses mengelola pustaka (masukan (<i>input</i>)/keluaran (<i>output</i>))
	Deskripsi	: pengarang pustaka = pengarang pengarang= <i>*string*</i>
8.	Nama	: pencarian anggota

	Digunakan di	: proses mengelola anggota (masukan (<i>input</i>))
	Deskripsi	: pencarian = kata kunci + kategori pencarian kata kunci = <i>*string*</i> kategori = <i>*string*</i>
9.	Nama	: anggota
	Digunakan di	: proses mengelola anggota (masukan (<i>input</i>)/keluaran (<i>output</i>))
	Deskripsi	: anggota = id + nama + alamat + email + {telepon} ⁿ id = <i>*string*</i> nama = <i>*string*</i> alamat = <i>*string*</i> email = <i>*string*</i> telepon = <i>*string*</i>
10.	Nama	: anggota tanpa telepon
	Digunakan di	: proses mengelola anggota (masukan (<i>input</i>)/keluaran (<i>output</i>))
	Deskripsi	: anggota = id + nama + alamat + email id = <i>*string*</i> nama = <i>*string*</i> alamat = <i>*string*</i> email = <i>*string*</i>
11.	Nama	: telepon anggota
	Digunakan di	: proses mengelola anggota (masukan (<i>input</i>)/keluaran (<i>output</i>))
	Deskripsi	: telepon anggota = telepon anggota telepon anggota = <i>*string*</i>
12.	Nama	: pencarian pinjaman
	Digunakan di	: proses mengelola pinjaman (masukan (<i>input</i>))

	Deskripsi	:	pencarian = kata kunci + kategori pencarian kata kunci = <i>*string*</i> kategori = <i>*string*</i>
13.	Nama	:	peminjaman
	Digunakan di	:	proses mengelola peminjaman (masukan (<i>input</i>)/keluaran (<i>output</i>))
	Deskripsi	:	peminjaman = id anggota + id buku tanggal pinjam + tanggal kembali id anggota = <i>*string*</i> id buku = <i>*string*</i> tanggal pinjam = <i>*date*</i> tanggal kembali = <i>*date*</i>
14.	Nama	:	pencarian petugas
	Digunakan di	:	proses mengelola petugas (masukan (<i>input</i>))
	Deskripsi	:	pencarian = kata kunci + kategori pencarian kata kunci = <i>*string*</i> kategori = <i>*string*</i>
15.	Nama	:	petugas
	Digunakan di	:	proses mengelola petugas (masukan (<i>input</i>)/keluaran (<i>output</i>))
	Deskripsi	:	anggota = username + password + nama + no petugas + hak akses username = <i>*string*</i> password = <i>*string*</i> nama = <i>*string*</i> no petugas = <i>*string*</i> hak akses = <i>*string*</i>

Misal diambil contoh dari kamus data di atas untuk data peminjaman:

- data peminjaman digunakan pada proses mengelola peminjaman sebagai aliran data masuk dan aliran data keluar maka ditulis sebagai masukan (*input*) dan keluaran (*output*)
- pada deskripsi di atas bahwa data peminjaman terdiri dari id anggota, id buku, tanggal pinjam, dan tanggal kembali dimana id anggota dan id buku merupakan tipe data *string*, sedangkan tanggal pinjam dan tanggal kembali bertipe data *date*.



Kamus data harus bisa dipetakan dengan hasil perancangan basis data

5.6 Implementasi DFD ke dalam Program Studi Kasus

DFD digunakan sebagai diagram guna merancang fungsi-fungsi yang dijalankan didalam perangkat lunak nanti. Oleh karena itu, akan sangat penting adanya kesesuaian antara perancangan dengan kode program yang akan dibuat. Sehingga disaat perangkat lunak sudah tidak ditangani pengembang pertamanya maka perangkat lunak tetap dapat dikembangkan karena kesesuaian kode program dan rancangan dalam dokumentasi. Proses yang mengalami dekomposisi tidak diimplementasikan dalam kode program, yang diimplementasikan adalah proses-proses pada dekomposisinya. Berikut adalah proses-proses yang ada di dalam diagram DFD studi kasus beserta contoh kode program dalam *pseudocode*:

Proses Dalam DFD	Level DFD	Aliran Data Masuk	Keterangan Fungsi	Contoh Implementasi
login	1	• login	memvalidasi pengguna apakah merupakan petugas atau bukan	<pre>function login(username: string, password: string){ //proses //return valid/tidak }</pre>

Proses Dalam DFD	Level DFD	Aliran Data Masuk	Keterangan Fungsi	Contoh Implementasi
logout	1		menghapus status login petugas yang sudah login	<pre> procedurelogout() //proses) </pre>
memeriksa status login	1		memeriksa apakah pengguna merupakan petugas yang sudah login	<pre> functionmemeriksaStatusLogin() //proses //return status_login } </pre>
mencari pustaka	1	• pencarian pustaka	melakukan pencarian pustaka dan menampilkan hasilnya	<pre> proceduremencariPustaka(kata_kunci: string, kategori: string) //proses } </pre>
memasukkan pustaka	2	• status login • pustaka	memeriksa status login, jika valid maka memasukkan data pustaka ke dalam tabel pustaka dan tabel pengarang pustaka	<pre> procedurememasukkanPustaka(id: string, judul: string, pengarang: string[], penerbit: string, jumlah: int, tahun: int) if(memeriksaStatusLogin()) //proses } </pre>
mengubah pustaka	2	• status login • pustaka	memeriksa status login, jika valid maka mengubah data pustaka dengan id masukan di tabel pustaka dan tabel pengarang pustaka	<pre> proceduremengubahPustaka(id: string, judul: string, pengarang: string[], penerbit: string, jumlah: int, tahun: int) if(memeriksaStatusLogin()) //proses } </pre>
menghapus pustaka	2	• status login	memeriksa status login, jika valid	<pre> proceduremenghapusPustaka(id: string, judul: string, pengarang: </pre>

Proses Dalam DFD	Level DFD	Aliran Data Masuk	Keterangan Fungsi	Contoh Implementasi
		• pustaka	maka menghapus data pustaka dengan id masukan di tabel pustaka dan tabel pengarang pustaka	<pre> string[], penerbit: string, jumlah: int, tahun: int){ if(memeriksaStatusLogi n()){ //proses } } </pre>
melihat pustaka	2	• status login • pustaka	memeriksa status login, jika valid maka menampilkan data pustaka	<pre> proceduremelihatPustak a(){ if(memeriksaStatusLogi n()){ //proses } } </pre>
memasukkan anggota	2	• status login • anggota	memeriksa status login, jika valid maka memasukkan data anggotake dalam tabel anggota dan tabel telepon anggota	<pre> procedurememasukkanAng gota(id: string, nama: string, alamat: string, email: string, telepon: string[]){ if(memeriksaStatusLogi n()){ //proses } } </pre>
mengubah anggota	2	• status login • anggota	memeriksa status login, jika valid maka mengubah data anggota dengan id masukan di tabel anggota dan tabel telepon anggota	<pre> proceduremengubahAnggo ta(id: string, nama: string, alamat: string, email: string, telepon: string[]){ if(memeriksaStatusLogi n()){ //proses } } </pre>
menghapus anggota	2	• status login • anggota	memeriksa status login, jika valid	<pre> proceduremenghapusAngg ota(id: string, nama: string, alamat: string, email: string, </pre>

Proses Dalam DFD	Level DFD	Aliran Data Masuk	Keterangan Fungsi	Contoh Implementasi
			maka menghapus data anggota dengan id masukan di tabel anggota dan tabel telepon anggota	<pre> telepon: string[] if(memeriksaStatusLogin()){ //proses } </pre>
melihat anggota	2	- status login - anggota	memeriksa status login, jika valid maka menampilkan data anggota	<pre> procedure melihatAnggota() { if(memeriksaStatusLogin()){ //proses } } </pre>
mencari anggota	2	pencarian anggota	melakukan pencarian anggota dan menampilkan hasilnya	<pre> procedure mencariAnggota(kata_kunci: string, kategori: string) { //proses } </pre>
memasukkan peminjaman	2	- status login - peminjaman	memeriksa status login, jika valid maka memasukkan data peminjaman ke dalam tabel peminjaman	<pre> procedure memasukkanPeminjaman(id_anggota: string, id_pustaka: string, tgl_pinjam: date, tgl_kembali: date) { if(memeriksaStatusLogin()){ //proses } } </pre>
mengubah peminjaman	2	- status login - peminjaman	memeriksa status login, jika valid maka mengubah data peminjaman dengan id_anggota, id_pustaka, dan	<pre> procedure mengubahPeminjaman(id_anggota: string, id_pustaka: string, tgl_pinjam: date, tgl_kembali: date) { if(memeriksaStatusLogin()){ //proses } } </pre>

Proses Dalam DFD	Level DFD	Aliran Data Masuk	Keterangan Fungsi	Contoh Implementasi
			tgl_pinjam masukan di tabel peminjaman	
menghapus peminjaman	2	- status login - peminjam an	memeriksa status login, jika valid maka menghapus data peminjaman dengan id_anggota, id_pustaka, dan tgl_pinjam masukan di tabel peminjaman	<pre> procedure menghapusPeminjaman(id_anggota: string, id_pustaka: string, tgl_pinjam: date, tgl_kembali: date){ if(memeriksaStatusLogin()){ //proses } } </pre>
melihat peminjaman	2	- status login - peminjam an	memeriksa status login, jika valid maka menampilkan data peminjaman	<pre> procedure melihatPeminjaman() { if (memeriksaStatusLogin()){ //proses } } </pre>
mencari peminjaman	2	pencarian peminjam an	melakukan pencarian peminjaman dan menampilkan hasilnya	<pre> procedure mencariPeminjaman(kata_kunci: string, kategori: string){ //proses } </pre>
memasukkan petugas	2	- status login - petugas	memeriksa status login, jika valid maka memasukkan data petugas ke dalam tabel petugas	<pre> procedure memasukkanPetugas(username: string, password: string, nama: string, no_petugas: string, hak_akses: string){ if (memeriksaStatusLogin()){ //proses } } </pre>
mengubah	2	status	memeriksa	<pre> procedure mengubahPetug </pre>

Proses Dalam DFD	Level DFD	Aliran Data Masuk	Keterangan Fungsi	Contoh Implementasi
petugas		login • petugas	status login, jika valid maka mengubah data petugas dengan username masukan di tabel petugas	<pre> as(username: string, password: string, nama: string, no_petugas: string, hak_akses: string){ if (memeriksaStatusLogi n()){ //proses } } </pre>
menghapus petugas	2	• status login • petugas	memeriksa status login, jika valid maka menghapus data petugas dengan username masukan di tabel petugas	<pre> proceduremenghapusPetu gas(username: string, password: string, nama: string, no_petugas: string, hak_akses: string){ if (memeriksaStatusLogi n()){ //proses } } </pre>
melihat petugas	2	• status login • petugas	memeriksa status login, jika valid maka menampilkan data petugas	<pre> proceduremelihatPetuga s(){ if (memeriksaStatusLogi n()){ //proses } } </pre>
mencari petugas	2	• pencarian petugas	melakukan pencarian petugas dan menampilkan hasilnya	<pre> procedure mencariPetugas(kata_ku ncl: string, kategori: string){ //proses } </pre>

Perancangan DFD bukan hanya dirancang, tapi juga diimplementasikan dalam pengembangan aplikasi agar selalu ada kesesuaian antara implementasi dan perancangan.



1. Apa yang dimaksud dengan pemrograman terstruktur?
2. Apa kelebihan dan kekurangan paradigma pemrograman terstruktur?
3. Apa yang dimaksud dengan DFD (*Data Flow Diagram*)?
4. Apa yang dimaksud dengan *Context Diagram*?
5. Bagaimana tahapan melakukan perancangan sistem menggunakan DFD?
6. Apa yang dimaksud dengan Kamus Data?
7. Bagaimana keterhubungan antara Kamus Data dengan Perancangan Basis Data?
8. Bisakah DFD digunakan untuk menggambarkan sistem berorientasi objek? Jelaskan alasannya!
9. Buatlah perancangan pemrograman terstruktur menggunakan DFD dan Kamus Data untuk studi kasus Sistem Informasi Apotek!

6

Pemrograman Berorientasi Objek



Overview

Bab ini berisi tentang pengenalan, sejarah perkembangan, dan konsep dasar rekayasa perangkat lunak untuk pemrograman berorientasi objek. Selain itu, pada bab ini juga disebutkan beberapa perbedaan antara pemrograman terstruktur dan pemrograman berorientasi objek. Penjelasan lebih detail tentang rekayasa perangkat lunak dengan menggunakan pendekatan berorientasi objek akan dijelaskan pada bab-bab selanjutnya.

6.1 Pengertian Pemrograman Berorientasi Objek

Metodologi berorientasi objek adalah suatu strategi pembangunan perangkat lunak yang mengorganisasikan perangkat lunak sebagai kumpulan objek yang berisi data dan operasi yang diberlakukan terhadapnya. Metodologi berorientasi objek merupakan suatu cara bagaimana sistem perangkat lunak dibangun melalui pendekatan objek secara sistematis. Metode berorientasi objek didasarkan pada penerapan prinsip-prinsip pengelolaan kompleksitas. Metode berorientasi objek meliputi rangkaian aktivitas analisis berorientasi objek, perancangan berorientasi objek, pemrograman berorientasi objek, dan pengujian berorientasi objek.

Pada saat ini, metode berorientasi objek banyak dipilih karena metodologi lama banyak menimbulkan masalah seperti adanya kesulitan pada saat mentransformasi hasil dari satu tahap pengembangan ke tahap berikutnya, misalnya pada metode pendekatan terstruktur, jenis aplikasi yang dikembangkan saat ini berbeda dengan masa lalu. Aplikasi yang dikembangkan pada saat ini sangat beragam (aplikasi bisnis, *real-time*, *utility*, dan sebagainya) dengan *platform* yang berbeda-beda, sehingga menimbulkan tuntutan kebutuhan metodologi pengembangan yang dapat mengakomodasi ke semua jenis aplikasi tersebut.

Keuntungan menggunakan metodologi berorientasi objek adalah sebagai berikut:

1. meningkatkan produktivitas
karena kelas dan objek yang ditemukan dalam suatu masalah masih dapat dipakai ulang untuk masalah lainnya yang melibatkan objek tersebut (*reusable*).
2. kecepatan pengembangan
karena sistem yang dibangun dengan baik dan benar pada saat analisis dan perancangan akan menyebabkan berkurangnya kesalahan pada saat pengkodean.

3. kemudahan pemeliharaan
karena dengan model objek, pola-pola yang cenderung tetap dan stabil dapat dipisahkan dan pola-pola yang mungkin sering berubah-ubah.
4. adanya konsistensi
karena sifat pewarisan dan penggunaan notasi yang sama pada saat analisis, perancangan maupun pengkodean.
5. meningkatkan kualitas perangkat lunak
karena pendekatan pengembangan lebih dekat dengan dunia nyata dan adanya konsistensi pada saat pengembangannya, perangkat lunak yang dihasilkan akan mampu memenuhi kebutuhan pemakai serta mempunyai sedikit kesalahan.

Saat ini sudah banyak bahasa pemrograman berorientasi objek. Banyak orang berpikir bahwa pemrograman berorientasi objek identik dengan bahasa Java. Memang bahasa Java merupakan bahasa yang paling konsisten dalam mengimplementasikan paradigma pemrograman berorientasi objek. Namun sebenarnya bahasa pemrograman yang mendukung pemrograman berorientasi objek tidak hanya bahasa Java.



Java merupakan bahasa pemrograman yang paling konsisten dalam mengimplementasikan paradigma pemrograman berorientasi objek.

Berikut beberapa contoh bahasa pemrograman yang mendukung pemrograman berorientasi objek:

- Bahasa Pemrograman Smalltalk
Smalltalk merupakan salah satu bahasa pemrograman yang dikembangkan untuk mendukung pemrograman berorientasi objek mulai tahun 1978. Smalltalk memiliki berbagai versi,

versi yang banyak dikenal adalah Smalltalk-80 yang dibuat pada tahun 1980.

- Bahasa Pemrograman Eiffel

Eiffel merupakan bahasa pemrograman yang dikembangkan untuk mendukung pemrograman berorientasi objek mulai tahun 1985 oleh Bertrand Meyer dan *compiler* Eiffel selesai dibuat pada tahun 1987. Eiffel memiliki sintaks mirip dengan sintaks pada bahasa pemrograman Pascal. Eiffel merupakan bahasa pemrograman objek murni karena semua kode programnya dibungkus di dalam kelas.

- Bahasa Pemrograman C++

Bahasa pemrograman C++ merupakan pengembangan lebih lanjut dari bahasa pemrograman C untuk mendukung pemrograman berorientasi objek. Keterkaitan antara bahasa C dan C++ dapat dilihat pada gambar berikut.



Gambar 27 Keterkaitan Bahasa Pemrograman C dan C++

- Bahasa Pemrograman (web) PHP

PHP dibuat pertama kali oleh seorang perancang perangkat lunak (*software engineering*) yang bernama Rasmus Lerdoff. Rasmus Lerdoff membuat halaman *web* PHP pertamanya pada tahun 1994. PHP4 dengan versi-versi akhir menuju PHP5 sudah mendukung pemrograman berorientasi objek. PHP merupakan bahasa pemrograman yang digunakan untuk pemrograman *web*.



PHP saat ini sudah mendukung pemrograman berorientasi objek

- Bahasa Pemrograman Java

Java dikembangkan oleh perusahaan Sun Microsystems. Java menurut definisi dari Sun Microsystems adalah nama untuk sekumpulan teknologi untuk membuat dan menjalankan perangkat lunak pada komputer *standalone* ataupun pada lingkungan jaringan. Java 2 adalah generasi kedua dari Java *platform*.

Java berdiri di atas sebuah mesin *interpreter* yang diberi nama *Java Virtual Machine* (JVM). JVM inilah yang akan membaca *bytecode* dalam *file* .class dari suatu program sebagai representasi langsung program yang berisi bahasa mesin. Oleh karena itu bahasa Java disebut sebagai bahasa pemrograman yang *portable* karena dapat dijalankan pada berbagai sistem operasi, asalkan pada sistem operasi tersebut terdapat JVM.

Java merupakan bahasa pemrograman objek murni karena semua kode programnya dibungkus dalam kelas. Saat ini Sun Microsystems sudah diakuisisi Oracle Corporation sehingga pengembangan Java diteruskan oleh Oracle Corporation.

6.2 Konsep Dasar Berorientasi Objek

Pendekatan berorientasi objek merupakan suatu teknik atau cara pendekatan dalam melihat permasalahan dan sistem (sistem perangkat lunak, sistem informasi, atau sistem lainnya). Pendekatan berorientasi objek akan memandang sistem yang akan dikembangkan sebagai suatu kumpulan objek yang berkorespondensi dengan objek-objek dunia nyata.

Ada banyak cara untuk mengabstraksikan dan memodelkan objek-objek tersebut, mulai dari abstraksi objek, kelas, hubungan antar kelas sampai abstraksi sistem. Saat mengabstraksikan dan memodelkan objek, data dan proses-proses yang dimiliki oleh objek akan dienkapsulasi (dibungkus) menjadi satu kesatuan.

Dalam rekayasa perangkat lunak, konsep pendekatan berorientasi objek dapat diterapkan pada tahap analisis, perancangan, pemrograman, dan pengujian perangkat lunak. Ada berbagai teknik yang dapat digunakan pada masing-masing tahap tersebut, dengan aturan dan alat bantu pemodelan tertentu.

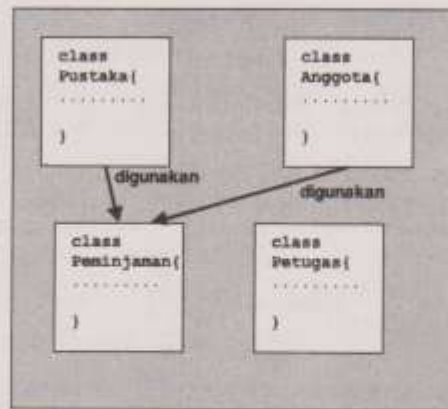
Sistem berorientasi objek merupakan sebuah sistem yang dibangun dengan berdasarkan metode berorientasi objek adalah sebuah sistem yang komponennya dibungkus (dienkapsulasi) menjadi kelompok data dan fungsi. Setiap komponen dalam sistem tersebut dapat mewarisi atribut dan sifat dan komponen lainnya, dan dapat berinteraksi satu sama lain.

Berikut ini adalah beberapa konsep dasar yang harus dipahami tentang metodologi berorientasi objek:

- *Kelas (class)*

Kelas adalah kumpulan objek-objek dengan karakteristik yang sama. Kelas merupakan definisi statik dan himpunan objek yang sama yang mungkin lahir atau diciptakan dari kelas tersebut. Sebuah kelas akan mempunyai sifat (atribut), kelakuan (operasi/metode), hubungan (relationship) dan arti. Suatu kelas dapat diturunkan dari kelas yang lain, dimana atribut dan kelas semula dapat diwariskan ke kelas yang baru.

Secara teknis, kelas adalah sebuah struktur tertentu dalam pembuatan perangkat lunak. Kelas merupakan bentuk struktur pada kode program yang menggunakan metodologi berorientasi objek. Ilustrasi dari sebuah kelas dapat dilihat pada gambar berikut.



Gambar 28 Ilustrasi Kelas

Kelas secara fisik adalah berkas atau *file* yang berisi kode program, di mana kode program merupakan semua hal yang terkait dengan nama kelas. Berikut adalah bentuk fisik dari sebuah kelas dalam *pseudocode*:

```

class Pustaka{
//atribut
    id : string
    judul : string
    pengarang : string[]
    penerbit : string
    jumlah : int
    tahun : int

//metode
    procedure setId(id: string){
        this.id = id
    }

    function getId(){
        return id
    }

    .....

    procedure queryCariPustaka(kata_kunci: string, kategori:
        string){
//proses
        query = "SELECT * FROM INTO tpustaka WHERE ....."
        execute(query)
    }
}
  
```

```

procedure queryMemasukkanPustaka(id: string, judul: string,
    pengarang: string[], penerbit: string, jumlah: int, tahun:
    int){
    //proses
    query = "INSERT INTO tpustaka(....) VALUES (...)"
    execute(query)
}

procedure queryMengubahPustaka(id: string, judul: string,
    pengarang: string[], penerbit: string, jumlah: int, tahun:
    int){
    //proses
    query = "UPDATE tpustaka SET ....."
    execute(query)
}

procedure queryMenghapusPustaka(id: string, judul: string,
    pengarang: string[], penerbit: string, jumlah: int, tahun:
    int){
    //proses
    query = "DELETE FROM tpustaka WHERE ....."
    execute(query)
}

procedure melihatPustaka(){
    //proses
    query = "SELECT * FROM tpustaka ....."
    execute(query)
}

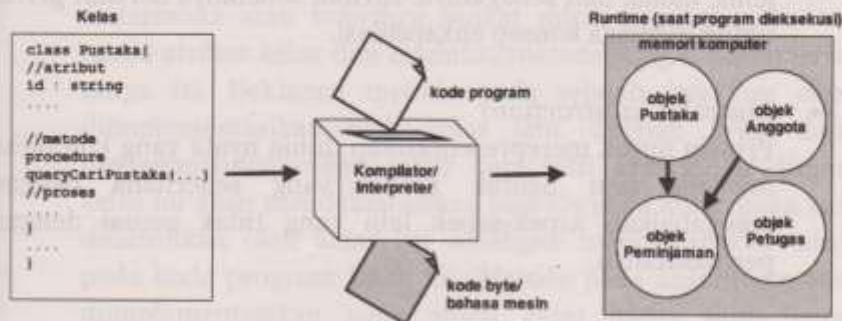
```

- **Objek (*object*)**

Objek adalah abstraksi dan sesuatu yang mewakili dunia nyata seperti benda, manusia, satuan organisasi, tempat, kejadian, struktur, status, atau hal-hal lain yang bersifat abstrak. Objek merupakan suatu entitas yang mampu menyimpan informasi (status) dan mempunyai operasi (kelakuan) yang dapat diterapkan atau dapat berpengaruh pada status objeknya. Objek mempunyai siklus hidup yaitu diciptakan, dimanipulasi, dan dihancurkan.

Secara teknis, sebuah kelas saat program dieksekusi maka akan dibuat sebuah objek. Objek dilihat dari segi teknis adalah elemen pada saat *runtime* yang akan diciptakan, dimanipulasi, dan dihancurkan saat eksekusi sehingga sebuah objek hanya ada saat sebuah program dieksekusi, jika masih

dalam bentuk kode, disebut sebagai kelas jadi pada saat *runtime* (saat sebuah program dieksekusi), yang kita punya adalah objek, di dalam teks program yang kita lihat hanyalah kelas. Ilustrasi kelas dan objek dapat dilihat pada gambar berikut.



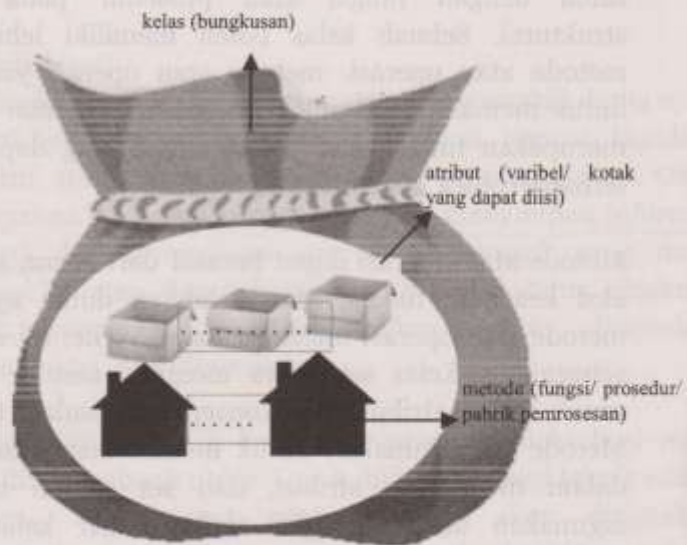
Gambar 29 Ilustrasi Kelas dan Objek

- Metode (*method*)

Operasi atau metode atau *method* pada sebuah kelas hampir sama dengan fungsi atau prosedur pada metodologi struktural. Sebuah kelas boleh memiliki lebih dari satu metode atau operasi. metode atau operasi yang berfungsi untuk memanipulasi objek itu sendiri. Operasi atau metode merupakan fungsi atau transformasi yang dapat dilakukan terhadap objek atau dilakukan oleh objek.

Metode atau operasi dapat berasal dari *event*, aktivitas atau aksi keadaan, fungsi, atau kelakuan dunia nyata. Contoh metode atau operasi misalnya Read, Write, Move, Copy, dan sebagainya. Kelas sebaiknya memiliki metode get dan set untuk setiap atribut agar konsep enkapsulasi tetap terjaga. Metode get digunakan untuk memberikan akses kelas lain dalam mengakses atribut, dan set adalah metode yang digunakan untuk mengisi atribut, agar kelas lain tidak mengakses atribut secara langsung.

- **Atribut (*attribute*)**
Atribut dari sebuah kelas adalah variabel global yang dimiliki sebuah kelas. Atribut dapat berupa nilai atau elemen-elemen data yang dimiliki oleh objek dalam kelas objek. Atribut dipunyai secara individual oleh sebuah objek, misalnya berat, jenis, nama, dan sebagainya. Atribut sebaiknya bersifat privat untuk menjaga konsep enkapsulasi.
- **Abstraksi (*abstraction*)**
Prinsip untuk merepresentasikan dunia nyata yang kompleks menjadi satu bentuk model yang sederhana dengan mengabaikan aspek-aspek lain yang tidak sesuai dengan permasalahan.
- **Enkapsulasi (*encapsulation*)**
Pembungkusan atribut data dan layanan (operasi-operasi) yang dipunyai objek untuk menyembunyikan implementasi dan objek sehingga objek lain tidak mengetahui cara kerjanya. Misalkan seperti pada ilustrasi berikut:



Gambar 30 Ilustrasi Enkapsulasi

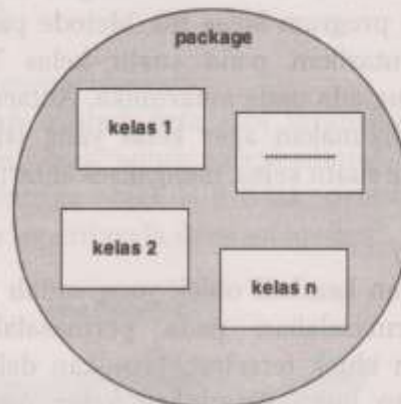
- **Pewarisan (*inheritance*)**
Mekanisme yang memungkinkan satu objek mewarisi sebagian atau seluruh definisi dan objek lain sebagai bagian dan dirinya.
- **Antarmuka (*interface*)**
Antarmuka atau *interface* sangat mirip dengan kelas, tapi tanpa atribut kelas dan memiliki metode yang dideklarasikan tanpa isi. Deklarasi metode pada sebuah *interface* dapat diimplementasikan oleh kelas lain. Sebuah kelas dapat mengimplementasikan lebih dari satu antarmuka dimana kelas ini akan mendeklarasikan metode pada antarmuka yang dibutuhkan oleh kelas itu sekaligus mendefinisikan isinya pada kode program kelas itu. Metode pada antarmuka yang diimplementasikan pada suatu kelas harus sama persis dengan yang ada pada antarmuka. Antarmuka atau *interface* biasanya digunakan agar kelas yang lain tidak mengakses langsung ke suatu kelas, mengakses antarmukanya.
- **Reusabilily**
Pemanfaatan kembali objek yang sudah didefinisikan untuk suatu permasalahan pada permasalahan lainnya yang melibatkan objek tersebut. Misalkan dalam sebuah aplikasi peminjaman buku diperlukan kelas Anggota, maka ketika membuat aplikasi penyewaan VCD, kelas anggota ini bisa digunakan kembali dengan sedikit perubahan untuk aplikasi penyewaan VCD tanpa harus membuat dari awal kembali.
- **Generalisasi dan Spesialisasi**
Menunjukkan hubungan antara kelas dan objek yang umum dengan kelas dan objek yang khusus. Misalnya kelas yang lebih umum (generalisasi) adalah kendaraan darat dan kelas khususnya (spesialisasi) adalah mobil, motor, dan kereta.
- **Komunikasi Antar Objek**
Komunikasi antar-objek dilakukan lewat pesan (*message*) yang dikirim dan satu objek ke objek lainnya.

- **Polimorfisme (*polymorphism*)**

Kemampuan suatu objek untuk digunakan di banyak tujuan yang berbeda dengan nama yang sama sehingga menghemat baris program.

- **Package**

Package adalah sebuah kontainer atau kemasan yang dapat digunakan untuk mengelompokkan kelas-kelas sehingga memungkinkan beberapa kelas yang bernama sama disimpan dalam *package* yang berbeda. Ilustrasi dari sebuah *package* dapat dilihat pada gambar berikut.



Gambar 31 *Package*

6.3 Perbandingan Pendekatan OO dan Terstruktur

Perbedaan yang paling dasar dari pendekatan terstruktur dan pendekatan OO (*Object Oriented*) atau berorientasi objek adalah pada metode berorientasi fungsi atau aliran data/*Data Flow Diagram* (DFD) /pendekatan terstruktur, dekomposisi permasalahan dilakukan berdasarkan fungsi atau proses secara hirarki, mulai dari konteks sampai proses-proses yang paling kecil. Sementara pada metode berorientasi objek, dekomposisi permasalahan dilakukan berdasarkan objek-objek yang ada dalam sistem.

Pendekatan terstruktur berkembang cukup pesat pada tahun 1990an karena pendekatan terstruktur mempunyai kelebihan dalam kemudahan untuk memahami sistem. Konsep tentang dekomposisi permasalahan mulai dari paling dasar (diagram konteks) sampai paling detail cukup memudahkan dalam pemahaman bentuk sistem khususnya bagi *user* yang mempunyai pemahaman tentang sistem cukup rendah.

Pendekatan berorientasi objek saat ini berkembang cukup pesat karena mempunyai kelebihan dalam peningkatan produktivitas karena mempunyai *reusability* yang cukup tinggi dibanding dengan pendekatan lain. Kelas-kelas dalam pemrograman berorientasi objek dapat dengan mudah dimanfaatkan untuk sistem lain yang dikembangkan.

Ilustrasi perbandingan pendekatan berorientasi objek dengan pendekatan terstruktur dapat dilihat pada gambar berikut:



Gambar 32 Ilustrasi Perbandingan OO vs Terstruktur



1. Sebutkan kelebihan dan kekurangan metodologi berorientasi objek!
2. Jelaskan secara singkat sejarah perkembangan metodologi berorientasi objek!
3. Ilustrasikan dalam dunia nyata, apa yang disebut
 - a. Kelas
 - b. Objek
 - c. *Method*
 - d. Atribut
 - e. *Interface*
 - f. *Package*
4. Mengapa saat ini metodologi berorientasi objek berkembang lebih pesat dibandingkan dengan metode-metode yang lain dalam bidang rekayasa perangkat lunak?
5. Sebutkan paradigma lain dalam bahasa pemrograman selain paradigma berorientasi objek, bandingkan, dan sebutkan masing-masing kekurangan dan kelemahannya!
6. Mengapa bahasa pemrograman PHP generasi terbaru berkembang menjadi bahasa pemrograman berorientasi objek?

7

Analisis dan Desain Berorientasi Objek



Overview

Bab ini menjelaskan mengenai beberapa contoh metode dalam melakukan analisis dan desain berorientasi objek yang mulai berkembang pada sekitar tahun 1990-an. Beberapa metode tersebut menjadi acuan pengembangan UML. Pembahasan tentang UML akan dilakukan pada bab-bab selanjutnya. Metode tersebut antara lain sebagai berikut:

1. CRC (*Component, Responsibility, Collaborator*)
2. Metode Booch
3. OMT (*Object Modelling Technique*)

4. OOSE (*Object-Oriented Software Engineering*)
5. Metode Coad Yourdon

Selain itu pada bab ini juga dijelaskan secara singkat tentang CASE tools untuk pendukung kegiatan analisis dan desain berorientasi objek serta kategori-kategori CASE tools.

Bab ini juga berisi penjelasan tentang RUP, tahapan dalam RUP, dan penggunaannya. Setelah membaca bab ini, mahasiswa diharapkan mampu untuk melakukan proses pengembangan perangkat lunak secara iteratif sesuai dengan RUP.

7.1 Analisis Berorientasi Objek

Analisis berorientasi objek atau *Object Oriented Analysis* (OOA) adalah tahapan untuk menganalisis spesifikasi atau kebutuhan akan sistem yang akan dibangun dengan konsep berorientasi objek, apakah benar kebutuhan yang ada dapat diimplementasikan menjadi sebuah sistem berorientasi objek. Ilustrasi dapat dilihat pada gambar berikut:



Gambar 33 Ilustrasi OOA

Analisis ini sebaiknya dilakukan oleh orang-orang yang benar-benar memahami implementasi sistem yang berbasis atau berorientasi

objek, karena tanpa pemahaman itu maka sistem yang dihasilkan bisa jadi tidak realistis jika diimplementasikan dengan berbasis objek.

OOA biasanya menggunakan kartu CRC (*Component, Responsibility, Collaborator*) untuk membangun kelas-kelas yang akan digunakan atau menggunakan UML (*Unified Modeling Language*) pada bagian diagram *use case*, diagram kelas, dan diagram objek.

Sebelum perkembangan UML ada beberapa bahasa pemodelan yang berkembang untuk memodelkan sistem berorientasi objek. Bahasa pemodelan berorientasi objek pada awal perkembangannya (generasi awal) di antaranya adalah:

1. CRC

CRC (*Component, Responsibility, Collaborator*) diperkenalkan oleh Kent Beck dan Ward Cunningham pada tahun 1989 sebagai bagian dari *Object-Oriented Programming, Systems, Languages and Applications* (OOPSLA). CRC dibuat untuk bakal yang akan menjadi kelas yang dianalisis. Berikut adalah contoh sebuah kartu CRC:

Class Name: Person	
Superclasses:	
Subclasses:	
Responsibilities	Collaborators
Gender Age Calculate age at given year Cook capability	Cuisine, Receipe

yang dalam bahasa Indonesia adalah sebagai berikut:

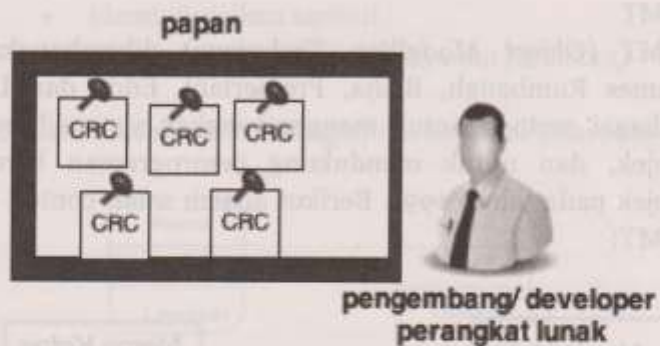
Nama Kelas: Manusia	
Kelas Orang Tua:	
Kelas Anak:	
Tanggung Jawab	Kelas Terkait
Jenis kelamin:	Masakan, Resep

Usia	
Menghitung usia pada	
tahun yang diberikan	
Kemampuan memasak	

Di mana:

- **Nama Kelas (*Class Name*)**
Merupakan nama yang diberikan pada sebuah kelas
- **Kelas Orang Tua (*Superclasses*)**
Merupakan kelas orang tua (dalam hubungan pewarisan) atau kelas super dari kelas yang dibuat CRC-nya
- **Kelas Anak (*Subclasses*)**
Merupakan kelas anak (dalam hubungan pewarisan) atau sub kelas dari kelas yang dibuat CRC-nya
- **Tanggung Jawab (*Responsibilities*)**
Merupakan isi atribut dan operasi yang harus ada dalam kelas yang dibuat CRC-nya
- **Kolaborator/Kelas yang terkait (*Collaborators*)**
Merupakan kelas yang terkait (untuk bekerjasama) dengan kelas yang sedang dibuat CRC-nya tetapi bukan merupakan kelas orang tua atau kelas anak. Terkait dalam hubungan memakai dan dipakai di dalam kelas.

CRC dibuat per kelas. Keterkaitan antarkelas dapat dilihat pada kolom kelas super, subkelas, dan kolaborator/kelas terkait. Dalam sebuah analisis pembuatan kelas, dapat terdiri dari banyak kartu CRC. Ilustrasi penggunaan CRC dapat dilihat pada gambar berikut:



Gambar 34 Ilustrasi Penggunaan CRC

CRC awalnya dibuat di kertas, lalu ditempelkan di papan atau di meja di ruang diskusi pengembang/developer perangkat lunak.

2. Metode Booch

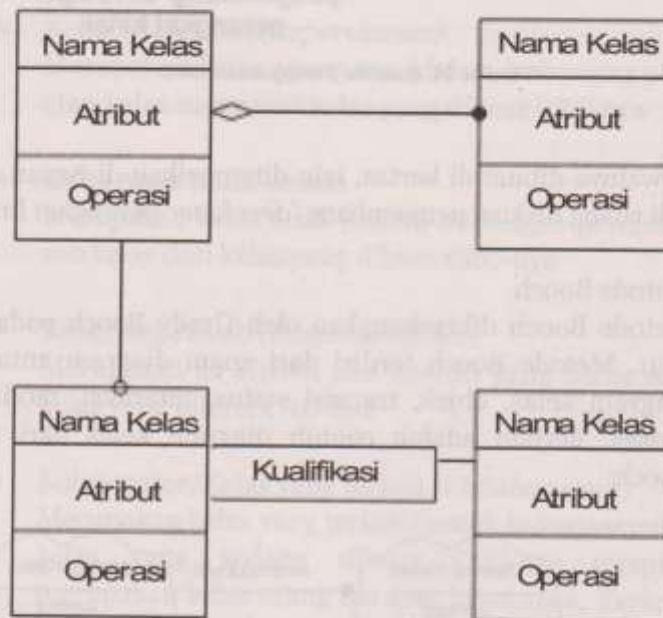
Metode Booch dikembangkan oleh Grady Booch pada tahun 1991. Metode Booch terdiri dari enam diagram antara lain diagram kelas, objek, transisi status, interaksi, modul, dan proses. Berikut adalah contoh diagram kelas dari metode Booch:



Gambar 35 Contoh diagram Booch

3. OMT

OMT (*Object Modelling Technique*) dikembangkan oleh James Rumbaugh, Blaha, Premerlani, Eddy dan Lorensen sebagai metode untuk mengembangkan sistem berorientasi objek, dan untuk mendukung pemrograman berorientasi objek pada tahun 1991. Berikut adalah salah contoh diagram OMT:



Gambar 36 Contoh diagram OMT

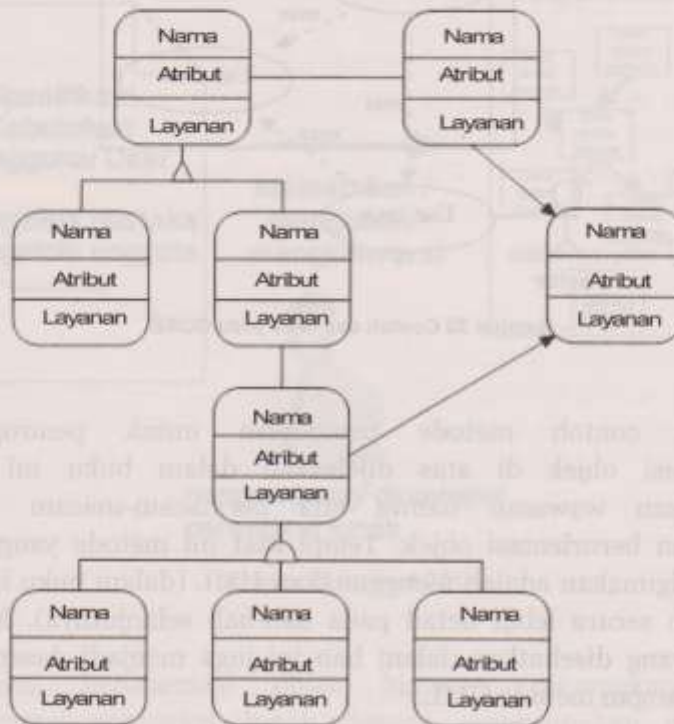
4. Metode Coad Yourdon

Metode Coad Yourdon diperkenalkan pada tahun 1991. Metode Coad Yourdon menyediakan sebuah diagram kelas. Diagram kelas pada metode ini dibuat dengan langkah-langkah sebagai berikut:

- Mendefinisikan kelas dan objek
- Mengidentifikasi struktur kelas dan objek
- Mendefinisikan subjek nama kelas

- Mendefinisikan atribut
- Mendefinisikan operasi/layanan (*services*)

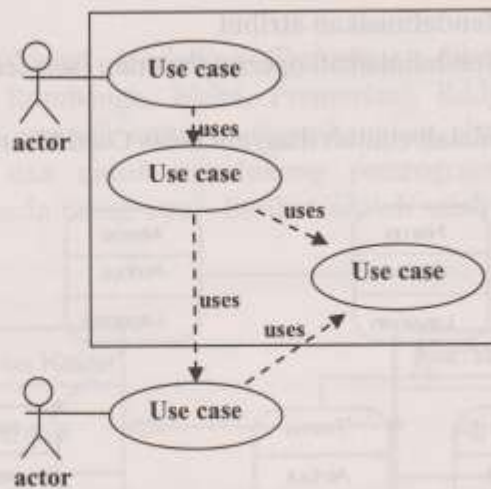
Berikut adalah contoh diagram kelas Coad Yourdon:



Gambar 37 Contoh diagram kelas Coad Yourdon

5. OOSE

OOSE (*Object-Oriented Software Engineering*) dikembangkan oleh Ivar Jacobson pada tahun 1992. OOSE adalah metodologi desain berorientasi objek yang mulai melibatkan *use case* (penggunaan kasus). Berikut adalah contoh diagram *use case* pada metodologi OOSE:



Gambar 38 Contoh use case pada OOSE

Beberapa contoh metode pemodelan untuk pemrograman berorientasi objek di atas dijelaskan dalam buku ini untuk memberikan wawasan bahwa ada bermacam-macam metode pemodelan berorientasi objek. Tetapi saat ini metode yang paling banyak digunakan adalah menggunakan UML (dalam buku ini akan dijelaskan secara lebih detail pada bab-bab selanjutnya). Metode-metode yang disebutkan dalam bab ini juga menjadi dasar dalam pengembangan metode UML.

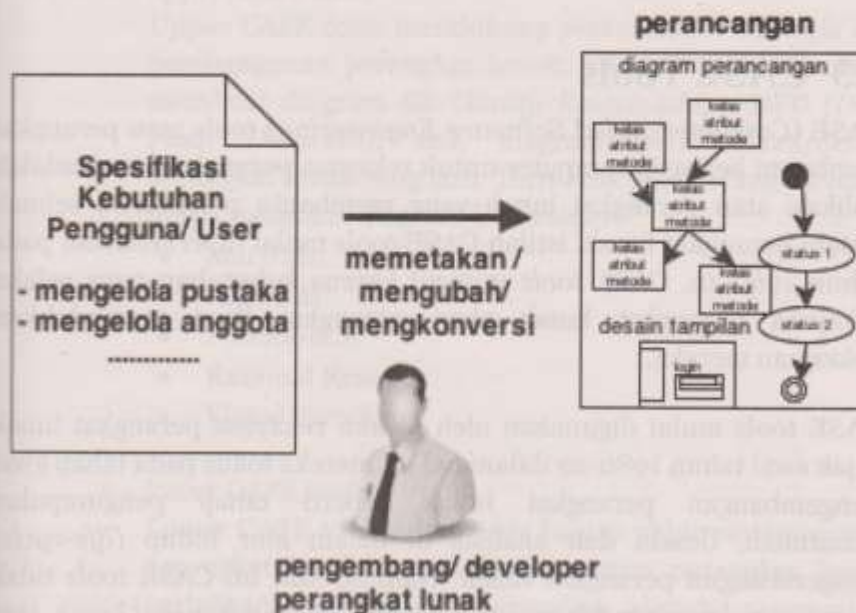


Saat ini pemodelan yang paling sering digunakan pada pemrograman berorientasi objek adalah menggunakan UML.

7.2 Desain Berorientasi Objek

Desain berorientasi objek atau *Object Oriented Design* (OOD) adalah tahapan perantara untuk memetakan spesifikasi atau kebutuhan sistem yang akan dibangun dengan konsep berorientasi objek ke desain pemodelan agar lebih mudah diimplementasikan dengan

pemrograman berorientasi objek. Ilustrasi OOD dapat dilihat pada gambar berikut:



Gambar 39 Ilustrasi OOD

Pemodelan berorientasi objek biasanya dituangkan dalam dokumentasi perangkat lunak dengan menggunakan perangkat pemodelan berorientasi objek, di antaranya adalah UML (*Unified Modeling Language*). Kendala dan permasalahan pembangunan sistem berorientasi objek biasanya dapat dikenali dalam tahap ini.

OOA dan OOD dalam proses yang berulang-ulang seringkali memiliki batasan yang samar, sehingga kedua tahapan ini sering juga disebut OOAD (*object oriented analysis and design*) atau dalam bahasa Indonesia berarti Analisis dan Desain Berorientasi Objek.

OOA dan OOD sering kali memiliki batasan yang samar, sehingga biasanya disebutkan langsung

menjadi OOAD (*Object Oriented Analysis and Design*)

7.3 CASE Tools

CASE (*Computer-Aided Software Engineering*) tools atau perangkat pembantu berbasis komputer untuk rekayasa perangkat lunak adalah aplikasi atau perangkat lunak yang membantu pembuatan sebuah sistem perangkat lunak. istilah CASE tools mulai diperkenalkan pada tahun 1960-an. CASE tools muncul karena kebutuhan para pelaku rekayasa perangkat lunak akan perangkat yang memudahkan pekerjaan mereka.

CASE tools mulai digunakan oleh pelaku rekayasa perangkat lunak sejak awal tahun 1980-an dalam hal ini mereka fokus pada tahap awal pengembangan perangkat lunak seperti tahap pengumpulan kebutuhan, desain dan analisis di dalam alur hidup (*life-cycle*) pengembangan perangkat lunak. Namun, saat ini CASE tools tidak hanya berfokus pada tahap awal pengembangan perangkat lunak, tapi sudah ke semua bagian pengembangan dan perbaikan perangkat lunak.

Perangkat lunak yang termasuk CASE tools dapat merupakan perangkat lunak dalam tahap analisis, desain, dokumentasi, maupun implementasi ke dalam kode program dan pengujian program. CASE tools termasuk editor desain perangkat lunak, editor kode program, kompiler, *debugger*, perangkat pembangunan sistem perangkat lunak, dan lain sebagainya. CASE juga dapat mengacu pada metode yang didedikasikan pada disiplin rekayasa sistem informasi yang bersifat otomatis (tidak manual). CASE pada dasarnya digunakan untuk membangun kualitas perangkat lunak yang memiliki performansi efektif.

CASE tools dapat dikelompokkan menjadi empat dimensi yang berbeda sebagai berikut:

1. Pendukung alur hidup perangkat lunak (*life cycle support*)

Dibagi menjadi dua buah kelompok yaitu

- *upper CASE tools*

Upper CASE tools mendukung perencanaan strategis dan pembangunan perangkat lunak, misalnya aplikasi untuk membuat diagram ER (*Entity Relationship*), DFD (*Data Flow Diagram*), dan diagram perencanaan/desain perangkat lunak yang lain. Termasuk juga perangkat lunak untuk membuat diagram UML seperti:

- StarUML
- Argo UML
- PoseidonCE
- Rational Rose
- Visual Paradigm

- *lower CASE tools.*

Lower CASE tools fokus pada bagian akhir pembangunan perangkat lunak seperti implementasi perangkat lunak, perbaikan kesalahan perangkat lunak, pengujian, konstruksi dan integrasi perangkat lunak, pemeliharaan perangkat lunak, perekayasaan perangkat lunak kembali (*reengineering*), atau *reverse engineering* (perbaikan rekayasa perangkat lunak diawali dari perangkat lunak yang sudah jadi).

2. Dimensi integrasi (*integration dimension*)

dimensi ini fokus pada integrasi komponen perangkat lunak, seperti misalnya *framework* (kerangka kerja), integrasi lingkungan pengembangan perangkat lunak, dan lain sebagainya.

3. Dimensi konstruksi (*construction dimension*)

dimensi ini fokus pada proses konstruksi perangkat lunak, misalkan untuk rekayasa ulang perangkat lunak (*reengineering*), *reverse engineering*, atau konstruksi awal perangkat lunak.

4. Dimensi CASE berbasis keilmuan (*knowledge-based CASE*) dimensi ini fokus pada perangkat untuk membantu perangkat lunak memiliki basis keilmuan (biasanya berupa perangkat lunak pemodelan simulasi proses bisnis yang diinginkan) misalkan untuk membangun sistem pendukung keputusan (*Decision Support System*) atau sistem pakar (*Expert System*)

Banyaknya CASE tools yang dapat digunakan membuat perlunya ada standarisasi CASE tools yang baik itu sendiri untuk menghindari hal-hal berikut:

- standarisasi yang tidak memadai yang dapat menyebabkan integrasi komponen antar yang menggunakan CASE tools satu dan lainnya tidak dapat dilakukan karena perbedaan standar
- harapan yang tidak realistis karena tidak menggunakan CASE tools sehingga batasan-batasan yang seharusnya tidak dilakukan dapat dibatasi jika menggunakan CASE tools
- waktu implementasi yang lambat
- kontrol penyimpanan yang lemah, misalkan kontrol pada penyimpanan dokumentasi, kode program versi terbaru dan lain sebagainya yang terkait dengan perangkat lunak.



CASE tools adalah alat bantu untuk pengembangan perangkat lunak agar hasilnya lebih baik dan waktu pengerjaannya lebih singkat.

7.4 RUP

Unified Process atau dikenal juga dengan proses iteratif dan *incremental* merupakan sebuah proses pengembangan perangkat lunak yang dilakukan secara iteratif (berulang) dan inkremental (bertahap dengan progres menaik). Iteratif bisa dilakukan di dalam setiap tahap, atau iteratif tahap pada proses pengembangan perangkat lunak untuk menghasilkan perbaikan fungsi yang inkremental (bertambah menaik) di mana setiap interaksi akan

memperbaiki iterasi berikutnya. Salah satu *Unified Process* yang terkenal adalah RUP (*Rational Unified Process*).

RUP (*Rational Unified Process*) adalah pendekatan pengembangan perangkat lunak yang dilakukan berulang-ulang (*iterative*), fokus pada arsitektur (*architecture-centric*), lebih diarahkan berdasarkan penggunaan kasus (*use case driven*). RUP merupakan proses rekayasa perangkat lunak dengan pendefinisian yang baik (*well defined*) dan penstrukturan yang baik (*well structured*). RUP menyediakan pendefinisian struktur yang baik untuk alur hidup proyek perangkat lunak. RUP adalah sebuah produk proses perangkat lunak yang dikembangkan oleh *Rational Software* yang diakuisisi oleh IBM di bulan Februari 2003.

Proses pengulangan/iteratif pada RUP secara global dapat dilihat pada gambar berikut:



Gambar 40 Proses iteratif RUP



RUP (*Rational Unified Process*) adalah tahapan pengembangan sistem secara iteratif khusus untuk pemrograman berorientasi objek.

7.4.1 Kelebihan RUP

Pendekatan iteratif/pengulangan dari RUP dapat mengkomodir beberapa kelemahan pengembangan perangkat lunak tanpa menggunakan konsep pengulangan, misalnya pada pengembangan perangkat lunak *waterfall*, berikut adalah hal-hal yang dapat diatasi oleh RUP dibanding *waterfall*:

1. RUP mengakomodasi perubahan kebutuhan perangkat lunak. Kebutuhan untuk mengubah dan menambah fitur karena perubahan teknologi atau keinginan pelanggan (*customer*) merupakan salah satu kendala yang sering dialami pengembangan perangkat lunak yang berimbas pada terlambatnya waktu penyelesaian perangkat lunak. Keterlambatan ini dapat menyebabkan ketidakpuasan di sisi pelanggan (*customer*) dan pengembang menjadi frustrasi. Pengembangan secara iteratif fokus tim dalam membangun dan mendemonstrasikan perangkat lunak untuk beberapa minggu berikutnya sehingga memaksa untuk fokus pada kebutuhan yang lebih penting.
2. Integrasikanlah sebuah proses besar dan cepat ("big bang") di akhir proyek.
Menempatkan integrasi di bagian akhir proyek biasanya memakan waktu proyek yang cukup banyak, bisa mencapai 40 persen dari waktu proyek. Untuk mencegah hal ini maka pendekatan secara iteratif (pengulangan) dapat memecah proyek menjadi bagian iterasi kecil yang diakhiri dengan integrasi kecil yang nantinya digabungkan menjadi integrasi besar.
3. Risiko biasanya ditemukan atau dialamatkan selama pada proses integrasi awal.
Pendekatan integrasi pada RUP mengurangi resiko pada iterasi awal dimana saat semua komponen diuji.

4. Manajemen berarti membuat perubahan taktik pada produk
Taktik produk misalnya pengembangan dengan waktu singkat akan menghasilkan produk dengan fungsi yang terbatas akan dapat cepat digunakan oleh *user* sehingga memperkenalkan produk lebih cepat ke masyarakat dibandingkan produk kompetitor lain yang masih sedang dikembangkan.
5. Mendukung fasilitas penggunaan kembali
Lebih mudah untuk mengidentifikasi bagian umum yang sering digunakan dalam aplikasi jika diimplementasikan secara iterasi daripada mengidentifikasi pada saat perencanaan saja. Peninjauan kembali pada iterasi awal dapat membuat arsitek perangkat lunak untuk menandai peluang penggunaan kembali (*reuse*) dan kemudian mengembangkan kode umum yang lebih baik atau mapan pada iterasi berikutnya.
6. Kecacatan dapat ditemukan dan diperbaiki pada beberapa iterasi menghasilkan arsitektur yang baik dan aplikasi berkualitas tinggi
Kecacatan dideteksi lebih baik mulai pada iterasi awal daripada pada tahap pengujian. Sehingga kejadian seperti bottleneck dapat ditemukan tidak pada saat aplikasi sudah dikirimkan ke pelanggan (*customer*).
7. Lebih baik menggunakan menggunakan "anggota proyek" dibandingkan susunan secara seri pada tim proyek
Pada pendekatan *waterfall* anggota tim bekerja secara seri, seperti seorang analis bekerja untuk menganalisis kebutuhan sistem lalu memberikan hasil analisis ke desainer untuk melakukan desain sistem, kemudian desainer memberikan desain ke *programmer* dan *programmer* mengirimkan aplikasi ke pelanggan (*customer*). Hal ini dapat menyebabkan kesalahpahaman akan hasil sebelumnya misalkan antara analis kebutuhan dan *programmer*.

Selain itu juga dapat menyebabkan kesalahpahaman akan siapa yang harusnya bertanggung jawab jika terjadi kecacatan

atau kesalahan. Hal ini dapat terjadi karena pada pendekatan *waterfall* seorang analis hanya bekerja pada waktu analisis kebutuhan saja, desainer hanya pada waktu desain saja, dan *programmer* hanya pada saat implementasi dan pengujian program. Dengan menggunakan mekanisme "anggota proyek" maka setiap iterasi akan terjadi kerjasama antar anggota proyek untuk saling memperbaiki yang menjadi tanggung jawabnya.

8. Anggota tim belajar selama proyek berjalan

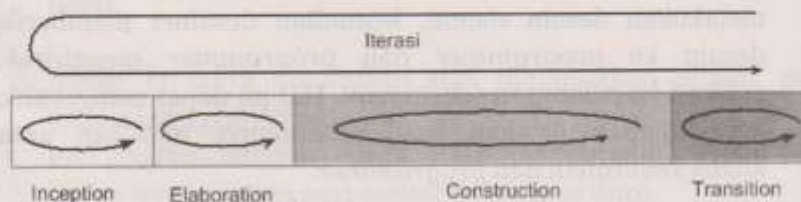
Anggota proyek memiliki peluang belajar dari kesalahan selama siklus pengembangan perangkat lunak dan memperbaiki kesalahan pada iterasi berikutnya. Kesalahan yang terjadi pada iterasi sebelumnya dapat diperbaiki pada iterasi berikutnya.

9. Pengembangan perangkat lunak dapat diperbaiki seiring proses pengembangan perangkat lunak

Setiap akhir proses iterasi tidak hanya ada peninjauan mengenai target produk tetapi juga peninjauan pada kesalahan yang terjadi pada iterasi sebelumnya agar dapat diperbaiki pada iterasi berikutnya.

7.4.2 FaseRUP

RUP memiliki empat buah tahap atau fase yang dapat dilakukan pula secara iteratif. Berikut ini adalah gambar alur hidup RUP.



Gambar 41 Alur hidup RUP

Berikut ini penjelasan untuk setiap fase pada RUP.

1. *Inception* (permulaan)

Tahap ini lebih pada memodelkan proses bisnis yang dibutuhkan (*business modeling*) dan mendefinisikan kebutuhan akan sistem yang akan dibuat (*requirements*). Berikut adalah tahap yang dibutuhkan pada tahap ini:

- memahami ruang lingkup dari proyek (termasuk pada biaya, waktu, kebutuhan, resiko dan lain sebagainya)
- membangun kasus bisnis yang dibutuhkan

Hasil yang diharapkan dari tahap ini adalah memenuhi *Lifecycle Objective Milestone* (batas/tonggak objektif dari siklus) dengan kriteria berikut:

- Umpan balik dari pendefinisian ruang lingkup, perkiraan biaya, dan perkiraan jadwal.
- Kebutuhan dimengerti dengan pasti (dapat dibuktikan) dan sejalan dengan kasus primer yang dibutuhkan.
- Kredibilitas dari perkiraan biaya, perkiraan jadwal, penentuan skala prioritas, resiko, dan proses pengembangan.
- Ruang lingkup purwarupa (*prototype*) yang akan dikembangkan
- Membangun garis dasar dengan membandingkan perencanaan aktual dengan perencanaan yang direncanakan.

Jika pada akhir tahap ini target yang diinginkan tidak dicapai maka dapat dibatalkan atau diulang kembali setelah dirancang ulang agar kriteria yang diinginkan dapat dicapai. Batas/tonggak objektif digunakan untuk mendeteksi apakah sebuah kebutuhan akan sistem dapat diimplementasikan atau tidak.

2. *Elaboration* (perluasan/perencanaan)

Tahap ini lebih difokuskan pada perencanaan arsitektur sistem. Tahap ini juga dapat mendeteksi apakah arsitektur sistem yang diinginkan dapat dibuat atau tidak. Mendeteksi resiko yang mungkin terjadi dari arsitektur yang dibuat. Tahap ini lebih pada analisis dan desain sistem serta implementasi sistem yang fokus pada purwarupa sistem (*prototype*).

Hasil yang diharapkan dari tahap ini adalah memenuhi *Lifecycle Architecture Milestone* (batas/tonggak arsitektur dari siklus) dengan kriteria berikut:

- model kasus yang digunakan (*use case*) dimana kasus dan aktor yang terlibat telah diidentifikasi dan sebagian besar kasus harus dikembangkan. Model *use case* harus 80 persen lengkap dibuat.
- deskripsi dari arsitektur perangkat lunak dari proses pengembangan sistem perangkat lunak telah dibuat.
- rancangan arsitektur yang dapat diimplementasikan dan mengimplementasikan *use case*
- kasus bisnis atau proses bisnis dan daftar risiko yang sudah mengalami perbaikan (revisi) telah dibuat
- rencana pengembangan untuk seluruh proyek telah dibuat
- purwarupa (*prototype*) yang dapat didemonstrasikan untuk mengurangi setiap resiko teknis yang diidentifikasi.

Jika pada akhir tahap ini target yang diinginkan tidak dicapai maka dapat dibatalkan atau diulang kembali. Batas/tonggak arsitektur digunakan untuk mendeteksi apakah sebuah kebutuhan akan sistem dapat diimplementasikan atau tidak melalui pembuatan arsitektur.

3. *Construction* (konstruksi)

Tahap ini fokus pada pengembangan komponen dan fitur-fitur sistem. Tahap ini lebih pada implementasi dan pengujian sistem yang fokus pada implementasi perangkat lunak pada kode program. Tahap ini menghasilkan produk perangkat

lunak dimana menjadi syarat dari *Initial Operational Capability Milestone* atau batas/tonggak kemampuan operasional awal.

4. *Transition* (transisi)

Tahap ini lebih pada *deployment* atau instalasi sistem agar dapat dimengerti oleh *user*. Tahap ini menghasilkan produk perangkat lunak dimana menjadi syarat dari *Initial Operational Capability Milestone* atau batas/tonggak kemampuan operasional awal. Aktivitas pada tahap ini termasuk pada pelatihan *user*, pemeliharaan dan pengujian sistem apakah sudah memenuhi harapan *user*.

Produk perangkat lunak juga disesuaikan dengan kebutuhan yang didefinisikan pada tahap *inception*. Jika semua kriteria objektif terpenuhi maka dianggap sudah memenuhi *Product Release Milestone* (batas/tonggak peluncuran produk) dan pengembangan perangkat lunak selesai dilakukan.

Akhir dari keempat fase ini adalah produk perangkat lunak yang sudah lengkap. Keempat fase pada RUP dijalankan secara berurutan dan iteratif dimana setiap iterasi dapat digunakan untuk memperbaiki iterasi berikutnya.



1. Apa perbedaan antara analisis berorientasi objek dengan desain berorientasi objek?
2. Jelaskan bagaimana sejarah perkembangan metode analisis dan desain berorientasi objek!
3. Bandingkan metode OMT dan OOSE, kemudian sebutkan masing-masing kelebihan dan kekurangannya!
4. Bilamana sebaiknya CASE *tools* digunakan?
5. Jelaskan bagaimana sejarah berkembangnya CASE *tools*!
6. Sebutkan dan jelaskan minimal 2 contoh metode analisis dan desain berorientasi objek selain yang sudah dijelaskan dalam buku ini!
7. Sebutkan contoh-contoh CASE *tools* yang saat ini beredar di pasaran! Setiap dimensi minimal terdapat 1 contoh.
8. Apa pengertian RUP?
9. Bagaimana penggunaan RUP?
10. Sebutkan metode pengembangan perangkat lunak selain RUP, sebutkan masing-masing kelebihan dan kekurangannya!
11. Kapan sebaiknya kita menerapkan RUP?
12. Apa keterkaitan antara RUP dengan metode OMT dan OOSE? Jelaskan!

8

Pemodelan dan UML



Overview

Bab ini membahas tentang kompleksitas pengembangan perangkat lunak dan perlunya dilakukan pemodelan untuk membantu. Salah satu pemodelan yang saat ini paling banyak digunakan adalah UML. UML (*Unified Modeling Language*) adalah salah standar bahasa yang banyak digunakan di dunia industri untuk mendefinisikan *requirement*, membuat analisis & desain, serta menggambarkan arsitektur dalam pemrograman berorientasi objek. Isi bab ini meliputi:

1. Kompleksitas perangkat lunak
2. Pemodelan
3. Pengenalan tentang UML
4. Sejarah UML
5. Diagram UML
6. Penjelasan singkat masing-masing diagram UML

8.1 Kompleksitas Pengembangan Perangkat Lunak

Mengelola pengembangan perangkat lunak bukanlah hal yang mudah. Secara logika sama dengan mengelola banyak kepala yang memiliki tingkat pemahaman dan pemikiran yang berbeda untuk membuat sebuah benda. Semakin banyak kepala yang harus disatukan maka semakin sulit mengelolanya.

Kompleksitas sebuah perangkat lunak dapat dilihat dari hal-hal berikut:

- Kompleksitas domain atau permasalahan perangkat lunak
Pendefinisian fungsi-fungsi pada perangkat dan pendefinisian penanganan kasus-kasus yang mungkin di dalam sebuah fungsi bukanlah hal yang mudah. Sering permasalahan yang belum didefinisikan pada spesifikasi muncul begitu sebuah perangkat lunak sudah masuk ke tahap implementasi atau pengkodean. Hal seperti ini dapat menyebabkan proses kembali ke tahap analisis atau sering langsung diputuskan pada saat implementasi tanpa memperbaiki dokumen analisis perangkat lunak, secara konsep hal ini dapat menyebabkan ketidakkonsistenan antara dokumen dan perangkat lunak. Belum lagi permasalahan perbedaan interpretasi pemahaman spesifikasi oleh orang yang mengembangkan aplikasi yang terkadang tidak ditanyakan, tapi membuat asumsi sendiri.
- Kesulitan mengelola proses pengembangan perangkat lunak
Pengembangan perangkat lunak biasa dilakukan secara tim. Oleh karena itu diperlukan koordinasi yang cukup tinggi diantara anggota tim. Jika koordinasi kurang maka kesalahpahaman interpretasi akan banyak terjadi sehingga bisa jadi perangkat lunak yang dibangun tidak pernah selesai.
- Kemungkinan fleksibilitas perubahan perangkat lunak
Sering kasus pada dunia nyata bahwa tim yang mengerjakan bukanlah orang yang memiliki kepentingan terhadap aplikasi.

Sehingga tercipta hubungan klien dan developer dimana klien akan melakukan permintaan aplikasi dengan fungsi-fungsi yang dia butuhkan dan *developer* berkewajiban membuat aplikasi yang diminta oleh klien. Semakin terbatasnya pengetahuan klien mengenai teknologi informasi dan semakin banyaknya klien yang perlu dimintai pertimbangan mengenai kebutuhan aplikasi maka akan semakin tinggi kemungkinan perubahan spesifikasi di tengah proses pengembangan jika tidak ada perjanjian spesifikasi yang mengikat. Hal inilah yang sering menyebabkan pengembangan aplikasi mengalami kemoloran waktu.

- Permasalahan karakteristik bagian-bagian perangkat lunak secara diskrit

Dalam pembangunan perangkat lunak sering dilakukan dengan beberapa orang atau tim. Maka perangkat lunak akan dibagi-bagi menjadi bagian-bagian yang harus dikerjakan orang-orang di dalam tim. Jika permasalahan bagian-bagian perangkat lunak secara diskrit tidak terdefinisi dengan benar atau dipahami berbeda oleh setiap orang yang ada di dalam tim, maka kemungkinan saat bagian-bagian ini digabungkan akan terjadi banyak kesalahan atau *error*. Maka dari itu koordinasi dan komunikasi yang baik di dalam sebuah tim sangat dibutuhkan.

Karena berbagai masalah dan risiko yang mungkin timbul di dalam pengembangan perangkat lunak maka perlu adanya perencanaan dan pemodelan perangkat lunak.

8.2 Pemodelan

Pemodelan adalah gambaran dari realita yang simpel dan dituangkan dalam bentuk pemetaan dengan aturan tertentu. Pemodelan dapat menggunakan bentuk yang sama dengan realitas misalnya jika seorang arsitek ingin memodelkan sebuah gedung yang akan dibangun maka dia akan memodelkannya dengan membuat sebuah

maket (tiruan) arsitektur gedung yang akan dibangun di mana maket itu akan dibuat semirip mungkin dengan desain gedung yang akan dibangun agar arsitektur gedung yang diinginkan dapat terlihat. Seperti yang kita ketahui bahwa manusia akan lebih memahami suatu hal dengan menggunakan visual agar sekelompok manusia yang berkepentingan dapat mengerti bagaimanakah ide yang akan dikerjakan. Pemodelan juga banyak digunakan untuk merencanakan suatu hal agar kegagalan dan risiko yang mungkin terjadi dapat diminimalisasi.

Pada dunia pembangunan perangkat lunak sistem informasi juga diperlukan pemodelan. Pemodelan perangkat lunak digunakan untuk mempermudah langkah berikutnya dari pengembangan sebuah sistem informasi sehingga lebih terencana. Seperti halnya maket, pemodelan pada pembangunan perangkat lunak digunakan untuk memvisualkan perangkat lunak yang akan dibuat.

Pemodelan perangkat lunak memiliki beberapa abstraksi, misalnya sebagai berikut:

- petunjuk yang terfokus pada proses yang dimiliki oleh sistem
- spesifikasi struktur secara abstrak dari sebuah sistem (belum detail)
- spesifikasi lengkap dari sebuah sistem yang sudah final
- spesifikasi umum atau khusus sistem
- bagian penuh atau parsial dari sebuah sistem

Perangkat pemodelan adalah suatu model yang digunakan untuk menguraikan sistem menjadi bagian-bagian yang dapat diatur dan mengomunikasikan ciri konseptual dan fungsional kepada pengamat. Peran perangkat pemodelan:

- Komunikasi
Perangkat pemodelan dapat digunakan sebagai alat komunikasi antara pemakai dengan analis sistem maupun *developer* dalam pengembangan sistem.
- Eksperimentasi
Pengembangan sistem yang bersifat "*trial and error*".

- **Prediksi**
Model meramalkan bagaimana suatu sistem akan bekerja

Salah satu perangkat pemodelan adalah *Unified Modeling Language* (UML).

8.3 Pengenalan UML

Pada perkembangan teknologi perangkat lunak, diperlukan adanya bahasa yang digunakan untuk memodelkan perangkat lunak yang akan dibuat dan perlu adanya standarisasi agar orang di berbagai negara dapat mengerti pemodelan perangkat lunak. Seperti yang kita ketahui bahwa menyatukan banyak kepala untuk menceritakan sebuah ide dengan tujuan untuk memahami hal yang sama tidaklah mudah, oleh karena itu diperlukan sebuah bahasa pemodelan perangkat lunak yang dapat dimengerti oleh banyak orang.

Banyak orang yang telah membuat bahasa pemodelan pembangunan perangkat lunak sesuai dengan teknologi pemrograman yang berkembang pada saat itu, misalnya yang sempat berkembang dan digunakan oleh banyak pihak adalah *Data Flow Diagram* (DFD) untuk memodelkan perangkat lunak yang menggunakan pemrograman prosedural atau struktural, kemudian juga ada *State Transition Diagram* (STD) yang digunakan untuk memodelkan sistem *real time* (waktu nyata).

Pada perkembangan teknik pemrograman berorientasi objek, muncullah sebuah standarisasi bahasa pemodelan untuk pembangunan perangkat lunak yang dibangun dengan menggunakan teknik pemrograman berorientasi objek, yaitu *Unified Modeling Language* (UML). UML muncul karena adanya kebutuhan pemodelan visual untuk menspesifikasikan, menggambarkan, membangun, dan dokumentasi dari sistem perangkat lunak. UML merupakan bahasa visual untuk pemodelan dan komunikasi mengenai sebuah sistem dengan menggunakan diagram dan teks-teks pendukung.

UML hanya berfungsi untuk melakukan pemodelan. Jadi penggunaan UML tidak terbatas pada metodologi tertentu, meskipun pada kenyataannya UML paling banyak digunakan pada metodologi berorientasi objek.

Seperti yang kita ketahui bahwa banyak hal di dunia sistem informasi yang tidak dapat dibakukan, semua tergantung kebutuhan, lingkungan dan konteksnya. Begitu juga dengan perkembangan penggunaan UML bergantung pada level abstraksi penggunaannya. Jadi belum tentu pandangan yang berbeda dalam penggunaan UML adalah suatu yang salah, tapi perlu ditelaah dimanakah UML digunakan dan hal apa yang ingin divisualkan. Secara analogi jika dengan bahasa yang kita gunakan sehari-hari, belum tentu penyampaian bahasa dengan puisi adalah hal yang salah. Sistem informasi bukanlah ilmu pasti, maka jika ada banyak perbedaan dan interpretasi di dalam bidang sistem informasi merupakan hal yang sangat wajar.

8.4 Sejarah UML

Bahasa pemrograman berorientasi objek yang pertama dikembangkan dikenal dengan nama Simula-67 yang dikembangkan pada tahun 1967. Bahasa pemrograman ini kurang berkembang dan dikembangkan lebih lanjut, namun dengan kemunculannya telah memberikan sumbangan yang besar pada developer pengembang bahasa pemrograman berorientasi objek selanjutnya.

Perkembangan aktif dari pemrograman berorientasi objek mulai menggeliat ketika berkembangnya bahasa pemrograman Smalltalk pada awal 1980-an yang kemudian diikuti dengan perkembangan bahasa pemrograman berorientasi objek yang lainnya seperti C objek, C++, Eiffel, dan CLOS. Secara aktual, penggunaan bahasa pemrograman berorientasi objek pada saat itu masih terbatas, namun telah banyak menarik perhatian di saat itu. Sekitar lima tahun setelah Smalltalk berkembang, maka berkembang pula metode pengembangan berorientasi objek. Metode yang pertama diperkenalkan

oleh Sally Shlaer dan Stephen Mellor (Shlaer-Mellor, 1988) dan Peter Coad dan Edward Yourdon (Coad-Yourdon, 1991), diikuti oleh Grady Booch (Booch, 1991), James R. Rumbaugh, Michael R. Blaha, William Lorensen, Frederick Eddy, William Premerlani (Rumbaugh-Blaha-Premerlani-Eddy-Lorensen, 1991), dan masih banyak lagi.

Buku terkenal yang juga berkembang selanjutnya adalah karangan Ivar Jacobson (Jacobson, 1992) yang menerangkan perbedaan pendekatan yang fokus pada *use case* dan proses pengembangan. Sekitar lima tahun kemudian muncul buku yang membahas mengenai metodologi berorientasi objek yang diikuti dengan buku-buku yang lainnya. Di dalamnya juga membahas mengenai konsep, definisi, notasi, terminologi, dan proses mengenai metodologi berorientasi objek.

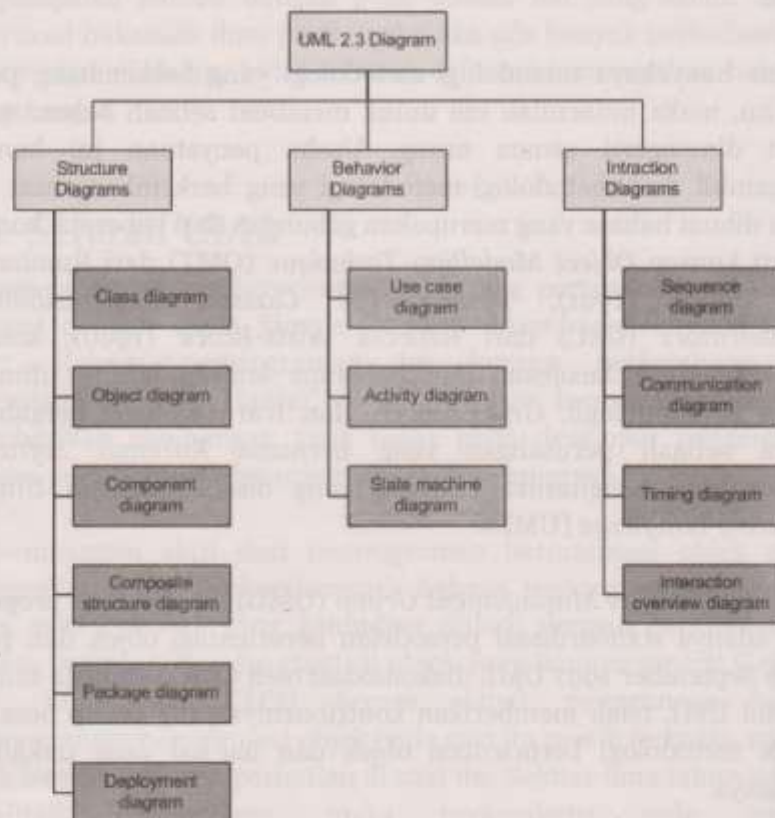
Karena banyaknya metodologi-metodologi yang berkembang pesat saat itu, maka muncullah ide untuk membuat sebuah bahasa yang dapat dimengerti semua orang. Usaha penyatuan ini banyak mengambil dari metodologi-metodologi yang berkembang saat itu. Maka dibuat bahasa yang merupakan gabungan dari beberapa konsep seperti konsep *Object Modelling Technique* (OMT) dari Rumbaugh dan Booch (1991), konsep *The Classes, Responsibilities, Collaborators* (CRC) dari Rebecca Wirfs-Brock (1990), konsep pemikiran Ivar Jacobson, dan beberapa konsep lainnya dimana James R. Rumbaugh, Grady Booch, dan Ivar Jacobson bergabung dalam sebuah perusahaan yang bernama *Rational Software Corporation* menghasilkan bahasa yang disebut dengan *Unified Modeling Language* (UML).

Pada 1996, *Object Management Group* (OMG) mengajukan proposal agar adanya standardisasi pemodelan berorientasi objek dan pada bulan September 1997 UML diakomodasi oleh OMG sehingga sampai saat ini UML telah memberikan kontribusinya yang cukup besar di dalam metodologi berorientasi objek dan hal-hal yang terkait di dalamnya.

Secara fisik, UML adalah sekumpulan spesifikasi yang dikeluarkan oleh OMG. UML terbaru adalah UML 2.3 yang terdiri dari 4 macam spesifikasi, yaitu *Diagram Interchange Specification*, UML Infrastructure, UML Superstructure, dan Object Constraint Language (OCL). Seluruh spesifikasi tersebut dapat diakses di website <http://www.omg.org>.

8.5 Diagram UML

Pada UML 2.3 terdiri dari 13 macam diagram yang dikelompokkan dalam 3 kategori. Pembagian kategori dan macam-macam diagram tersebut dapat dilihat pada gambar di bawah.



Gambar 42 Diagram UML

Berikut ini penjelasan singkat dari pembagian kategori tersebut.

- *Structure diagrams* yaitu kumpulan diagram yang digunakan untuk menggambarkan suatu struktur statis dari sistem yang dimodelkan.
- *Behavior diagrams* yaitu kumpulan diagram yang digunakan untuk menggambarkan kelakuan sistem atau rangkaian perubahan yang terjadi pada sebuah sistem.
- *Interaction diagrams* yaitu kumpulan diagram yang digunakan untuk menggambarkan interaksi sistem dengan sistem lain maupun interaksi antar subsistem pada suatu sistem.

Penjelasan tentang masing-masing diagram akan dilakukan pada sub bab-sub bab berikutnya.



Perbedaan simbol UML dapat terjadi karena perbedaan perangkat lunak/CASE tools yang digunakan. CASE tools untuk membuat diagram-diagram UML misalnya:

- StarUML (*open source*)
- Argo UML
- PoseidonCE
- Rational Rose
- Visual Paradigm (*shareware*)

8.6 Class Diagram

Diagram kelas atau *class diagram* menggambarkan struktur sistem dari segi pendefinisian kelas-kelas yang akan dibuat untuk membangun sistem. Kelas memiliki apa yang disebut atribut dan metode atau operasi.

- atribut merupakan variabel-variabel yang dimiliki oleh suatu kelas

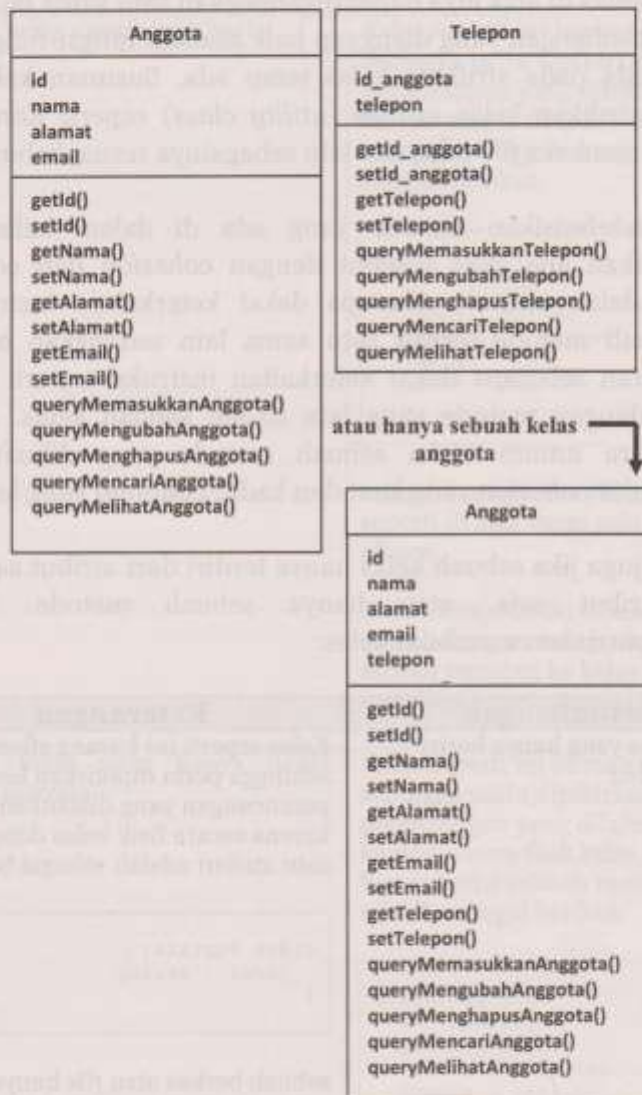
- operasi atau metode adalah fungsi-fungsi yang dimiliki oleh suatu kelas

Diagram kelas dibuat agar pembuat program atau *programmer* membuat kelas-kelas sesuai rancangan di dalam diagram kelas agar antara dokumentasi perancangan dan perangkat lunak sinkron. Banyak berbagai kasus, perancangan kelas yang dibuat tidak sesuai dengan kelas-kelas yang dibuat pada perangkat lunak, sehingga tidaklah ada gunanya lagi sebuah perancangan karena apa yang dirancang dan hasil jadinya tidak sesuai.

Kelas-kelas yang ada pada struktur sistem harus dapat melakukan fungsi-fungsi sesuai dengan kebutuhan sistem sehingga pembuat perangkat lunak atau *programmer* dapat membuat kelas-kelas di dalam program perangkat lunak sesuai dengan perancangan diagram kelas. Susunan struktur kelas yang baik pada diagram kelas sebaiknya memiliki jenis-jenis kelas berikut:

- Kelas main
Kelas yang memiliki fungsi awal dieksekusi ketika sistem dijalankan.
- Kelas yang menangani tampilan sistem (*view*)
Kelas yang mendefinisikan dan mengatur tampilan ke pemakai.
- Kelas yang diambil dari pendefinisian *use case* (*controller*)
Kelas yang menangani fungsi-fungsi yang harus ada diambil dari pendefinisian *use case*, kelas ini biasanya disebut dengan kelas proses yang menangani proses bisnis pada perangkat lunak.
- Kelas yang diambil dari pendefinisian data (*model*)
Kelas yang digunakan untuk memegang atau membungkus data menjadi sebuah kesatuan yang diambil maupun akan disimpan ke basis data. Semua tabel yang dibuat di basis data dapat dijadikan kelas, namun untuk tabel dari hasil relasi atau atribut multivalued pada ERD dapat dijadikan kelas tersendiri dapat juga tidak asalkan pengaksesannya dapat dipertanggungjawabkan atau tetap ada di dalam perancangan

kelas. Misalkan dalam tabel TTelepon dan TAnggota pada studi kasus maka perancangan kelas dapat mengandung kelas Telepon dan Anggota atau perancangan kelas hanya terdiri dari kelas Anggota di mana di dalamnya ada sebuah atribut berupa larik (*array*) bertipe *string* dengan nama telepon. Ilustrasinya dapat dilihat pada gambar berikut:



Gambar 43 Perancangan kelas data untuk tabel dari atribut *multivalue*

Hal terpenting adalah kolom telepon tetap dapat diakses dari perancangan kelas. Kelas data biasanya adalah kelas yang terkait dengan pengaksesan tabel pada basis data sesuai dengan nama kelasnya, misalnya kelas Anggota maka akan digunakan untuk mengakses tabel yang menyimpan data anggota.

Jenis-jenis kelas di atas juga dapat digabungkan satu sama lain sesuai dengan pertimbangan yang dianggap baik asalkan fungsi-fungsi yang sebaiknya ada pada struktur kelas tetap ada. Susunan kelas juga dapat ditambahkan kelas utilitas (*utility class*) seperti Koneksi ke basis data, membaca *file* teks, dan lain sebagainya sesuai kebutuhan.

Dalam mendefinisikan metode yang ada di dalam kelas perlu memperhatikan apa yang disebut dengan *cohesion* dan *coupling*. *Cohesion* adalah ukuran seberapa dekat keterkaitan instruksi di dalam sebuah metode terkait satu sama lain sedangkan *coupling* adalah ukuran seberapa dekat keterkaitan instruksi antara metode yang satu dengan metode yang lain dalam sebuah kelas. Sebagai aturan secara umum maka sebuah metode yang dibuat harus memiliki kadar *cohesion* yang kuat dan kadar *coupling* yang lemah.

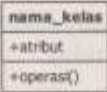
Perhatikan juga jika sebuah kelas hanya terdiri dari atribut saja, atau sebuah atribut saja, atau hanya sebuah metode. Berikut pertimbangan dalam membuat kelas:

Pertimbangan	Keterangan
sebuah kelas yang hanya berisi sebuah atribut	Kelas seperti ini kurang efisien sehingga perlu dipikirkan kembali perancangan yang dilakukan, karena secara fisik kelas dengan satu atribut adalah sebagai berikut: <pre>class Pustaka{ judul : string }</pre> sebuah berkas atau <i>file</i> hanya berisi seperti di atas tentu saja tidak efisien.

Pertimbangan	Keterangan
	Cara memperbaiki rancangan adalah dengan menggabungkan satu atribut tersebut ke kelas lain yang memiliki keterkaitan lebih dekat.
sebuah kelas yang hanya berisi atribut	Kelas seperti ini kurang efisien sehingga perlu dipikirkan kembali perancangan yang dilakukan, karena secara fisik kelas dengan hanya berisi atribut saja adalah sebagai berikut: <pre data-bbox="691 800 1073 1010">class Pustaka{ id : string judul : string penerbit : string jumlah : int tahun : int }</pre> sebuah berkas atau <i>file</i> hanya berisi seperti di atas tentu saja tidak efisien. Cara memperbaiki rancangan adalah dengan menggabungkan atribut tersebut ke kelas lain yang memiliki keterkaitan lebih dekat.
sebuah kelas yang hanya berisi sebuah metode	Kelas seperti ini kurang efisien sehingga perlu dipikirkan kembali perancangan yang dilakukan, karena secara fisik kelas dengan hanya berisi sebuah metode saja adalah sebagai berikut: <pre data-bbox="691 1524 1073 1734">class Pustaka{ id : string judul : string penerbit : string jumlah : int tahun : int }</pre>

Pertimbangan	Keterangan
	<pre> procedure queryMelihatPustaka() { query = "SELECT" execute(query) } </pre> sebuah berkas atau <i>file</i> hanya berisi seperti di atas tentu saja tidak efisien. Cara memperbaiki rancangan adalah dengan menggabungkan atribut dan metode tersebut ke kelas lain yang memiliki keterkaitan lebih dekat.

Berikut adalah simbol-simbol yang ada pada diagram kelas:

Simbol	Deskripsi
kelas 	kelas pada struktur sistem
antarmuka / <i>interface</i> 	sama dengan konsep <i>interface</i> dalam pemrograman berorientasi objek
asosiasi / <i>association</i> 	relasi antarkelas dengan makna umum, asosiasi biasanya juga disertai dengan <i>multiplicity</i>
asosiasi berarah / <i>directed association</i> 	relasi antarkelas dengan makna kelas yang satu digunakan oleh kelas yang lain, asosiasi biasanya juga disertai dengan <i>multiplicity</i>
generalisasi 	relasi antarkelas dengan makna generalisasi-spesialisasi (umum khusus)
kebergantungan / <i>dependency</i>	relasi antarkelas dengan makna

Simbol	Deskripsi
	kebergantungan antarkelas
agregasi / <i>aggregation</i> 	relasi antarkelas dengan makna semua-bagian (<i>whole-part</i>)



Arah panah relasi pada diagram kelas mengarah pada diagram kelas yang lebih besar kontrolnya atau yang dipakai.

8.7 Object Diagram

Diagram objek menggambarkan struktur sistem dari segi penamaan objek dan jalannya objek dalam sistem. Pada diagram objek harus dipastikan semua kelas yang sudah didefinisikan pada diagram kelas harus dipakai objeknya, karena jika tidak, pendefinisian kelas itu tidak dapat dipertanggungjawabkan. Diagram objek juga berfungsi untuk mendefinisikan contoh nilai atau isi dari atribut tiap kelas.

Untuk apa mendefinisikan sebuah kelas sedangkan pada jalannya sistem, objeknya tidak pernah dipakai. Hubungan *link* pada diagram objek merupakan hubungan memakai dan dipakai dimana dua buah objek akan dihubungkan oleh *link* jika ada objek yang dipakai oleh objek lainnya.

Berikut adalah simbol-simbol yang ada pada diagram objek:

Simbol	Deskripsi
Objek	objek dari kelas yang berjalan saat sistem dijalankan

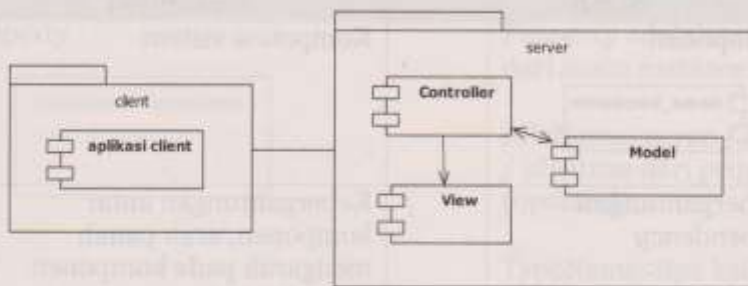
Simbol	Deskripsi
nama_objek : nama_kelas atribut = nilai	
Link 	relasi antar objek

Penulisan nilai sama dengan penulisan pada kamus data DFD. Misalkan jika sebuah atribut dapat berisi lebih dari satu string maka akan ditulis dengan lambang {"nilai1", "nilai2", ..., "nilain"}, atau misalkan ingin menuliskan bahwa nilai sebuah atribut string dapat diisi nilai1 atau nilai2 maka akan ditulis ["nilai1" | "nilai2"].

8.8 Component Diagram

Diagram komponen atau *component diagram* dibuat untuk menunjukkan organisasi dan ketergantungan diantara kumpulan komponen dalam sebuah sistem. Diagram komponen fokus pada komponen sistem yang dibutuhkan dan ada di dalam sistem. Diagram komponen juga dapat digunakan untuk memodelkan hal-hal berikut:

- *source code* program perangkat lunak
- komponen *executable* yang dilepas ke *user*
- basis data secara fisik
- sistem yang harus beradaptasi dengan sistem lain
- *framework* sistem, *framework* pada perangkat lunak merupakan kerangka kerja yang dibuat untuk memudahkan pengembangan dan pemeliharaan aplikasi, contohnya seperti Struts dari Apache yang menggunakan prinsip desain Model-View-Controller (MVC) dimana *source code* program dikelompokkan berdasarkan fungsinya seperti pada gambar berikut:



Gambar 44 Ilustrasi Framework

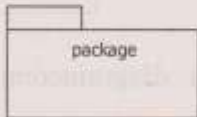
Di mana *controller* berisi *source code* yang menangani *request* dan validasi, *model* berisi *source code* yang menangani manipulasi data dan *business logic*, dan *view* berisi *source code* yang menangani tampilan.

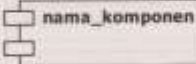
Komponen dasar yang biasanya ada dalam suatu sistem adalah sebagai berikut:

- Komponen *user interface* yang menangani tampilan
- Komponen *bussiness procesiing* yang menangani fungsi-fungsi proses bisnis
- Komponen *data* yang menangani manipulasi data
- Komponen *security* yang menangani keamanan sistem

Komponen lebih terfokus pada penggolongan secara umum fungsi-fungsi yang diperlukan.

Berikut adalah simbol-simbol yang ada pada diagram komponen:

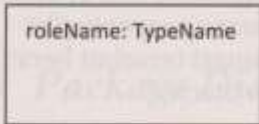
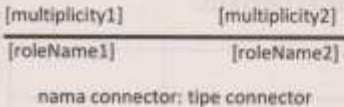
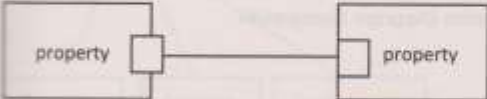
Simbol	Deskripsi
Package 	<i>package</i> merupakan sebuah bungkus dari satu atau lebih komponen

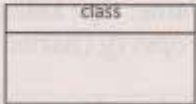
Simbol	Deskripsi
Komponen  nama_komponen	Komponen sistem
Kebergantungan <i>dependency</i> 	Kebergantungan antar komponen, arah panah mengarah pada komponen yang dipakai
Antarmuka / <i>interface</i>  nama_interface	sama dengan konsep <i>interface</i> pada pemrograman berorientasi objek, yaitu sebagai antarmuka komponen agar tidak mengakses langsung komponen
Link 	relasi antar komponen

8.9 Composite Structure Diagram

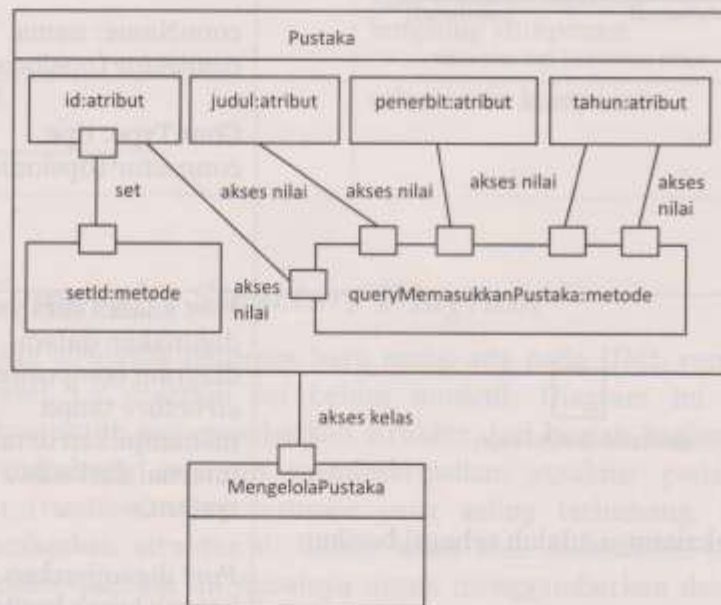
Composite structure diagram baru mulai ada pada UML versi 2.0. Pada versi 1.x diagram ini belum muncul. Diagram ini dapat digunakan untuk menggambarkan struktur dari bagian-bagian yang saling terhubung maupun mendeskripsikan struktur pada saat berjalan (*runtime*) dari *instance* yang saling terhubung. Dapat menggambarkan struktur di dalam kelas atau kolaborasi. Contoh penggunaan diagram ini misalnya untuk menggambarkan deskripsi dari setiap bagian mesin yang saling terkait untuk menjalankan fungsi mesin tersebut, menggambarkan aliran data *router* pada jaringan computer, dan lain-lain.

Berikut adalah simbol-simbol yang ada pada *diagram composite structure*:

Simbol	Deskripsi
Property 	<i>Property</i> adalah satu set dari suatu <i>instance</i>. roleName: peran / nama / identitas dari <i>property</i> (opsional) TypeName: tipe kelas dari <i>property</i> (harus ada)
Connector 	<i>Connector</i> adalah cara komunikasi dari 2 buah <i>instance</i>. connName: nama <i>connector</i> (opsional) ConnType: tipe <i>connector</i> (opsional)
Port  portName:EntityName[n] pemakaiannya adalah sebagai berikut: 	<i>Port</i> adalah cara yang digunakan dalam diagram <i>composite structure</i> tanpa menampilkan detail internal dari suatu system. <i>Port</i> digambarkan dalam bentuk kotak kecil yang menempel atau di dalam suatu <i>property</i>. <i>Port</i> digambarkan menempel <i>property</i> jika fungsi tersebut dapat diakses <i>public</i>.

Simbol	Deskripsi
	Sedangkan <i>port</i> digambarkan di dalam suatu <i>property</i> jika fungsi tersebut bersifat <i>protected</i> .
class 	Kelas; jika yang akan dijabarkan strukturnya adalah sebuah kelas

Contoh diagram *composite* adalah sebagai berikut:



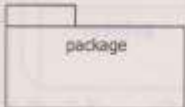
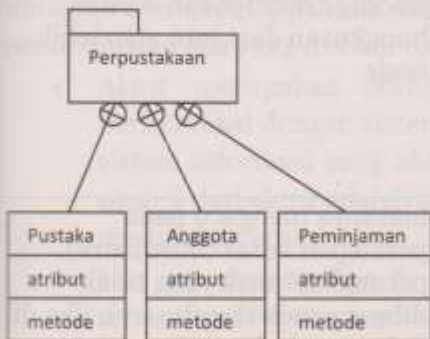
Gambar 45 Contoh Diagram *Composite*

Maksud dari diagram di atas adalah bahwa atribut id, judul, penerbit dan tahun diakses nilainya oleh metode `queryMemasukkanPustaka` dan atribut id diakses pada metode `setId` dengan memberikan nilai

kepada atribut id. Kelas Pustaka diakses oleh kelas MengelolaPustaka.

8.10 Package Diagram

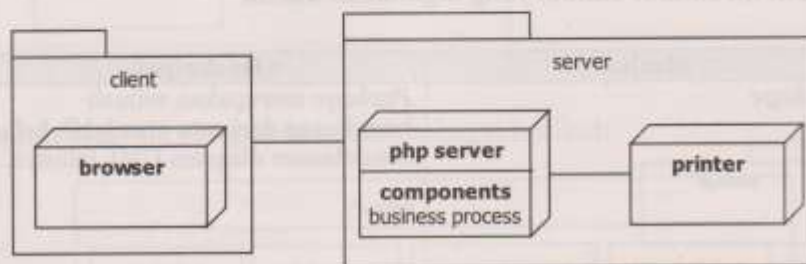
Package diagram menyediakan cara mengumpulkan elemen-elemen yang saling terkait dalam diagram UML. Hampir semua diagram dalam UML dapat dikelompokkan menggunakan *packagediagram*. Berikut ini simbol-simbol yang digunakan dalam

Simbol	Deskripsi
<i>Package</i> 	<i>Package</i> merupakan sebuah bungkusan dari satu atau lebih kelas atau elemen diagram UML lainnya.
Elemen dalam <i>package</i> digambarkan di dalam <i>package</i> 	
Elemen dalam <i>package</i> digambarkan di luar <i>package</i> 	

8.11 Deployment Diagram

Diagram *deployment* atau *deployment diagram* menunjukkan konfigurasi komponen dalam proses eksekusi aplikasi. Diagram *deployment* juga dapat digunakan untuk memodelkan hal-hal berikut:

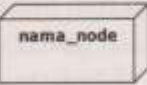
- sistem tambahan (*embedded system*) yang menggambarkan rancangan *device*, *node*, dan *hardware*.
- sistem *client/server* misalnya seperti gambar berikut:

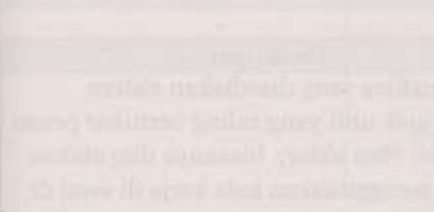
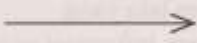


Gambar 46 Diagram Deployment Sistem Client / Server

- sistem terdistribusi murni
- rekayasa ulang aplikasi

Berikut adalah simbol-simbol yang ada pada diagram *deployment*:

Simbol	Deskripsi
Package 	<i>package</i> merupakan sebuah bungkusan dari satu atau lebih <i>node</i>
Node 	biasanya mengacu pada perangkat keras (<i>hardware</i>), perangkat lunak yang tidak dibuat sendiri (<i>software</i>), jika ada dalam <i>node</i> disertakan

Simbol	Deskripsi
	komponen untuk mengkonsistenkan rancangan maka komponen yang diikutsertakan harus sesuai dengan komponen yang telah didefinisikan sebelumnya pada diagram komponen
Kebergantungan / <i>dependency</i> 	Kebergantungan antar <i>node</i> , arah panah mengarah pada <i>node</i> yang dipakai
Link 	relasi antar <i>node</i>

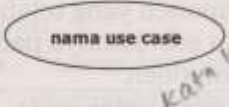
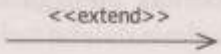
8.12 Use Case Diagram

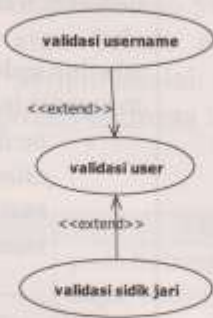
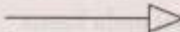
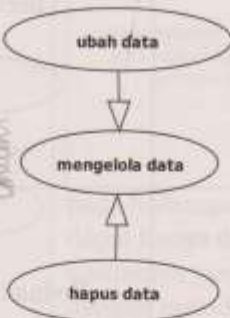
Use case atau diagram *use case* merupakan pemodelan untuk melakukan (*behavior*) sistem informasi yang akan dibuat. *Use case* mendeskripsikan sebuah interaksi antara satu atau lebih aktor dengan sistem informasi yang akan dibuat. Secara kasar, *use case* digunakan untuk mengetahui fungsi apa saja yang ada di dalam sebuah sistem informasi dan siapa saja yang berhak menggunakan fungsi-fungsi itu.

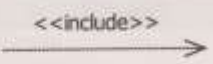
Syarat penamaan pada *use case* adalah nama didefinisikan sesimpel mungkin dan dapat dipahami. Ada dua hal utama pada *use case* yaitu pendefinisian apa yang disebut aktor dan *use case*.

- Aktor merupakan orang, proses, atau sistem lain yang berinteraksi dengan sistem informasi yang akan dibuat di luar sistem informasi yang akan dibuat itu sendiri, jadi walaupun simbol dari aktor adalah gambar orang, tapi aktor belum tentu merupakan orang.
- *Use case* merupakan fungsionalitas yang disediakan sistem sebagai unit-unit yang saling bertukar pesan antar unit atau aktor.

Berikut adalah simbol-simbol yang ada pada diagram *use case*:

Simbol	Deskripsi
<i>Use case</i> 	fungsi yang disediakan sistem sebagai unit-unit yang saling bertukar pesan antar unit atau aktor; biasanya dinyatakan dengan menggunakan kata kerja di awal di awal frase nama <i>use case</i>
Aktor / actor 	orang, proses, atau sistem lain yang berinteraksi dengan sistem informasi yang akan dibuat di luar sistem informasi yang akan dibuat itu sendiri, jadi walaupun simbol dari aktor adalah gambar orang, tapi aktor belum tentu merupakan orang; biasanya dinyatakan menggunakan kata benda di awal frase nama aktor
Asosiasi / association 	komunikasi antara aktor dan <i>use case</i> yang berpartisipasi pada <i>use case</i> atau <i>use case</i> memiliki interaksi dengan aktor
Ekstensi / extend 	relasi <i>use case</i> tambahan ke sebuah <i>use case</i> dimana <i>use case</i> yang ditambahkan dapat berdiri sendiri walau tanpa <i>use case</i> tambahan itu; mirip dengan prinsip <i>inheritance</i> pada pemrograman berorientasi objek; biasanya <i>use case</i> tambahan memiliki nama depan yang sama dengan <i>use case</i> yang ditambahkan, misal

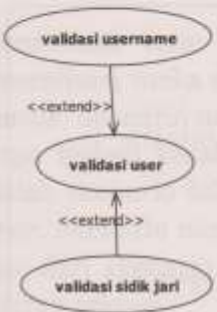
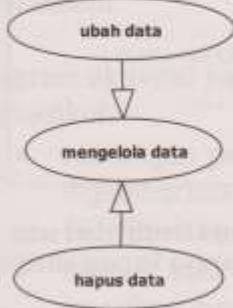
Simbol	Deskripsi
	 arah panah mengarah pada <i>use case</i> yang ditambahkan; biasanya <i>use case</i> yang menjadi <i>extend</i>-nya merupakan jenis yang sama dengan <i>use case</i> yang menjadi induknya
Generalisasi / <i>generalization</i> 	Hubungan generalisasi dan spesialisasi (umum - khusus) antara dua buah <i>use case</i> dimana fungsi yang satu adalah fungsi yang lebih umum dari lainnya, misalnya:  arah panah mengarah pada <i>use case</i> yang menjadi generalisasinya (umum)
Menggunakan / <i>include</i> / <i>uses</i>	relasi <i>use case</i> tambahan ke sebuah <i>use case</i> di mana <i>use case</i> yang ditambahkan memerlukan <i>use case</i> ini untuk menjalankan fungsinya atau sebagai syarat

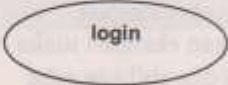
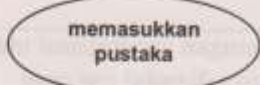
Simbol	Deskripsi
	dijalankan <i>use case</i> ini ada dua sudut pandang yang cukup besar mengenai include di <i>use case</i>: <i>include</i> berarti <i>use case</i> yang ditambahkan akan selalu dipanggil saat <i>use case</i> tambahan dijalankan, misal pada kasus berikut:  <i>include</i> berarti <i>use case</i> yang tambahan akan selalu melakukan pengecekan apakah <i>use case</i> yang ditambahkan telah dijalankan sebelum <i>use case</i> tambahan dijalankan, misal pada kasus berikut:  Kedua interpretasi di atas dapat dianut salah satu atau keduanya tergantung pada pertimbangan dan interpretasi yang dibutuhkan.



Arah panah relasi pada *use case* mengarah pada *use case* yang lebih besar kontrolnya atau yang dipakai.

Use casenantinya akan menjadi kelas proses pada diagram kelas sehingga perlu dipertimbangkan penamaan yang dilakukan apakah sudah layak menjadi kelas atau belum sesuai dengan aturan pendefinisian kelas yang baik. Berikut adalah aturan perubahan *use case* yang layak menjadi kelas proses sehingga layak sebagai dijadikan *use case*:

Hubungan	Keterangan
Ekstensi / <i>extend</i>  <pre> graph TD A([validasi username]) -- "<<extend>>" --> B([validasi user]) B -- "<<extend>>" --> C([validasi sidik jari]) </pre>	pada hubungan ekstensi maka dapat hanya diambil <i>use case</i> induknya yang dijadikan kelas dengan metode berupa <i>use case</i> ekstensinya <pre> class ValidasiUser{ //atribut procedure validasiUsername() { //proses } procedure validasiSidikJari() { //proses } } </pre>
Generalisasi / <i>generalization</i>  <pre> graph TD A([ubah data]) -- "generalization" --> B([mengelola data]) C([hapus data]) -- "generalization" --> B </pre>	pada hubungan generalisasi maka dapat hanya diambil <i>use case</i> umumnya yang dijadikan kelas dengan metode berupa <i>use case</i> khususnya <pre> class MengelolaData{ //atribut procedure ubahData() { //proses } } </pre>

Hubungan	Keterangan
	<pre> procedure hapusData(){ //proses } </pre>
<i>use case</i> yang berdiri sendiri 	metode yang mungkin bisa ada di dalam kelas proses login adalah sebagai berikut: <pre> class Login{ //atribut procedure login(){ //proses } procedure logout(){ //proses } } </pre>
<i>use case</i> yang kurang tepat sebagai sebuah <i>use case</i> yang berdiri sendiri 	<pre> class MemasukkanPustaka{ //atribut procedure memasukkanPustaka(){ //proses } } </pre> kelas yang hanya terdiri dari satu metode sebenarnya kurang efisien

Setiap *use case* dilengkapi dengan skenario. Skenario *use case* adalah alur jalannya proses *use case* dari sisi aktor dan sistem. Berikut adalah format tabel skenario *use case*:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
Skenario Alternatif	

Skenario *use case* dibuat per *use case* terkecil, misalkan untuk generalisasi maka skenario yang dibuat adalah *use case* yang lebih khusus. Skenario normal adalah skenario bila sistem berjalan normal tanpa terjadi kesalahan atau *error*. Sedangkan skenario alternatif adalah skenario bila sistem tidak berjalan normal, atau mengalami *error*. Skenario normal dan skenario alternatif dapat lebih dari satu. Alur dari skenario inilah yang nantinya menjadi dasar pembuatan diagram sekuen.

8.13 Activity Diagram

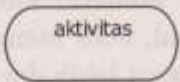
Diagram aktivitas atau *activity diagram* menggambarkan *workflow* (aliran kerja) atau aktivitas dari sebuah sistem atau proses bisnis atau menu yang ada pada perangkat lunak. Yang perlu diperhatikan disini adalah bahwa diagram aktivitas menggambarkan aktivitas sistem bukan apa yang dilakukan aktor, jadi aktivitas yang dapat dilakukan oleh sistem.

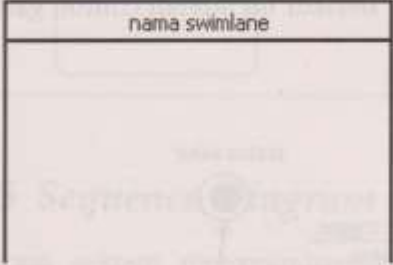
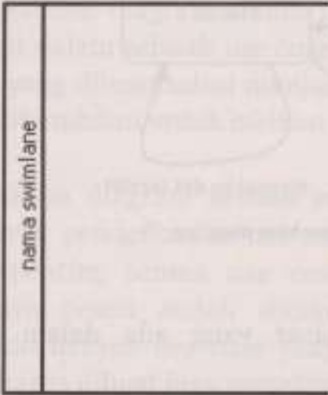
Diagram aktivitas juga banyak digunakan untuk mendefinisikan hal-hal berikut:

- rancangan proses bisnis dimana setiap urutan aktivitas yang digambarkan merupakan proses bisnis sistem yang didefinisikan
- urutan atau pengelompokan tampilan dari sistem / *user interface* dimana setiap aktivitas dianggap memiliki sebuah rancangan antarmuka tampilan

- rancangan pengujian dimana setiap aktivitas dianggap memerlukan sebuah pengujian yang perlu didefinisikan kasus ujinya
- rancangan menu yang ditampilkan pada perangkat lunak

Berikut adalah simbol-simbol yang ada pada diagram aktivitas:

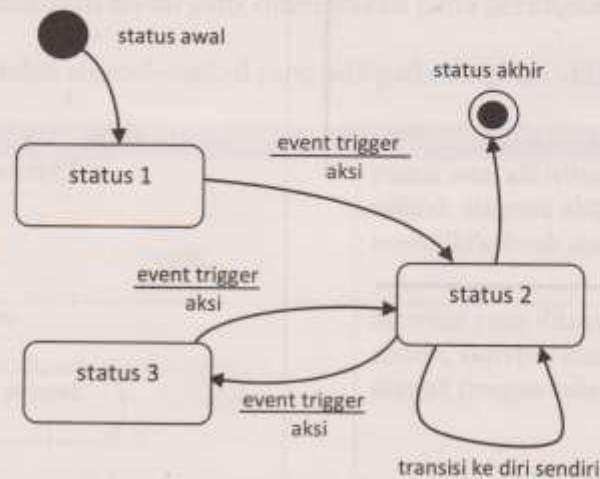
Simbol	Deskripsi
status awal 	status awal aktivitas sistem, sebuah diagram aktivitas memiliki sebuah status awal
aktivitas 	aktivitas yang dilakukan sistem, aktivitas biasanya diawali dengan kata kerja
percabangan / <i>decision</i> 	asosiasi percabangan dimana jika ada pilihan aktivitas lebih dari satu
penggabungan / <i>join</i> 	asosiasi penggabungan dimana lebih dari satu aktivitas digabungkan menjadi satu
status akhir 	status akhir yang dilakukan sistem, sebuah diagram aktivitas memiliki sebuah status akhir
swimlane	memisahkan organisasi bisnis yang bertanggung jawab terhadap aktivitas yang terjadi

Simbol	Deskripsi
 atau 	

8.14 State Machine Diagram

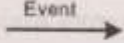
State machine diagram atau *statechart diagram* atau dalam bahasa Indonesia disebut diagram mesin status atau sering juga disebut diagram status digunakan untuk menggambarkan perubahan status atau transisi status dari sebuah mesin atau sistem atau objek. Jika diagram sekuen digunakan untuk interaksi antar objek maka diagram status digunakan untuk interaksi di dalam sebuah objek. Perubahan tersebut digambarkan dalam suatu graf berarah. *State machine diagram* merupakan pengembangan dari diagram *Finite State Automata* dengan penambahan beberapa fitur dan konsep baru. Diagram *Finite State Automata* (FSA) ini biasanya diajarkan dalam mata kuliah Automata.

State machine diagram cocok digunakan untuk menggambarkan alur interaksi pengguna dengan sistem. Berikut ini adalah contoh gambar diagram mesin status.



Gambar 47 Contoh *state machine diagram*

Berikut ini komponen-komponen dasar yang ada dalam *state machine diagram*:

Simbol	Deskripsi
Start / Status Awal (<i>Initial State</i>) 	<i>Start</i> atau <i>initial state</i> adalah <i>state</i> atau keadaan awal pada saat sistem mulai hidup.
End / Status Akhir (<i>Final State</i>) 	<i>End</i> atau <i>final state</i> adalah <i>state</i> keadaan akhir dari daur hidup suatu sistem.
Event 	<i>Event</i> adalah kegiatan yang menyebabkan berubahnya status mesin.
State	<i>State</i> atau status adalah keadaan

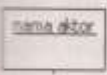
Simbol	Deskripsi
	sistem pada waktu tertentu. <i>State</i> dapat berubah jika ada <i>event</i> tertentu yang memicu perubahan tersebut.

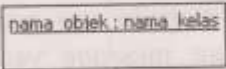
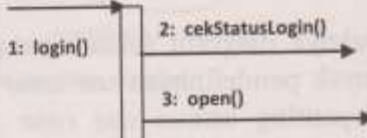
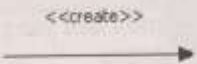
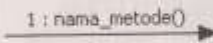
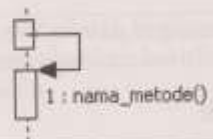
8.15 Sequence Diagram

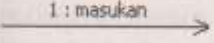
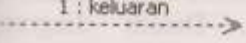
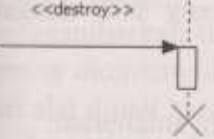
Diagram sekuen menggambarkan kelakuan objek pada *use case* dengan mendeskripsikan waktu hidup objek dan *message* yang dikirimkan dan diterima antar objek. Oleh karena itu untuk menggambar diagram sekuen maka harus diketahui objek-objek yang terlibat dalam sebuah *use case* beserta metode-metode yang dimiliki kelas yang diinstansiasi menjadi objek itu. Membuat diagram sekuen juga dibutuhkan untuk melihat skenario yang ada pada *use case*.

Banyaknya diagram sekuen yang harus digambar adalah minimal sebanyak pendefinisian *use case* yang memiliki proses sendiri atau yang penting semua *use case* yang telah didefinisikan interaksi jalannya pesan sudah dicakup pada diagram sekuen sehingga semakin banyak *use case* yang didefinisikan maka diagram sekuen yang harus dibuat juga semakin banyak.

Berikut adalah simbol-simbol yang ada pada diagram sekuen:

Simbol	Deskripsi
Aktor  atau  tanpa waktu aktif	orang, proses, atau sistem lain yang berinteraksi dengan sistem informasi yang akan dibuat di luar sistem informasi yang akan dibuat itu sendiri, jadi walaupun simbol dari aktor adalah gambar orang, tapi aktor belum tentu merupakan orang; biasanya dinyatakan menggunakan kata benda di awal frase nama aktor

Simbol	Deskripsi
Garis hidup / <i>lifeline</i> 	menyatakan kehidupan suatu objek
Objek 	menyatakan objek yang berinteraksi pesan
Waktu aktif 	menyatakan objek dalam keadaan aktif dan berinteraksi, semua yang terhubung dengan waktu aktif ini adalah sebuah tahapan yang dilakukan di dalamnya, misalnya  maka cekStatusLogin() dan open() dilakukan di dalam metode login() Aktor tidak memiliki waktu aktif
Pesan tipe create 	menyatakan suatu objek membuat objek yang lain, arah panah mengarah pada objek yang dibuat
Pesan tipe call 	menyatakan suatu objek memanggil operasi/metode yang ada pada objek lain atau dirinya sendiri, 

Simbol	Deskripsi
	arah panah mengarah pada objek yang memiliki operasi/metode, karena ini memanggil operasi/metode maka operasi/metode yang dipanggil harus ada pada diagram kelas sesuai dengan kelas objek yang berinteraksi
Pesan tipe send 	menyatakan bahwa suatu objek mengirimkan data/masukan/informasi ke objek lainnya, arah panah mengarah pada objek yang dikirim
Pesan tipe return 	menyatakan bahwa suatu objek yang telah menjalankan suatu operasi atau metode menghasilkan suatu kembalian ke objek tertentu, arah panah mengarah pada objek yang menerima kembalian
Pesan tipe destroy 	menyatakan suatu objek mengakhiri hidup objek yang lain, arah panah mengarah pada objek yang diakhiri, sebaiknya jika ada create maka ada destroy

Penomoran pesan berdasarkan urutan interaksi pesan. Penggambaran letak pesan harus berurutan, pesan yang lebih atas dari lainnya adalah pesan yang berjalan terlebih dahulu.

Semua metode di dalam kelas harus ada di dalam diagram kolaborasi atau sekuen, jika tidak ada berarti perancangan metode di dalam kelas itu kurang baik. Hal ini dikarenakan ada metode yang tidak dapat dipertanggungjawabkan kegunaannya.

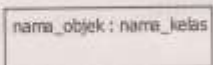
8.16 Communication Diagram

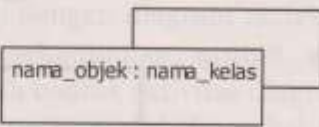
Communication diagram atau diagram komunikasi pada UML versi 2.x adalah penyederhanaan dari diagram kolaborasi (*collaboration diagram*) pada UML versi 1.x. *Collaboration diagram* sudah tidak muncul lagi pada UML versi 2.x. Diagram komunikasi sebenarnya adalah diagram kolaborasi tetapi dibuat untuk tiap sekuen.

Diagram komunikasi menggambarkan interaksi antar objek/bagian dalam bentuk urutan pengiriman pesan. Diagram komunikasi merepresentasikan informasi yang diperoleh dari Diagram Kelas, Diagram Sekuen, dan Diagram *Use Case* untuk mendeskripsikan gabungan antara struktur statis dan tingkah laku dinamis dari suatu sistem.

Diagram komunikasi mengelompokkan *message* pada kumpulan diagram sekuen menjadi sebuah diagram. Dalam diagram komunikasi yang dituliskan adalah operasi/metode yang dijalankan antara objek yang satu dan objek lainnya secara keseluruhan, oleh karena itu dapat diambil dari jalannya interaksi pada semua diagram sekuen. Penomoran metode dapat dilakukan berdasarkan urutan dijalankannya metode/operasi diantara objek yang satu dengan objek lainnya atau objek itu sendiri.

Berikut adalah simbol-simbol yang ada pada diagram kolaborasi:

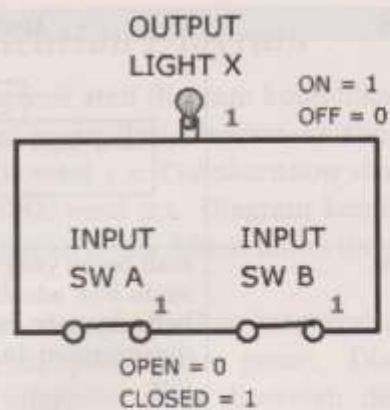
Simbol	Deskripsi
Objek 	objek yang melakukan interaksi pesan
Link 	relasi antar-objek yang menghubungkan objek satu dengan lainnya atau dengan dirinya sendiri

Simbol	Deskripsi
	
Arah pesan / stimulus 	arah pesan yang terjadi, jika pada suatu <i>link</i> ada dua arah pesan yang berbeda maka arah juga digambarkan dua arah pada dua sisi <i>link</i> 

8.17 Timing Diagram

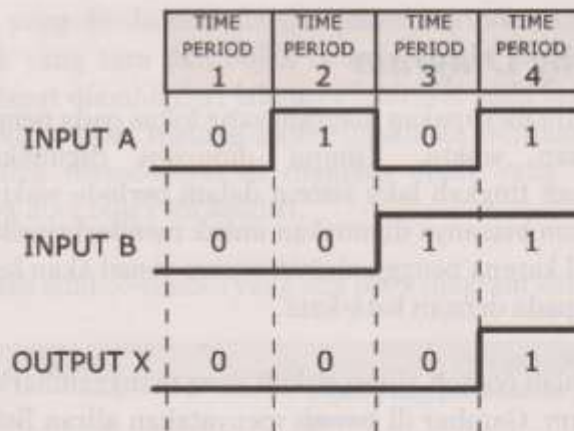
Timing diagram merupakan diagram yang fokus pada penggambaran terkait batasan waktu. *Timing diagram* digunakan untuk menggambarkan tingkah laku sistem dalam periode waktu tertentu. *Timing diagram* biasanya digunakan untuk mendeskripsikan operasi dari alat digital karena penggambaran secara visual akan lebih mudah dipahami daripada dengan kata-kata.

Berikut ini adalah contoh aliran sirkuit yang menggambarkan operasi *timing diagram*. Gambar di bawah menyatakan aliran listrik. Status OPEN = 0 artinya *switch* dalam posisi terbuka (tidak terhubung), sedangkan status CLOSED = 1 adalah posisi *switch* terhubung. Lampu akan menyala, status ON = 1, jika kedua *switch* terhubung.



Gambar 48 Contoh aliran sirkuit

Berikut ini contoh gambar pemanfaatan *timing diagram* dalam menggambarkan seluruh status dalam gambar aliran sirkuit di atas.



Gambar 49 Contoh *timing diagram*

Aliran waktu pada *timing diagram* dibaca dari kiri ke kanan. Pada gambar *timing diagram* di atas dapat terlihat bahwa lampu akan menyala jika kedua *switch* atau INPUT A dan INPUT B dalam posisi CLOSED = 1. Jika salah satu *switch* tidak terhubung maka lampu tidak akan menyala.

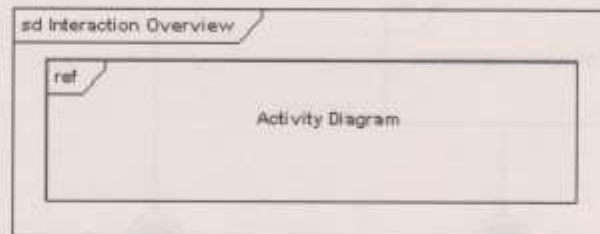
8.18 Interaction Overview Diagram

Interaction overview diagram mirip dengan diagram aktivitas yang berfungsi untuk menggambarkan sekumpulan urutan aktivitas. *Interaction Overview Diagram* adalah bentuk aktivitas diagram yang setiap titik merepresentasikan diagram interaksi. Interaksi diagram dapat meliputi diagram sekuen, diagram komunikasi, *interaction overview diagram*, dan timing diagram.

Hampir semua notasi pada *interaction overview diagram* sama dengan notasi pada diagram aktivitas. Sebagai contoh *initial*, *final*, *decision*, *merge*, *fork*, dan *join nodes* sama seperti pada diagram aktivitas. Tambahan pada *interaction overview diagram* adalah *interaction occurrence* dan *interaction element*.

1. Interaction Occurrence

Interaction occurrence atau kejadian interaksi adalah referensi untuk diagram interaksi yang ada. Sebuah *interaction occurrence* ditunjukkan sebagai *frame* referensi (*frame* dengan tulisan "*ref*" di pojok kiri atas). Nama diagram yang sedang direfensikan ditunjukkan pada tengah *frame*. Berikut adalah gambar contoh dari *interaction occurrence*:



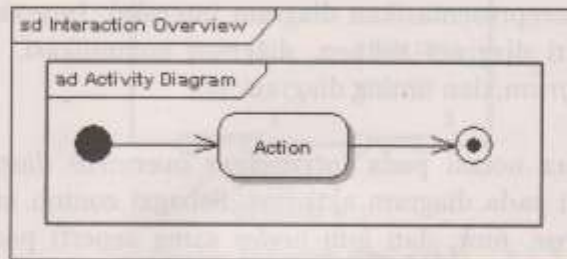
Gambar 50 Contoh *interaction occurrence*

sd di atas kependekan dari *sequence diagram*.

2. Interaction Element

Interaction element atau elemen interaksi mirip *interaction occurrence*. Perbedaannya adalah di dalam *interaction*

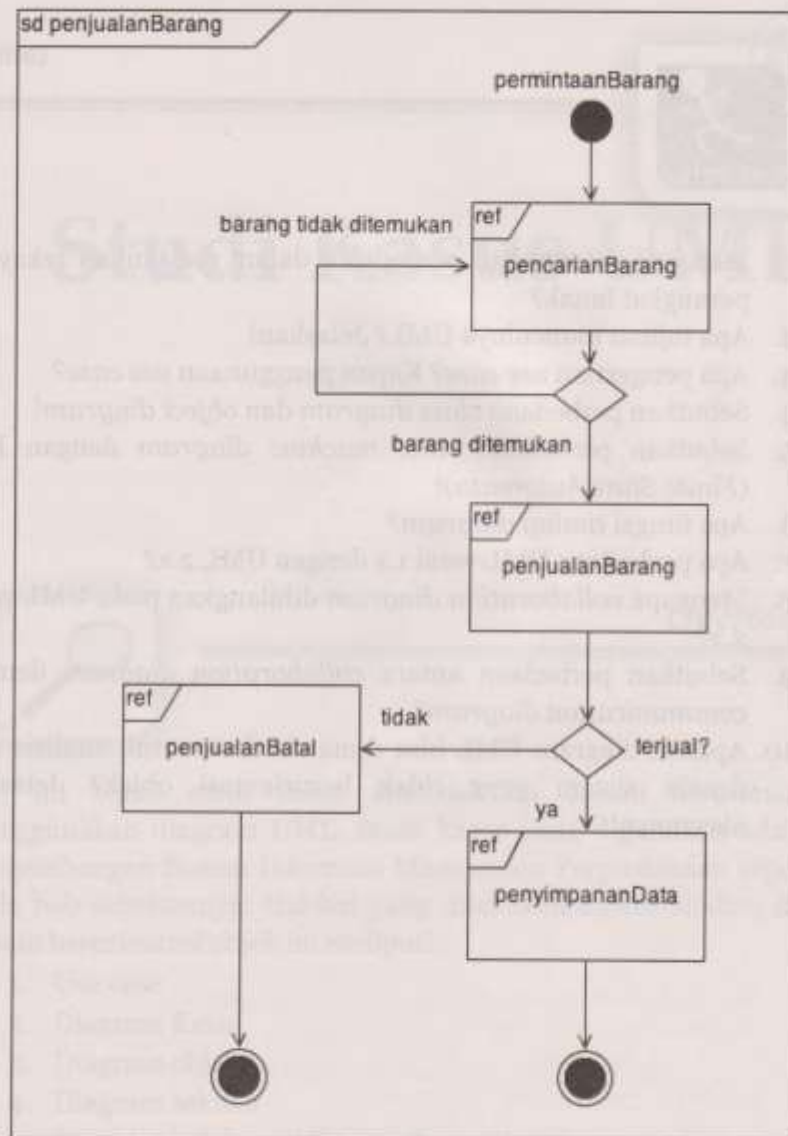
element menampilkan isi diagram yang direferensikan secara langsung, sedangkan *interaction occurrence* hanya menampilkan nama diagram yang direferensikan. Berikut adalah gambar contoh dari *interaction element*:



Gambar 51 Contoh *interaction element*

Berikut ini adalah contoh *interaction overview diagram* yang menampilkan kontrol-kontrol pada diagram aktivitas (*fork*, *join*, *merge*, dan lain-lain) dengan abstraksi subproses digambarkan menggunakan *interaction occurrence*.





Gambar 52 Contoh *interaction overview diagram*



1. Mengapa dibutuhkan pemodelan dalam melakukan rekayasa perangkat lunak?
2. Apa tujuan munculnya UML? Jelaskan!
3. Apa pengertian *use case*? Kapan penggunaan *use case*?
4. Sebutkan perbedaan *class diagram* dan *object diagram*!
5. Sebutkan perbedaan *state machine diagram* dengan FSA (*Finite State Automata*)!
6. Apa fungsi *timing diagram*?
7. Apa perbedaan UML versi 1.x dengan UML 2.x?
8. Mengapa *collaboration diagram* dihilangkan pada UML versi 2.x?
9. Sebutkan perbedaan antara *collaboration diagram* dengan *communication diagram*?
10. Apakah diagram UML bisa dimanfaatkan untuk analisis dan desain sistem yang tidak berorientasi objek? Jelaskan alasannya!

9

Studi Kasus UML



Overview

Bab ini berisi studi kasus analisis dan desain berorientasi menggunakan diagram UML. Studi kasus yang digunakan adalah pengembangan Sistem Informasi Manajemen Perpustakaan seperti pada bab sebelumnya. Hal-hal yang dilakukan dalam analisis dan desain berorientasi objek ini meliputi:

1. Use case
2. Diagram Kelas
3. Diagram objek
4. Diagram sekuen
5. Diagram kolaborasi/komunikasi
6. Diagram status
7. Diagram aktivitas
8. Diagram komponen
9. Diagram *deployment*

9.1 Use Case

1. Definisi Aktor

Berikut adalah deskripsi pendefinisian aktor pada sistem informasi manajemen perpustakaan:

No	Aktor	Deskripsi
1.	Petugas perpustakaan	orang yang bertugas dan memiliki hak akses untuk melakukan operasi pengelolaan data pustaka, anggota, dan proses peminjaman pustaka
2.	Anggota/pengunjung perpustakaan	anggota adalah orang yang diperbolehkan meminjam pustaka sesuai dengan hak aksesnya, sedangkan pengunjung hanya memiliki hak akses melihat pustaka dan membaca di perpustakaan tanpa memiliki hak untuk meminjam pustaka.

2. Definisi Use case

Berikut adalah deskripsi pendefinisian *use case* pada sistem informasi manajemen perpustakaan:

No	Use case	Deskripsi
1.	Validasi	merupakan proses pengecekan hak akses siapa yang berhak mengakses proses pengelolaan data perpustakaan yang dalam kasus ini ada petugas perpustakaan login wajib untuk fungsi-fungsi yang berkaitan dengan akses perubahan ke basis data, oleh karena itu fungsi-fungsi yang melakukan perubahan basis data harus mengecek validasi <i>user</i> yang mengakses fungsi-fungsi ini

No	Use case	Deskripsi
		validasi merupakan generalisasi dari proses <i>login</i> , <i>logout</i> , dan memeriksa status login
2.	Login	merupakan proses untuk melakukan login petugas perpustakaan
3.	Logout	merupakan proses untuk melakukan <i>logout</i> petugas perpustakaan
4.	Memeriksa Status Login	merupakan proses untuk memeriksa apakah pengguna sistem informasi sudah melakukan login atau belum
5.	Mengelola pustaka	mengelola pustaka merupakan proses generalisasi yang meliputi lima buah proses pengelolaan data pustaka yaitu memasukkan pustaka, mengubah pustaka, menghapus pustaka, mencari pustaka, dan melihat pustaka.
6.	Memasukkan pustaka	merupakan proses memasukkan data pustaka ke dalam basis data
7.	Mengubah pustaka	merupakan proses mengubah data pustaka yang ada di basis data
8.	Menghapus pustaka	merupakan proses menghapus data pustaka yang ada di basis data
9.	Mencari pustaka	merupakan proses mencari data pustaka yang ada di basis data
10.	Melihat pustaka	merupakan proses menampilkan data pustaka yang ada di basis data
11.	Mengelola anggota	mengelola data anggota merupakan proses generalisasi yang meliputi lima buah proses pengelolaan data

No	Use case	Deskripsi
		anggota yaitu memasukkan anggota, mengubah anggota, menghapus anggota, mencari anggota, dan melihat anggota
12.	Memasukkan anggota	merupakan proses memasukkan data anggota ke dalam basis data
13.	Mengubah anggota	merupakan proses mengubah data anggota yang ada di basis data
14.	Menghapus anggota	merupakan proses menghapus data anggota yang ada di basis data
15.	Mencari anggota	merupakan proses mencari data anggota yang ada di basis data
16.	Melihat anggota	merupakan proses menampilkan data anggota yang ada di basis data
17.	Mengelola peminjaman	mengelola data peminjaman merupakan proses generalisasi yang meliputi lima buah proses pengelolaan data peminjaman yaitu memasukkan peminjaman, mengubah peminjaman, menghapus peminjaman, mencari peminjaman, dan melihat peminjaman
18.	Memasukkan peminjaman	merupakan proses memasukkan data peminjaman ketika ada anggota yang meminjam pustaka
19.	Mengubah peminjaman	merupakan proses mengubah data peminjaman yang dapat dilakukan untuk mengubah status peminjaman begitu pustaka dikembalikan
20.	Menghapus peminjaman	merupakan proses menghapus data peminjaman jika ternyata data sudah terlalu banyak, atau peminjaman

No	Use case	Deskripsi
		tidak jadi dilakukan
21.	Mencari peminjaman	merupakan proses mencari data peminjaman
22.	Melihat peminjaman	merupakan proses melihat data peminjaman
23.	Mengelola petugas	mengelola data petugas merupakan proses generalisasi yang meliputi lima buah proses pengelolaan data petugas yaitu memasukkan petugas, mengubah petugas, menghapus petugas, mencari petugas, dan melihat petugas
24.	Memasukkan petugas	merupakan proses memasukkan data petugas ke dalam basis data
25.	Mengubah petugas	merupakan proses mengubah data petugas yang ada di basis data
26.	Menghapus petugas	merupakan proses menghapus data petugas yang ada di basis data
27.	Mencari petugas	merupakan proses mencari data petugas yang ada di basis data
28.	Melihat petugas	merupakan proses menampilkan data petugas yang ada di basis data

3. Skenario Use Case

Berikut adalah skenario jalannya masing-masing *use case* yang telah didefinisikan sebelumnya:

Nama *Use case*: Login

Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
1. Memasukkan <i>username</i> dan <i>password</i>	
	2. Memeriksa valid tidaknya data masukan dengan memeriksa ke tabel <i>petugas</i>
	3. Masuk ke aplikasi pengelolaan data perpustakaan
Skenario Alternatif	
1. Memasukkan <i>username</i> dan <i>password</i>	
	2. Memeriksa valid tidaknya data masukan
	3. Menampilkan pesan <i>login tidak valid</i>
4. Memasukkan <i>username</i> dan <i>password</i> yang valid	
	5. Memeriksa <i>valid</i> tidaknya data masukan
	6. Masuk ke aplikasi pengelolaan data perpustakaan

Nama *Use case*: Logout

Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
1. Memilih menu <i>logout</i>	
	2. Melakukan <i>logout</i>

Nama *Use case*: Memeriksa Status Login

Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	

Aksi Aktor	Reaksi Sistem
	1. Memeriksa ke variabel <i>session</i> sebagai penanda <i>login</i> apakah petugas sudah login
	2. Mengembalikan status <i>login</i> , sudah login atau belum

Nama *Use case*: Memasukkan pustaka

Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status <i>login</i>
2. Memasukkan data pustaka sesuai kolom yang ada	
	3. Memeriksa <i>valid</i> tidaknya data masukan
	4. Menyimpan data pustaka ke basis data
	5. Menampilkan pesan sukses disimpan
Skenario Alternatif	
	1. Memeriksa status <i>login</i>
2. Memasukkan data pustaka sesuai kolom yang ada	
	3. Memeriksa <i>valid</i> tidaknya data masukan
	4. Mengeluarkan pesan bahwa data masukan tidak <i>valid</i>
5. Memperbaiki data masukan yang tidak	

Aksi Aktor	Reaksi Sistem
<i>valid</i>	
	6. Memeriksa valid tidaknya data masukan
	7. Menyimpan data pustaka ke basis data
	8. Menampilkan pesan sukses disimpan

Nama *Use case*: Mengubah pustaka
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data pustaka yang akan diubah
	4. Menampilkan data pustaka yang dicari (belum semua kolom data pustaka ditampilkan dan bisa banyak data pustaka yang memenuhi data pencarian)
5. Memilih data pustaka yang akan diubah	
	6. Menampilkan semua kolom data pustaka yang akan diubah
7. Mengubah data pustaka	
	8. Memeriksa valid tidaknya data masukan
	9. Menyimpan data yang telah diubah ke basis data
	10. Menampilkan pesan bahwa data sukses disimpan
Skenario Alternatif	

Aksi Aktor	Reaksi Sistem
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data pustaka yang akan diubah
	4. Menampilkan data pustaka yang dicari (belum semua kolom data pustaka ditampilkan dan bisa banyak data pustaka yang memenuhi data pencarian)
5. Memilih data pustaka yang akan diubah	
	6. Menampilkan semua kolom data pustaka yang akan diubah
7. Mengubah data pustaka	
	8. Memeriksa valid tidaknya data masukan
	9. Menampilkan pesan bahwa data masukan tidak valid
10. Memperbaiki data masukan yang diubah dan tidak valid	
	11. Memeriksa valid tidaknya data masukan
	12. Menyimpan data yang telah diubah ke basis data
	13. Menampilkan pesan bahwa data sukses disimpan

Nama Use case: Menghapus pustaka

Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	

Aksi Aktor	Reaksi Sistem
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data pustaka yang akan dihapus
	4. Menampilkan data pustaka yang dicari (belum semua kolom data pustaka ditampilkan dan bisa banyak data pustaka yang memenuhi data pencarian)
5. Memilih data pustaka yang akan dihapus	
	6. Menampilkan pesan konfirmasi apakah data akan benar-benar dihapus
7. Mengeklik pilihan setuju data dihapus	
	8. Menghapus data pustaka dari basis data
	9. Menampilkan pesan bahwa data sukses dihapus
Skenario Alternatif	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data pustaka yang akan dihapus
	4. Menampilkan data pustaka yang dicari (belum semua kolom data pustaka ditampilkan dan bisa banyak data pustaka yang memenuhi data pencarian)
5. Memilih data pustaka	

Aksi Aktor	Reaksi Sistem
yang akan dihapus	
	6. Menampilkan pesan konfirmasi apakah data akan benar-benar dihapus
7. Mengeklik pilihan tidak setuju data dihapus	
	8. Kembali ke form pencarian pustaka

Nama Use case: Mencari pustaka

Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
1. Memasukkan kata kunci dan kategori pencarian	
	2. Mencari data pustaka yang akan dicari
	3. Menampilkan data pustaka yang dicari (belum semua kolom data pustaka ditampilkan dan bisa banyak data pustaka yang memenuhi data pencarian)
4. Memilih pustaka yang dicari	
	5. Menampilkan data pustaka (semua kolom) dari pustaka yang dipilih
Skenario Alternatif	
1. Memasukkan kata kunci dan kategori pencarian	
	2. Mencari data pustaka yang akan dicari
	3. Menampilkan pesan data pustaka tidak ada
4. Memasukkan kata kunci dan kategori pencarian	
	5. Mencari data pustaka

Aksi Aktor	Reaksi Sistem
	yang akan dicari
	6. Menampilkan data pustaka yang dicari (belum semua kolom data pustaka ditampilkan dan bisa banyak data pustaka yang memenuhi data pencarian)
7. Memilih pustaka yang dicari	
	8. Menampilkan data pustaka (semua kolom) dari pustaka yang dipilih

Nama *Use case*: Melihat pustaka
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
	2. Menampilkan data pustaka yang dicari (belum semua kolom data pustaka ditampilkan dan bisa banyak data pustaka yang memenuhi data pencarian)
3. Memilih pustaka yang dicari	
	4. Menampilkan data pustaka (semua kolom) dari pustaka yang dipilih

Nama *Use case*: Memasukkan anggota
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan data anggota sesuai kolom	

Aksi Aktor	Reaksi Sistem
yang ada	
	3. Memeriksa valid tidaknya data masukan
	4. Menyimpan data anggota ke basis data
	5. Menampilkan pesan sukses disimpan
Skenario Alternatif	
	1. Memeriksa status login
2. Memasukkan data anggota sesuai kolom yang ada	
	3. Memeriksa valid tidaknya data masukan
	4. Mengeluarkan pesan bahwa data masukan tidak valid
5. Memperbaiki data masukan yang tidak valid	
	6. Memeriksa valid tidaknya data masukan
	7. Menyimpan data anggota ke basis data
	8. Menampilkan pesan sukses disimpan

Nama Use case: Mengubah anggota

Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data anggota yang akan diubah
	4. Menampilkan data anggota yang dicari (belum semua kolom

Aksi Aktor	Reaksi Sistem
	data anggota ditampilkan dan bisa banyak data anggota yang memenuhi data pencarian)
5. Memilih data anggota yang akan diubah	
	6. Menampilkan semua kolom data anggota yang akan diubah
7. Mengubah data anggota	
	8. Memeriksa valid tidaknya data masukan
	9. Menyimpan data yang telah diubah ke basis data
	10. Menampilkan pesan bahwa data sukses disimpan
Skenario Alternatif	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data anggota yang akan diubah
	4. Menampilkan data anggota yang dicari (belum semua kolom data anggota ditampilkan dan bisa banyak data anggota yang memenuhi data pencarian)
5. Memilih data anggota yang akan diubah	
	6. Menampilkan semua kolom data anggota yang akan diubah
7. Mengubah data anggota	

Aksi Aktor	Reaksi Sistem
	8. Memeriksa valid tidaknya data masukan
	9. Menampilkan pesan bahwa data masukan tidak valid
10. Memperbaiki data masukan yang diubah dan tidak valid	
	11. Memeriksa valid tidaknya data masukan
	12. Menyimpan data yang telah diubah ke basis data
	13. Menampilkan pesan bahwa data sukses disimpan

Nama Use case: Menghapus anggota
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data anggota yang akan dihapus
	4. Menampilkan data anggota yang dicari (belum semua kolom data anggota ditampilkan dan bisa banyak data anggota yang memenuhi data pencarian)
5. Memilih data anggota yang akan dihapus	
	6. Menampilkan pesan konfirmasi apakah data akan benar-benar

Aksi Aktor	Reaksi Sistem
	dihapus
7. Mengklik pilihan setuju data dihapus	
	8. Menghapus data anggota dari basis data
	9. Menampilkan pesan bahwa data sukses dihapus
Skenario Alternatif	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data anggota yang akan dihapus
	4. Menampilkan data anggota yang dicari (belum semua kolom data anggota ditampilkan dan bisa banyak data anggota yang memenuhi data pencarian)
5. Memilih data anggota yang akan dihapus	
	6. Menampilkan pesan konfirmasi apakah data akan benar-benar dihapus
7. Mengklik pilihan tidak setuju data dihapus	
	8. Kembali ke form pencarian anggota

Nama *Use case*: Mencari anggota
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan kata kunci	

Aksi Aktor	Reaksi Sistem
dan kategori pencarian	
	3. Mencari data anggota yang akan dicari
	4. Menampilkan data anggota yang dicari (belum semua kolom data anggota ditampilkan dan bisa banyak data anggota yang memenuhi data pencarian)
5. Memilih anggota yang dicari	
	6. Menampilkan data anggota (semua kolom) dari anggota yang dipilih
Skenario Alternatif	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data anggota yang akan dicari
	4. Menampilkan pesan data anggota tidak ada
5. Memasukkan kata kunci dan kategori pencarian	
	6. Mencari data anggota yang akan dicari
	7. Menampilkan data anggota yang dicari (belum semua kolom data anggota ditampilkan dan bisa banyak data anggota yang memenuhi data pencarian)
8. Memilih anggota yang dicari	
	9. Menampilkan data

Aksi Aktor	Reaksi Sistem
	anggota (semua kolom) dari anggota yang dipilih

Nama *Use case*: Melihat anggota
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
	2. Menampilkan data anggota yang dicari (belum semua kolom data anggota ditampilkan dan bisa banyak data anggota yang memenuhi data pencarian)
3. Memilih anggota yang dicari	
	4. Menampilkan data anggota (semua kolom) dari anggota yang dipilih

Nama *Use case*: Memasukkan peminjaman
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan data peminjaman sesuai kolom yang ada	
	3. Memeriksa valid tidaknya data masukan
	4. Menyimpan data peminjaman ke basis data
	5. Menampilkan pesan sukses disimpan
Skenario Alternatif	
	1. Memeriksa status login

Aksi Aktor	Reaksi Sistem
2. Memasukkan data peminjaman sesuai kolom yang ada	
	3. Memeriksa valid tidaknya data masukan
	4. Mengeluarkan pesan bahwa data masukan tidak valid
5. Memperbaiki data masukan yang tidak valid	
	6. Memeriksa valid tidaknya data masukan
	7. Menyimpan data peminjaman ke basis data
	8. Menampilkan pesan sukses disimpan

Nama *Use case*: Mengubah peminjaman
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data peminjaman yang akan diubah
	4. Menampilkan data peminjaman yang dicari (belum semua kolom data peminjaman ditampilkan dan bisa banyak data peminjaman yang memenuhi data pencarian)
5. Memilih data peminjaman yang akan	

Aksi Aktor	Reaksi Sistem
diubah	
	6. Menampilkan semua kolom data peminjaman yang akan diubah
7. Mengubah data peminjaman	
	8. Memeriksa valid tidaknya data masukan
	9. Menyimpan data yang telah diubah ke basis data
	10. Menampilkan pesan bahwa data sukses disimpan
Skenario Alternatif	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data peminjaman yang akan diubah
	4. Menampilkan data peminjaman yang dicari
5. Mengubah data peminjaman	
	6. Memeriksa valid tidaknya data masukan
	7. Menampilkan pesan bahwa data masukan tidak valid
8. Memperbaiki data masukan yang diubah dan tidak valid	
	9. Memeriksa valid tidaknya data masukan
	10. Menyimpan data yang telah diubah ke basis data
	11. Menampilkan pesan

Aksi Aktor	Reaksi Sistem
	bahwa data sukses disimpan

Nama *Use case*: Menghapus peminjaman
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data peminjaman yang akan dihapus
	4. Menampilkan data peminjaman yang dicari (belum semua kolom data peminjaman ditampilkan dan bisa banyak data peminjaman yang memenuhi data pencarian)
5. Memilih data peminjaman yang akan dihapus	
	6. Menampilkan pesan konfirmasi apakah data akan benar-benar dihapus
7. Mengeklik pilihan setuju data dihapus	
	8. Menghapus data peminjaman dari basis data
	9. Menampilkan pesan bahwa data sukses dihapus
Skenario Alternatif	
	1. Memeriksa status login
2. Memasukkan kata kunci	

Aksi Aktor	Reaksi Sistem
dan kategori pencarian	
	3. Mencari data peminjaman yang akan dihapus
	4. Menampilkan data peminjaman yang dicari (belum semua kolom data peminjaman ditampilkan dan bisa banyak data peminjaman yang memenuhi data pencarian)
5. Memilih data peminjaman yang akan dihapus	
	6. Menampilkan pesan konfirmasi apakah data akan benar-benar dihapus
7. Mengeklik pilihan tidak setuju data dihapus	
	8. Kembali ke form pencarian peminjaman

Nama *Use case*: Mencari peminjaman

Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data peminjaman yang akan dicari
	4. Menampilkan data peminjaman yang dicari (belum semua kolom data peminjaman ditampilkan dan bisa

Aksi Aktor	Reaksi Sistem
	banyak data peminjaman yang memenuhi data pencarian)
5. Memilih peminjaman yang dicari	
	6. Menampilkan data peminjaman (semua kolom) dari peminjaman yang dipilih
Skenario Alternatif	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data peminjaman yang akan dicari
	4. Menampilkan pesan data peminjaman tidak ada
5. Memasukkan kata kunci dan kategori pencarian	
	6. Mencari data peminjaman yang akan dicari
	7. Menampilkan data peminjaman yang dicari (belum semua kolom data peminjaman ditampilkan dan bisa banyak data peminjaman yang memenuhi data pencarian)
8. Memilih peminjaman yang dicari	
	9. Menampilkan data peminjaman (semua kolom) dari peminjaman yang dipilih

Nama Use case: Melihat peminjaman

Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
	2. Menampilkan data peminjaman yang dicari (belum semua kolom data peminjaman ditampilkan dan bisa banyak data peminjaman yang memenuhi data pencarian)
3. Memilih peminjaman yang dicari	
	4. Menampilkan data peminjaman (semua kolom) dari peminjaman yang dipilih

Nama Use case: Memasukkan petugas

Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan data petugas sesuai kolom yang ada	
	3. Memeriksa valid tidaknya data masukan
	4. Menyimpan data petugas ke basis data
	5. Menampilkan pesan sukses disimpan
Skenario Alternatif	
	1. Memeriksa status login
2. Memasukkan data petugas sesuai kolom yang ada	
	3. Memeriksa valid tidaknya data masukan

Aksi Aktor	Reaksi Sistem
	4. Mengeluarkan pesan bahwa data masukan tidak valid
5. Memperbaiki data masukan yang tidak valid	
	6. Memeriksa valid tidaknya data masukan
	7. Menyimpan data petugas ke basis data
	8. Menampilkan pesan sukses disimpan

Nama Use case: Mengubah petugas
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data petugas yang akan diubah
	4. Menampilkan data petugas yang dicari (belum semua kolom data petugas ditampilkan dan bisa banyak data petugas yang memenuhi data pencarian)
5. Memilih data petugas yang akan diubah	
	6. Menampilkan semua kolom data petugas yang akan diubah
7. Mengubah data petugas	
	8. Memeriksa valid tidaknya data masukan
	9. Menyimpan data yang telah diubah ke basis

Aksi Aktor	Reaksi Sistem
	data
	10. Menampilkan pesan bahwa data sukses disimpan
Skenario Alternatif	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data petugas yang akan diubah
	4. Menampilkan data petugas yang dicari (belum semua kolom data petugas ditampilkan dan bisa banyak data petugas yang memenuhi data pencarian)
5. Memilih data petugas yang akan diubah	
	6. Menampilkan semua kolom data petugas yang akan diubah
7. Mengubah data petugas	
	8. Memeriksa valid tidaknya data masukan
	9. Menampilkan pesan bahwa data masukan tidak valid
10. Memperbaiki data masukan yang diubah dan tidak valid	
	11. Memeriksa valid tidaknya data masukan
	12. Menyimpan data yang telah diubah ke basis data
	13. Menampilkan pesan bahwa data sukses disimpan

Nama Use case: Menghapus anggota

Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data petugas yang akan dihapus
	4. Menampilkan data petugas yang dicari (belum semua kolom data petugas ditampilkan dan bisa banyak data petugas yang memenuhi data pencarian)
5. Memilih data petugas yang akan dihapus	
	6. Menampilkan pesan konfirmasi apakah data akan benar-benar dihapus
7. Mengeklik pilihan setuju data dihapus	
	8. Menghapus data petugas dari basis data
	9. Menampilkan pesan bahwa data sukses dihapus
Skenario Alternatif	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data petugas yang akan dihapus
	4. Menampilkan data petugas yang dicari (belum semua kolom data petugas ditampilkan dan bisa banyak data

Aksi Aktor	Reaksi Sistem
	petugas yang memenuhi data pencarian)
5. Memilih data petugas yang akan dihapus	
	6. Menampilkan pesan konfirmasi apakah data akan benar-benar dihapus
7. Mengklik pilihan tidak setuju data dihapus	
	8. Kembali ke <i>form</i> pencarian petugas

Nama Use case: Mencari petugas
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
2. Memasukkan kata kunci dan kategori pencarian	
	3. Mencari data petugas yang akan dicari
	4. Menampilkan data petugas yang dicari (belum semua kolom data petugas ditampilkan dan bisa banyak data petugas yang memenuhi data pencarian)
5. Memilih petugas yang dicari	
	6. Menampilkan data petugas (semua kolom) dari petugas yang dipilih
Skenario Alternatif	
	7. Memeriksa status login
8. Memasukkan kata kunci dan kategori pencarian	
	9. Mencari data petugas

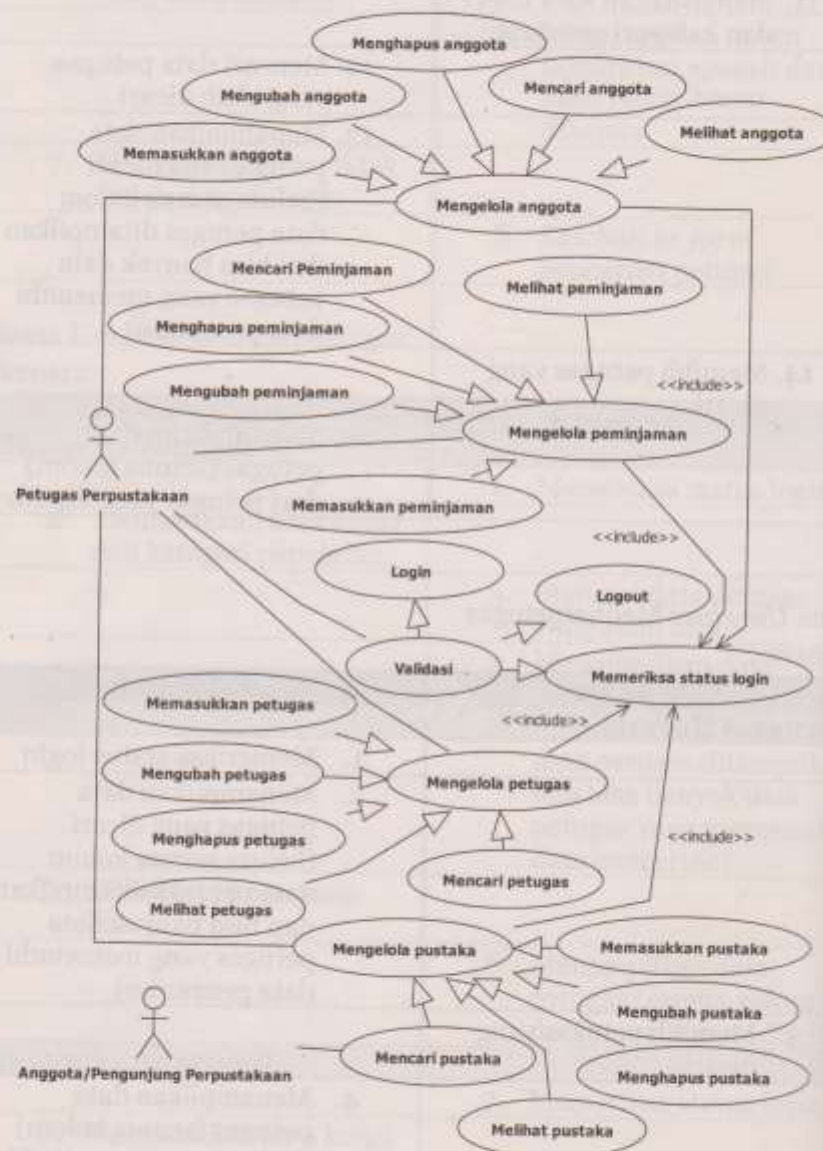
Aksi Aktor	Reaksi Sistem
	yang akan dicari
	10. Menampilkan pesan data petugas tidak ada
11. Memasukkan kata kunci dan kategori pencarian	
	12. Mencari data petugas yang akan dicari
	13. Menampilkan data petugas yang dicari (belum semua kolom data petugas ditampilkan dan bisa banyak data petugas yang memenuhi data pencarian)
14. Memilih petugas yang dicari	
	15. Menampilkan data petugas (semua kolom) dari petugas yang dipilih

Nama *Use case*: Melihatpetugas
Skenario:

Aksi Aktor	Reaksi Sistem
Skenario Normal	
	1. Memeriksa status login
	2. Menampilkan data petugas yang dicari (belum semua kolom data petugas ditampilkan dan bisa banyak data petugas yang memenuhi data pencarian)
3. Memilih petugas yang dicari	
	4. Menampilkan data petugas (semua kolom) dari petugas yang dipilih

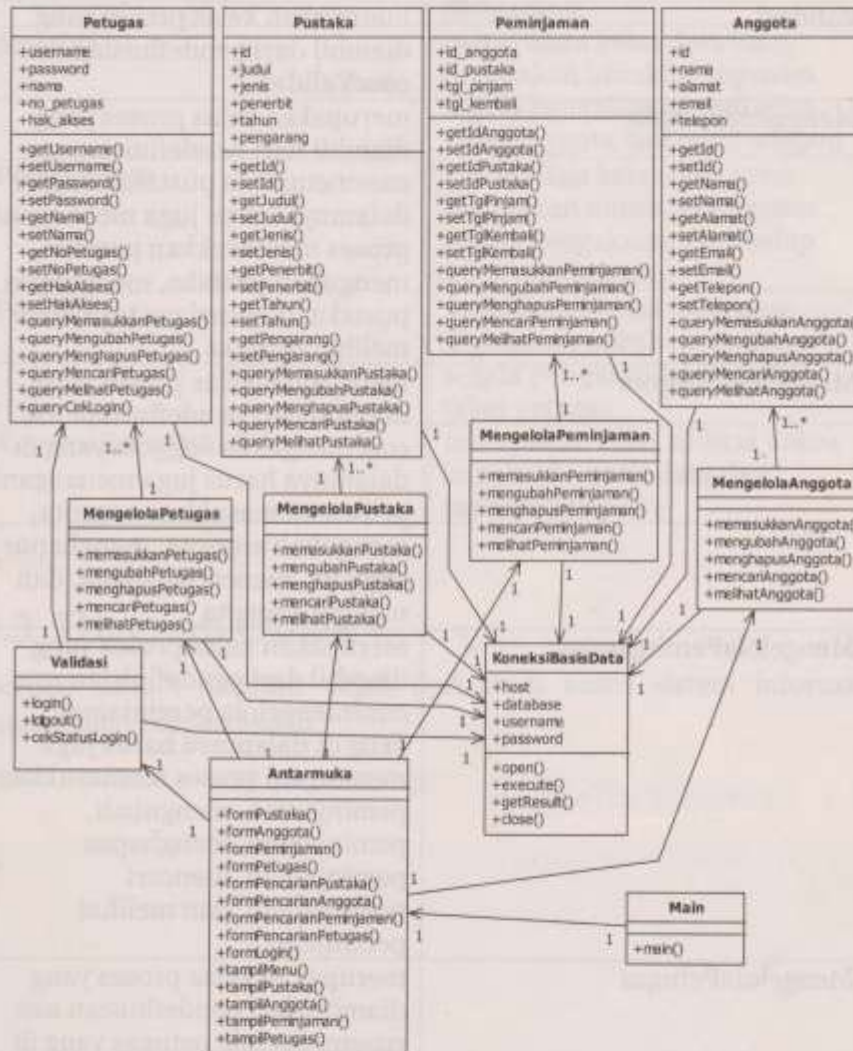
4. Diagram Use case

Berikut adalah diagram *use case* dari sistem informasi manajemen perpustakaan:



Gambar 53 Diagram Use case Perpustakaan

9.2 Diagram Kelas



Gambar 54 Diagram Kelas Studi Kasus

Keterangan:

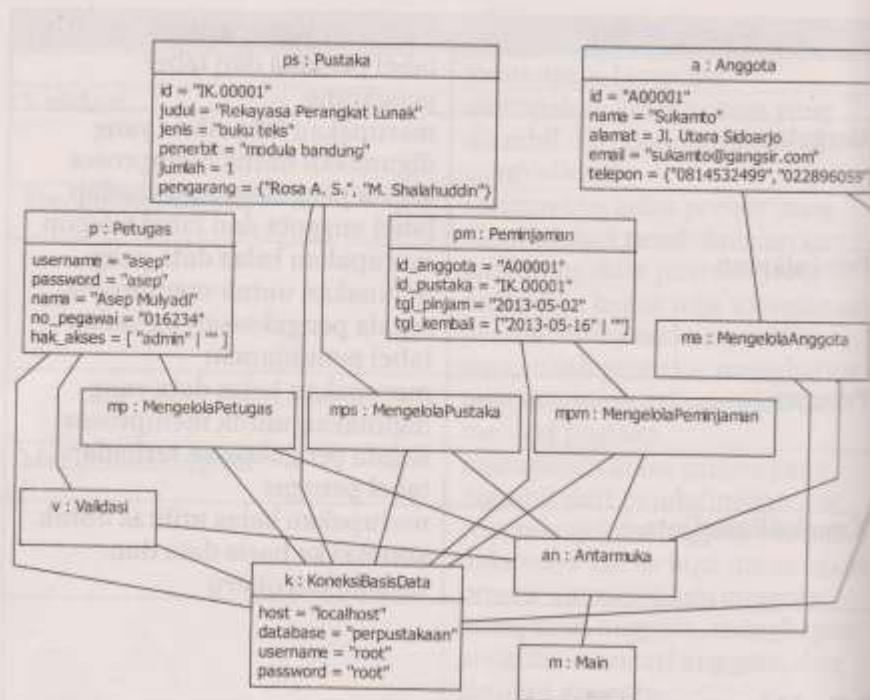
Nama Kelas	Keterangan
Main	merupakan kelas main
Antarmuka	merupakan kelas yang

Nama Kelas	Keterangan
Validasi	menangani tampilan merupakan kelas proses yang diambil dari pendefinisian <i>use case</i> Validasi
MengelolaPustaka	merupakan kelas proses yang diambil dari pendefinisian <i>use case</i> mengelola pustaka yang di dalamnya harus juga menangani proses memasukkan pustaka, mengubah pustaka, menghapus pustaka, mencari pustaka, dan melihat pustaka
MengelolaAnggota	merupakan kelas proses yang diambil dari pendefinisian <i>use case</i> mengelola anggota yang di dalamnya harus juga menangani proses memasukkan anggota, mengubah anggota, menghapus anggota, mencari anggota, dan melihat anggota
MengelolaPeminjaman	merupakan kelas proses yang diambil dari pendefinisian <i>use case</i> mengelola peminjaman yang di dalamnya harus juga menangani proses memasukkan peminjaman, mengubah peminjaman, menghapus peminjaman, mencari peminjaman, dan melihat peminjaman
MengelolaPetugas	merupakan kelas proses yang diambil dari pendefinisian <i>use case</i> mengelola petugas yang di dalamnya harus juga menangani proses memasukkan petugas, mengubah petugas, menghapus petugas, mencari petugas, dan melihat petugas
Pustaka	merupakan kelas data yang digunakan untuk memproses segala pengaksesan terhadap

Nama Kelas	Keterangan
	tabel pustaka dan tabel pengarang
Anggota	merupakan kelas data yang digunakan untuk memproses segala pengaksesan terhadap tabel anggota dan tabel telepon
Peminjaman	merupakan kelas data yang digunakan untuk memproses segala pengaksesan terhadap tabel peminjaman
Petugas	merupakan kelas data yang digunakan untuk memproses segala pengaksesan terhadap tabel petugas
KoneksiBasisData	merupakan kelas utilitas untuk koneksi ke basis data dan melakukan <i>query</i>

9.3 Diagram Objek

Berikut adalah diagram objek dari studi kasus sistem informasi manajemen perpustakaan:

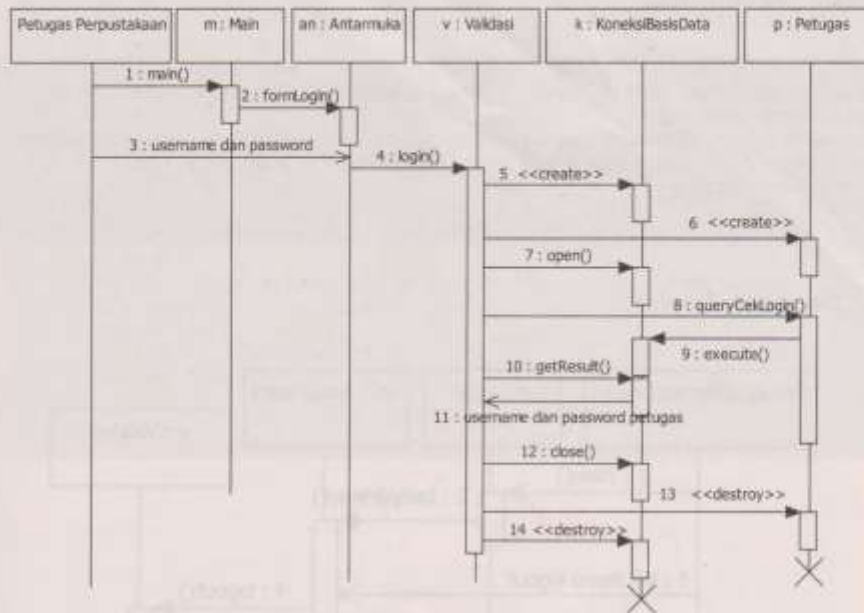


Gambar 55 Diagram Objek Studi Kasus

9.4 Diagram Sekuen

Berikut adalah diagram sekuen dari sistem informasi manajemen perpustakaan:

1. Use case: Login



Gambar 1 Diagram sekuen - Login

Dari diagram sekuen di atas maka urutan proses pada metode *login* di kelas Validasi adalah sebagai berikut (*dalam pseudocode*):

```

procedure login(username: string, password: string){
    KoneksiBasisData k = new KoneksiBasisData // create
    Petugas p = new Petugas // create
    k.open()
    p.queryCekLogin(k, username, password)
    if(k.getResult() <> NULL){
        SESSION[status] = "valid"
    }else{
        SESSION[status] = "invalid"
    }
    k.close()
    delete(p) //destroy
    delete(k) //destroy
}

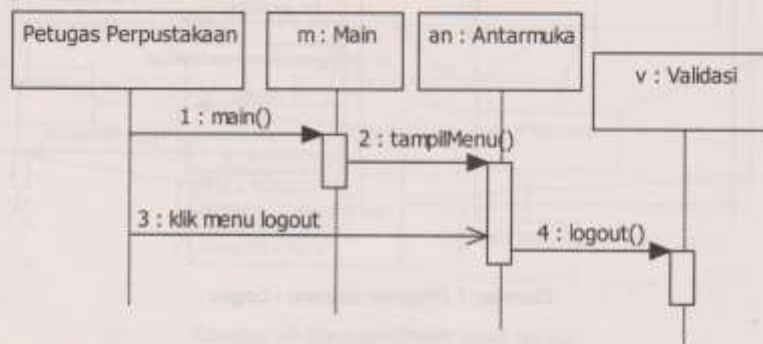
```

Metode *execute* dijalankan di dalam metode *queryCekLogin()* dan hasil dari *query* yaitu username dan password petugas didapat dari

metode getResult(). Berikut adalah tahapan metode queryCekLogin() dalam *pseudocode*:

```
procedure queryCekLogin(k: KoneksiBasisData, username: string, password: string){
    query = "SELECT username, password FROM tpetugas where username='" + username + "' AND password='" + password + "' ";
    k.execute(query)
}
```

2. Use case: Logout



Gambar 57 Diagram Sekuen untuk Use Case Logout

Proses logout hanya mengosongkan session status login.

3. Use case: Memeriksa status login

Proses memeriksa status login berisi untuk memeriksa apakah pengguna perangkat lunak sudah melakukan *login*. Proses ini digunakan oleh *use case* lain sehingga akan menjadi bagian diagram sekuen dari *use case* lain yang menggunakannya.

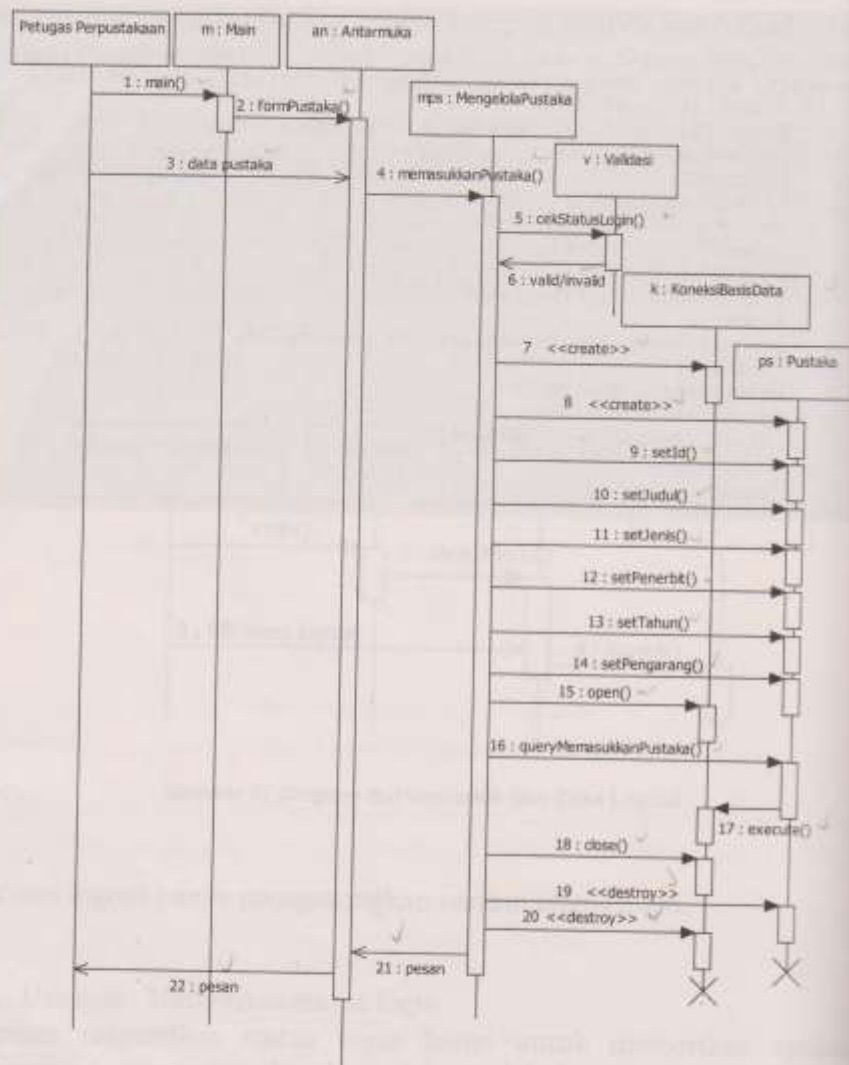
4. Use case: Memasukkan pustaka

Dari gambar diagram sekuen di bawah ini maka tahapan yang ada di dalam metode memasukkanPustaka() adalah sebagai berikut (dalam *pseudocode*):


```

function memasukkanPustaka(id: string, judul: string, jenis: string,
penerbit: string, tahun: string, pengarang: string[], v: Validasi){
    if(v.cekStatusLogin()){
        KoneksiBasisData k = new KoneksiBasisData //create
        Pustaka ps = new Pustaka //create
        ps.setId(id)
        ps.setJudul(judul)
        ps.setJenis(jenis)
        ps.setPenerbit(penerbit)
        ps.setTahun(tahun)
        ps.setPengarang(pengarang)
        k.open()
        ps.queryMemasukkanPustaka(k) //k.execute ada didalamnya
        k.close()
        delete(ps) //destroy
        delete(k) //destroy
        return "Penyimpanan Berhasil"
    }
}

```

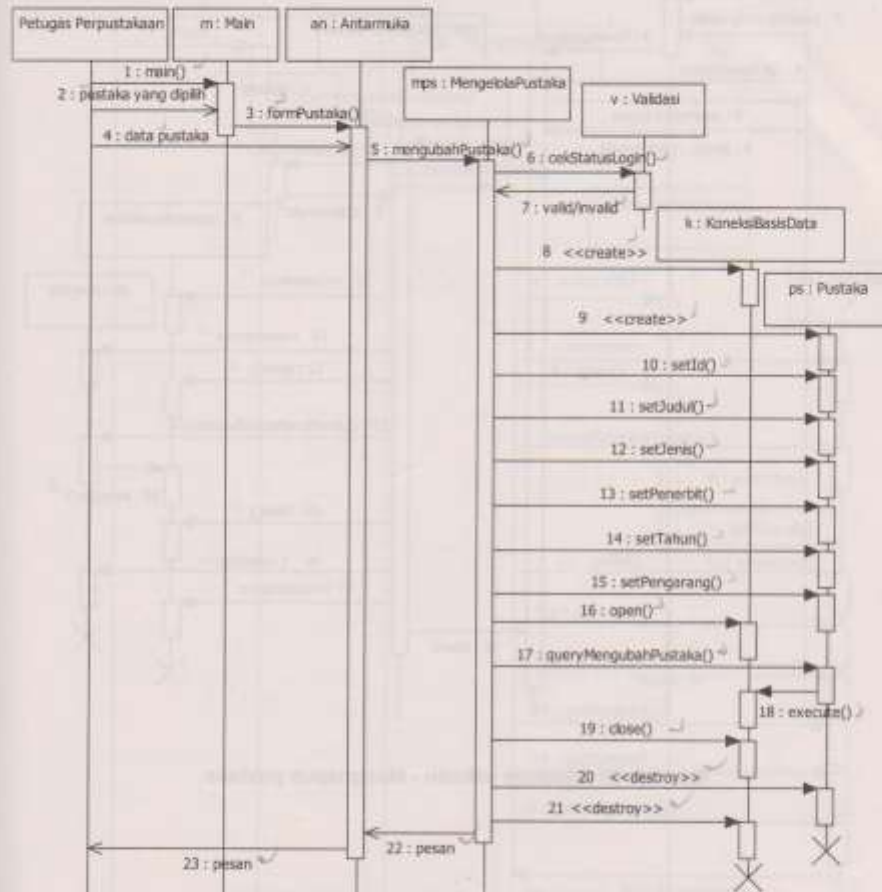


Gambar 58 Diagram sekuen - Memasukkan pustaka

5. Use case: Mengubah pustaka

Proses Mengubah pustaka diawali dengan proses Mencari pustaka yang dapat dilihat pada diagram sekuen Mencari pustaka

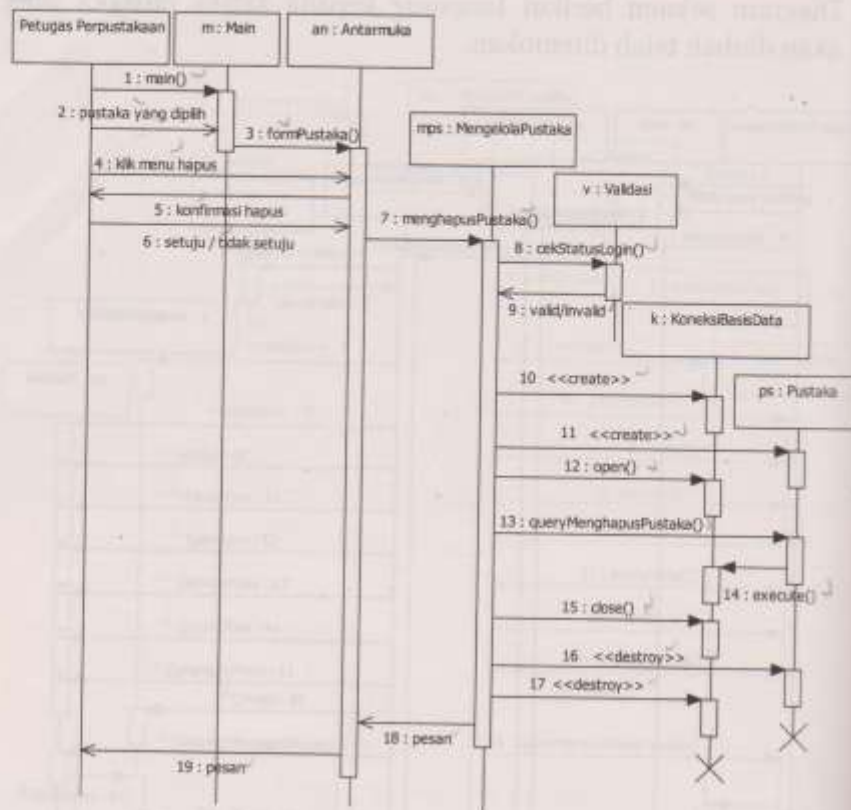
Diagram sekuen berikut langsung kepada ketika pustaka yang akan diubah telah ditemukan.



Gambar 59 Diagram sekuen - Mengubah pustaka

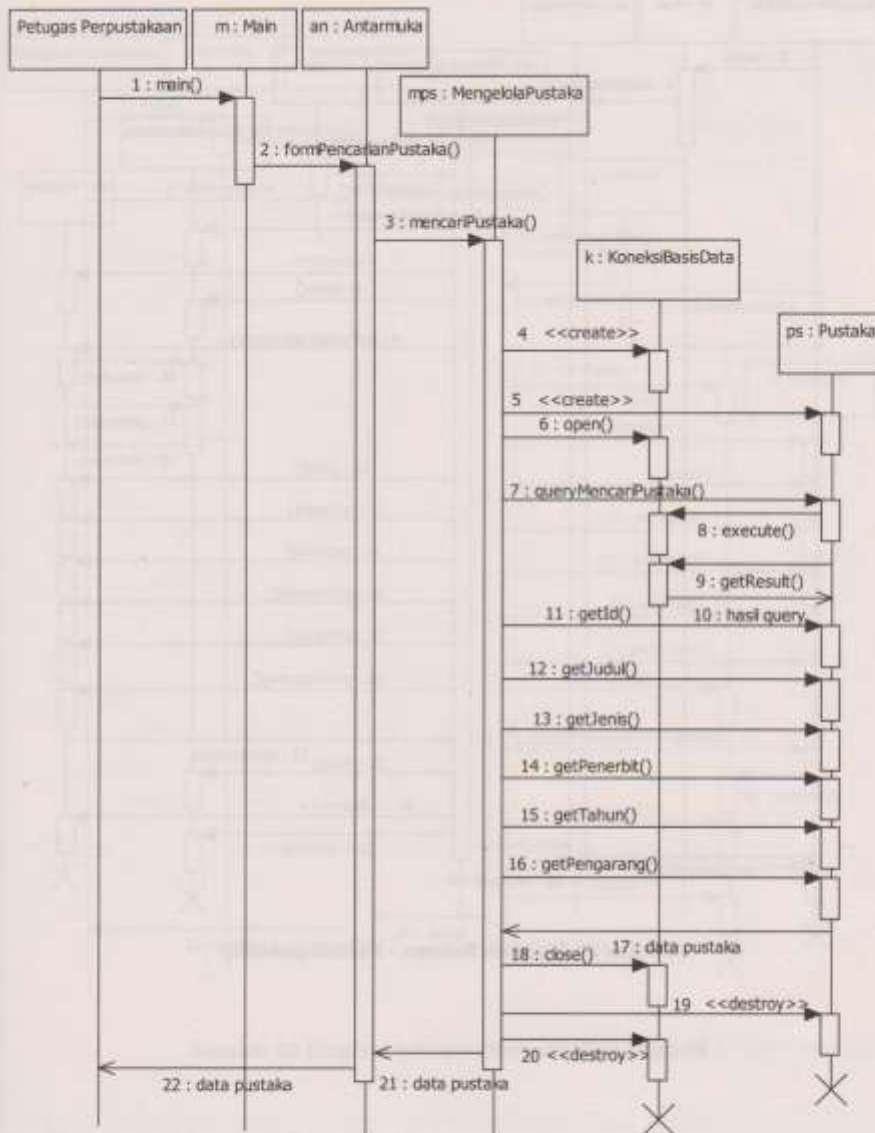
Use case: Menghapus pustaka

Proses Menghapus pustaka diawali dengan proses Mencari pustaka yang dapat dilihat pada diagram sekuen Mencari pustaka. Diagram sekuen berikut langsung kepada ketika pustaka yang akan dihapus telah ditemukan.



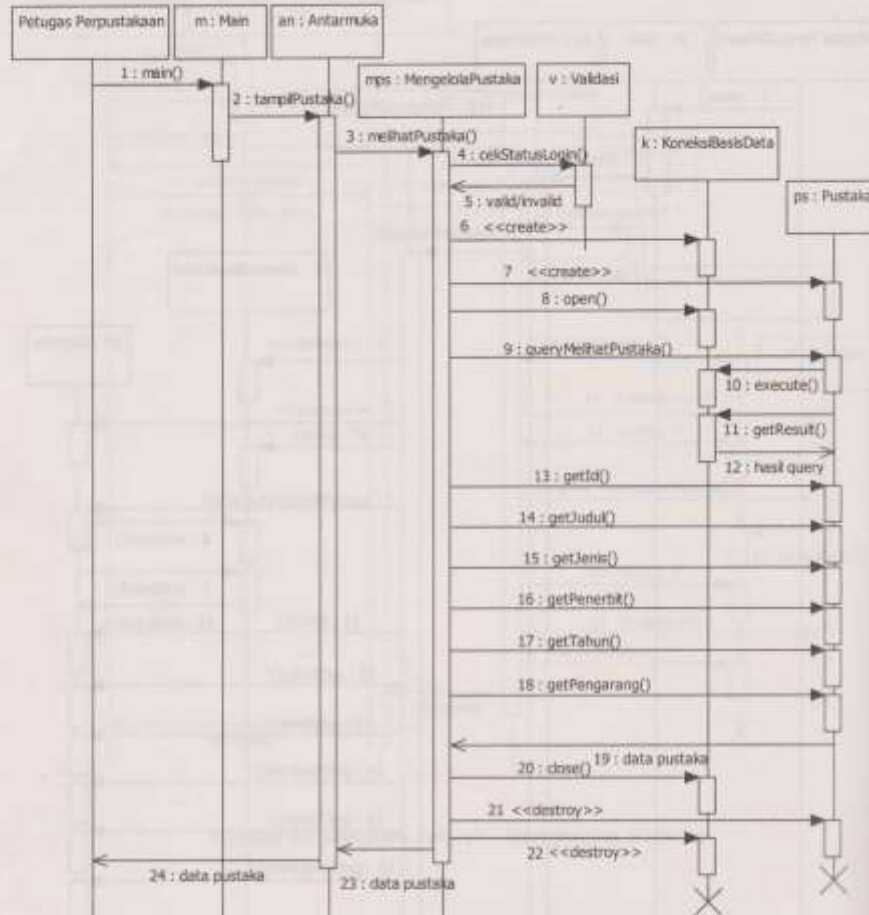
Gambar 60 Diagram sekuen - Menghapus pustaka

6. Use case: Mencari pustaka



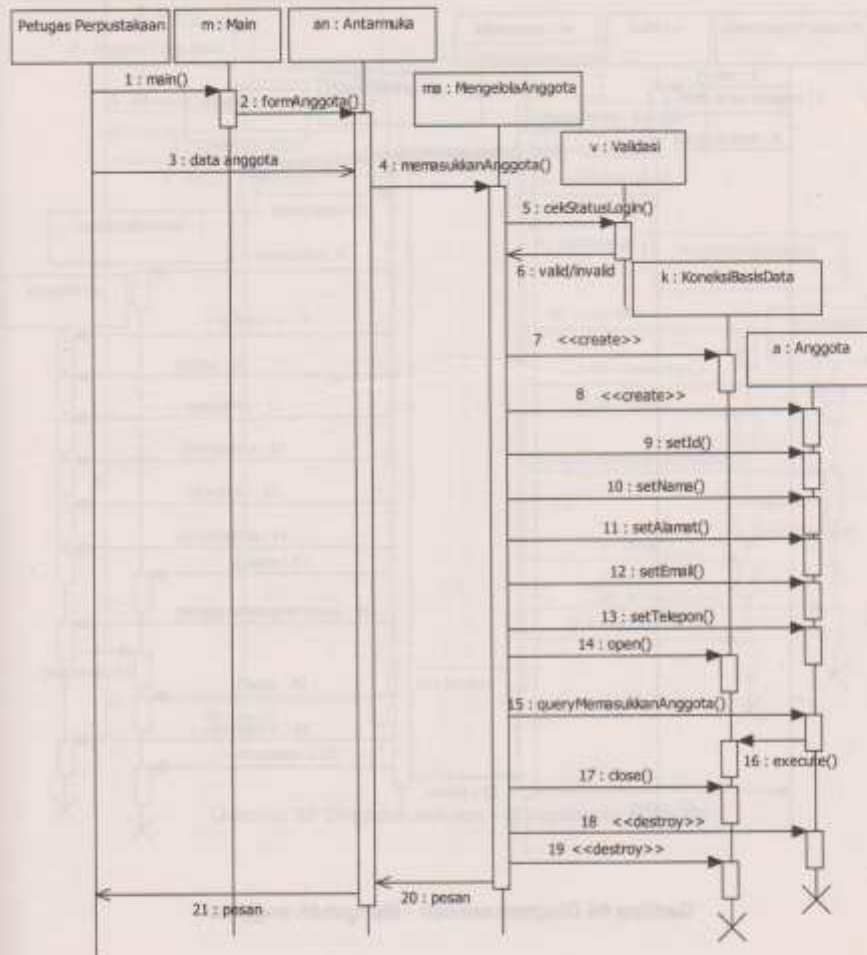
Gambar 61 Diagram Sekuan - Mencari pustaka

7. Use case: Melihat pustaka



Gambar 62 Diagram Sekuen - Melihat pustaka

8. Use case: Memasukkan anggota

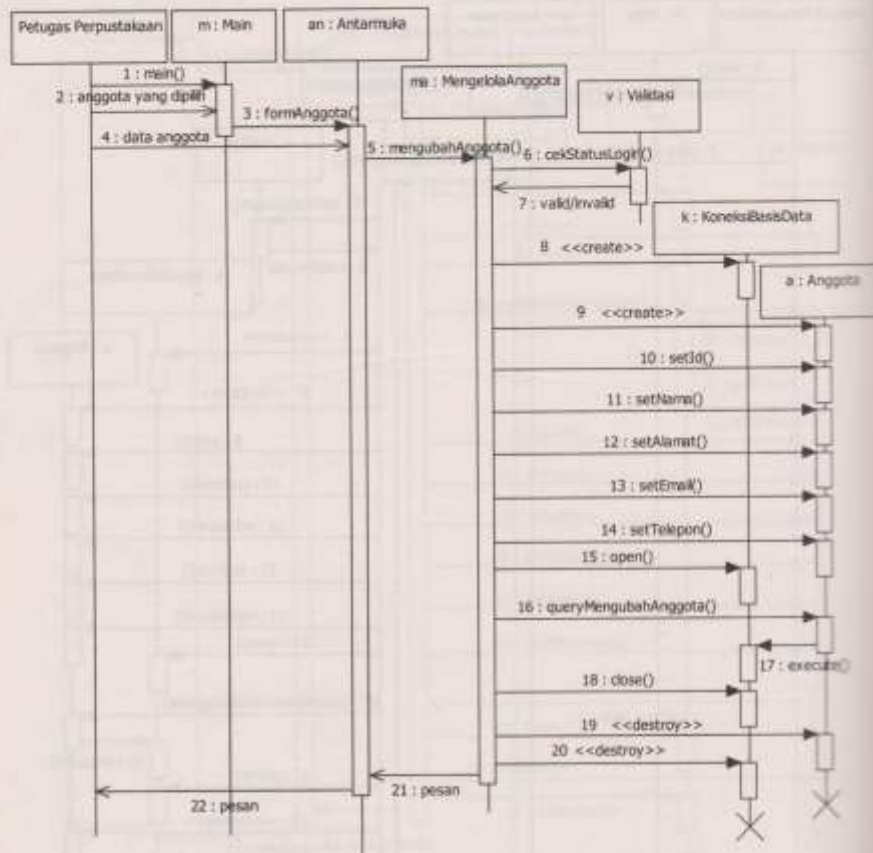


Gambar 63 Diagram sekuen - Memasukkan anggota

9. Use case: Mengubah anggota

Proses Mengubah anggota diawali dengan proses Mencari anggota yang dapat dilihat pada diagram sekuen Mencari anggota. Diagram

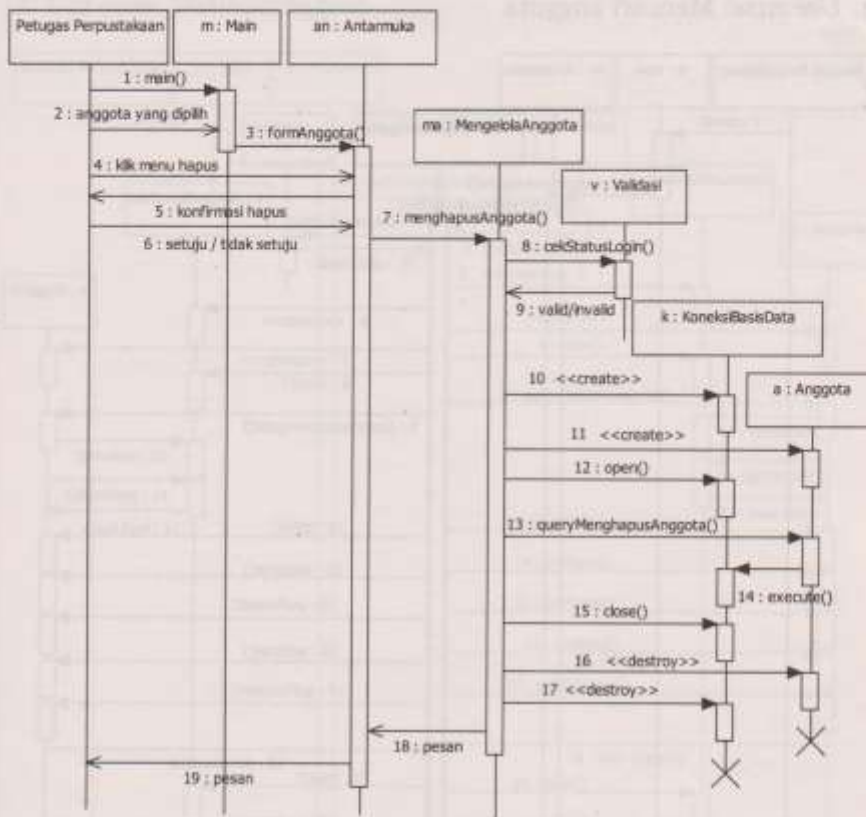
sekuen berikut langsung kepada ketika anggota yang akan diubah telah ditemukan.



Gambar 64 Diagram sekuen - Mengubah anggota

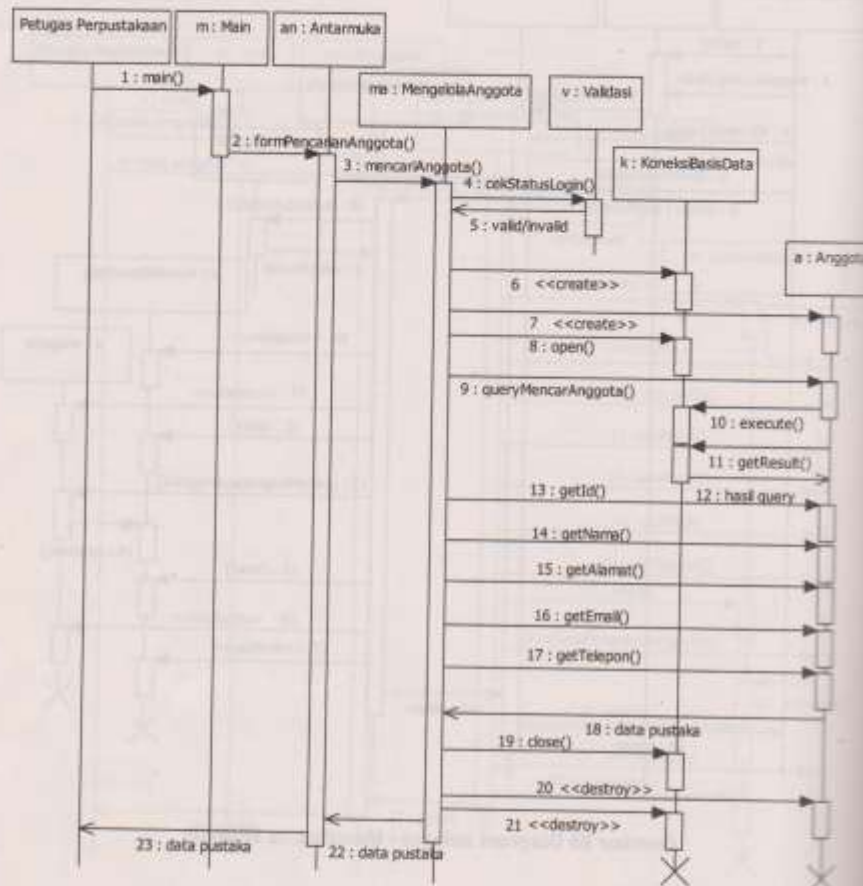
10. Use case: Menghapus anggota

Proses Menghapus anggota diawali dengan proses Mencari anggota yang dapat dilihat pada diagram sekuen Mencari anggota. Diagram sekuen berikut langsung kepada ketika anggota yang akan dihapus telah ditemukan.



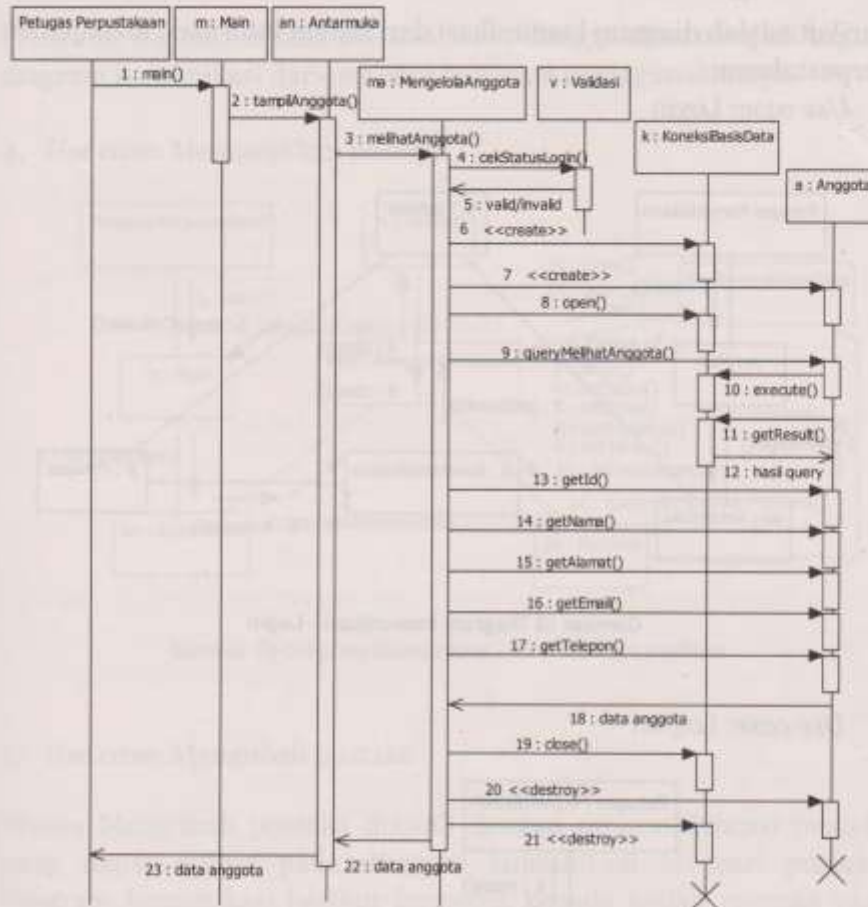
Gambar 65 Diagram sekuen - Menghapus anggota

11. Use case: Mencari anggota



Gambar 66 Diagram sekuen - Mencari anggota

12. Use case: Melihat anggota



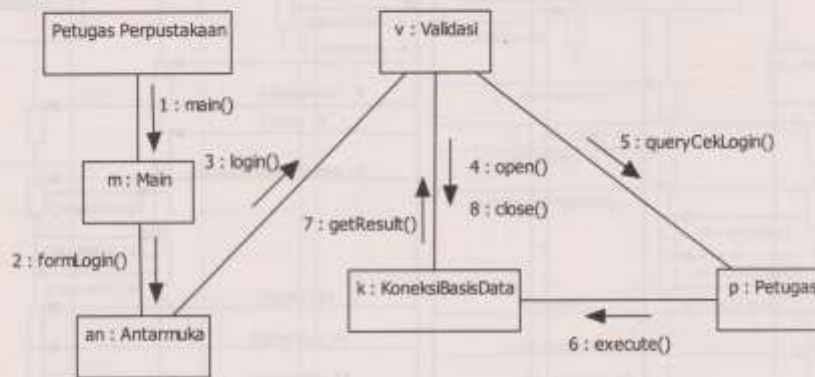
Gambar 67 Diagram sekuen - Mengubah peminjaman

Diagram sekuen dari *use case* yang terkait dengan peminjaman dan petugas memiliki kesamaan dengan diagram sekuen dari *use case* yang terkait dengan anggota.

9.5 Diagram Komunikasi

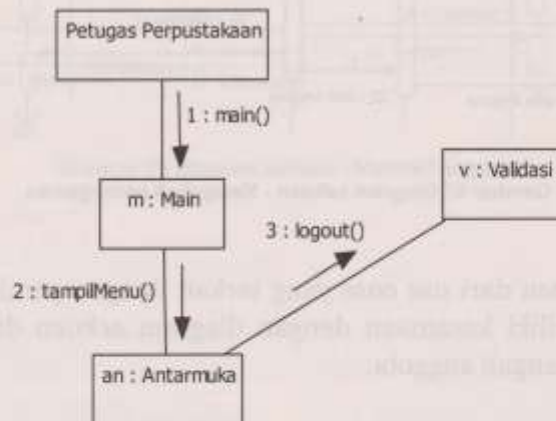
Berikut adalah diagram komunikasi dari sistem informasi manajemen perpustakaan:

1. Use case: Login



Gambar 68 Diagram komunikasi - Login

2. Use case: Logout

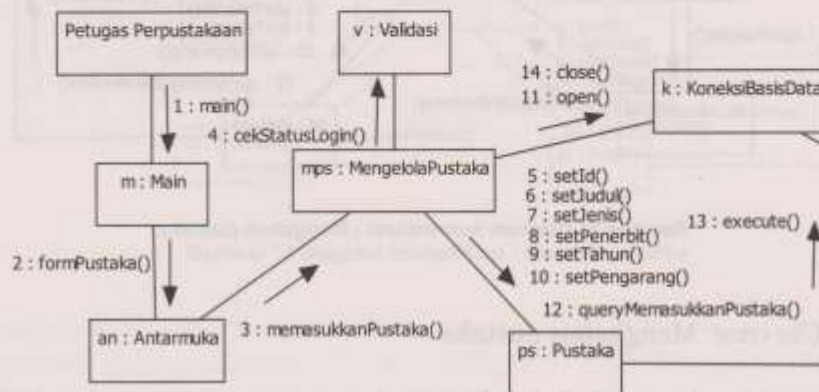


Gambar 69 Diagram komunikasi - Logout

3. Use case: Memeriksa status login

Proses ini digunakan oleh *use case* lain sehingga akan menjadi bagian diagram komunikasi dari *use case* lain yang menggunakannya.

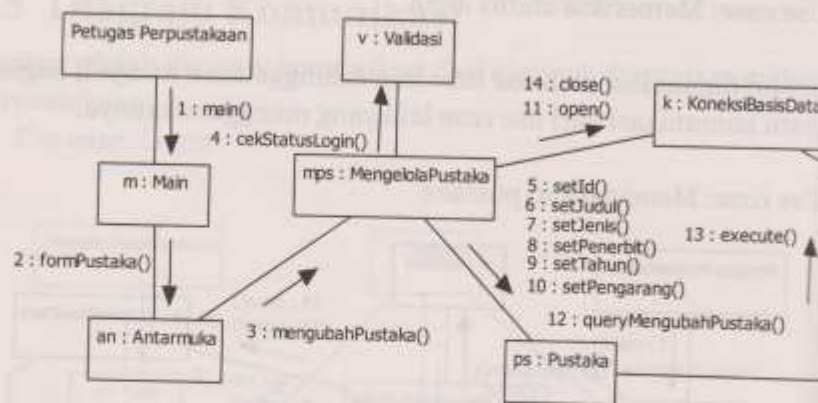
4. Use case: Memasukkan pustaka



Gambar 70 Diagram komunikasi - Memasukkan pustaka

5. Use case: Mengubah pustaka

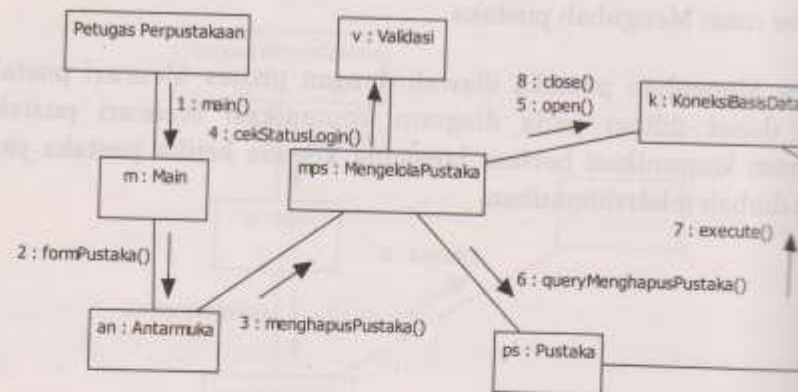
Proses Mengubah pustaka diawali dengan proses Mencari pustaka yang dapat dilihat pada diagram komunikasi Mencari pustaka. Diagram komunikasi berikut langsung kepada ketika pustaka yang akan diubah telah ditemukan.



Gambar 71 Diagram komunikasi - Mengubah pustaka

6. Use case: Menghapus pustaka

Proses Menghapus pustaka diawali dengan proses Mencari pustaka yang dapat dilihat pada diagram komunikasi Mencari pustaka. Diagram komunikasi berikut langsung kepada ketika pustaka yang akan dihapus telah ditemukan.



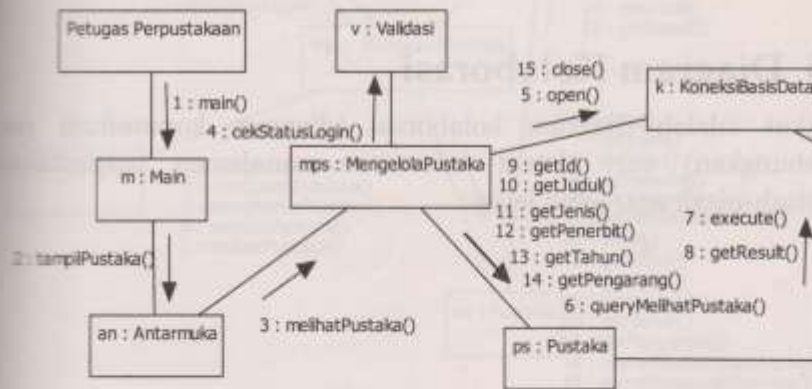
Gambar 72 Diagram komunikasi - Menghapus pustaka

Use case: Mencari pustaka



Gambar 73 Diagram komunikasi - Mencari pustaka

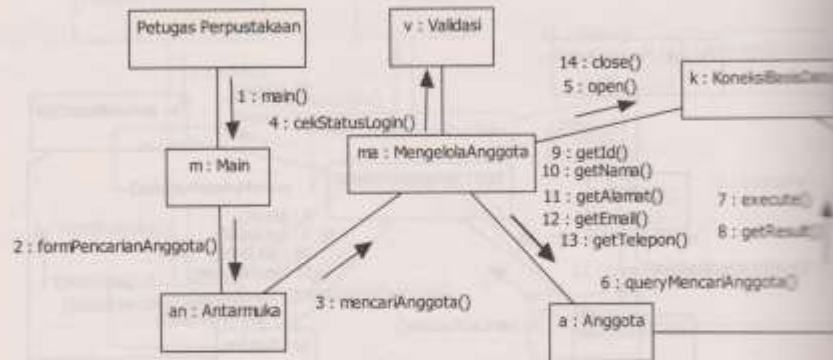
Use case: Melihat pustaka



Gambar 74 Diagram komunikasi - Melihat pustaka

Diagram komunikasi dari *use case* yang terkait anggota mirip dengan diagram komunikasi yang terkait pustaka, kecuali *use case* Mencari anggota yang diperlukan adanya proses memeriksa status login.

9. Use case: Mencari anggota



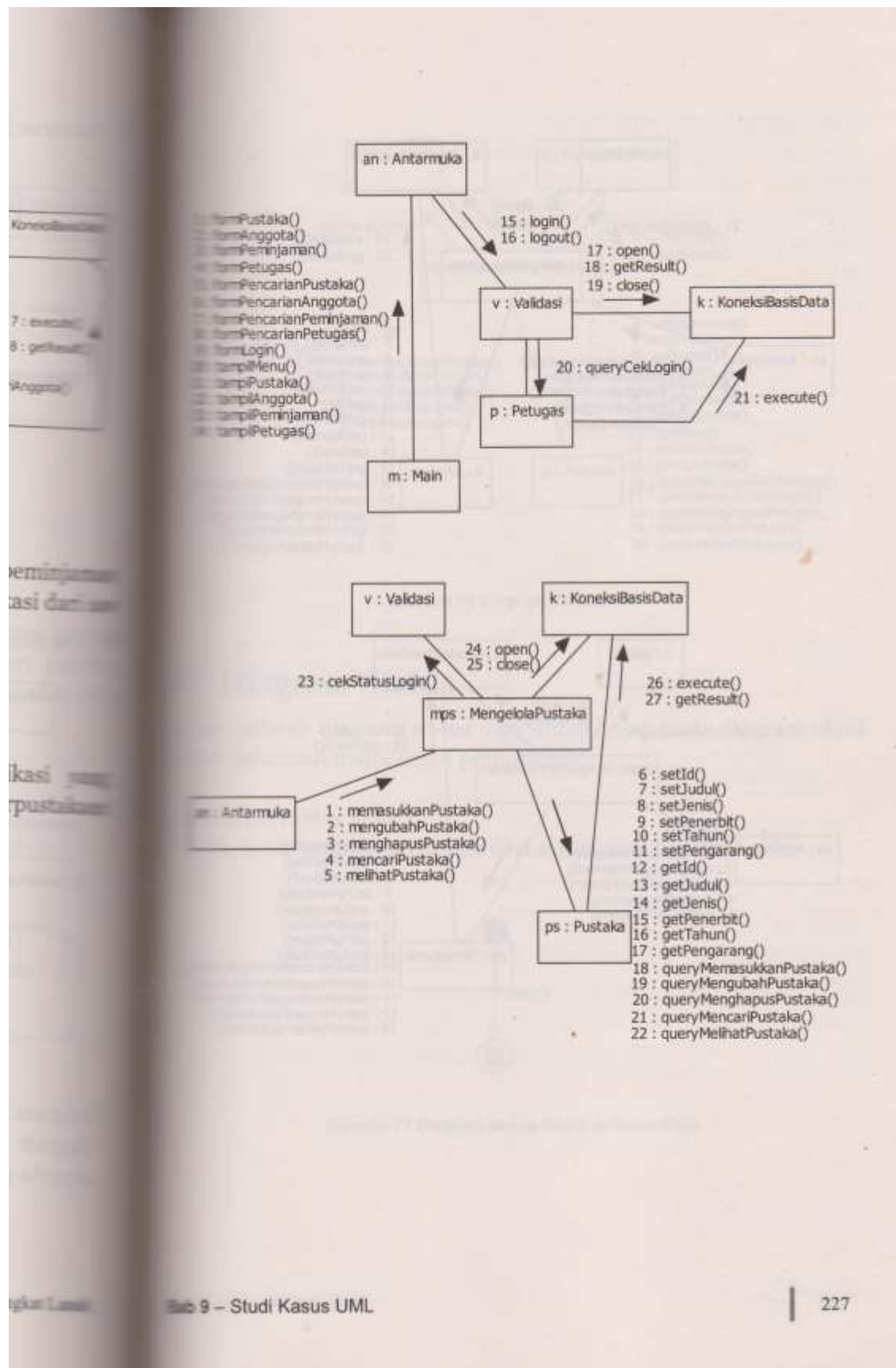
Gambar 75 Diagram komunikasi - Mencari anggota

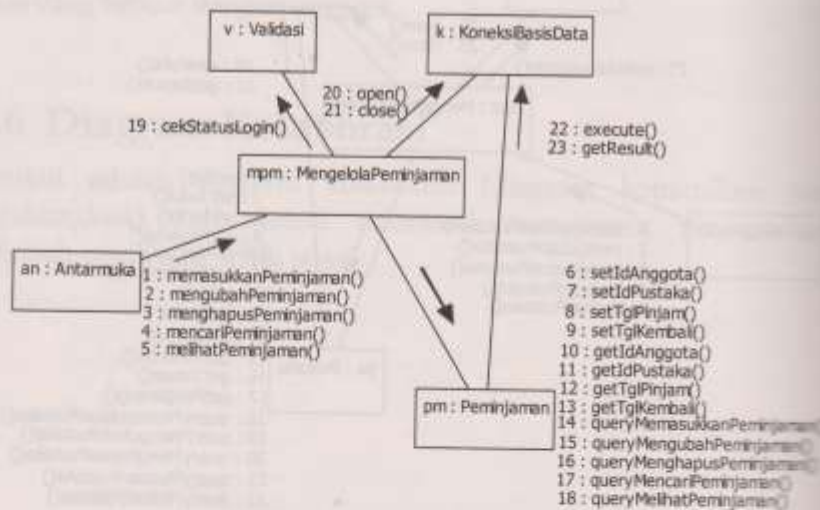
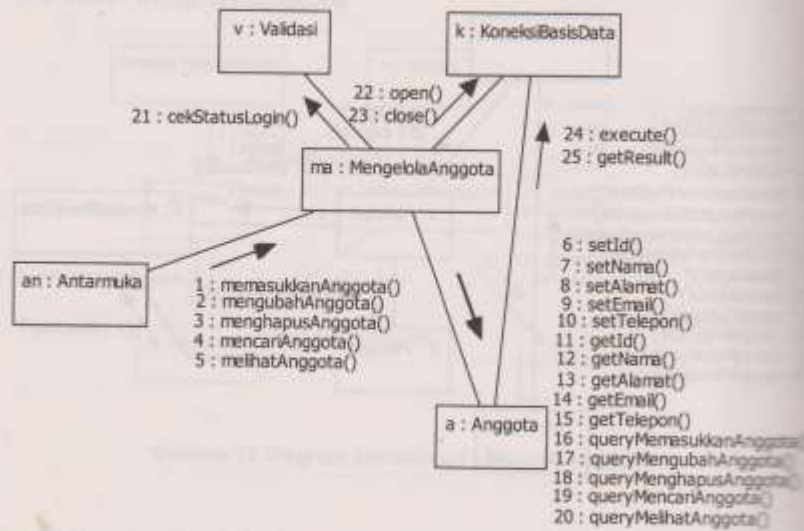
Diagram komunikasi dari *use case* yang terkait dengan peminjaman dan petugas memiliki kesamaan dengan diagram komunikasi dari *use case* yang terkait dengan anggota.

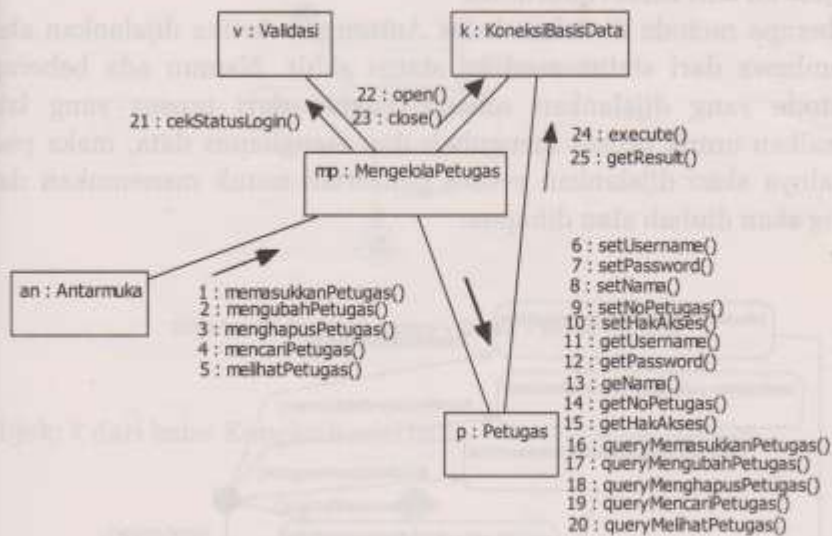
9.6 Diagram Kolaborasi

Berikut adalah diagram kolaborasi (diagram komunikasi yang digabungkan) dari sistem informasi manajemen perpustakaan (dipisah-pisah agar lebih jelas):









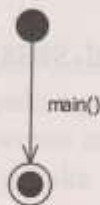
Gambar 76 Diagram Kolaborasi

9.7 Diagram Status

Berikut adalah diagram status dari setiap objek pada diagram objek sistem informasi manajemen perpustakaan:

Objekm dari kelas Main

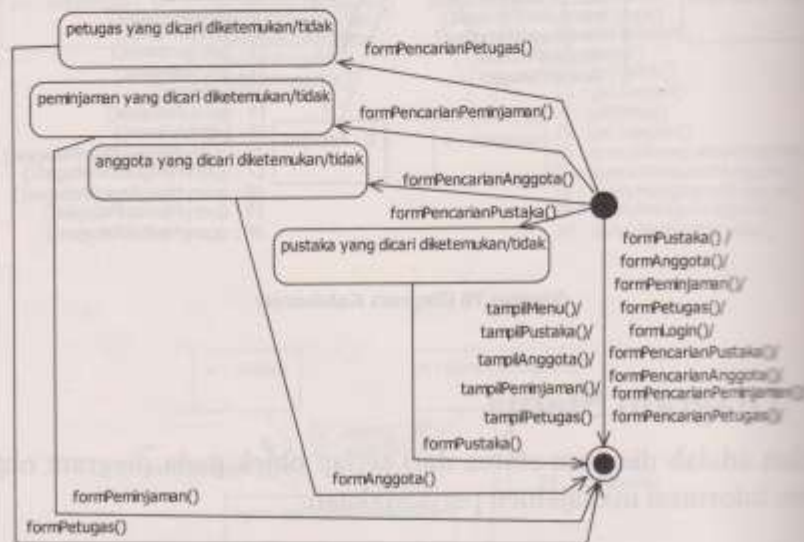
Metode main membawa transisi dari status awal ke status akhir



Gambar 77 Diagram status Objek m Kelas Main

Objek: an dari kelas Antarmuka

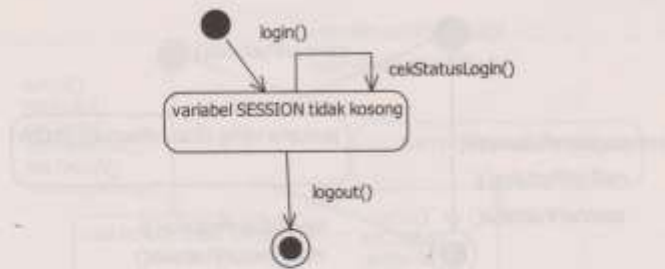
Beberapa metode di dalam kelas Antarmuka ketika dijalankan akan membawa dari status awal ke status akhir. Namun ada beberapa metode yang dijalankan sebagai bagian dari proses yang lain. misalkan untuk proses mengubah dan menghapus data, maka pada awalnya akan dijalankan proses pencarian untuk menemukan data yang akan diubah atau dihapus.



Gambar 78 Diagram status Objek an Kelas Antarmuka

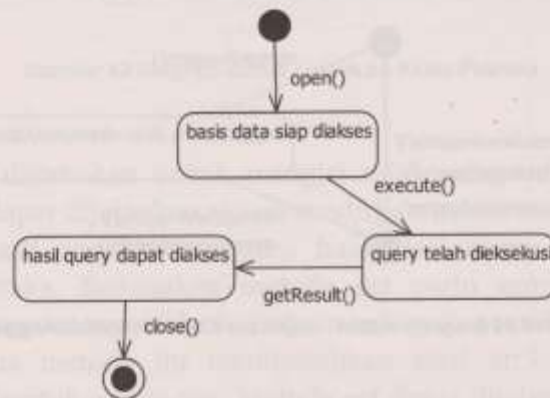
Objek: v dari kelas Validasi

Metode login mengisi variabel SESSION sebagai penanda bahwa status telah login.



Gambar 79 Diagram status - Objek v Kelas Validasi

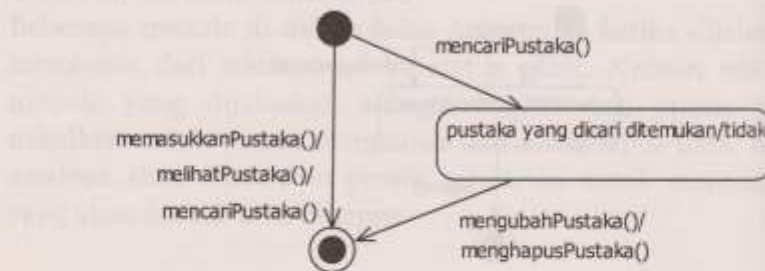
Objek: k dari kelas KoneksiBasisData



Gambar 80 Diagram status - Objek k Kelas KoneksiBasisData

Objek: mps dari kelas MengelolaPustaka

Proses mencari pustaka merupakan proses yang dapat berdiri sendiri maupun sebagai bagian dari proses mengubah atau menghapus data pustaka untuk menemukan pustaka mana yang akan diubah atau dihapus.



Gambar 81 Diagram status - Objek mps Kelas MengelolaPustaka

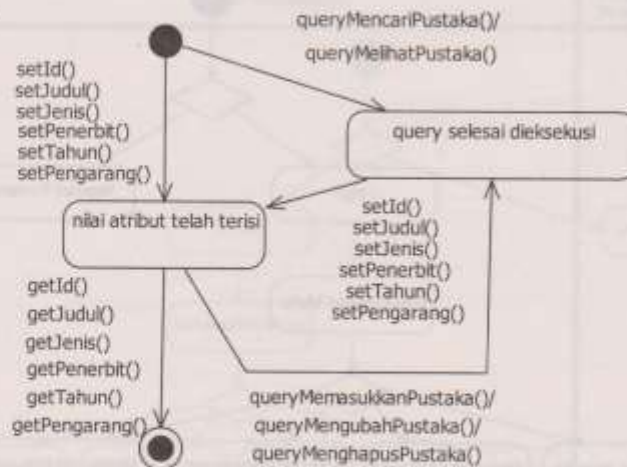
Objek: ma dari kelas MengelolaAnggota



Gambar 82 Diagram status - objek ma Kelas MengelolaAnggota

untuk objek mpm dari kelas MengelolaPeminjaman dan objek mp dari kelas MengelolaPetugas memiliki kesamaan diagram status dengan diagram status objek mps dari kelas MengelolaPustaka dan objek ma dari kelas MengelolaAnggota.

Objek: ps dari kelas Pustaka



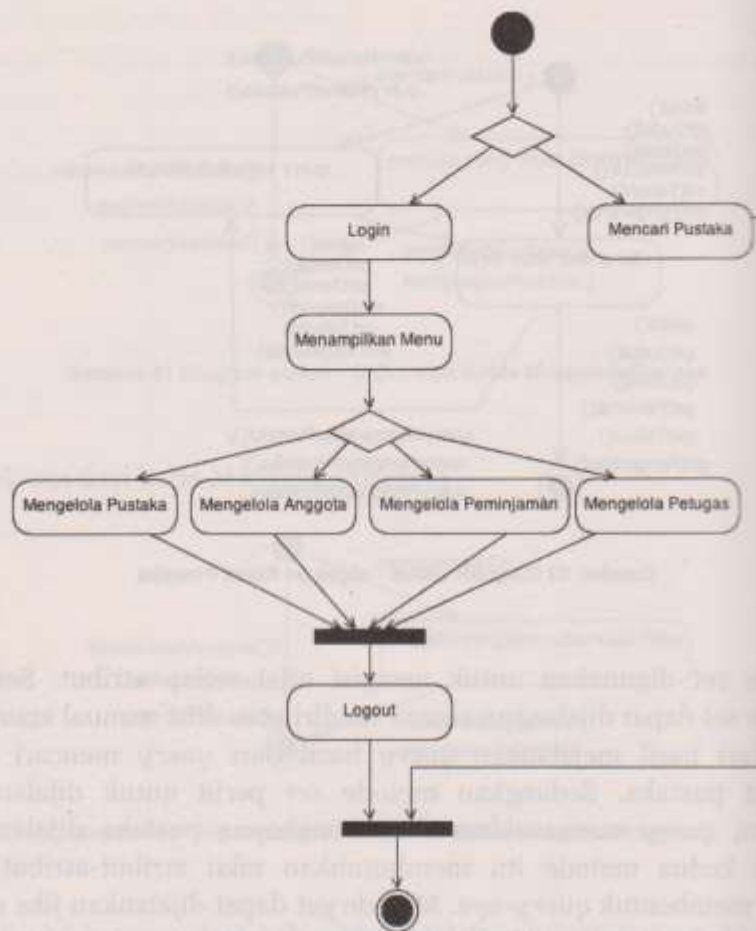
Gambar 83 Diagram status - objek ps Kelas Pustaka

Metode set digunakan untuk mengisi nilai setiap atribut. Semua metode set dapat dijalankan secara sendiri atau diisi manual ataupun diisi dari hasil menjalankan *query* hasil dari *query* mencari dan melihat pustaka. Sedangkan metode set perlu untuk dijalankan sebelum *query* memasukkan dan menghapus pustaka dijalankan karena kedua metode itu membutuhkan nilai atribut-atribut itu dalam membentuk *query*-nya. Metode get dapat dijalankan jika nilai atribut-atribut telah terisi.

Diagram status untuk objek dari kelas data yang lain mirip dengan diagram status pada objek ps dari kelas Pustaka.

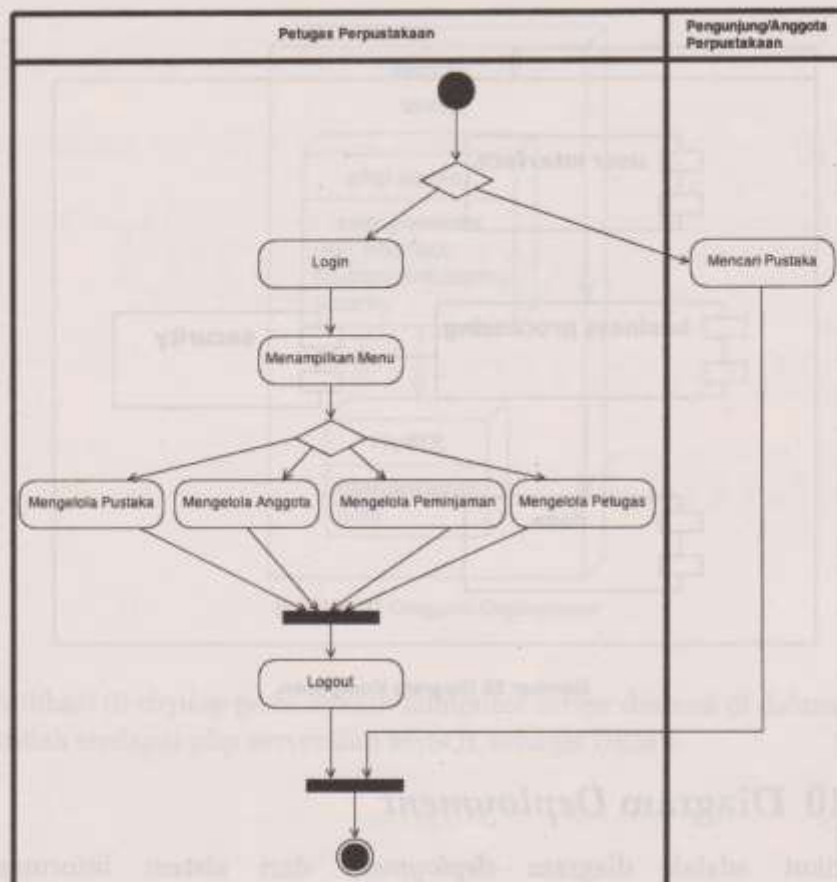
9.8 Diagram Aktivitas

Berikut adalah diagram aktivitas dari sistem informasi manajemen perpustakaan:



Gambar 84 Diagram Interaksi Studi Kasus

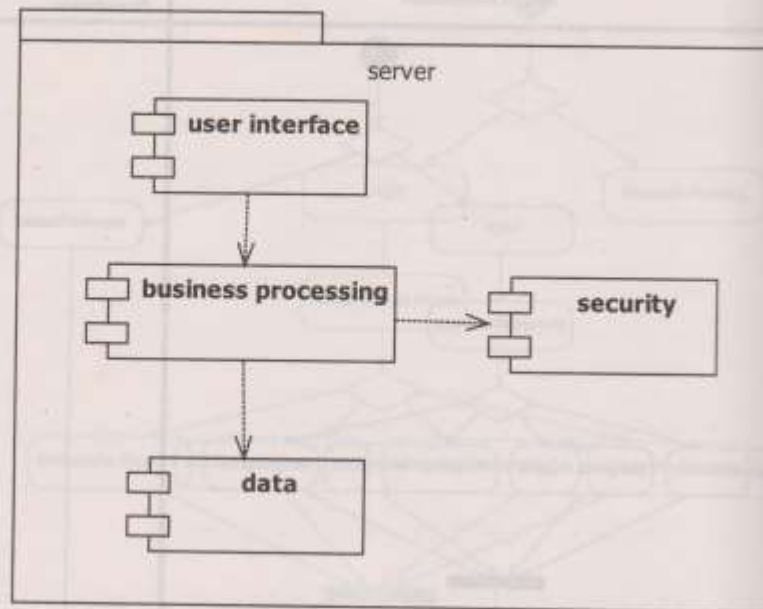
Jika digambar dengan menggunakan *swimlane* maka akan menjadi sebagai berikut:



Gambar 85 Diagram Aktivitas dengan Swimlane

9.9 Diagram Komponen

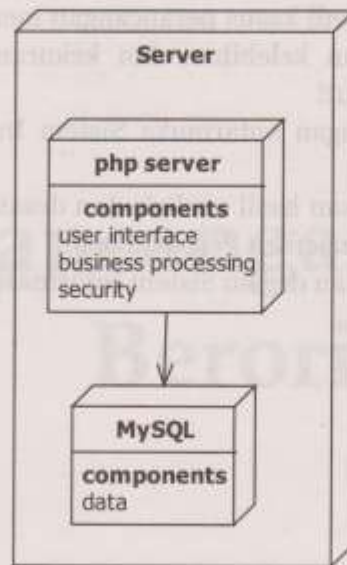
Berikut adalah diagram komponen dari sistem informasi manajemen perpustakaan:



Gambar 86 Diagram Komponen

9.10 Diagram *Deployment*.

Berikut adalah diagram *deployment* dari sistem informasi manajemen perpustakaan:



Gambar 87 Diagram *Deployment*

Aplikasi di-*deploy* pada sebuah komputer server dimana di dalamnya sudah terdapat php server dan MySQL sebagai DBMS.



Latihan

1. Apa fungsi penggambaran diagram *use case* dalam analisis berorientasi objek?
2. Kapan sebaiknya diagram sekuen, diagram kolaborasi, diagram komponen, dan diagram *deployment* digunakan dalam analisis dan desain berorientasi objek? Sebutkan alasannya!

3. Bandingkan studi kasus perancangan menggunakan DFD dan UML, sebutkan kelebihan dan kekurangan masing-masing metode tersebut!
4. Buat perancangan antarmuka Sistem Informasi Manajemen Perpustakaan!
5. Implementasikan hasil analisis dan desain studi kasus Sistem Informasi Manajemen Perpustakaan!
6. Buat analisis dan desain Sistem Informasi Apotek!



9.20 Diagram Deployment

Diagram deployment menunjukkan distribusi fisik dari sistem informasi, termasuk perangkat keras, perangkat lunak, dan jaringan yang digunakan untuk menjalankan sistem.

Contoh:



Diagram deployment digunakan untuk menunjukkan distribusi fisik dari sistem informasi, termasuk perangkat keras, perangkat lunak, dan jaringan yang digunakan untuk menjalankan sistem.

Diagram deployment digunakan untuk menunjukkan distribusi fisik dari sistem informasi, termasuk perangkat keras, perangkat lunak, dan jaringan yang digunakan untuk menjalankan sistem.

10

Perancangan Pola Berorientasi Objek



Overview

Bab ini membahas perancangan pola berorientasi objek. Pola tersebut terdiri dari 2 macam yaitu *design pattern* dan *anti pattern*.

- *Design pattern* adalah pola solusi yang sudah teruji untuk menyelesaikan permasalahan pemrograman.
- *Anti pattern* adalah pola perbaikan terhadap sistem yang sebelumnya dikembangkan dengan kurang baik.

10.1 Design Pattern

Design Pattern (Pola Desain) adalah suatu bagian dari teknologi pemrograman berorientasi objek. Pola Desain adalah solusi yang sudah teruji (*best practice*) untuk menyelesaikan permasalahan pemrograman. Pola Desain merupakan sebuah pola atau cara untuk mendesain komponen-komponen dalam pemrograman berorientasi objek yang baik sehingga komponen-komponen yang ada dapat digunakan kembali untuk aplikasi yang lain. Pola Desain dimulai dengan menelusuri solusi-solusi yang baik dari suatu masalah untuk menghasilkan pola yang baik.

Cara membuat sebuah pola desain komponen yang baik adalah dengan memperhatikan kemungkinan perubahan sistem yang mungkin terjadi di masa mendatang. Pola Desain penting untuk dipelajari agar dapat merancang program berorientasi objek yang dapat digunakan kembali untuk aplikasi lain dan memudahkan kerja tim jika sudah menggunakan sebuah pola pembuatan program berorientasi objek yang sudah standar (mengurangi perbedaan cara berpikir antara anggota tim). Pola yang baik memperhatikan *cohesion* (keeratan keterkaitan metode dalam sebuah kelas) dan *coupling* (keeratan hubungan antar kelas).

Design Pattern (Pola Desain) sebaiknya dipelajari ketika pemrograman berorientasi objek telah dipahami. Pola desain bertujuan agar komponen program bersifat *reusable* (dapat digunakan kembali berulang-ulang). *Framework* merupakan rancangan arsitektur perangkat lunak yang digunakan sebagai standar pembuatan program dan memudahkan pengguna dalam memelihara perangkat lunak yang dibuat dengan menggunakan *framework*. *Framework* yang berorientasi objek adalah salah satu bagian implementasi pola desain. Subbab-subbab berikutnya akan membahas pola-pola desain yang telah ada.

Pada sebuah pola biasanya memiliki atribut-atribut (hal-hal yang perlu diperhatikan) sebagai berikut:

- Nama (*Name*)
Nama yang diberikan pada pola
- Tujuan (*Intent*)
Merupakan pemaparan singkat mengenai permasalahan yang terjadi dan tujuan yang ingin dicapai
- Alias (*Also Known As*)
Nama alias yang diberikan pada pola, nama alias bersifat opsional (tidak harus ada)
- Motivasi (*Motivation*)
Sebuah skenario yang menjelaskan permasalahan perancangan dan solusi yang diberikan pola desain
- Kemampuan diaplikasikan (*Applicability*)
Spesifikasi situasi yang sesuai untuk penerapan pola
- Struktur (*Structure*)
Gambar struktur yang menjelaskan hubungan kelas dan objek dalam pola
- Partisipasi (*Participants*)
Berbagai kelas dan / atau objek yang berpartisipasi dan berperan dalam pola
- Kolaborasi (*Collaborations*)
Bagaimana kolaborasi kerjasama dari partisipan untuk melaksanakan perannya masing-masing
- Konsekuensi (*Consequences*)
Bagaimana pencapaian tujuan yang dilakukan pola dan konsekuensi penerapan pola
- Implementasi (*Implementation*)
Berbagai hal yang berkaitan dengan penerapan pola

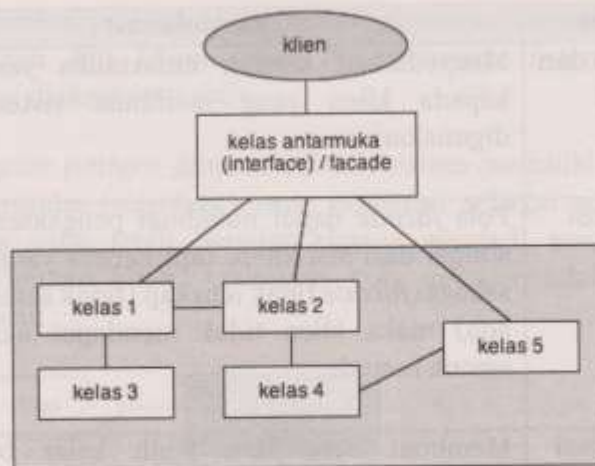
- Contoh Kode Program (*Sample Code*)
Contoh kode program yang merepresentasikan penerapan pola
- Contoh Penggunaan Pola (*Known Uses*)
Contoh-contoh penggunaan pola pada sistem yang riil
- Pola-pola Lain yang Terkait (*Related Pattern*)
Pola-pola lain yang berhubungan dengan pola ini

Pola Desain dibagi menjadi beberapa kelompok sebagai berikut:

1. Pola yang Berbasis pada Proses Pembuatan (*Creational Patterns*)
Pola disusun dengan menitikberatkan komponen-komponen (objek-objek) apa saja yang perlu untuk dibuat
2. Pola yang Berbasis pada Struktur (*Structural Patterns*)
Pola disusun dengan menitikberatkan pada struktur spesifik komponen-komponen (kesesuaian objek dengan tindakan dan kemampuan yang diharapkan) pemrograman berorientasi objek

Contohnya adalah:

- a. *Facade pattern* dimana sebuah sistem memiliki antarmuka universal atau *unified* sebagai pembatas antara *user/kliendengan* sistem. Arsitektur pola *facade* adalah sebagai berikut:



Gambar 1 Arsitektur pola facade

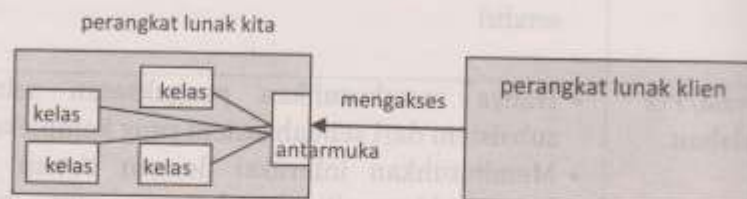
Klien dapat berupa kelas lain atau sistem lain.

Berikut deskripsi dari pola facade:

Atribut	Penjelasan
Nama	Pola Facade (Facade Pattern)
Tujuan	• kebutuhan struktur kelas dalam sistem menjadi lebih simpel dengan pengaksesan melalui satu kelas • kebutuhan mendefinisikan antarmuka (interface) sendiri
Motivasi/Permasalahan	• Hanya membutuhkan pengaksesan sebuah subsistem dari sebuah sistem yang kompleks • Membutuhkan interaksi dengan sistem lain secara khusus/tidak dengan cara biasa (misalkan butuh ada gateway)
Kemampuan diaplikasikan	Menyediakan antarmuka/interface antara sistem dan klien

Atribut	Penjelasan
Partisipasi dan kolaborasi	Menyediakan sebuah antarmuka yang simpel kepada klien yang membuat sistem mudah digunakan
Konsekuensi	Pola <i>facade</i> dapat membuat pengaksesan sistem simpel dari sisi klien, tapi karena yang berperan sebagai <i>facade</i> tidak lengkap (tidak semua metode ada) maka klien tidak mendapat akses untuk semua metode
Implementasi	Membuat satu atau lebih kelas baru yang memenuhi syarat sebagai <i>facade</i>

Pola *Facade* biasanya juga digunakan untuk menjaga keamanan dari perangkat lunak kita agar klien tidak mudah dalam memanipulasi perangkat lunak kita. Pola ini biasanya digunakan untuk dua buah perangkat lunak yang akan dihubungkan atau di masa mendatang memiliki kemungkinan untuk dihubungkan dengan perangkat lunak lain. Agar perangkat lunak yang terhubung pada perangkat lunak kita (klien) tidak terlalu banyak memiliki akses pada kelas-kelas di dalam perangkat lunak kita. Ilustrasinya dapat dilihat pada gambar berikut:



Gambar 2 Gambar Ilustrasi Pola *Facade*

Pola ini sesuai diimplementasikan jika hanya perangkat lunak klien yang mengakses perangkat lunak kita, tapi perangkat lunak kita tidak membutuhkan akses apa pun ke perangkat lunak klien. Contoh lain

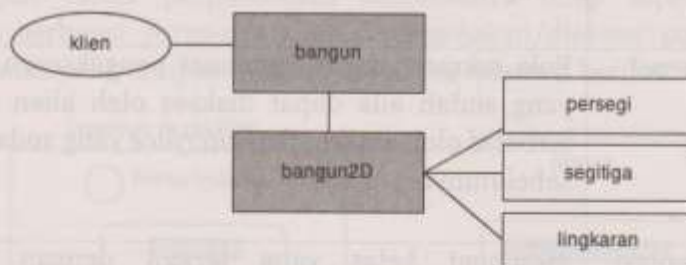
dari implementasi pola ini adalah teknologi *webservice* atau RSS atau API (misalnya API (*Application Programming Interface*) pada Twitter) yang diakses satu arah.

- b. *Adapter pattern* dimana sebuah sistem memiliki kelas atau antarmuka (*interface*) yang berperan sebagai adaptor dari kelas yang lebih bersifat khusus menjadi sesuai dengan permintaan klien. Arsitektur pola *adapter* adalah sebagai berikut:



Gambar 3 Arsitektur pola *adapter*

Contoh penerapan arsitektur pola *adapter* adalah sebagai berikut:



Gambar 4 Contoh penerapan arsitektur pola *adapter*

Klien menginginkan tampilan yang tampak adalah dengan sebutan bangun, sedangkan perangkat lunak yang ada sudah memiliki tiga kelas persegi, segitiga, dan lingkaran maka dibuatlah tampilan bangun di sisi klien dan dibuat tampilan bangun 2D untuk mengadaptasi kelas yang ada. Klien dapat berupa kelas lain atau sistem lain.

Berikut deskripsi dari pola *adapter*:

Atribut	Penjelasan
Nama	Pola <i>Adapter</i> (<i>Adapter Pattern</i>)
Tujuan	Mencocokkan kebutuhan akan objek dengan antarmuka/ <i>interface</i> yang dibutuhkan klien
Motivasi/Permasalahan	- Sistem memiliki data dan perilaku yang benar tetapi memiliki antarmuka (<i>interface</i>) yang salah sehingga butuh ada kelas lain untuk mengadaptasi kesalahan yang sudah terjadi - Biasanya terjadi pada sistem dengan keterhubungan <i>extend</i>
Kemampuan diaplikasikan	Menyediakan antarmuka/ <i>interface</i> sebagai batasan yang dibutuhkan
Partisipasi dan kolaborasi	Menyediakan sebuah antarmuka untuk mengadaptasi kebutuhan klien
Konsekuensi	Pola <i>adapter</i> dapat membuat pengaksesan kelas yang sudah ada dapat diakses oleh klien tanpa terbatas oleh antarmuka/ <i>interface</i> yang sudah ada sebelumnya
Implementasi	Membuat kelas yang terkait dengan kelas sebelumnya dimana kelas yang terkait berisi antarmuka metode kelas yang sudah ada agar memenuhi kebutuhan klien

Pola *adapter* biasanya digunakan jika klien meminta tampilan yang berbeda dengan tampilan yang sudah ada. Maka agar tidak membuat banyak usaha merombak perangkat lunak yang ada maka dibuatlah sebuah tampilan di sisi klien untuk mengadaptasi permintaan klien

dan sebuah tampilan di sisi pengembang untuk memperjelas transisi yang terjadi. Pola ini sering dipakai jika pengembang perangkat lunak mencoba memakai kembali kelas-kelas yang telah ada (*reusable*) tapi disesuaikan dengan permintaan klien sehingga tampilan yang tampak sesuai dengan permintaan klien tetapi tetap dapat dimengerti strukturnya oleh pengembang. Contoh ilustrasi penggunaannya dapat dilihat pada gambar berikut:



Gambar 5 Ilustrasi Implementasi Pola Adapter

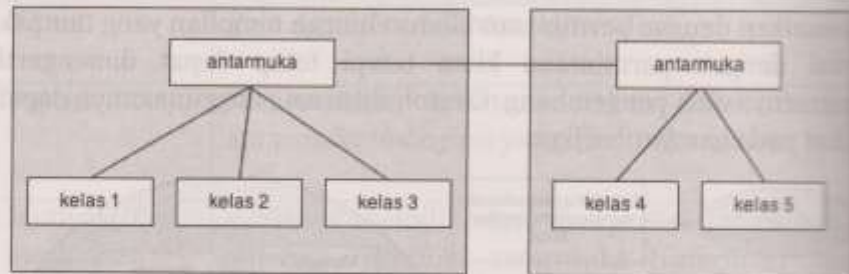
Perangkat lunak perpustakaan sebelumnya tetap dipergunakan desain berbagai *form*-nya dan dipergunakan/diakses pada kelas antarmuka yang tampak di klien. Misalkan Ilustrasi berikut:



Gambar 6 Ilustrasi Cara Kerja Adaptor dan Antarmuka di Sisi Klien

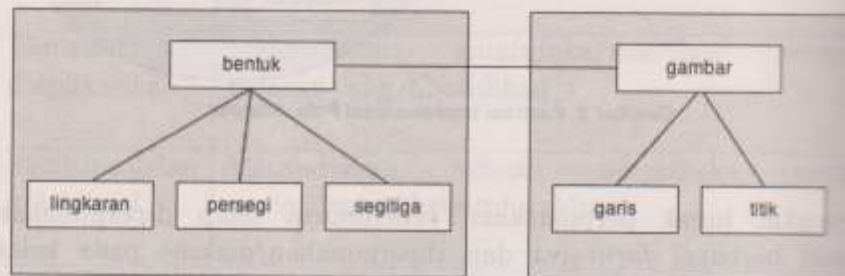
- c. *Bridge pattern* dimana sebuah subsistem memiliki kelas atau antarmuka (*interface*) yang berperan sebagai penghubung

antara subsistem yang satu dengan subsistem yang lain. Arsitektur pola *bridge* adalah sebagai berikut:



Gambar 7 Arsitektur pola *bridge*.

Misalkan:



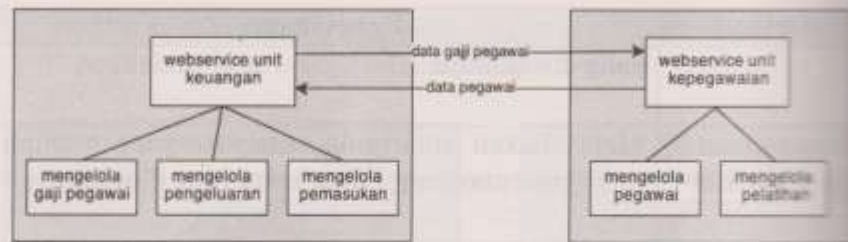
Gambar 8 Contoh arsitektur pola *bridge*

Berikut adalah deskripsi dari pola *bridge*:

Atribut	Penjelasan
Nama	Pola Penghubung(<i>Bridge Pattern</i>)
Tujuan	Menghubungkan dua objek yang dibutuhkan oleh dua atau lebih objek yang berada di bawahnya
Motivasi/Permasalahan	Kelas yang menjadi penghubung tidak boleh menyebabkan penambahan jumlah kelas yang diwakilinya dan mampu mewakili semua kelas

Atribut	Penjelasan
	yang diwakilinya
Kemampuan diaplikasikan	Menyediakan antarmuka/ <i>interface</i> yang mampu merepresentasikan semua kelas yang diwakilinya
Partisipasi dan kolaborasi	Menyediakan antarmuka untuk mewakili kumpulan kelas lain
Konsekuensi	Memperpanjang pengaksesan objek-objek yang diwakilinya
Implementasi	Mengapsulasi dengan membuat kelas abstrak yang mewakili kelas-kelas yang diwakilinya

Pola *bridge* biasanya digunakan untuk dua perangkat lunak yang saling terhubung namun membutuhkan akses dua arah. Misalkan perangkat lunak yang membutuhkan pertukaran data dimana dari kedua perangkat lunak itu memiliki DBMS yang berbeda, maka biasanya sering menggunakan teknologi *webservice* di kedua perangkat lunak yang saling terhubung tersebut. Sebagai contoh riil misalkan ada dua buah unit, yaitu unit keuangan yang bertanggung jawab memberikan gaji pada karyawan dan unit kepegawaian. Kedua unit itu memiliki perangkat lunak yang berbeda. Unit keuangan membutuhkan data pegawai dan unit kepegawaian membutuhkan data jumlah gaji pegawai. Maka kedua perangkat lunak itu harus saling menarik data. Agar keamanan tetap terjaga dan agar unit lain tidak terlalu banyak memiliki akses pada perangkat lunak unit lain, maka dipakailah teknologi *webservice* di kedua perangkat lunak untuk saling mengirim data yang dibutuhkan. Berikut ilustrasinya:

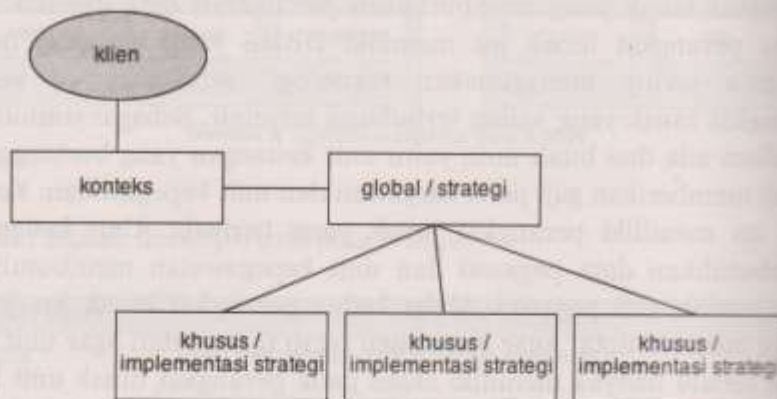


Gambar 9 Ilustrasi Implementasi Pola Bridge

3. Pola yang Berbasis pada Kebiasaan (*Behavioral patterns*)

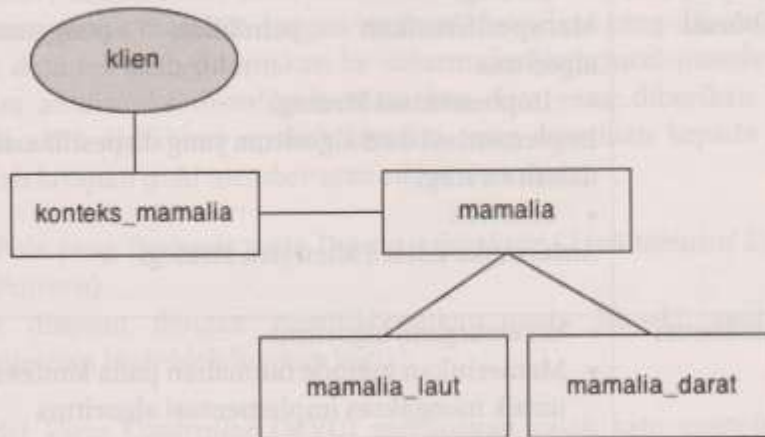
Pola disusun dengan menitikberatkan pada spesifikasi kelakuan sistem aplikasi yang akan dibuat apakah pola yang dibuat sudah sesuai dengan kelakuan atau kemampuan aplikasi yang diharapkan.

- a. *Strategy pattern* merupakan sebuah antarmuka (*interface*) yang berperan sebagai penggeneralisasi dari kelas yang lebih bersifat khusus. Arsitektur pola *strategy* adalah sebagai berikut:



Gambar 10 Arsitektur pola strategy

Misalkan:



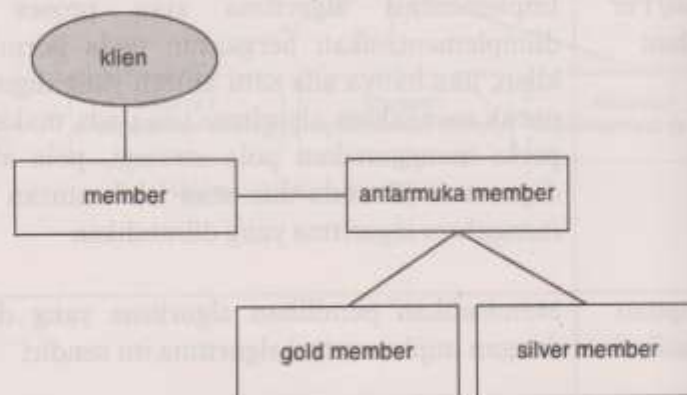
Gambar 11 Contoh arsitektur pola *strategy*

Berikut adalah deskripsi dari pola strategi:

Atribut	Penjelasan
Nama	Pola Strategi(<i>Strategy Pattern</i>)
Tujuan	Mengizinkan penggunaan aturan atau metode yang berbeda tergantung pada konteksnya
Motivasi/Permasalahan	Implementasi algoritma atau proses yang diimplementasikan bergantung pada permintaan klien, jika hanya ada satu aturan yang digunakan untuk mengakses algoritma yang ada maka tidak perlu menggunakan pola strategi, pola strategi digunakan jika ada dua atau lebih aturan untuk mengakses algoritma yang dibutuhkan
Kemampuan diaplikasikan	Memisahkan pemilihan algoritma yang diakses dengan implementasi algoritma itu sendiri

Atribut	Penjelasan
Partisipasi dan kolaborasi	• Strategi Menspesifikasikan pemilihan penggunaan algoritma • Implementasi Strategi Implementasi dari algoritma yang dispesifikasikan dalam strategi • Konteks Antarmuka antara klien dan strategi
Konsekuensi	• kekeluargaan algoritma • Memerlukan metode tambahan pada konteks untuk mengakses implementasi algoritma
Implementasi	Membuat kelas yang menggunakan algoritma (konteks) dan kelas strategi yang memiliki metode abstrak untuk menspesifikasikan pengaksesan implementasi algoritma

Pola strategi digunakan jika klien akan mengakses sebuah layanan dengan satu nama namun sebenarnya dari sisi penyedia layanan sebenarnya ada beberapa layanan yang dapat diakses dengan satu layanan di sisi klien. Misalkan ilustrasi di bawah ini:



Gambar 12 Ilustrasi Implementasi Pola Strategi

Dari ilustrasi di atas misalkan klien akan mengakses layanan untuk *member*, maka di sisi klien hanya akan melihat layanan itu sebagai layanan untuk *member* dengan memasukkan data yang dibutuhkan. Lalu data itu akan diteruskan ke antarmuka (*interface*) *member*. Di dalam antarmuka *member* berdasarkan data yang diberikan klien maka akan dipilih apakah layanan yang diberikan kepada klien adalah layanan *gold member* atau *silver member*.

4. Pola yang Berbasis pada Desain Arsitektur (*Architectural Design Pattern*)

Pola disusun dengan menitikberatkan pada hirarki arsitektur (pemisahan blok-blok lingkup kerja).

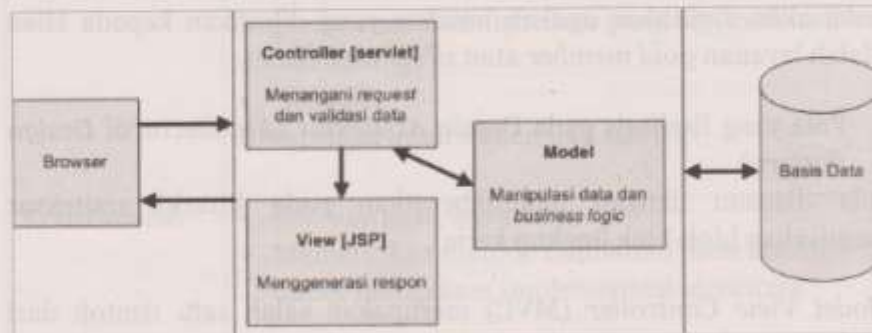
Model View Controller (MVC) merupakan salah satu contoh dari *Architectural Design Pattern*. Pada saat ini banyak *framework* yang didedikasikan pemrograman berorientasi objek.

Apa itu *framework*? *Framework* adalah kerangka kerja yang memudahkan *programmer* untuk membuat sebuah aplikasi sehingga *programmer* akan lebih mudah melakukan perubahan (*customize*) terhadap aplikasinya dan dapat memakainya kembali untuk aplikasi lain yang sejenis. Apa yang disebut sebagai *framework* biasanya sudah memiliki aplikasi implementasinya, maka beda antara pola desain dan *framework* adalah pola desain merupakan sebuah konsep sedangkan *framework* bisa merupakan aplikasi hasil implementasi dari pola desain atau aplikasi kerangka kerja yang tidak berasal dari implementasi sebuah konsep.

Contoh-contoh *framework* yang sering digunakan antara lain sebagai berikut:

- PHP: CodeIgniter, Smarty, cakePHP, Zend, Symfony, Yii, dan lain-lain
- Java: Spring, Hibernate, Struts, dan lain-lain
- Ruby: Ruby on Rails, dan lain-lain
- Python: Django, dan lain-lain
- .NET: ASP .NET MVC Framework, dan lain-lain

Beberapa *framework* untuk membangun aplikasi web berbasis Java telah banyak dibuat, misalnya Struts dari Apache. Struts menggunakan prinsip desain *Model-View Controller* (MVC). Berikut adalah arsitektur dari MVC:



Gambar 13 Arsitektur Struts

Strut sebenarnya merupakan implementasi dari MVC yang berupa pustaka-pustaka (*library*) standar Struts. Konsep MVC bertujuan agar sebuah aplikasi dapat mudah dipelihara oleh orang-orang di dalam tim pengembangan yang berbeda spesifikasi pekerjaan, misalnya *Database Administrator* (DBA) untuk mengurus masalah basis data, blok *controller* untuk *programmer*, dan blok *view* untuk desainer antarmuka (*interface designer*). Struts dapat digunakan untuk membangun sebuah aplikasi web *enterprise* yang memiliki banyak proses kompleks. Komponen dasar dari Struts adalah:

- *framework* dasar
Menyediakan fungsi dasar MVC dan kumpulan blok dasar dari aplikasi. Pondasi dari *framework* dasar adalah *servlet* yang memegang kontrol (*controller servlet*) dan sisanya adalah kelas-kelas yang menangani fungsi-fungsi yang diperlukan.
- pustaka tag JSP (*Java Server Pages*)
Struts dibungkus dengan beberapa pustaka tag JSP sebagai berikut:
 - **HTML** digunakan untuk menggenerasi *form* yang berinteraksi dengan API Struts.

- **Bean** digunakan untuk bekerja dengan objek *Java Bean* misalnya mengakses nilai *bean*.
 - **Logic** digunakan untuk implementasi kondisi logik dalam JSP.
 - **Nested** digunakan untuk penanganan level *nested* dari tag HTML, *Bean*, dan logik.
- *tiles plugin*
Struts dibungkus dengan *tiles subframework*. *Tiles* adalah kumpulan *framework template* JSP yang memfasilitasi penggunaan kembali kode-kode yang telah ada.
 - *plugin validator*
Struts dibungkus dengan *subframework validator* yang menyediakan kumpulan *framework* untuk kebutuhan validasi data pada klien maupun *server*.

Struts dapat di-download pada <http://jakarta.apache.org/struts/>. Pada *site* tersebut juga terdapat keterangan lebih lanjut mengenai Struts dan penggunaanya dengan Apache Tomcat.



MVC merupakan salah satu contoh *architectural design pattern*. Saat ini banyak *framework* yang menggunakan prinsip desain MVC

Masih banyak lagi *design pattern* yang belum di bahas di buku ini karena jumlahnya sangat banyak. *Design pattern* sangat menarik untuk dipelajari jika akan mengembangkan perangkat lunak yang cukup kompleks atau mengalami perkembangan yang cukup kompleks di masa mendatang.

10.2 Anti Pattern

Anti pattern (pola kebalikan) merupakan pola yang dihasilkan dari penelusuran solusi-solusi yang buruk yang pernah digunakan untuk mencari solusi yang baik. *Anti pattern* lebih kepada mengambil pelajaran pada kegagalan-kegagalan yang pernah dialami sebelumnya dan bagaimana cara memperbaikinya. *Anti pattern* lebih bersifat pada perbaikan perangkat lunak berorientasi objek yang sudah ada atau proses pengembangan yang sudah berjalan untuk diperbaiki dengan proses-proses tertentu yang sesuai dengan permasalahan yang dialami. Langkah-langkah membuat *anti pattern* yang baik:

- Tidak menganggap separuh langkah buruk yang sudah ada sebagai sesuatu yang buruk tapi dipandang sebagai sebuah bagian langkan untuk menghasilkan pola yang positif untuk kedepannya.
- Mencoba mencari tahu apa yang salah dengan pola sebelumnya untuk menghasilkan pola yang baik (menangani kesalahan pola yang sudah ada).

Berikut adalah contoh-contoh kesalahan yang digunakan untuk memperbaiki pola pemrograman berorientasi objek:

1. *Spaghetti code* (kode spageti)

Banyak kode program yang tidak efisien dan diulang-ulang. Biasanya terjadi karena *programmer* membuat program dengan asal jalan tanpa memahami konsep pemrograman itu sendiri.

Penyebab terjadinya *spaghetti code*:

- *lone ranger programmer* (pembuat program solo, tanpa tim)
- dokumentasi kuno dan tidak lengkap
- melibatkan pembuat program yang sering ragu-ragu

Cara memperbaiki (*refactoring*):

memilah-milah fungsi yang digunakan ke dalam sub kelas atau membuat super kelas untuk memilah kode.

2. *Stovepipe system* (sistem pipa kompor)
Keterkaitan antar kelas tidak jelas, sehingga ada beberapa kelas yang sebenarnya secara implisit melakukan fungsi yang sama.

Penyebab *Stovepipe system*:

- Impelementasi kode program tidak sesuai dengan dokumentasi
- Analisis yang tidak terbiasa dengan integrasi dan aspek-aspek kunci dari pembuatan perangkat lunak

Cara memperbaiki (*refactoring*):

Mengelompokkan fungsi-fungsi pada kelas menggunakan antarmuka dengan memperhatikan keterkaitan antar kelas.

3. *Analysis paralysis*

Memperlihatkan ketidakseimbangan upaya pada fase analisis dari sebuah proyek, hal ini terjadi karena proses analisis dilakukan oleh banyak orang yang kurang baik koordinasi timnya.

Penyebab *Analysis paralysis*:

- Koordinasi tim analisis yang tidak baik
- Pemaksaan penyelesaian analisis sebelum proses desain
- Terpaku pada paradigma baru dan sudut pandang yang berbeda-beda

Cara memperbaiki (*refactoring*):

- Tidak menggunakan banyak analisis
- Memperkerjakan arsitek perangkat lunak profesional
- Menyediakan prototipe untuk analisis

4. *Design by committee*

Desain kelas-kelas yang digunakan tidak detail sehingga menimbulkan asumsi sendiri pada permasalahan teknis detail dari anggota tim proyek. Hal ini terjadi biasanya karena desain aplikasi dilakukan pada pertemuan-pertemuan anggota tim dengan kepemimpinan pertemuan yang kurang baik.

Penyebab *Design by committee*:

- Dokumen terlalu rumit

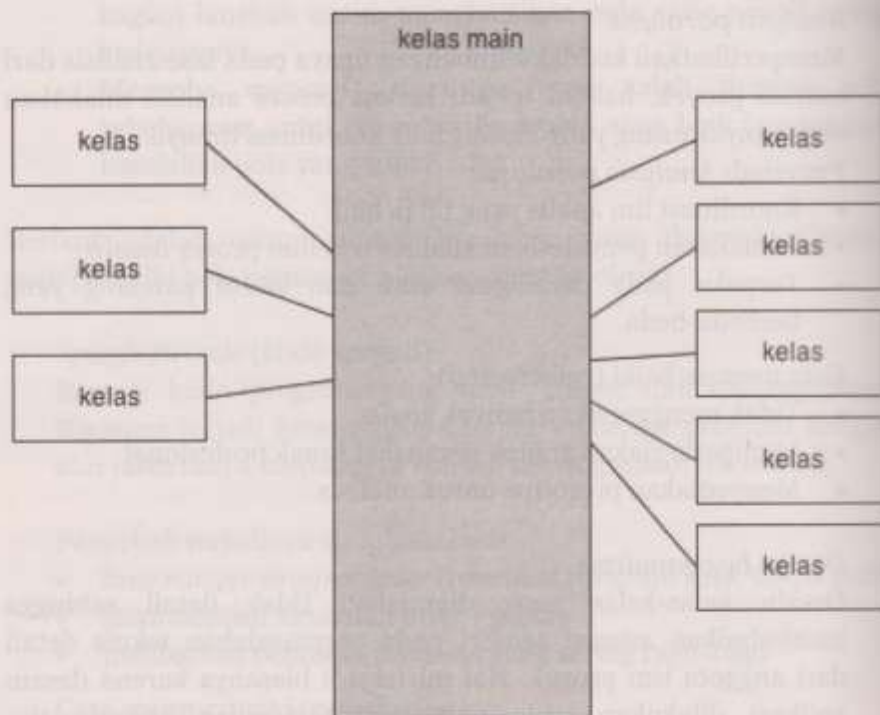
- Pertemuan yang tidak efektif
- Tidak mampu menentukan prioritas fitur
- Tidak ada arsitek perangkat lunak

Cara memperbaiki (*refactoring*):

- Mengefektifkan pertemuan dengan tim yang efektif
- Ada arsitek perangkat lunak

5. *God class*

Ada kelas yang memiliki peranan sangat dominan dalam perancangan kelas dalam sebuah aplikasi sehingga hampir semua kelas yang dibuat bergantung secara langsung maupun tidak langsung pada kelas ini. *God class* biasanya berisi kode program yang sangat panjang karena harus menangani banyak hal.



Gambar 14 Ilustrasi *god class*

Penyebab *God Class*:

- *Single* kelas dengan sejumlah besar atribut, operasi, atau keduanya. Sebuah kelas dengan 60 atau lebih atribut dan operasi mengindikasikan terjadinya *God Class* (kelas Tuhan)
- Tidak memperhatikan kohesi (*cohesion*) atau keterkaitan atribut dan operasi di dalam kelas.
- Berpikiran bahwa kelas yang memegang kontrol hanya satu
- Tidak ada desain berorientasi objek

Cara memperbaiki (*refactoring*):

Mengategorikan atau mengelompokkan atribut-atribut dan operasi-operasi di dalam kelas yang menjadi *God Class* menggunakan prinsip *cohesion* dan *coupling* untuk penyusunan kelas-kelas baru sebagai penguraian dari *God Class*.

6. *Mythical man month*

Desain atau struktur program yang digunakan hanya bergantung pada keputusan satu orang di dalam tim, hal ini akan menimbulkan ketidakdetailan struktur desain aplikasi apalagi jika orang yang dominan tidak memiliki kemampuan yang baik.

Penyebab *Mythical man month*:

- Orang yang memegang peranan penting bukan pembuat program (*programmer*) yang andal
- Menambah banyak orang di tengah proyek karena ketidakmampuan tim yang menjalankan proyek.

Cara memperbaiki (*refactoring*):

- Membuat tim atau sub tim dengan ukuran yang ideal dan memiliki orang berkemampuan baik sebagai pemegang peranan penting dalam tim
- Menyerahkan proyek kepada *developer* berbakat daripada membangun sendiri perangkat lunak dengan staf yang tidak berbakat

7. *Death march project*

Desain dihasilkan dari perkiraan proyek yang tidak realistik, misalnya waktu yang tidak realistik untuk mengembangkan aplikasi, spesifikasi kemampuan aplikasi yang tidak realistik,

sehingga dapat dikatakan desain seperti ini memang ditujukan untuk kegagalan.

Penyebab *Death march project*:

- Desain dilakukan oleh orang yang kurang memiliki kemampuan untuk mendesain perangkat lunak
- Kepala proyek tidak memiliki pengetahuan yang baik mengenai pembangunan perangkat lunak

Cara memperbaiki (*refactoring*):

Memberikan pekerjaan desain dan memimpin proyek kepada orang yang memiliki kemampuan dalam pembangunan perangkat lunak sehingga semua manajemen proyek dapat realistis

Semua kesalahan di atas dapat berakibat fatal yaitu sistem aplikasi tidak dipakai lagi karena akan diganti baru. Hal ini terjadi karena struktur aplikasi sangat sulit untuk dipelihara jika ada perubahan maupun tambahan pada spesifikasi sistem aplikasi.

Anti pattern mencoba menghubungkan apa yang terjadi di dunia nyata dalam pengembangan perangkat lunak dengan perancangan pola yang baik dengan tujuan dapat menyelesaikan permasalahan-permasalahan yang ada pada dunia pengembangan perangkat lunak. *Anti pattern* biasanya diselesaikan dengan perekayasaan kembali (*reengineering*) agar aplikasi dapat lebih mudah untuk dipelihara (*software maintenance*) salah satunya adalah proses *refactoring* dimana perancangan kode program diperbaiki dan disesuaikan dengan kaidah-kaidah pemrograman berorientasi objek yang benar.



Pengembangan perangkat lunak sebaiknya dilakukan dengan baik sejak awal proses agar tidak perlu melakukan *anti pattern*

11

Manajemen Proyek Perangkat Lunak



Overview

Bab ini membahas tentang manajemen proyek perangkat lunak. Pengembangan perangkat lunak biasanya dikerjakan dalam bentuk proyek. Ada hal-hal yang harus dikelola agar perangkat lunak yang dihasilkan memenuhi kualitas yang diharapkan dengan waktu, biaya, dan sumber daya sesuai dengan perencanaan awal.

Pada bab ini juga dibahas tentang hal-hal yang dilakukan dalam pengujian perangkat lunak agar menghasilkan perangkat lunak dengan kualitas sesuai yang diharapkan.

11.1 Pengertian Manajemen Proyek Perangkat Lunak

Proyek adalah urutan kegiatan yang unik, kompleks, dan saling terkait, memiliki satu tujuan, dan tujuan harus diselesaikan dalam waktu tertentu, sesuai anggaran, dan memenuhi spesifikasi. Manajemen/pengelolaan proyek perangkat lunak bertujuan agar perangkat lunak yang dibuat sampai ke tangan pelanggan (*customer*) tepat waktu dan sesuai dengan harapan pelanggan (*customer*).

Proyek perangkat lunak dapat didapatkan dari proses-proses berikut:

- Memiliki koneksi atau rekan yang sedang membutuhkan rekanan untuk mengerjakan proyek teknologi informasi (*Information Technology (IT)*).
- Mencari proyek perangkat lunak dari internet. Beberapa situs luar negeri menyediakan iklan para pencari rekanan proyek perangkat lunak untuk mengerjakan perangkat lunak yang mereka butuhkan. Biasanya orang yang mendapat proyek dari internet paling tidak dapat berbahasa inggris dan menjaga reputasi karena jika tidak orang asing sering tidak bisa percaya lagi.
- Pihak swasta yang sedang membutuhkan rekanan untuk mengerjakan proyek teknologi informasi (*Information Technology (IT)*).
- Penunjukan langsung dari instansi pemerintah yang sedang membutuhkan rekanan untuk mengerjakan proyek teknologi informasi (*Information Technology (IT)*).
- Proses pelelangan oleh instansi pemerintah yang membutuhkan rekanan untuk mengerjakan proyek teknologi informasi (*Information Technology (IT)*).

Macam-macam proyek IT yang dapat diperoleh adalah sebagai berikut:

- pengadaan perangkat keras (*hardware*)
- pengembangan perangkat lunak (*software*)
- pemeliharaan perangkat lunak (*software*)
- konsultasi IT (biasanya dilakukan oleh konsultan IT)

Salah satu proyek IT adalah pengembangan dan pemeliharaan perangkat lunak. Manajemen proyek yang akan dibahas pada buku ini lebih terfokus pada proyek perangkat lunak.

Manajemen proyek perangkat lunak sangat dibutuhkan karena permasalahan yang sering timbul pada proyek perangkat lunak tanpa pengelolaan yang baik adalah pembengkakan biaya proyek dan waktu pengerjaan tidak sesuai rencana (molor waktu). Padahal waktu dan biaya biasanya sudah dianggarkan oleh pelanggan (*customer*) dan *developer* perangkat lunak harus mampu mengelolanya agar target perangkat lunak yang akan dibuat dapat dicapai.

Aktifitas manajemen perangkat lunak adalah sebagai berikut:

- Manajemen aktivitas proyek

Aktifitas yang dilakukan pada proyek perangkat lunak adalah sebagai berikut:

- menulis proposal
- perencanaan dan penjadwalan
- pembiayaan proyek
- pengawasan dan peninjauan proyek
- seleksi dan evaluasi anggota proyek
- penulisan laporan dan presentasi

- Perencanaan Proyek

Perencanaan proyek biasanya terdiri dari:

- organisasi proyek
- analisis risiko
- kebutuhan perangkat keras dan perangkat lunak

- pembagian kerja
- penjadwalan proyek
- mekanisme pengawasan dan pelaporan

- **Penjadwalan Proyek**

Hal-hal yang perlu dilakukan pada penjadwalan proyek adalah sebagai berikut:

- membagi kerja proyek menjadi lebih kecil dan memperkirakan waktu, sumber daya, dan personel untuk melakukan tiap bagian kerja
- mengatur urutan pembagian kerja
- meminimalisasi kebergantungan setiap bagian kerja agar tidak terjadi banyak waktu kosong (*delay*) karena saling menunggu bagian kerja lain selesai lebih dulu
- penjadwalan yang baik bergantung pada intuisi dan pengalaman pengelola proyek

Permasalahan yang sering timbul pada saat penjadwalan proyek perangkat lunak:

- memperkirakan waktu, sumber daya, dan biaya bukanlah hal yang mudah, diperlukan intuisi yang tajam dan pengalaman
- pembagian kerja tidak proporsional pada personel
- penambahan orang ke dalam proyek di tengah atau di belakang jalannya proyek dapat membuat proyek tidak tepat waktu karena harus terlalu banyak berkomunikasi untuk membuat orang yang baru masuk mengerti permasalahan yang terjadi
- masalah tidak terduga akan selalu terjadi

- **Manajemen risiko**

Manajemen risiko fokus pada mengidentifikasi risiko dan membuat perencanaan yang dapat meminimalisir risiko yang mungkin terjadi pada proyek perangkat lunak. risiko yang biasanya terjadi pada proyek perangkat lunak adalah sebagai berikut:

- risiko Proyek
 risiko pada penjadwalan dan sumber daya (biasanya kurang karena ketidaksesuaian dengan perencanaan). risiko proyek yang mungkin terjadi adalah sebagai berikut:
 - pergantian orang di dalam proyek
 - perubahan pengelolaan
 - tidak tersedianya perangkat keras
- risiko Produk
 risiko yang terjadi pada proyek dapat membuat perangkat lunak yang dihasilkan tidak sesuai harapan. risiko produk yang mungkin terjadi adalah sebagai berikut:
 - perubahan spesifikasi/kebutuhan produk
 - keterlambatan waktu pengumpulan kebutuhan (biasanya terjadi karena pelanggan (*customer*) tidak siap dengan apa yang sebenarnya mereka butuhkan)
 - besarnya sistem tidak dapat diperkirakan
 - perangkat pendukung proyek tidak ada
- risiko Bisnis
 risiko bisnis dapat menyebabkan masalah pada organisasi proyek dan proses produksi perangkat lunak. risiko bisnis yang sering terjadi adalah sebagai berikut:
 - perubahan teknologi
 - persaingan produk

Aktifitas yang dilakukan pada manajemen risiko adalah sebagai berikut:

- identifikasi risiko (*risk identification*)
 mengidentifikasi risiko proyek, risiko produk, dan risiko bisnis
- analisis risiko (*risk analysis*)
 memperkirakan konsekuensi dari risiko yang mungkin terjadi
- perencanaan risiko (*risk planning*)
 membuat perencanaan untuk meminimalisasi risiko
- pengawasan risiko (*risk monitoring*)

membuat mekanisme pengawasan risiko sepanjang proyek berjalan

Objek-objek yang harus dikelola dalam sebuah proyek perangkat lunak adalah sebagai berikut:

- ▲ orang
orang di sini mengacu pada anggota tim pengembang perangkat lunak
- ▲ produk
produk disini adalah perangkat lunak yang akan dihasilkan dari proses pengembangan
- ▲ proses
proses disini adalah proses pengembangan perangkat lunak
- ▲ proyek
proyek disini adalah proyek perangkat lunak itu sendiri agar berjalan dengan baik dan memenuhi target yang diinginkan



Objek-objek yang harus dikelola dalam manajemen proyek perangkat lunak meliputi orang, produk, proses, dan proyek.

Mengepalai atau memimpin sebuah proyek perangkat lunak bukanlah merupakan hal yang mudah. Diperlukan ketajaman analisis untuk menganalisis semua kebutuhan proyek yang dapat menimbulkan risiko proyek. Terjadinya risiko dalam proyek perangkat lunak dapat menyebabkan biaya membengkak atau waktu pengerjaan yang molor berkepanjangan. Maka dari itu diperlukan pengalaman yang baik untuk menanggulangi semua risiko yang mungkin muncul.

Kemampuan untuk menjalankan sebuah proyek perangkat lunak adalah sebagai berikut:

- ▲ mendefinisikan lingkup proyek perangkat lunak

- ▲ memperkirakan sumber daya yang dibutuhkan untuk menyelesaikan proyek perangkat lunak
- ▲ memimpin peninjauan (*review*) terhadap dokumentasi dan kode program
- ▲ mengumpulkan kebutuhan perangkat lunak dan membuat spesifikasi kebutuhan perangkat lunak
- ▲ mengelola desain, pemrograman, dan pengujian perangkat lunak dengan efektif
- ▲ membimbing proyek jika proyek mengalami masalah kualitas perangkat lunak
- ▲ mengelola sumber daya luar proyek (*outsorce*)
- ▲ membuat perubahan yang efektif pada jalannya proyek di dalam organisasi.

Oleh karena itu seorang pemimpin proyek sebaiknya adalah orang yang berpengalaman dan memiliki kredibilitas yang baik.

11.2 Perencanaan Proyek

Hal-hal yang harus masuk dalam perencanaan proyek adalah sebagai berikut:

- ▲ Pernyataan kerja yang merepresentasikan produk seperti spesifikasi perangkat lunak, rencana pengujian perangkat lunak, perencanaan kode program perangkat lunak, perencanaan laporan kesalahan/kecacatan (*defect*), dan semua perencanaan pekerjaan selama proyek berjalan.
- ▲ Daftar semua sumber daya (*resources*) yang dibutuhkan untuk pada rekayasa perangkat lunak beserta ketersediaannya.

Semua kemungkinan sumber daya harus diperhitungkan dengan detail, masalah kemampuan sumber daya dan ketersediaannya. Semakin banyak sumber daya yang dilibatkan dengan kemampuan yang kurang maka tidak akan membuat proyek berhasil, tapi kekurangan sumber daya juga dapat memperburuk kondisi proyek. Terkadang lebih baik

melibatkan sumber daya secukupnya dengan kemampuan sesuai dengan yang dibutuhkan untuk menjalankan proyek. Sumber daya di sini dapat berupa sumber daya manusia ataupun perangkat kerja.

- ▲ Rincian struktur pekerjaan dan kumpulan perkiraan pekerjaan yang dilakukan selama proses rekayasa perangkat lunak.

Rincian ini juga harus dilakukan secara detail, jangan sampai ada sebuah pekerjaan yang tidak diprediksi harus dikerjakan di tengah proyek, hal ini dapat menyebabkan kebingungan siapakah yang harus bertanggung jawab pada pekerjaan itu. Jika sumber daya manusia untuk pekerjaan lain dipekerjakan untuk mengerjakan pekerjaan yang tiba-tiba muncul maka pekerjaan lain akan menjadi terbengkalai dan proyek menjadi molor.

- ▲ Perencanaan jadwal dan biaya proyek.

Hal ini harus didasari dengan perencanaan yang sangat detail dan estimasi yang tepat. Jika perencanaan jadwal tidak tepat maka bisa jadi jadwal yang telah ditetapkan menjadi molor pada kenyataannya. Molornya jadwal dapat berimbas pada membengkaknya biaya (terutama jika mempergunakan pekerja *outsourcing*) dan rusaknya kredibilitas atau reputasi pekerja proyek perangkat lunak.

Maka dari itu rincian penjadwalan atas pekerjaan yang harus dikerjakan perlu dilakukan sedetail mungkin. Begitu pula jika terjadi pembengkakan biaya maka akan terjadi kebingungan siapakah yang harus menanggung biaya sedangkan pelanggan (*customer*) berpegang pada kontrak kerja di awal proyek yang biasanya juga berisi biaya yang harus dibayar.

- ▲ Perencanaan risiko yang mengidentifikasi setiap risiko yang mungkin dihadapi dan merencanakan bagaimana risiko itu akan ditangani. Rincian daftar risiko harus diidentifikasi sedetail mungkin agar jika risiko terjadi tidak menjadi kebingungan menanganinya. Risiko biasanya ditulis dalam

sebuah daftar risiko yang berhasil diidentifikasi, untuk setiap risiko diperkirakan kemungkinan terjadinya atau ditentukan skala prioritas terjadinya, lalu dibuatlah penanganan untuk setiap risiko yang berhasil diidentifikasi. Penanganan setiap risiko dapat direncanakan lebih dari satu penanganan untuk mengatasi jika terjadi kegagalan penanganan.

11.3 Pengujian Perangkat Lunak

Sebuah perangkat lunak perlu dijaga kualitasnya bahwa kualitas bergantung kepada kepuasan pelanggan (*customer*). Kualitas perangkat lunak perlu dijaga untuk keperluan sebagai berikut:

- ▲ agar dapat “survive” bertahan hidup di dunia bisnis perangkat lunak
- ▲ dapat bersaing dengan perangkat lunak yang lain
- ▲ penting untuk pemasaran global (*global marketing*)
- ▲ mengefektifkan biaya agar tidak banyak membuang perangkat lunak karena kegagalan pemasaran atau kegagalan produksi
- ▲ mempertahankan pelanggan (*customer*) dan meningkatkan keuntungan.

Sering perangkat lunak mengandung kesalahan (*error*) pada proses-proses tertentu pada saat perangkat lunak sudah berada di tangan *user*. Kesalahan-kesalahan (*error*) pada perangkat lunak ini sering disebut dengan “bug”. Untuk menghindari banyaknya *bug* maka diperlukan adanya pengujian perangkat lunak sebelum perangkat lunak diberikan ke pelanggan atau selama perangkat lunak masih terus dikembangkan.

Adanya *bug* adalah suatu yang biasa, bahkan di sebuah perangkat lunak yang sudah besar dan terkenal pun biasanya masih ada *bug*, sehingga tidak perlu merasa tersinggung atau bersedih jika masih ditemukan *bug* pada perangkat lunak yang dikembangkan. Yang bisa dilakukan pengembang perangkat lunak adalah meminimalisir *bug* dengan melakukan pengujian. Kelakuan perangkat lunak yang tidak sesuai dengan spesifikasi yang dibutuhkan bisa dianggap sebagai *bug*.

Pengujian diperlukan tidak hanya untuk meminimalisasi kesalahan secara teknis tapi juga kesalahan non teknis (misalnya pengujian pesan kesalahan sehingga *user* tidak bingung atau tidak mengerti dengan pesan kesalahan yang muncul, atau juga jika masukan dan keluaran yang diperlukan berkapasitas sangat besar).

Pengujian adalah satu set aktifitas yang direncanakan dan sistematis untuk menguji atau mengevaluasi kebenaran yang diinginkan. Aktifitas pengujian terdiri dari satu set atau sekumpulan langkah dimana dapat menempatkan desain kasus uji yang spesifik dan metode pengujian. Secara umum pola pengujian pada perangkat lunak adalah sebagai berikut:

- ▲ Pengujian dimulai dari level komponen hingga integrasi antar komponen menjadi sebuah sistem.
- ▲ Teknik pengujian berbeda-beda sesuai dengan berbagai sisi atau unit uji dalam waktu yang berbeda-beda pula bergantung pada pengujian pada bagian mana yang dibutuhkan.
- ▲ Pengujian dilakukan oleh pengembang perangkat lunak, dan jika untuk proyek besar, pengujian bisa dilakukan oleh tim uji yang tidak terkait dengan tim pengembang perangkat lunak (*independent test group* (ITG)).
- ▲ Pengujian dan penirkutuan (*debugging*) merupakan aktifitas yang berbeda, tapi penirkutuan (*debugging*) harus diakomodasi pada berbagai strategi pengujian. Pengujian lebih fokus untuk mencari adanya kesalahan (*error*) baik dari sudut pandang orang secara umum atau dari sudut pandang pengembang tanpa harus menemukan lokasi kesalahan pada kode program. Penirkutuan (*debugging*) adalah proses mencari lokasi kesalahan (*error*) pada kode program sehingga dapat segera diperbaiki oleh pembuat program (*programmer*).

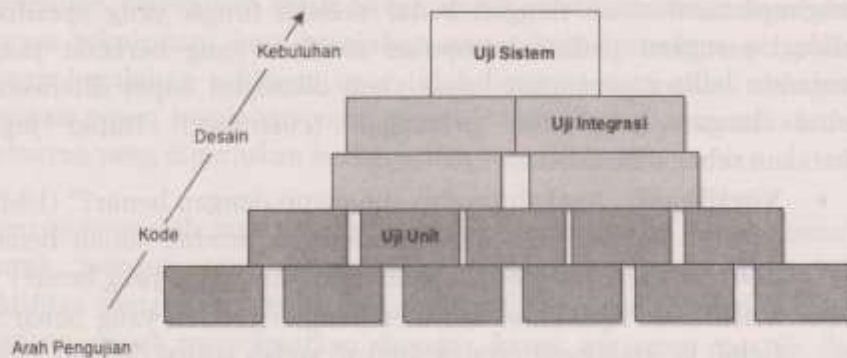
Pengujian perangkat lunak adalah sebuah elemen sebuah topik yang memiliki cakupan luas dan sering dikaitkan dengan verifikasi (*verification*) dan validasi (*validation*) (V&V). Verifikasi mengacu pada sekumpulan aktifitas yang menjamin bahwa perangkat lunak

mengimplementasikan dengan benar sebuah fungsi yang spesifik. Validasi mengacu pada sekumpulan aktifitas yang berbeda yang menjamin bahwa perangkat lunak yang dibangun dapat ditelusuri sesuai dengan kebutuhan pelanggan (*customer*). Dapat juga dikatakan sebagai berikut:

- **Verifikasi:** "Apakah produk dibangun dengan benar?" (lebih ke arah apakah proses pengembangan produk sudah benar dan telah berhasil mengimplementasikan fungsi yang benar)
- **Validasi:** "Apakah sudah membangun produk yang benar?" (lebih ke arah hasil produk apakah sudah sesuai dengan yang diinginkan)

Verifikasi dan validasi meliputi beberapa aktifitas yang sering disebut sebagai jaminan kualitas perangkat lunak (*Software Quality Assurance (SQA)*). Verifikasi dan validasi mencakup lingkup yang luas dari aktifitas SQA yang di dalamnya termasuk peninjauan teknis formal, audit kualitas dan konfigurasi, performansi pengawasan (*monitoring*), simulasi, studi kemungkinan (*feasibility*), peninjauan dokumentasi, peninjauan basis data, analisis algoritma, pengujian proses pengembangan perangkat lunak, pengujian kualifikasi, dan pengujian instalasi. Untuk melakukan pengujian diperlukan pendefinisian kasus uji (*test case*) yang jelas.

Pengujian untuk verifikasi dilakukan mulai dari lingkup yang kecil naik ke lingkup yang besar seperti pada gambar berikut:



Gambar 102 Hirarki pengujian sistem

Gambar di atas menunjukkan tahap pengujian pada level program di tangan pengembang perangkat lunak. Tahapan pengujian yang secara keseluruhan adalah sebagai berikut:



Gambar 103 Pengujian perangkat lunak

Pengujian diawali dari pengujian unit. Unit disini bisa berupa kumpulan fungsi atau prosedur yang memiliki keterkaitan pada pemrograman terstruktur (misalkan unit untuk menuliskan atau membaca data di basis data) atau kelas pada pemrograman berorientasi objek. Unit juga dapat berupa modul atau dikenal juga sebagai *package*. Setelah unit-unit selesai diuji maka dilakukan pengujian integrasi.



Pengujian sistem harus dilakukan bertahap sejak awal pengembangan, jika pengujian hanya di akhir maka dapat dipastikan kualitas sistemnya jelek.

Pengujian integrasi sebaiknya dilakukan secara bertahap, tidak dilakukan secara satu tahap langsung di akhir untuk menghindari kesulitan penelusuran jika terjadi kesalahan (*error*). Pengujian integrasi lebih pada pengujian penggabungan dari dua atau lebih unit pada perangkat lunak. Setelah pengujian integrasi maka dilakukan pengujian sistem dimana unit-unit proses yang sudah diintegrasikan diuji dengan antarmuka yang sudah dibuat sehingga pengujian ini dimaksudkan untuk menguji sistem perangkat lunak secara keseluruhan dan diuji secara satu sistem (tidak terpisah-pisah lagi).

Setelah pengujian sistem selesai dilakukan maka dapat dilakukan pengujian penerimaan perangkat lunak oleh pelanggan (*customer*) atau *user* (pemakai perangkat lunak). Pengujian penerimaan digunakan untuk mengetahui kepuasan pelanggan atau *user* terhadap perangkat lunak yang sudah dibuat. Jika pelanggan sudah puas dengan perangkat lunak, maka perangkat lunak dapat diserahkan kepada pelanggan (*customer*).

Pengujian untuk validasi memiliki beberapa pendekatan sebagai berikut:

- ▲ *Black-Box Testing* (pengujian kotak hitam)
yaitu menguji perangkat lunak dari segi spesifikasi fungsional tanpa menguji desain dan kode program. Pengujian dimaksudkan untuk mengetahui apakah fungsi-fungsi, masukan, dan keluaran dari perangkat lunak sesuai dengan spesifikasi yang dibutuhkan.

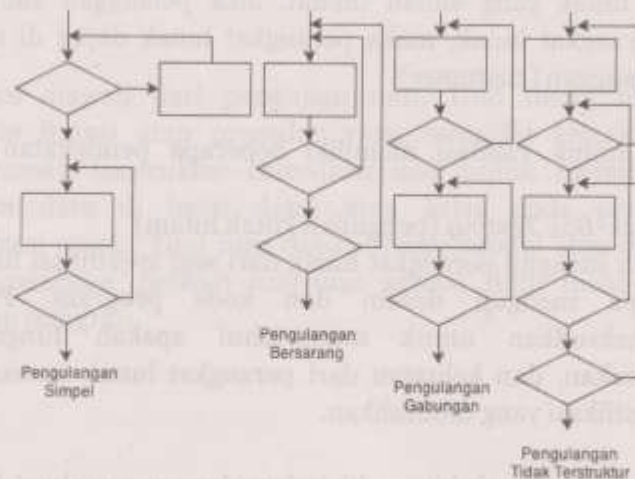
Pengujian kotak hitam dilakukan dengan membuat kasus uji yang bersifat mencoba semua fungsi dengan memakai

perangkat lunak apakah sesuai dengan spesifikasi yang dibutuhkan. Kasus uji yang dibuat untuk melakukan pengujian kotak hitam harus dibuat dengan kasus benar dan kasus salah, misalkan untuk kasus proses login maka kasus uji yang dibuat adalah:

- jika user memasukkan nama pemakai (*username*) dan kata sandi (*password*) yang benar
- jika user memasukkan nama pemakai (*username*) dan kata sandi (*password*) yang salah, misalnya nama pemakai benar tapi kata sandi salah, atau sebaliknya, atau keduanya salah.

★ *White-Box Testing* (pengujian kotak putih)

yaitu menguji perangkat lunak dari segi desain dan kode program apakah mampu menghasilkan fungsi-fungsi, masukan, dan keluaran yang sesuai dengan spesifikasi kebutuhan. Pengujian kotak putih dilakukan dengan memeriksa logik dari kode program. Pembuatan kasus uji bisa mengikuti standar pengujian dari standar pemrograman yang seharusnya. Contoh dari pengujian kotak putih misalkan menguji alur (dengan menelusuri) pengulangan (*looping*) pada logika pemrograman seperti ilustrasi berikut:



Gambar 104 Contoh pengujian kotak putih

Pengujian terhadap dokumentasi yang dibuat juga harus dilakukan agar dokumentasi yang dibuat tetap konsisten dengan perangkat lunak yang dibuat.

11.3.1 Pengujian Unit

Pengujian unit fokus pada usaha verifikasi pada unit yang terkecil pada desain perangkat lunak (komponen atau modul perangkat lunak). Setiap unit perangkat lunak diuji agar dapat diperiksa apakah aliran masukan (*input*) dan keluaran (*output*) dari unit sudah sesuai dengan yang diinginkan. Pengujian unit biasanya dilakukan saat kode program dibuat. Karena dalam sebuah perangkat lunak banyak memiliki unit-unit kecil maka untuk menguji unit-unit kecil ini biasanya dibuat program kecil (*main program*) untuk menguji unit-unit perangkat lunak.

Unit disini secara fisik dapat berupa prosedur atau fungsi, sekumpulan prosedur atau fungsi yang ada dalam satu berkas (*file*) jika dalam pemrograman terstruktur, atau kelas, bisa juga kumpulan kelas dalam satu *package* dalam pemrograman berorientasi objek. Ilustrasi pengujian unit adalah sebagai berikut:



Gambar 105 Ilustrasi pengujian unit

Setiap unit diuji menggunakan sebuah program pengujian yang khusus dibuat untuk menguji sebuah unit menggunakan kumpulan kasus uji yang didefinisikan.

11.3.2 Pengujian Integrasi

Pengujian integrasi adalah sebuah teknis yang sistematis untuk mengonstruksi struktur program seiring dengan menggabungkan fungsi program dengan antarmukanya. Pengujian terintegrasi bertujuan untuk mempergunakan komponen unit program yang sudah diuji dan membangun struktur seperti yang telah didesain sebelumnya.

Ada sebuah kecenderungan pada pengujian integrasi dimana integrasi tidak dilakukan secara bertahap, tapi langsung dilakukan pada akhir pengembangan perangkat lunak ("big bang"). Dengan menggunakan pendekatan "big bang" sebuah sistem diuji secara kesatuan sehingga sering ketika terjadi kesalahan (*error*) akan menemui kesulitan untuk menemukan dimana letak kesalahan (*error*) yang terjadi.

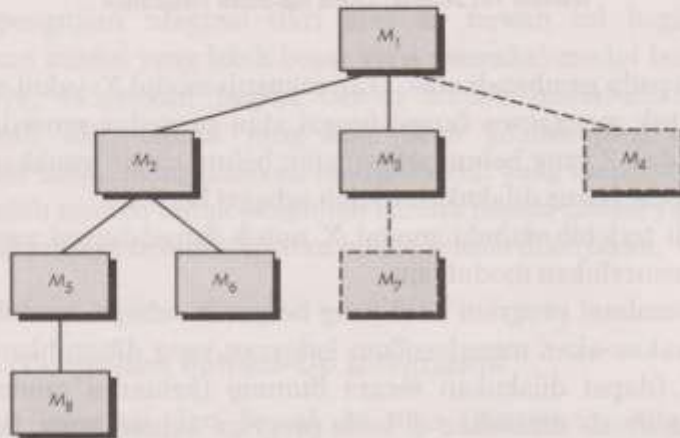
Integrasi secara bertahap (*incremental integration*) merupakan kebalikan dari pendekatan "big bang". Program dikonstruksi dan diuji secara bertahap sehingga kesalahan (*error*) yang terjadi lebih mudah diketahui letak kesalahannya dan diperbaiki. Antarmuka juga harus diuji secara lengkap sehingga diperlukan sebuah pengujian yang sistematis untuk menguji antarmuka beserta fungsi-fungsinya. Pengujian integrasi memiliki beberapa tipe strategi pengujian seperti sebagai berikut:

- ▲ pengujian integrasi dari atas ke bawah (*top-down integration*)
- ▲ pengujian integrasi dari bawah ke atas (*bottom-up integration*)
- ▲ pengujian integrasi regresi (*regression integration*)
- ▲ pengujian integrasi "asap" (*smoke integration*)
- ▲ pengujian integrasi "roti isi" (*sandwich integration*) yaitu mengombinasikan lebih dari satu strategi pengujian integrasi (misalnya pengujian dari atas ke bawah dan pengujian dari bawah ke atas)

Tipe-tipe strategi pengujian integrasi ini akan dibahas lebih detail pada subbab-subbab berikut.

11.3.2.1 Pengujian Top-Down Integration

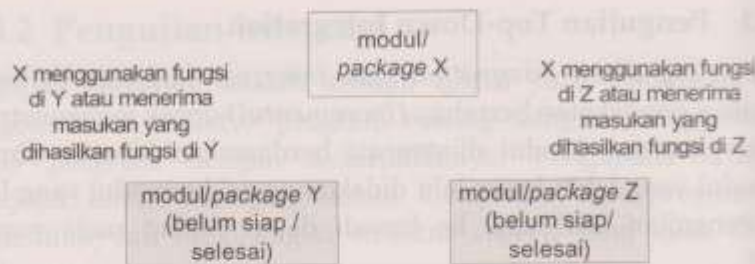
Pengujian *top-down integration* atau integrasi dari atas ke bawah merupakan pendekatan bertahap (*incremental*) untuk mengonstruksi struktur program. Modul diintegrasikan berdasarkan hirarki, dimulai dari modul yang lebih besar lalu didekomposisi ke modul yang lebih kecil. Pengujian dari atas ke bawah dapat dilihat pada gambar berikut:



Gambar 106 Ilustrasi hirarki *top-down integration*

Di mana urutan pengujian setiap modul sama dengan ilustrasi gambar di atas.

Pengujian dari atas ke bawah biasanya sering digunakan untuk menguji prototipe dimana sebuah prototipe biasanya pada awalnya dibuat secara sistem tapi berupa prototipe (alur kerja sudah dapat dilihat tapi sebenarnya fungsi-fungsi yang ada belum berjalan benar, fungsinya hanya untuk menunjukkan ke *user* atau pelanggan seperti apa sistem yang akan dihasilkan). Berikut adalah ilustrasi siklus dari pengujian dari atas ke bawah:



Gambar 107 Ilustrasi siklus *top-down integration*

Misalkan pada gambar di atas, akan menguji modul X, yakni modul X perlu untuk mengakses fungsi-fungsi atau prosedur-prosedur pada modul Y dan Z yang belum selesai atau belum siap digunakan, maka langkah yang harus dilakukan adalah sebagai berikut:

1. uji terlebih dahulu modul X untuk fungsi-fungsi yang tidak memerlukan modul lain.
2. membuat program kecil yang berperan sebagai modul Y yang seakan-akan menghasilkan keluaran yang dibutuhkan modul X (dapat dilakukan secara *dummy* (keluaran random atau *hardcode* dituliskan di kode program keluarannya, jadi tidak harus menggunakan modul Y yang sudah jadi)), misalkan jika membutuhkan keluaran berupa angka nilai mata uang:

```
//modul YDummy
function getSaldo() {
    return 5999999;
}
```

maka modul X dapat menggunakan modul Y sebagai berikut:

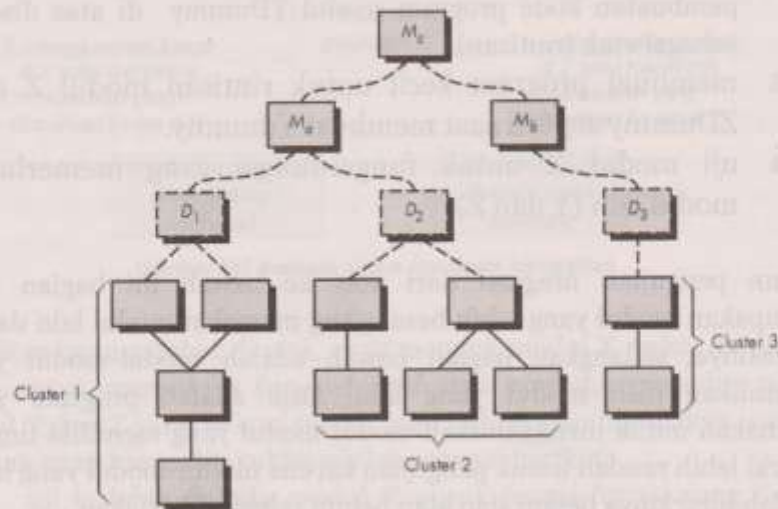
```
//modul X
include YDummy;
main() {
    .....
    print(getSaldo());
    .....
}
```


- pembuatan kode program modul YDummy di atas disebut sebagai *stub* (rintisan).
3. membuat program kecil untuk rintisan modul Z atau ZDummy seperti saat membuat Ydummy.
 4. uji modul X untuk fungsi-fungsi yang memerlukan modul lain (Y dan Z).

Dalam pengujian ntegrasi dari atas ke bawah ini bagian atas merupakan modul yang lebih besar yang memakai modul lain dalam operasinya, sedangkan bagian bawah adalah modul-modul yang dibutuhkan oleh modul yang lain. *Stub* adalah program yang digunakan untuk menggantikan modul-modul yang memiliki tingkat hirarki lebih rendah untuk pengujian karena modul-modul yang lebih rendah hirarkinya belum siap atau belum selesai dikerjakan.

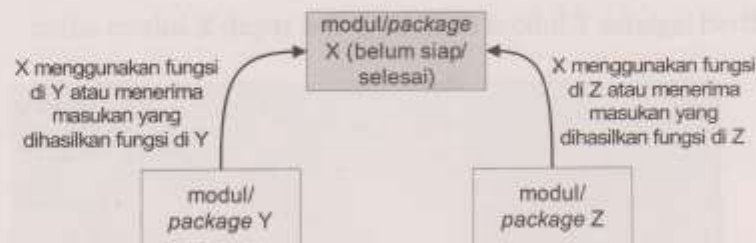
11.3.2.2 Pengujian Bottom-Up Integration

Pengujian integrasi dari bawah ke atas (*bottom-up integration*) memulai pengujian dari modul yang paling kecil ke modul yang lebih besar. Pengujian dari bawah ke atas sering dilakukan untuk pengembangan perangkat lunak yang tidak menggunakan alur prototipe sehingga perangkat lunak dibangun dari modul-modul yang kecil ke modul-modul yang besar sesuai dengan hirarki pemakaiannya. Ilustrasi pengujian dari bawah ke atas adalah sebagai berikut:



Gambar 108 Ilustrasi hirarki *bottom-up integration*

Beberapa modul yang memiliki hirarki paling rendah dibuat menjadi beberapa *cluster* (kelompok) berdasarkan keterkaitan fungsinya. Dari setiap *cluster* dibuat sebuah program untuk menguji setiap *cluster* dimana setiap program yang digunakan untuk menguji ini harus mampu menguji setiap fungsi yang ada di dalam *cluster*. Program yang digunakan untuk menguji fungsionalitas *cluster* disebut *driver*. Setelah setiap *cluster* selesai diuji dengan menggunakan program *driver* maka dilanjutkan pengujian dengan modul yang lebih tinggi hirarkinya. Ilustrasi pengujian tiap *cluster* adalah sebagai berikut:



Gambar 109 Ilustrasi pengujian tiap *cluster*

Misalkan pada gambar di atas, akan menguji modul Y dan Z, dimana modul Y dan Z diakses oleh modul X yang belum selesai atau belum siap digunakan, maka langkah yang harus dilakukan adalah sebagai berikut:

1. uji terlebih dahulu modul Y dan Z.
2. membuat program yang berperan sebagai modul X yang seakan-akan adalah modul X yang mengakses modul Y dan Z, misalkan sebagai berikut:

```
modul Y  
  
function getSaldo(){  
.....  
}
```

```
//modul Z  
  
function getNasabah(){  
.....  
}
```

maka modul XDummy dapat menggunakan modul Y dan modul Z sebagai berikut:

```
//modul XDummy  
  
include Y;  
include Z;  
  
main(){  
.....  
    print(getSaldo());  
.....  
    getNasabah();  
.....  
}
```

Pembuatan kode program modul XDummy di atas disebut sebagai *driver* (pemegang kontrol). *Driver* adalah program yang digunakan untuk menggantikan modul yang memiliki tingkat hirarki lebih tinggi

untuk pengujian karena modul-modul yang lebih tinggi hirarkinya belum siap atau belum selesai dikerjakan.

11.3.2.3 Pengujian Regression Integration

Setiap modul baru ditambahkan sebagai bagian dari integrasi maka kondisi perangkat lunak menjadi berubah. Hal ini mungkin saja menyebabkan masalah pada fungsionalitas yang sudah diuji sebelumnya. Pengujian regresi (*regression*) adalah eksekusi dari beberapa subset pengujian yang sudah terhubung atau saling terkait untuk menjamin bahwa modul yang baru masuk pengujian tidak mengubah fungsionalitas yang sudah diuji sebelumnya. Pengujian regresi dapat dilakukan secara manual dengan mengeksekusi sebuah subset untuk semua kasus uji dari subset itu atau bisa juga menggunakan perangkat (*tools*).

Pengujian regresi lebih sesuai menggunakan tiga kelompok kasus pengujian sebagai berikut:

- ▲ kelas kasus uji yang berisi contoh kasus pengujian yang dapat menguji semua fungsi perangkat lunak
- ▲ kelas kasus uji yang berisi kasus tambahan yang fokus pada fungsi perangkat lunak yang akan terpengaruh jika ada tambahan modul baru untuk diuji
- ▲ kelas kasus uji yang berisi kasus yang fokus pada komponen atau modul baru atau yang mengalami perubahan

Pengujian regresi sesuai digunakan untuk pengujian perangkat lunak yang memiliki struktur yang baik, karena jika digunakan pada perangkat lunak dengan struktur yang tidak baik maka bisa jadi lingkup kasus pengujian menjadi besar dan melebar. Pengujian regresi lebih fokus pada pengujian area dari komponen yang mengalami tambahan, perubahan, atau perbaikan.

11.3.2.4 Pengujian Smoke Integration

Pengujian “asap” (*smoke testing*) adalah sebuah pendekatan pengujian integrasi yang biasa digunakan ketika waktu pengerjaan perangkat lunak cukup singkat dan biasanya untuk komponen atau modul yang ditambahkan pada perangkat lunak. Pengujian asap awalnya berasal dari industri perangkat keras (*hardware*) dimana setelah sebuah komponen perangkat keras diubah atau diperbaiki maka perangkat keras akan dinyalakan, jika tidak ada asap yang keluar maka komponen itu berhasil lolos uji. Pengujian “asap” meliputi aktifitas-aktifitas berikut:

- ▲ mempersiapkan komponen atau modul perangkat lunak yang sudah ditranslasi menjadi kode program diintegrasikan dengan komponen lain yang terkait seperti berkas (*file*) data, pustaka (*libraries*), modul lain yang digunakan kembali (*reusable*), dan komponen rekayasa lainnya yang diperlukan untuk implementasi satu atau lebih fungsi perangkat lunak (merupakan komponen yang diubah atau diperbaiki)
- ▲ mempersiapkan sekumpulan pengujian yang didesain untuk menemukan kesalahan (*error*) yang menjaga perangkat lunak tetap memenuhi fungsinya
- ▲ mengintegrasikan kumpulan kode program, berkas (*file*) data, pustaka (*libraries*), modul lain yang digunakan kembali (*reusable*), dan komponen rekayasa lainnya yang diperlukan dengan kumpulan yang lain dan diuji per hari agar setiap penambahan komponen per hari dapat teruji, pendekatan yang dilakukan bisa menggunakan *top-down* atau *bottom-up*.

Kelebihan pengujian “asap” jika dilakukan pada perangkat lunak yang dikembangkan dengan waktu pendek adalah sebagai berikut:

- ▲ risiko integrasi dapat diminimalisasi karena pengujian dilakukan per hari sehingga ketidaksesuaian antar komponen dapat cepat terdeteksi
- ▲ kualitas produk dapat diperbaiki karena fokus pada pengujian fungsionalitas produk

- ▲ diagnosis kesalahan (*error*) dan perbaikan menjadi lebih mudah karena lebih jelas bahwa kesalahan biasanya muncul pada komponen yang baru saja diubah atau diperbaiki
- ▲ progres mudah untuk diperkirakan setiap harinya.

Pengujian "asap" lebih fokus pada apakah komponen yang diubah atau diperbaiki tidak menyebabkan kesalahan pada fungsionalitas perangkat lunak.



Latihan

1. Apa pengertian manajemen proyek perangkat lunak?
2. Hal-hal apa saja yang harus dikelola dalam manajemen perangkat lunak? Jelaskan!
3. Apa yang dimaksud dengan perencanaan proyek?
4. Apa yang dimaksud dengan pengujian perangkat lunak?
5. Mengapa harus dilakukan pengujian pada perangkat lunak?
6. Hal-hal apa saja yang harus diuji dalam perangkat lunak?
7. Apa perbedaan pengujian *black box* dengan pengujian *white box*? Jelaskan!
8. Apa yang dimaksud dengan:
 - a. Pengujian unit
 - b. Pengujian integrasi
 - c. Pengujian *top-down integration*
 - d. Pengujian *bottom-up integration*
 - e. Pengujian *regression integration*
 - f. Pengujian *smoke integration*

Daftar Pustaka

- _____. 2011. Data Modeling From Conceptual Model to DBMS. Enterprise Architect: Visual Modeling Platform (<http://www.sparxsystems.com>).
- _____. 2013. <https://www.draw.io>.
- _____. 2013. <http://www.visual-paradigm.com/VPGallery/diagrams/>
- _____. 2013. <http://pic.dhe.ibm.com/infocenter/rsarthlp/v8/index.jsp>.
- _____. 2013. <http://www.777icons.com>.
- _____. 2013. UML 2 Interaction Overview Diagram (http://www.sparxsystems.com.au/resources/uml2_tutorial/uml2_interactionoverviewdiagram.html).
- Barker, Richard. 1990. CASE Method: Entity Relationship Modelling. Addison-Wesley Professional.
- Booch, Grady. 1998. Object-oriented analysis and design with applications 2nd edition. Addison Wesley.
- Bowman, Kevin. 2004. System Analysis: A Beginner's Guide. Palgrave Macmillan.
- Brown, William J., Raphael C. Malveau, dkk. 1998. AntiPatterns Refactoring Software, Architectures, and Projects in Crisis. John Wiley & Sons, Inc.
- Greene, Jennifer dan Andrew Stellman. 2005. Applied Software Project Management. O'Reily.
- Hutcheson, Marnie L. 2003. Software Testing Fundamentals: Methods and Metrics. John Wiley & Sons.
- Kling, Rob, Howard Rosenbaum, dan Steve Sawyer. 2005. Understanding and Communicating Social Informatics: A Framework for Studying and Teaching The Human Contexts of

Information and Communication Technologies. Information Today Inc. New Jersey.

Knudson, Joan dan Ira Bitz. 1991. Project Management. Amacom.

Per Kroll, Philippe Kruchten. 2003. Rational Unified Process Made Easy: A Practitioner's Guide to the RUP. Addison Wesley.

Langer, Arthur M. 2008. Analysis and Design of Information Systems 3rd edition. Springer.

Patton, Ron. 2005. Software Testing. Sams Publishing.

Pressman, Roger S. 2001. Software Engineering: A Practitioner's Approach fifth edition. Mc Graw Hill.

Rumbaugh, James dkk. 1999. The Unified Modeling Language Reference Manual. Addison-Wesley.

S, Rosa A. dan M. Shalahuddin. 2010. Modul Pembelajaran Pemrograman Berorientasi Objek. Modula Bandung.

Shalahuddin, M. dan Rosa A. S. 2007. Belajar Pemrograman dengan Bahasa Pemrograman C++ dan Java: Dari Nol Menjadi Handal. Penerbit Informatika.

Shalahuddin, M. dan Rosa A. S. 2008. Analisa dan Desain Sistem Informasi. Politeknik Telkom.

Shalloway, Alan dan James R. Trott. 2004. Design Patterns Explained A New Perspective on Object-Oriented Design Second Edition. Addison-Wesley.

Sommerville, Ian. 2004. Software Engineering: 7th Edition. McGraw-Hill.

Song, Il-Yeol, Mary Evans dan E.K. Park. 1995. A Comparative Analysis of Entity Relationship Diagrams. Journal of Computer and Software Engineering Vol. 3 No. 4 pp. 427-459.

Team, Rational. Rational Unified Process: Best Practices for Software Development Teams. Rational: The Software Development Company.

US Department of Justice. 2003. Information Resources Management.

Wysocki, Robert K. dan Rudd McGary. 2003. Effective Project Management: Third Edition. John Wiley and Sons.

REKAYASA PERANGKAT LUNAK

Rosa A. S
M. Shalahuddin

TERSTRUKTUR dan BERORIENTASI OBJEK



Pada buku ini akan membahas tentang analisis dan desain sistem dengan disertai suatu studi kasus untuk memudahkan dalam pemahaman. Analisis dan desain sistem itu dimulai dengan analisis dan desain basis data, analisis dan desain sistem untuk pemrograman terstruktur dengan menggunakan DFD, dan analisis dan desain sistem untuk pemrograman berorientasi objek dengan menggunakan UML.

Setelah membaca studi kasus tersebut, pembaca diharapkan dapat memahami bagaimana melakukan analisis dan desain sistem untuk pemrograman terstruktur maupun pemrograman berorientasi objek. Untuk menghasilkan analisis dan desain perangkat lunak yang baik, seorang analis seharusnya memahami konsep pemrograman. Tanpa pemahaman konsep pemrograman yang baik, analis tidak mungkin menghasilkan perancangan perangkat lunak yang baik.

Pada buku ini juga dijelaskan mengenai tahapan-tahapan yang harus dilalui dalam rekayasa perangkat lunak serta penjelasan secara umum tentang manajemen proyek perangkat lunak.



Penerbit **INFORMATIKA**
Pemasaran: BI-OBSES
Pasar buku Palasari 62 Bandung 40264
Tel. (022) 7317812 Fax. (022) 7317896
www.biobses.com

ISBN: 978-602-1514-05-4



9 786021 514054